

THE ULTIMATE **ANIMATED**

JAVA

INTERVIEW
QUESTIONS

Crack Interviews with Visual Learning



Development > Programming Languages

The Ultimate Animated Java Interview Questions Course

Conquer Your Java Interview Prep with 180+ Animated Lectures, Code Walkthroughs, 1200+ Quiz Questions & Revision PDFs

New

Created by [Peerless Geek](#)

🕒 Last updated 6/2025 🌐 English 📄 English [CC]



Premium

Access this top-rated course, plus 26,000+ more top-rated courses, with a Udemy plan. [See Plans & Pricing](#)

5.0



15 ratings



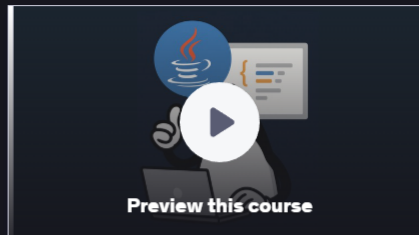
1,046

learners

What you'll learn

- ✓ Master Java concepts from basics to advanced with visual animations and hands-on coding.
- ✓ Confidently tackle Java interviews with 1,200+ MCQs designed to reinforce core concepts.
- ✓ Apply OOP concepts like inheritance, polymorphism, and encapsulation through real-world coding examples.
- ✓ Apply critical thinking to tackle unexpected interview questions by understanding core Java concepts rather than memorizing answers

Explore related topics



Preview this course

Personal

Teams

🔒 This Premium course is included in plans

Subscribe to Udemy's top courses

Get this course, plus 26,000+ of our top-rated courses, with Personal Plan. [Learn more](#)

Try Personal Plan for free

Starting at \$14.00 per month after trial

Cancel anytime

or

\$12.99

Add to cart

Buy now

30-Day Money-Back Guarantee

Full Lifetime Access

Share

Gift this course

Apply Coupon

Welcome, Fellow **Java Explorer**

Before we dive into the depths of interview questions and concepts, I want to take a moment to speak directly to you — the learner.

This guide isn't just a collection of answers. It's a carefully crafted summary of the full course — built with love, effort, and a passion for teaching. While it captures the core ideas and structure of over 180 Java interview questions, **some topics here are simplified or summarized** compared to the detailed explanations we covered in the video lectures.

So, if you've already completed the full course, this PDF will serve as a powerful revision and reinforcement tool. If not, I highly recommend going through the course first — to fully experience the animations, in-depth explanations, and hands-on clarity we explored together.

That said, I truly hope this resource helps you revise smarter and feel more confident in your interview preparation.

If you found it helpful, please consider leaving a review on Udemy. It may seem like a small gesture, but it genuinely motivates me to keep pushing the limits and continue creating better, clearer, and more meaningful learning experiences for you.

Until then, keep exploring, keep coding, and never stop striving for mastery.

With you in code and spirit,

Peerless Geek

Java Educator | Udemy Instructor

Contents

1. Java Basics & Core Concepts	9. Encapsulation
2. Data types and Wrapper classes	10. Inheritance
3. Variables, Constructors & Methods	11. Polymorphism
4. Operators	12. Association (Aggregation & Composition)
5. Strings	13. Abstraction
6. Arrays	14. Generics
7. Object-Oriented Programming (OOP) Basics	15. Collections Framework
8. Classes and Objects	16. Exception Handling
17. Miscellaneous	

Java Basics & Core Concepts

01

QUESTION

What is the difference between **JDK**, **JRE** and **JVM**? 

Explanation



From: The Ultimate Java Interview Questions Course on Udemy © Peerless Geek

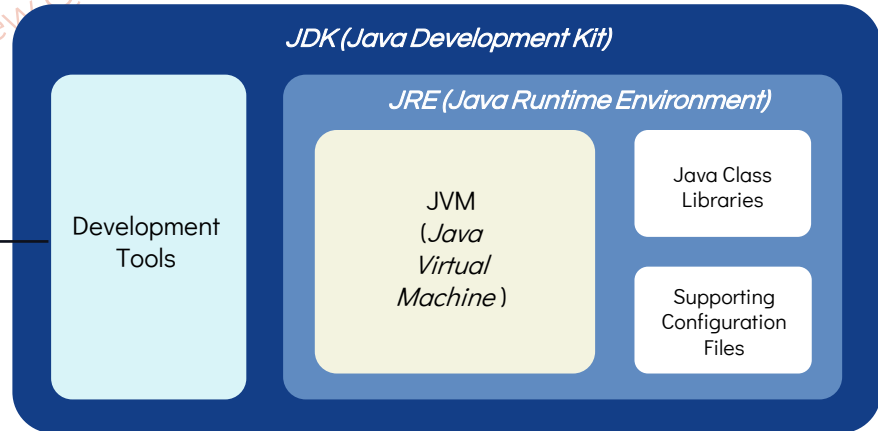
EXPLANATION

JDK (*Java Development Kit*)

The **JDK** (*Java Development Kit*) is a complete software development package for Java applications, which includes the **JRE** (*Java Runtime Environment*) along with **essential development tools** such as the compiler (*javac*), debugger, documentation tools (*javadoc*), and other utilities. It is required for developing Java applications and provides everything needed to write, compile, debug, and run Java code.

Development tools in the JDK are utilities that aid in writing, compiling, debugging, and running Java programs.

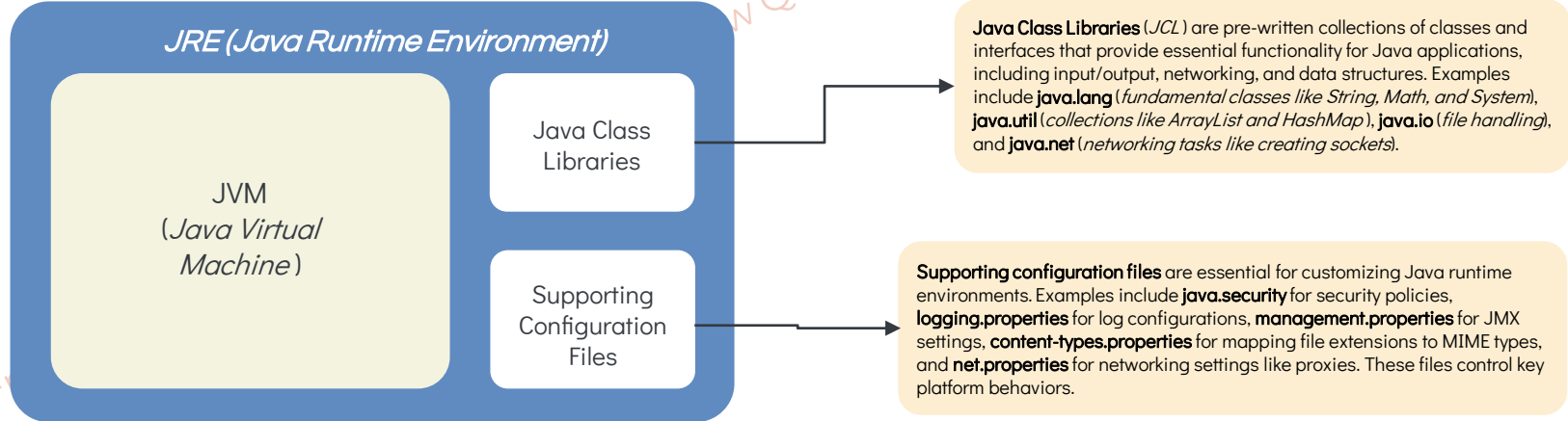
Examples include **javac** (compiler for converting source code to bytecode), **java** (launcher for executing applications), **jdb** (debugger for troubleshooting), **jar** (tool for packaging files into Java archives), and **javadoc** (documentation generator).



EXPLANATION

JRE (*Java Runtime Environment*)

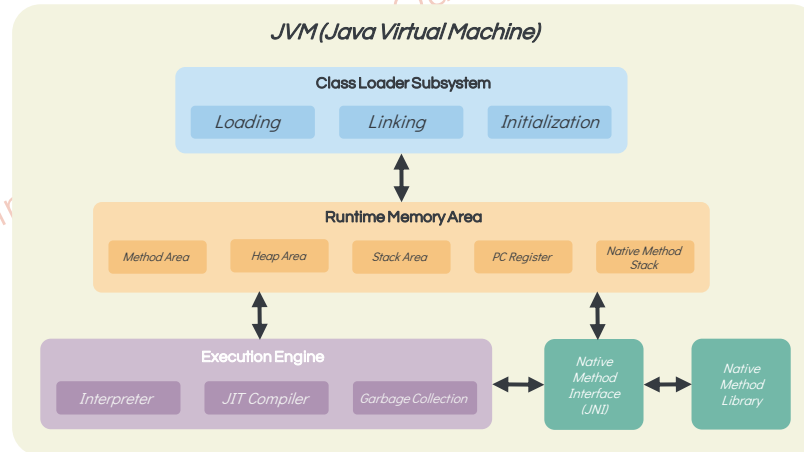
The **JRE** (*Java Runtime Environment*) is a software package that provides the necessary environment to run Java applications, containing everything needed to execute compiled Java programs, including the **Java Virtual Machine (JVM)** and **core libraries** and is designed for end users who only need to run Java applications, the JRE does not include development tools such as a compiler, which are part of the Java Development Kit (JDK).



EXPLANATION

JVM (*Java Virtual Machine*)

The **JVM** (*Java Virtual Machine*) is a core part of the Java platform that runs Java bytecode, enabling Java's "write once, run anywhere" feature. While Java programs are platform-independent, the JVM itself is platform-dependent and handles key tasks like class loading, memory management, code execution, garbage collection, security, and performance optimization via just-in-time (*JIT*) compilation, making it essential for running Java applications across different systems.



From: The Ultimate Anir

EXPLANATION

The Java Trinity: *JDK*, *JRE*, and *JVM*

JDK	JRE	JVM
JDK stands for Java Development Kit.	JRE stands for Java Runtime Environment.	JVM stands for Java Virtual Machine.
It is used for developing Java applications and provides a complete development environment.	It is used for running compiled Java applications on end-user machines.	It is responsible for executing Java bytecode and converting it to machine code.
It contains the complete <i>JRE plus additional development tools and utilities.</i>	It contains the <i>JVM plus the core libraries</i> needed to run Java applications.	It is an abstract machine specification that provides a runtime environment for Java bytecode.
It includes development tools like compiler (<i>javac</i>), debugger, documentation generator (<i>JavaDoc</i>), and other utilities.	It includes only the runtime libraries and utilities needed to run Java programs.	It does not include any tools as it is purely an execution engine.
It is designed for Java developers and programmers who need to create and run Java applications.	It is designed for end users who only need to run Java applications.	It is not directly used by either developers or end users as it's a component of JRE.

QUESTION

What is the JVM?



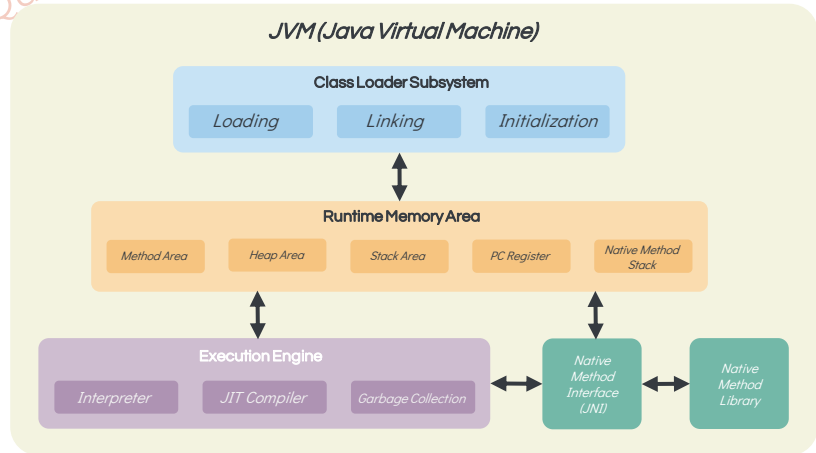
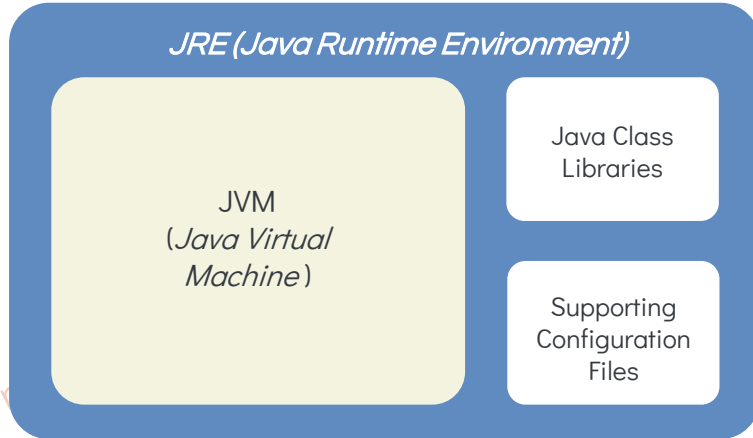
Explanation



EXPLANATION

Java Virtual Machine (*JVM*)

The **Java Virtual Machine** (*JVM*), a core part of the **Java Runtime Environment** (*JRE*), enables platform-independent execution of Java programs by running Java bytecode compiled from source code. Key components of the JVM include the Class Loader for loading class files, the Runtime Data Area for memory allocation, and the Execution Engine for translating bytecode into platform-specific machine code.

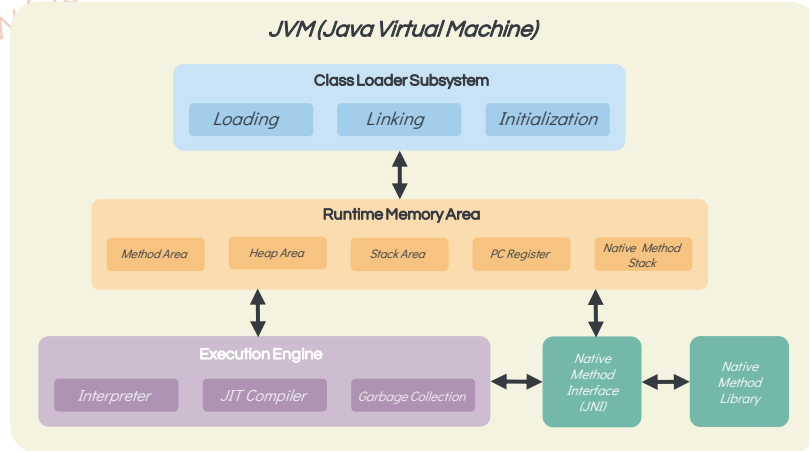
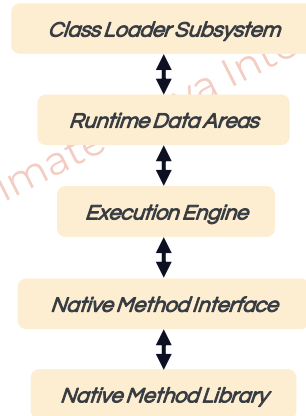


EXPLANATION

Java Virtual Machine (JVM)

The **Java Virtual Machine (JVM)**, a core part of the **Java Runtime Environment (JRE)**, enables platform-independent execution of Java programs by running Java bytecode compiled from source code. Key components of the JVM include the Class Loader for loading class files, the Runtime Data Area for memory allocation, and the Execution Engine for translating bytecode into platform-specific machine code.

Components of JVM



QUESTION

Is Java
platform-
independent? If
yes, how? 

Explanation

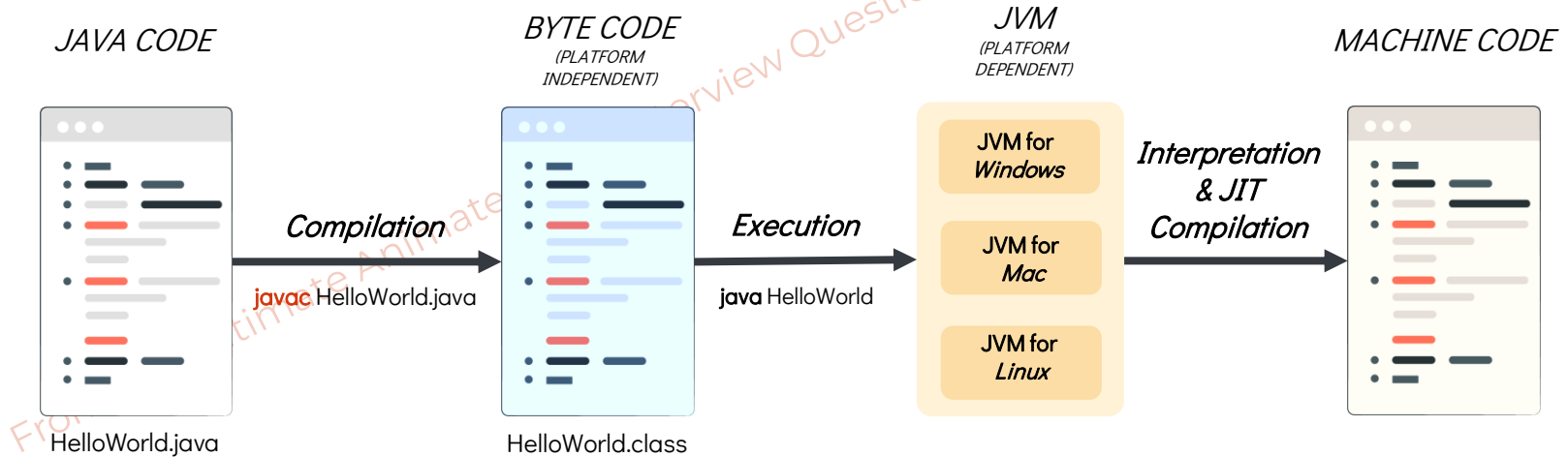


From: The Complete Java Interview Questions Course on Udemy © Peerless Geek

EXPLANATION

Java Bytecode: *The Key to Platform Independence*

Java is a **platform-independent** language because its compiler, **javac**, converts source code into platform-neutral bytecode, which can run on any system with a preinstalled Java Virtual Machine (JVM). While the JVM is platform-dependent, the bytecode can be executed on any system without modification, ensuring Java's portability across different platforms.



EXPLANATION

Java Bytecode: *The Key to Platform Independence*

Java code refers to the source code written by developers in the Java programming language. It is the human-readable, high-level code that follows Java's syntax rules and is saved in files with the .java extension.

Bytecode in Java is an intermediate, *platform-independent* code that the Java compiler generates after compiling a Java program. It is not directly executable by the hardware but is interpreted or executed by the Java Virtual Machine (JVM).

The **Java Virtual Machine (JVM)** is a runtime environment that allows Java programs (*and other programs written in JVM-compatible languages*) to be executed.

Machine code refers to the final, low-level code that the underlying hardware's CPU can execute. It is composed of binary instructions (*sequences of 0s and 1s*) specific to a particular processor architecture.

JAVA CODE

BYTE CODE
(PLATFORM
INDEPENDENT)

JVM
(PLATFORM
DEPENDENT)

MACHINE CODE



HelloWorld.java

Compilation

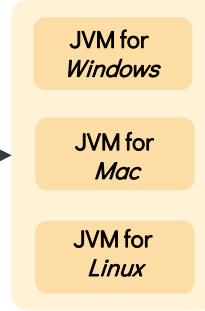
`javac HelloWorld.java`



HelloWorld.class

Execution

`java HelloWorld`



Interpretation
& JIT
Compilation



QUESTION

What is a class loader?



Explanation

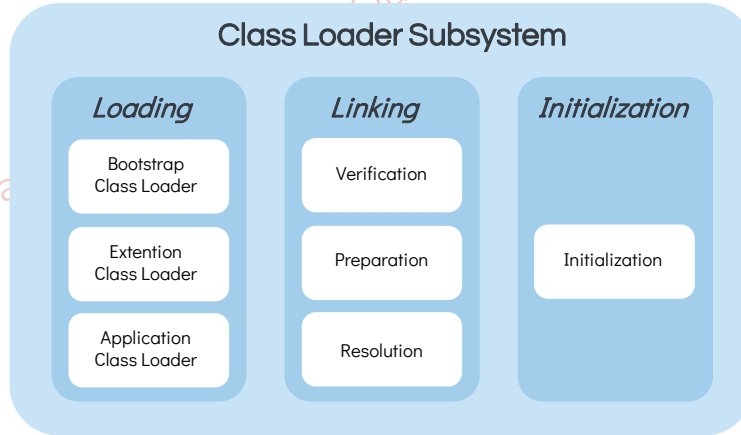


From: The Complete Java Interview Questions Course on Udemy © Peerless Geek

EXPLANATION

Class Loader Subsystem

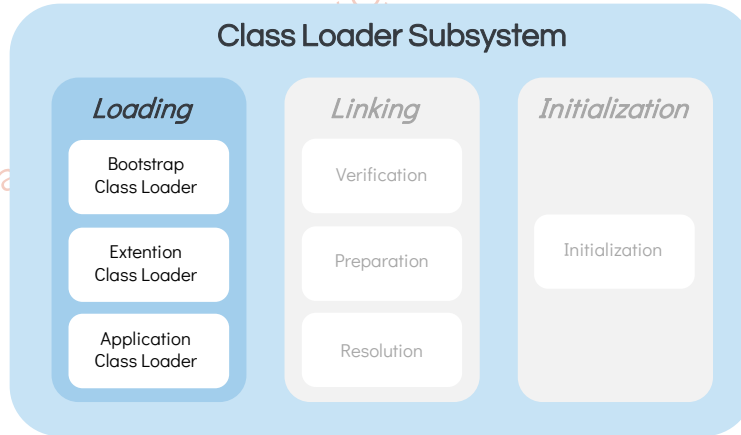
The **Class Loader Subsystem** in Java's JVM is a component responsible for dynamically loading **.class** files (*bytecode*) into the JVM at runtime, linking them by verifying, preparing, and resolving, and initializing them by executing their static initializers and initializing static variables. It facilitates Java's dynamic and platform-independent execution by ensuring classes are loaded only when needed and in a secure and modular way.



EXPLANATION

Loading in Class Loader Subsystem

Loading in Java's class loader subsystem is the *first phase* of the class loading process. During this phase, the class loader locates the **.class file** (from sources such as the *file system, JAR files, or network*), reads its bytecode, and creates a Class object in the Java heap. This process involves identifying the appropriate class loader (e.g., *Bootstrap, Extension, or Application class loader*). If the class cannot be found, a **ClassNotFoundException** is thrown.



EXPLANATION

Bootstrap Class Loader

Bootstrap Class Loader is a fundamental component of the Java Virtual Machine (JVM) that loads core Java classes, such as those in the *java.lang* package, from the Java Runtime Environment (JRE) into memory during program execution.

- It is the parent of all class loaders in the hierarchy, and unlike other class loaders, it is not a subclass of *java.lang.ClassLoader*, forming the foundation of the entire class-loading mechanism in Java.
- It loads the essential Java classes located in the `<JAVA_HOME>/jre/lib` directory (or `<JAVA_HOME>/jmods` for Java 9 and above), including fundamental packages like *java.lang* and *java.util*.
- It is implemented in native code (typically in *C* or *C++*) as part of the JVM itself, rather than being a Java object.
- It is initialized when the JVM starts, ensuring that the core classes required for the JVM's operation are available immediately.

EXPLANATION

Extension Class Loader

Extension Class Loader, also known as the **Platform Class Loader** in newer Java versions, loads classes from the JDK's extension directories, typically located in the lib/ext folder, making them available to all Java applications running on the platform.

- It is the parent of the **Application Class Loader** in the class loader hierarchy and a child of the **Bootstrap Class Loader**, inheriting its class-loading responsibilities and extending them to include classes from the extension directories.
- It loads classes from the Java Extensions directory, typically located at `<JAVA_HOME>/lib/ext`, or from any directory specified by the `java.ext.dirs` system property.
- It follows the delegation model, where it delegates class-loading requests to its parent class loader (*the Bootstrap Class Loader*) before attempting to load the class itself.
- It is fully visible to Java applications, and can be obtained using `ClassLoader.getSystemClassLoader()`, unlike the Bootstrap Class Loader.

EXPLANATION

Application Class Loader

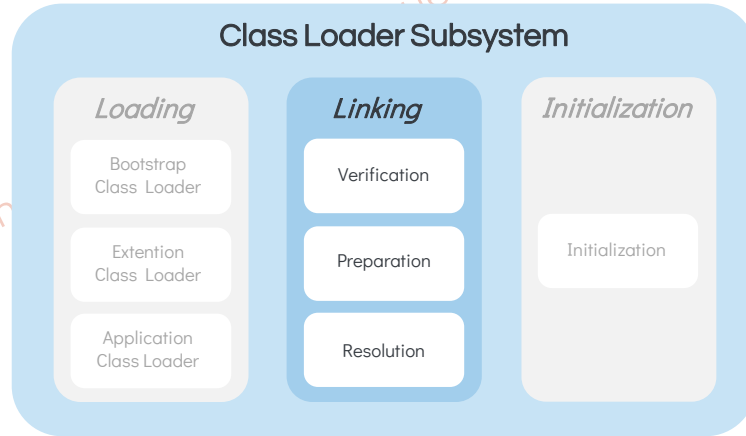
Application Class Loader, also known as the **System ClassLoader**, is a component of the Java Virtual Machine (JVM) that loads classes from the application's classpath, making them available for use during program execution.

- It is the child of the **Extension Class Loader** and the parent of any custom class loaders defined by the application.
- It is responsible for loading **user-defined classes** (e.g., `com.myapp.MyClass`), **libraries, third-party dependencies** (e.g., `org.apache.commons.lang3.StringUtils` from Apache Commons), and **resources from directories and JAR files** (e.g., `junit-4.13.2.jar` for unit testing) specified in the CLASSPATH environment variable or the `-cp/-classpath` command-line option.
- It follows the delegation model, where it delegates class-loading requests to its parent class loader (*the Extension Class Loader*) before attempting to load the class itself.
- It is fully visible to Java applications, and can be obtained using `ClassLoader.getSystemClassLoader()`, unlike the Bootstrap Class Loader.

EXPLANATION

Linking in Class Loader Subsystem

Linking in Java's Class Loader Subsystem is the **second phase** of the class loading process, where the JVM verifies the class for correctness, allocates memory for static fields with default values, and replaces symbolic references with direct memory references, ensuring the class is ready for execution.



EXPLANATION

Verification

Verification in the Class Loader's *linking phase* ensures a Java class file's bytecode is correct and adheres to JVM specifications and checks structural and semantic validity after loading and before preparation and resolution.

- It checks that the bytecode conforms to the JVM specification, ensuring it is properly formatted and adheres to the expected structure.
- It ensures that operations in the code are performed on appropriate data types, preventing type mismatches that could lead to runtime errors.
- It analyzes the control flow to confirm that all branches lead to valid locations and that there are no illegal jumps or unreachable code segments.
- It checks that the operand stack is used correctly, ensuring that stack operations (*pushes and pops*) are balanced and that the stack does not overflow or underflow during method execution.
- It enforces access restrictions, ensuring that code does not illegally access private or protected members of classes, thereby preserving encapsulation.
- It ensures that all objects and variables are properly initialized before use, preventing scenarios where uninitialized data could lead to unpredictable behavior.

EXPLANATION

Preparation

Preparation in the Class Loader's *linking phase* involves allocating and initializing memory for class variables to their default values and it occurs after verification and ensures the class is ready for resolution and execution.

- It ensures that when the initialization phase occurs, all static variables are in a consistent and predictable state, ready to be assigned their actual intended values.
- It involves allocating memory for all static fields of a class or interface in the memory area, which are shared among all instances of the class.
- It initializes static variables to their default values, such as 0 for numeric types, false for booleans, and null for reference types.
- It excludes final static fields with compile-time constants, as their values are set during initialization and embedded directly in the bytecode.
- It does not execute Java code; it only allocates memory and performs default initialization without running constructors or methods.

EXPLANATION

Resolution

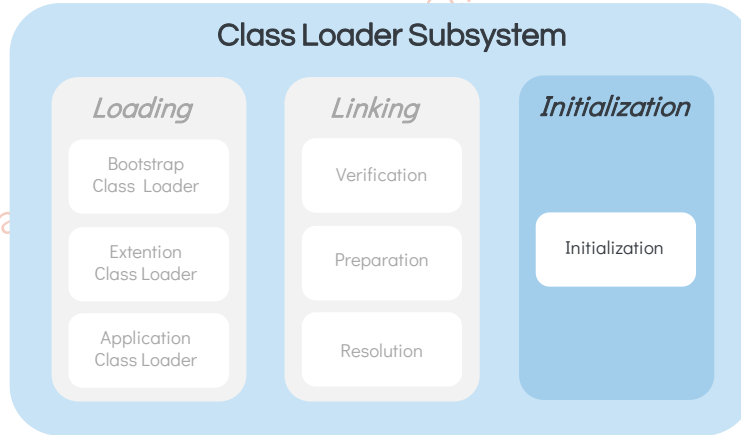
Resolution in the Class Loader's *linking phase* involves transforming symbolic references in the constant pool of a class or interface into direct memory references and it process occurs after verification and ensures the class is ready for efficient execution.

- It converts symbolic references (*such as class names, method names, and field names*) into direct memory references, allowing faster access during runtime.
- It performs resolution either eagerly (*at the time of class loading*) or lazily (*when the reference is first accessed*), depending on the implementation.
- It fails if the JVM cannot transform symbolic references into direct memory references, in which case it throws a **LinkageError** or one of its subclasses, such as **NoClassDefFoundError** or **IncompatibleClassChangeError**, to promptly detect issues with missing or incompatible classes.

EXPLANATION

Initialization in Class Loader Subsystem

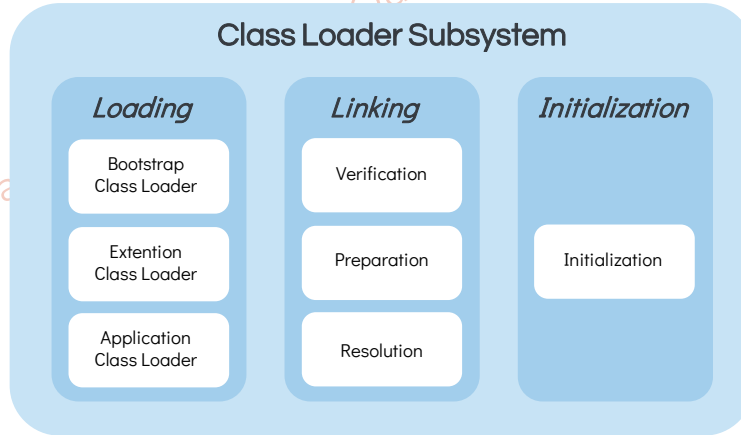
Initialization in Java's Class Loader Subsystem is the *final phase* of the class loading process, triggered by the class's first active use. During this phase, the JVM executes static initializers and assigns defined values to static variables, ensuring that all static fields and blocks are initialized in the order they are declared.



EXPLANATION

Class Loader Subsystem

The **Class Loader Subsystem** in Java's JVM is a component responsible for dynamically loading **.class** files (*bytecode*) into the JVM at runtime, linking them by verifying, preparing, and resolving, and initializing them by executing their static initializers and initializing static variables. It facilitates Java's dynamic and platform-independent execution by ensuring classes are loaded only when needed and in a secure and modular way.



QUESTION

What **memory storages** are available in the JVM? 

Explanation

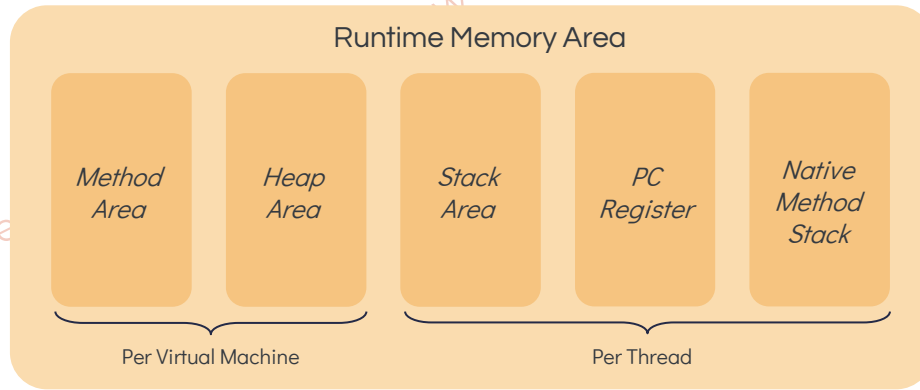


From: The Java Interview Questions Course on Udemy © Peerless Geek

EXPLANATION

Runtime Data Areas

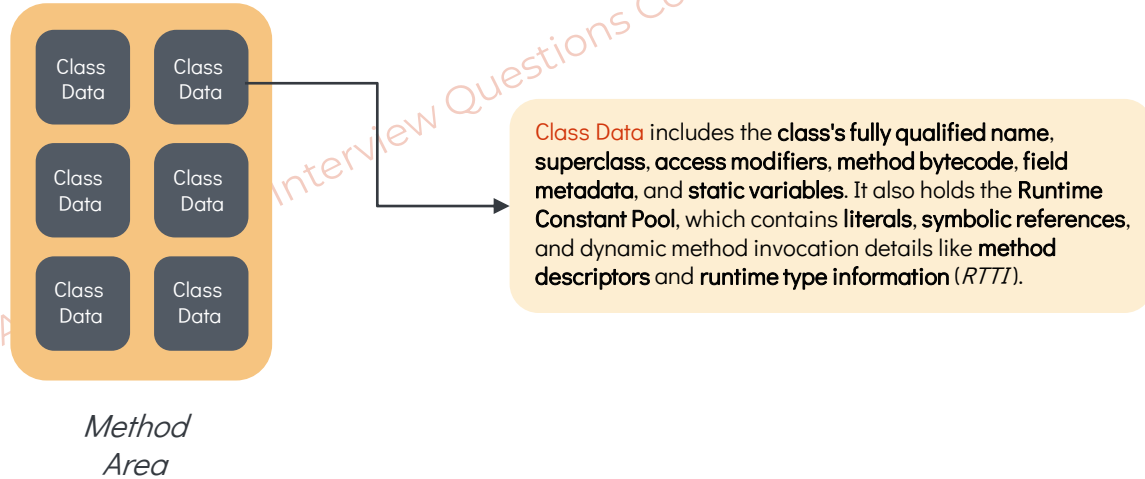
The **Runtime Data Areas** in Java's JVM are the memory regions allocated during the execution of a Java program, used to store data such as class structures, method data, object instances, local variables, and intermediate results. These areas include the **method area**, **heap**, **stack**, **program counter register**, and **native method stack**, and they ensure efficient memory management and execution of Java applications.



EXPLANATION

Method Area

Method Area in *Java's Runtime Memory Area* is a critical component responsible for storing class-level data that is shared among all threads.



EXPLANATION

Method Area

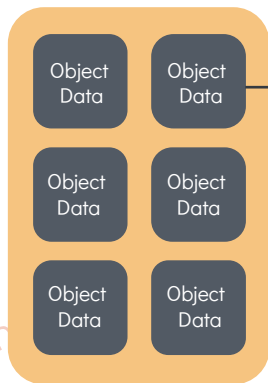
Method Area in *Java's Runtime Memory Area* is a critical component responsible for storing class-level data that is shared among all threads.

- It contains information about classes, including their fully qualified name, modifiers, fields, methods, and interfaces.
- It holds runtime constants such as string literals and symbolic references resolved during class loading or execution.
- It stores static variables defined in classes.
- It stores method-related information, such as method code (*bytecode*), and descriptors.
- It is created when the JVM starts and is destroyed only when the JVM exits.
- It is shared among all threads, meaning all threads access the same method data and class metadata.

EXPLANATION

Heap Area

Heap Area in *Java's Runtime Memory Area* is a critical memory region dedicated to the dynamic allocation of objects during the execution of a Java application and serves as the primary storage for all class instances (*objects*) and arrays.



Heap Area

Object Data includes **instance variables** and **metadata** that the JVM uses to manage objects in memory. When a new object is created in Java, its instance variables (*non-static fields that hold the object's unique state*) and object metadata (such as class information and synchronization details) are stored in the heap.

EXPLANATION

Heap Area

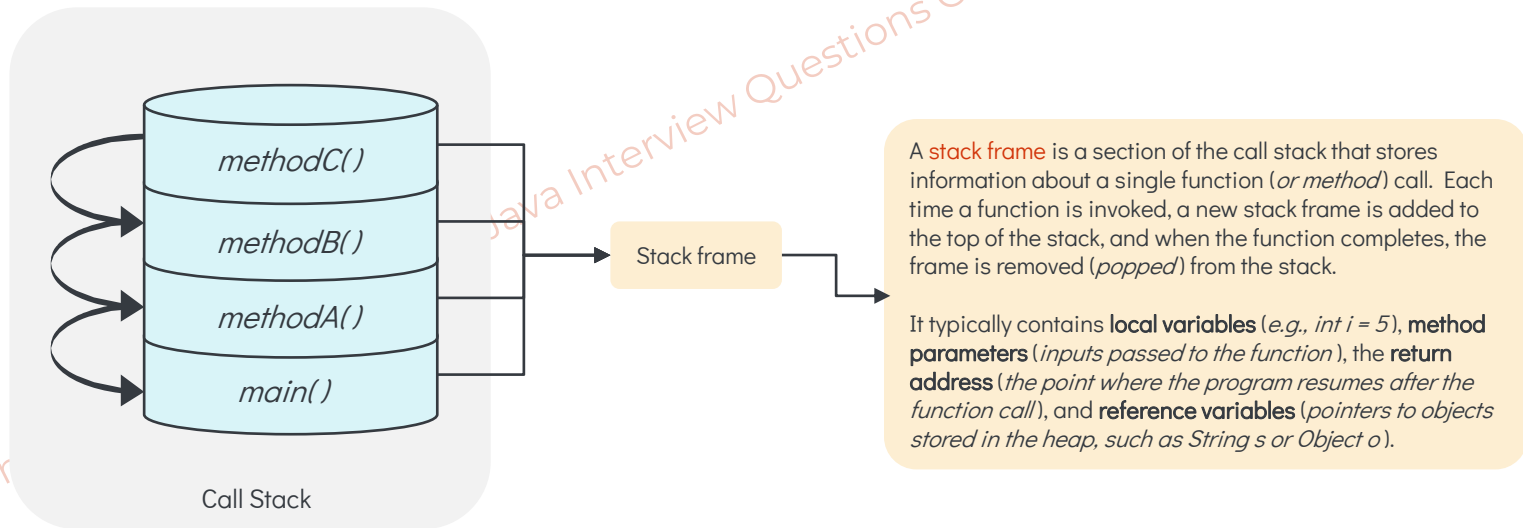
Heap Area in *Java's Runtime Memory Area* is a critical memory region dedicated to the dynamic allocation of objects during the execution of a Java application and serves as the primary storage for all class instances (*objects*) and arrays.

- It stores instances of classes, including objects, arrays, and user-defined class instances (created using the `new` keyword).
- It is divided into Young Generation (*new objects*), Old Generation (*long-lived objects*), and Permanent Generation (*class metadata, replaced by Metaspace in Java 8+*).
- It is automatically managed by garbage collection, where the JVM automatically reclaims memory from unused objects, reducing the risk of memory leaks and manual memory management errors.
- If the heap cannot accommodate new object allocations due to memory constraints, the JVM throws an **OutOfMemoryError**.
- It is a shared resource accessible by all threads within the application, facilitating inter-thread communication through shared objects.

EXPLANATION

Stack Area

Stack Area in Java's Runtime Memory is a fundamental thread-private region used for executing threads, with each thread having its own stack to store method frames, local variables, and partial results.



EXPLANATION

Stack Area

Stack Area in *Java's Runtime Memory* is a fundamental thread-private region used for executing threads, with each thread having its own stack to store method frames, local variables, and partial results.

- It holds a stack frame (*memory structure*) for each method call.
- It stores local variables and method parameters for each method execution in stack frames, with primitive types (*e.g., int, float*) stored directly, while references to objects are stored as memory addresses, pointing to objects in the heap.
- It keeps track of method invocation and return points by growing the stack with a new frame for each method call and removing the frame when the method completes.
- It operates on a *LIFO* principle, meaning that the last method to be called is the first to finish execution and be removed from the stack, ensuring that method calls follow the correct order of execution.
- Its memory is automatically managed: it is allocated when a method is called and deallocated when the method returns.
- Its size is usually limited, and if it exceeds its limit (*such as during deep recursion*), the JVM throws a **StackOverflowError**.

EXPLANATION

PC Register

PC Register in *Java's Runtime Memory Area*, is a thread-specific memory region that holds the address of the instruction currently being executed by the Java Virtual Machine.

- It stores the address of the current instruction being executed, enabling the JVM to keep track of the execution flow within a thread.
- It stores only the address of the next instruction, not data or variables, and is responsible for directing the flow of execution.
- It is updated to the address of a method's code when the method is called, and when the method completes, it is updated to the return address, resuming execution after the method call.
- Its value is undefined if the current method being executed is a native method (*i.e., implemented in a language other than Java*).
- It is not directly accessible to the program code, as it is managed by the CPU or JVM runtime and automatically updated during the instruction execution process.
- It is unique to each thread, ensuring that the execution flow of one thread does not interfere with another.

EXPLANATION

Native Method Stack

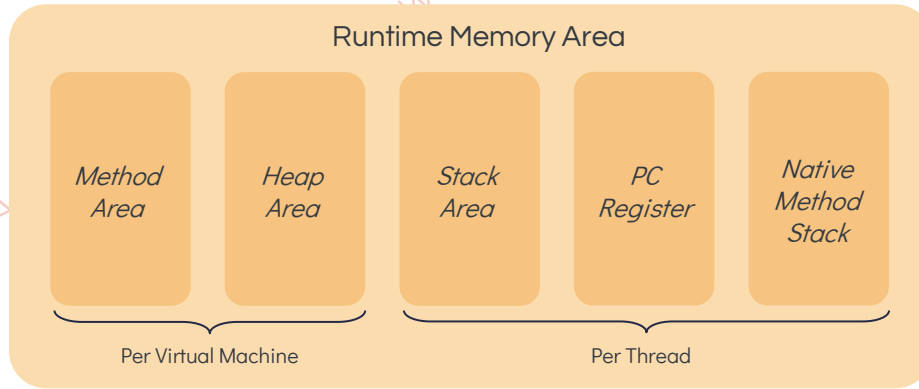
Native method stack in Java's *Runtime Memory Area*, is a memory area used to support the execution of native methods, which are methods written in languages like C or C++ (outside of Java) and interfaced with Java through the Java Native Interface (*JNI*).

- It is responsible for managing the execution of native methods, which are implemented outside the JVM, often in languages like C, C++, or assembly.
- It stores frames for native method invocations, including local variables, method arguments, and return addresses, enabling the execution of native code.
- It is used when Java calls native libraries, such as system libraries or DLLs, for features that are specific to the operating system or for performance optimizations.
- It is not subject to the JVM's garbage collection, with memory for native methods being managed by the operating system or native code itself.
- It is allocated for each thread in the JVM, ensuring that native method executions remain isolated across threads.

EXPLANATION

Runtime Data Areas

The **Runtime Data Areas** in Java's JVM are the memory regions allocated during the execution of a Java program, used to store data such as class structures, method data, object instances, local variables, and intermediate results. These areas include the **method area**, **heap**, **stack**, **program counter register**, and **native method stack**, and they ensure efficient memory management and execution of Java applications.



QUESTION

What is JIT?



Explanation



From: The Java Interview Questions Course on Udemy © Peerless Geek

EXPLANATION

Just-In-Time (JIT) compiler

The **JIT (Just-In-Time) compiler** is a component of the Java Virtual Machine (JVM) that improves the performance of Java applications by compiling bytecode into native machine code at runtime, rather than interpreting the bytecode, which allows the JVM to execute Java programs faster by eliminating the need to interpret bytecode every time the program is run.

- It translates bytecode into native machine code at runtime, meaning the translation happens as the program executes.
- It can selectively compile "**hot spots**" — frequently executed code paths — based on profiling data which ensures that the most used parts of the code are optimized.
- It caches the compiled code once a portion of code has been compiled, so it doesn't need to be recompiled the next time it is executed.
- It works in tandem with the JVM's garbage collector, ensuring that it does not interfere with memory management and works efficiently with the JVM heap.

QUESTION

What are the differences between Java and C++? 

Explanation



EXPLANATION

Java	C++
Java is platform-independent and runs on any system with a JVM.	C++ is platform-dependent and needs to be recompiled for each platform.
Its code must be inside a class and follows strict OOP principles.	It supports both procedural and object-oriented programming.
It does not support multiple inheritance with classes, but it supports multiple inheritance through interfaces.	It supports multiple inheritance through classes, which can cause complexity.
It does not use explicit pointers for security reasons.	It fully supports pointers for direct memory manipulation.
It enforces both checked and unchecked exceptions.	It only supports unchecked exceptions.
It offers extensive built-in APIs for networking, databases, and more.	It provides the Standard Template Library (STL) for data structures and algorithms.
It provides built-in thread support, documentation comments (<code>/** ... */</code>), and the unsigned right shift operator (<code>>>></code>).	It supports goto, pointers, call by reference, structures, unions, multiple inheritance, and virtual functions.
It uses automatic memory management with Garbage Collection.	It requires manual memory management using new and delete.
It is slower due to JVM overhead and garbage collection.	It is faster as it compiles directly to machine code.
It is safer with built-in memory management to avoid leaks.	It requires careful memory handling to prevent leaks and dangling pointers.
It is used for web apps, enterprise software, and Android development.	It is used for games, system software, and performance-critical applications.

QUESTION

Explain **public**
static void
main(String[]
args). 

Explanation

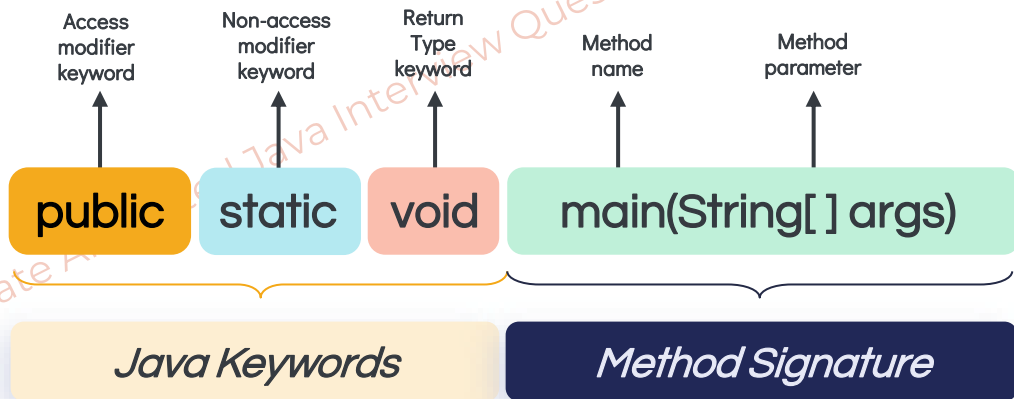


From: The Complete Java Interview Questions Course on Udemy © Peerless Geek

EXPLANATION

Understanding the *Main Method*

In Java, the **public static void main(String[] args)** method is the entry point of any standalone application, essential for execution and is declared **public** for accessibility, **static** to run without an object, and **void** as it returns no value, while **String[] args** allows command-line input.



EXPLANATION

Understanding the *Main Method*

public

- It is an access modifier that makes the main method accessible from anywhere.
- It ensures that the Java Virtual Machine (JVM), which needs to invoke the main method from outside the class, can access it.

static

- It means that the method belongs to the class, not an instance of the class.
- It allows the JVM to call the main method without creating an object of the class.

void

- It is the return type of the method.
- It means the method does not return any value.

main

- It is the name of the method that the JVM looks for as the starting point of a program.
- It is predefined and must be written exactly like this for the JVM to recognize it.

String[] args

- It is the parameter of the main method that represents an array of String values.
- It is used to pass command-line arguments to the program.

QUESTION

What happens if we don't declare the **main method** as **static**? 

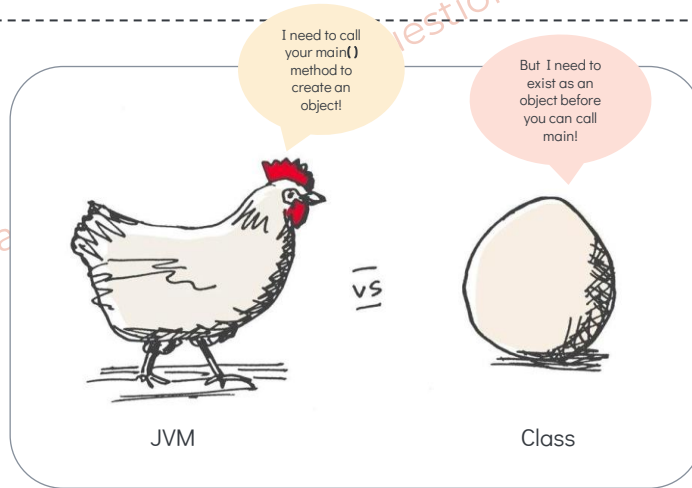
Explanation



EXPLANATION

The need for a *static* main() method

If the main method is not declared as *static*, the Java Virtual Machine (JVM) cannot invoke it without creating an instance of the class. Since the main method is the *entry point* and must run without any pre-existing object, the program will fail to execute and the JVM then throws an error like: "*Error: Main method not found in class Main, please define the main method as: public static void main(String[] args).*"



EXPLANATION

The need for a *static* main() method

If the main method is not declared as static, the Java Virtual Machine (JVM) cannot invoke it without creating an instance of the class. Since the main method is the *entry point* and must run without any pre-existing object, the program will fail to execute and the JVM then throws an error like: "*Error: Main method not found in class Main, please define the main method as: public static void main(String[] args).*"

```
public class Main {  
    // Incorrect main method: Missing the  
    'static' modifier  
    public void main(String[] args) {  
    }  
}
```



```
public class Main {  
    // Entry point of the program  
    public static void main(String[]  
args) {  
    }  
}
```



QUESTION

Will the program
run if we write
static public void
main? 

Explanation



From: The
Animated Java Interview Questions Course on Udemy © Peerless Geek

EXPLANATION

Java Main Method Modifiers: *public static* vs. *static public*

Yes, the Java program will run if you write `static public void main(String[] args)` because the order of modifiers like `public` and `static` doesn't matter in Java; both `public static` and `static public` are valid and the Java compiler recognizes the method as the entry point regardless of the order. However, the **CONVENTIONAL ORDER** is `public static void main(String[] args)`, and it's recommended to follow it for better readability and consistency with standard Java practices.



Fast Red Car



Red Fast Car

From: The Ultimate

EXPLANATION

Java Main Method Modifiers: *public static* vs. *static public*

Yes, the Java program will run if you write **static public void main(String[] args)** because the order of modifiers like public and static doesn't matter in Java; both public static and static public are valid and the Java compiler recognizes the method as the entry point regardless of the order. However, the **CONVENTIONAL ORDER** is **public static void main(String[] args)**, and it's recommended to follow it for better readability and consistency with standard Java practices.

```
public class Main {  
    // Entry point of the program  
    public static void main(String[] args) {  
    }  
}
```



```
public class Main {  
    // The order of 'static' and 'public'  
    // modifiers is interchangeable.  
    static public void main(String[] args) {  
    }  
}
```



QUESTION

Can you **run a code before** executing the main method? 

Explanation



EXPLANATION

Executing Code Before main(): *Static Blocks & Initialization*

Yes, you can run code before the main() method in Java using *static blocks* or *static variable initialization*. A **static block** is executed when the class is loaded, before the main() method runs, making it ideal for setup tasks. Similarly, **static variable initialization** allows code to be executed during class loading by calling a method or performing logic while assigning a static variable.

```
public class Main {  
    // Static block  
    static {  
        System.out.println("This runs  
before the main method.");  
    }  
  
    public static void main(String[]  
args) {  
        System.out.println("This is the  
main method.");  
    }  
}
```

```
public class Main {  
    // Static variable initialization  
    static String message = initMessage();  
  
    private static String initMessage() {  
        System.out.println("Initializing  
message...");  
        return "Hello";  
    }  
  
    public static void main(String[] args) {  
        System.out.println("Main method: " +  
message);  
    }  
}
```

QUESTION

What is the difference between `System.out`, `System.err`, and `System.in`? 

Explanation



From: The Java Interview Questions Course on Udemy © Peerless Geek

EXPLANATION

I/O Streams: *System.out*, *System.err*, and *System.in*

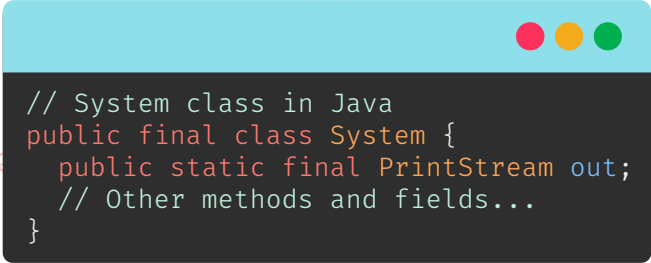
System.out, *System.err*, and *System.in* are standard I/O streams provided by the `java.lang.System` class, used to manage input and output for console applications.

From: The Ultimate Animated Java Interview Questions Course on Udemy © Peerless Geek

EXPLANATION

System.out

System.out is a **PrintStream** object used to output normal or standard messages to the console (*or terminal*). It prints data to the standard output stream and is generally used for normal program output, such as results and informational messages.



```
// System class in Java
public final class System {
    public static final PrintStream out;
    // Other methods and fields...
}
```

java.lang.System class

EXPLANATION

System.err

System.err is a **PrintStream** object used to output error messages or diagnostic information to the standard error stream. Like *System.out*, it prints data to the console (or *terminal*) but is specifically used to separate error messages from normal program output.

```
// System class in Java
public final class System {
    public static final PrintStream err;
    // Other methods and fields...
}
```

java.lang.System class

EXPLANATION

System.in

System.in is an **InputStream** object used for input from the console or other standard input sources (e.g., *keyboard or file input*). It reads data from the standard input stream and is often wrapped in higher-level classes like **Scanner** or **BufferedReader** for easier reading.

```
// System class in Java
public final class System {
    public static final InputStream in;
    // Other methods and fields...
}
```

*java.lang.***System** class

QUESTION

What is the difference between `print`, `println`, and `printf`? 

Explanation



EXPLANATION

Output Methods: *print*, *println*, and *printf*

The *print*, *println*, and *printf* methods belong to the **PrintStream** class, accessed via **System.out** (a *static member field*). The *print* method outputs text without a newline, *println* outputs text followed by a newline, and *printf* allows formatted output using format specifiers.

From: The Ultimate Animated Java Interview Questions Course on Udemy © Peerless Geek

EXPLANATION

print method

The **print** method outputs text without adding a newline (\n) at the end, so the cursor remains on the same line and is useful when you want subsequent output to continue on the same line.

```
// PrintStream class for handling
output streams
public class PrintStream {
    // Method to print a string to
    the output
    public void print(String s) {
        // Implementation goes here
    }
}
```

java.io.PrintStream class

EXPLANATION

println method

The **println** method outputs text and automatically appends a newline (\n) at the end, moving the cursor to the next line. Each call to println ensures the output appears on a new line, making it useful when you want output to appear on separate lines.

```
// PrintStream class for handling
output streams
public class PrintStream {
    // Method to print a string to
    the output
    public void println(String x) {
        // Implementation goes here
    }
}
```

java.io.PrintStream class

EXPLANATION

printf method

The **printf** method is used for formatted output, allowing you to format text using placeholders and format specifiers (*similar to printf in C/C++*). Common format specifiers include **%s for strings**, **%d for integers**, **%f for floating-point numbers**, and **%n for a platform-independent newline**. It does not add a newline automatically, so %n or \n must be used explicitly for a new line.

```
// PrintStream class for handling
output streams
public class PrintStream {
    // Method to print a string to the
    output
    public PrintStream printf(String
format, Object ... args) {
    // Implementation goes here
    }
}
```

java.io.PrintStream class

EXPLANATION

printf method

The **printf** method is used for formatted output, allowing you to format text using placeholders and format specifiers (*similar to printf in C/C++*). Common format specifiers include **%s for strings**, **%d for integers**, **%f for floating-point numbers**, and **%n for a platform-independent newline**. It does not add a newline automatically, so %n or \n must be used explicitly for a new line.

Common Format Specifiers

Specifier	Description	Example with Input	Output
%d	Integer	System.out.printf("%d", 42);	42
%f	Floating-point number	System.out.printf("%.2f", 3.14159);	3.14
%s	String	System.out.printf("%s", "Java");	Java
%c	Character	System.out.printf("%c", 'A');	A
%b	Boolean	System.out.printf("%b", true);	true
%x	Hexadecimal (<i>lowercase</i>)	System.out.printf("%x", 255);	ff
%o	Octal	System.out.printf("%o", 10);	12
%%	Literal % (<i>no argument required</i>)	System.out.printf("%%");	%

QUESTION

What are Packages in Java?

Explanation

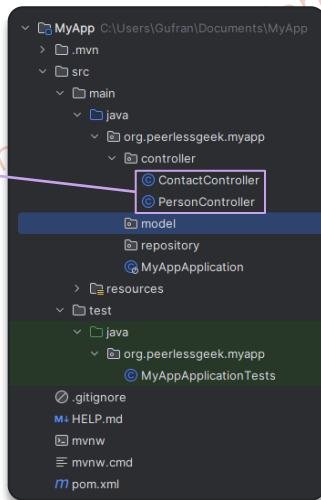


EXPLANATION

Package

A **package** is a way of organizing *classes*, *interfaces*, *enumerations*, and *sub-packages* in a structured manner and acts as a **namespace** (*a container that groups identifiers, such as classes*) that helps avoid name conflicts and makes it easier to locate and use classes.

Related Class



EXPLANATION

Package

A **package** is a way of organizing *classes, interfaces, enumerations, and sub-packages* in a structured manner and acts as a **namespace** (*a container that groups identifiers, such as classes*) that helps avoid name conflicts and makes it easier to locate and use classes.

- It is a way to organize and group related classes and interfaces.
- It provides a hierarchical structure (*similar to folders or directories*) for organizing codebases. For example, in *java.util*, *util* is a sub-package of *java*.
- Its naming follows the reverse domain convention (e.g., *com.google.gson*) to ensure globally unique names and is widely used in enterprise development.
- It is declared at the top of a Java file using the **package** keyword, followed by the package name. For example, **package com.google.gson;** declares that the *file* belongs to the *com.google.gson* package.
- Its containing classes or interfaces from other packages can be accessed using the **import** statement. For example, **import com.google.gson.Gson;** allows access to the **Gson** class from the *com.google.gson* package.
- It prevents name conflicts by creating a unique **namespace**. For instance, you can have a class named **Date** in both *java.util* and *java.sql* packages without conflict.
- It can be used to control the visibility of *Classes* within and outside it using access modifiers like **public** or *package-private* (default).

QUESTION

What are **Access Specifiers** and types of Access Specifiers? 

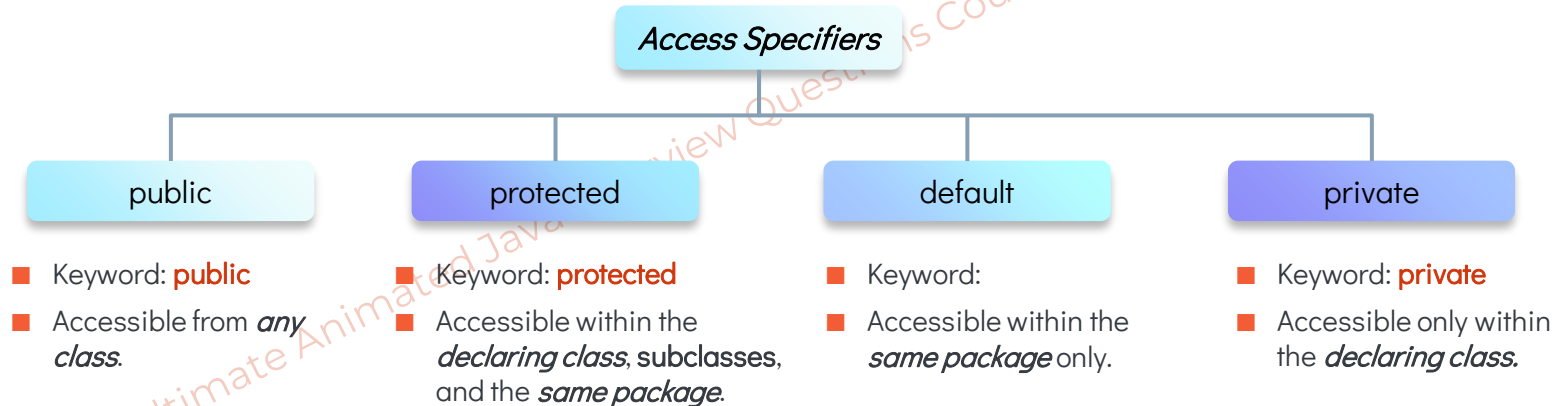
Explanation



EXPLANATION

Access Specifiers

Access specifiers (also known as **access modifiers**) define the visibility or scope of variables, methods, constructors, and classes and control how different parts of a program can access specific members of a class.



Note :

In Java, top-level classes can only be public or package-private, while inner classes can use all access modifiers (*private*, *protected*, *package-private*, *public*) for finer visibility control.

QUESTION

Why is Java, **not** a **pure** object-oriented language? 

Explanation

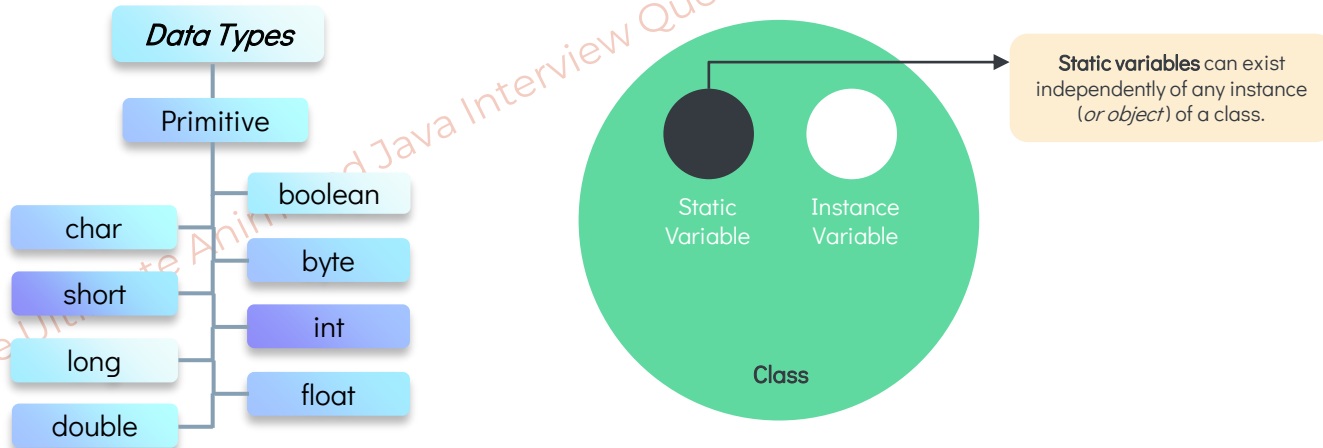


From: The Java Interview Questions Course on Udemy © Peerless Geek

EXPLANATION

Java: Object-Oriented but *Not Purely*!

Java is not a purely object-oriented language because it includes **primitive data types** (*byte, short, int, long, float, double, char, and boolean*) that are not objects, **allows static methods and variables** for access without object creation, **supports procedural programming**, and does not strictly enforce encapsulation since static variables can exist independently of objects.



Data types & Wrapper classes

02

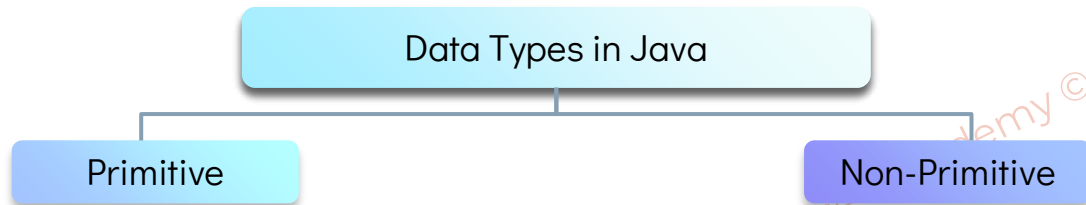
QUESTION

Explain different data types in Java.

Explanation

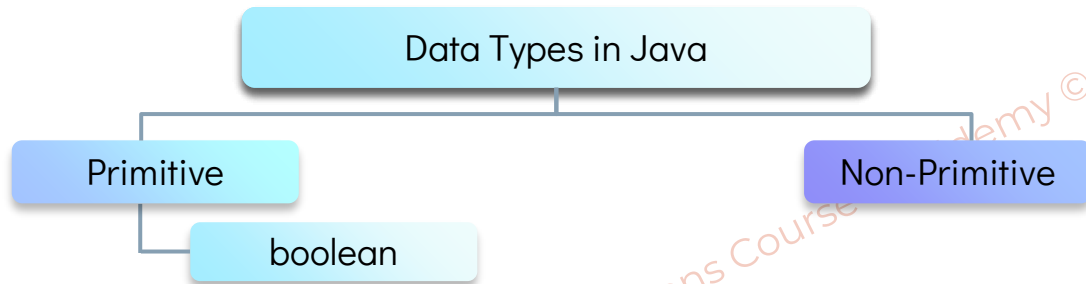


EXPLANATION



From: The Ultimate Animated Java Interview Questions Course by Peerless Geek Academy © Peerless Geek

EXPLANATION



From: The Ultimate Animated Java Interview Questions Course Academy © Peerless Geek

EXPLANATION

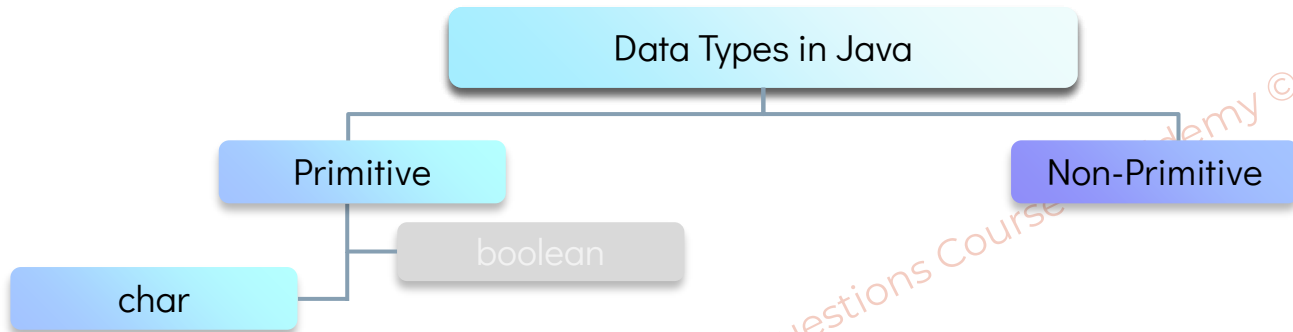
boolean

- Size: 1 byte
- Stores **true** or **false** values

$2^7 = 128$	$2^6 = 64$	$2^5 = 32$	$2^4 = 16$	$2^3 = 8$	$2^2 = 4$	$2^1 = 2$	$2^0 = 1$

8 Bits (1 byte)

EXPLANATION



From: The Ultimate Animated Java Interview Questions Course by Peerless Geek

EXPLANATION

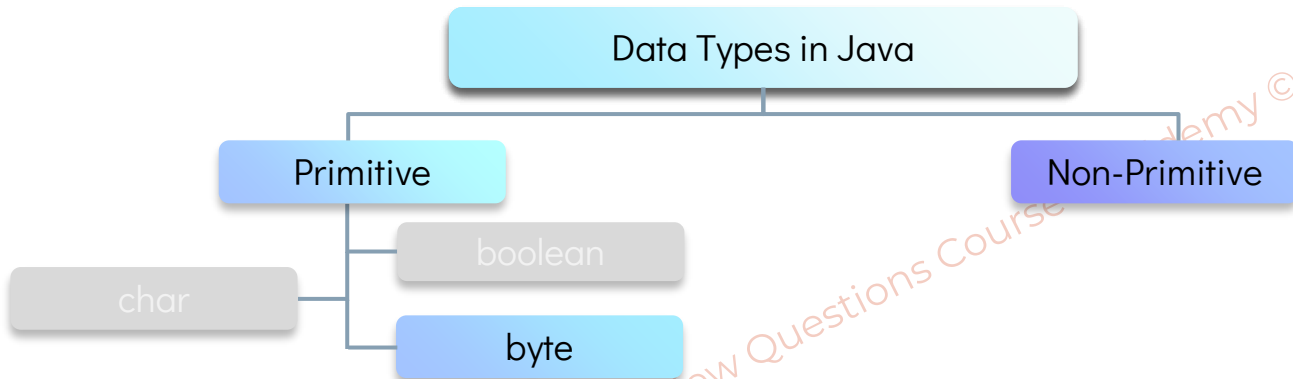
char

- Size: 2 bytes (16 bits)
- Stores a single 16-bit unsigned Unicode character (ex - 'a', '1', '@')
- Range: 0 to 65,535 (0 to $(2^{16} - 1)$)

32768	16384	8192	4096	2048	1024	512	256	128	64	32	16	8	4	2	1

16 Bits (2 bytes)

EXPLANATION



From: The Ultimate Animated Java Interview Questions Course by Academy © Peerless Geek

EXPLANATION

byte

- Size: 1 byte (8 bits)
- Stores an 8-bit signed two's complement integer
- Range: -128 to 127 (-2^7 to $2^7 - 1$)

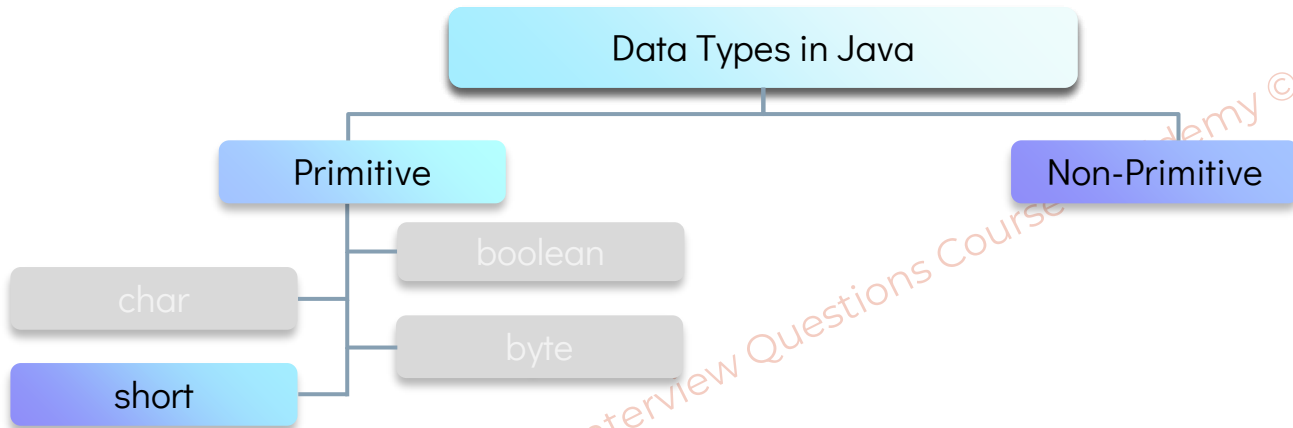
The remaining 7 bits are used to represent the absolute value.

$2^7 = 128$	$2^6 = 64$	$2^5 = 32$	$2^4 = 16$	$2^3 = 8$	$2^2 = 4$	$2^1 = 2$	$2^0 = 1$

8 Bits (1 byte)

↑
The most significant bit is used to represent the sign of the integer. 0 for positive and 1 for negative.

EXPLANATION



From: The Ultimate Animated Java Interview Questions Course by Academy © Peerless Geek

EXPLANATION

short

- Size: 2 bytes (16 bits)
- Stores an 16-bit signed two's complement integer
- Range: -32768 to 32767 (-2^{15} to $2^{15} - 1$)

The remaining 15 bits are used to represent the absolute value.

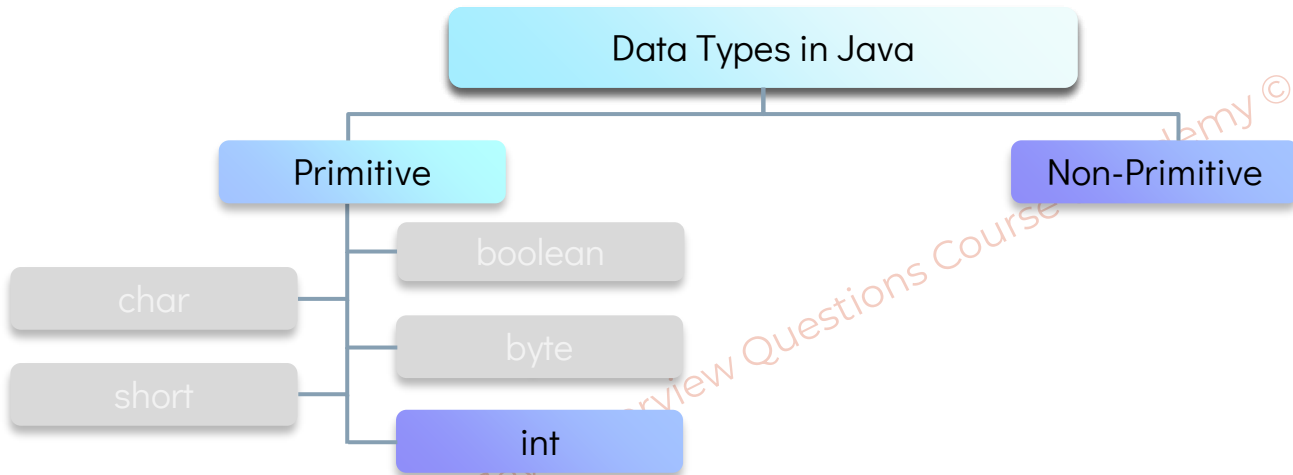
32768	16384	8192	4096	2048	1024	512	256	128	64	32	16	8	4	2	1



16 Bits (2 bytes)

The most significant bit is used to represent the sign of the integer. 0 for positive and 1 for negative.

EXPLANATION

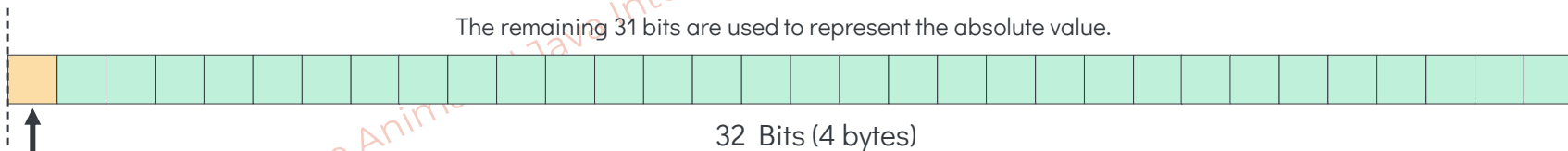


From: The Ultimate Animated Java Interview Questions Course by Jemy © Peerless Geek

EXPLANATION

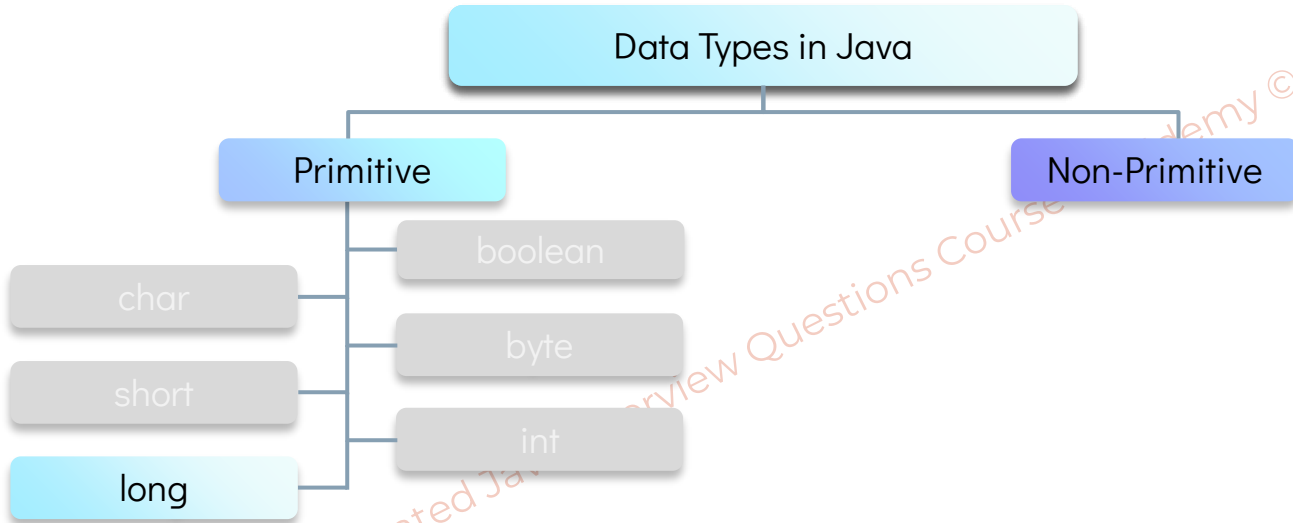
int

- Size: 4 bytes (32 bits)
- Stores an 32-bit signed two's complement integer
- Range: -2,147,483,648 to 2,147,483,647 (inclusive)
(-2^{31} to $2^{31} - 1$)



The most significant bit is used to represent the sign of the integer. 0 for positive and 1 for negative.

EXPLANATION

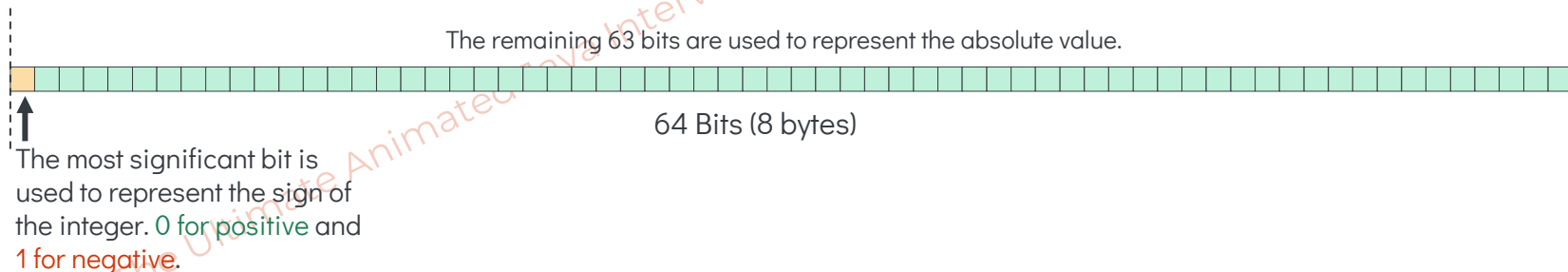


From: The Ultimate Animated Java Interview Questions Course by Academy © Peerless Geek

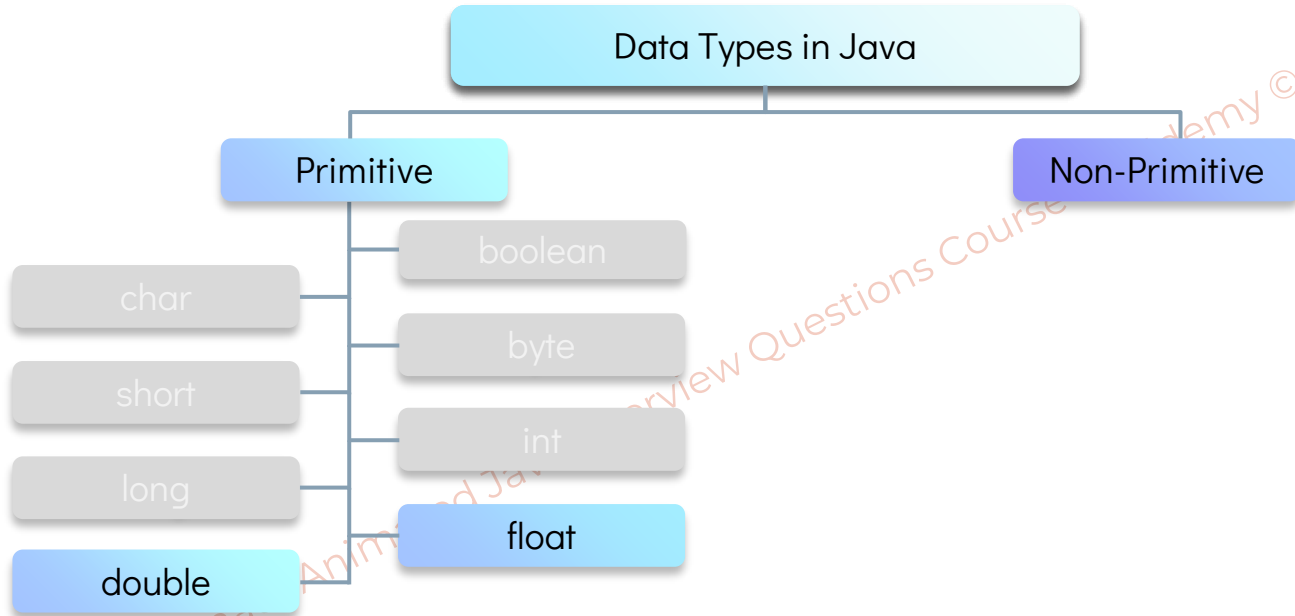
EXPLANATION

long

- Size: 8 bytes (64 bits)
- Stores an 64-bit signed two's complement integer
- Range: -9,223,372,036,854,775,808 to 9,223,372,036,854,775,807 (-2^{63} to $2^{63} - 1$)



EXPLANATION



EXPLANATION

float

- Size: 4 bytes (32 bits)
- Stores a single precision 32-bit IEEE 754 float-point
- Has a precision of 7 decimal digits
- Range: 1.4×10^{-45} to 3.4×10^{38}

double

- Size: 8 bytes (64 bits)
- Stores a double-precision 64-bit IEEE 754 float-point
- Has a precision of 15 decimal digits.
- Range: 4.9×10^{-324} to 1.8×10^{308}

```
public static void main(String[]  
args) {  
    float myFloat = 3.14f;  
    double myDouble = 3.14;  
}
```

EXPLANATION

primitive

boolean

- Size: 1 byte
- Stores **true** or **false** values

char

- Size: 2 bytes (16 bits)
- Stores a single 16-bit **unsigned Unicode character** (ex - 'a', '1', '@')
- Range: 0 to 65,535 (0 to $(2^{16} - 1)$)

byte

- Size: 1 byte (8 bits)
- Stores an 8-bit **signed two's complement** integer
- Range: -128 to 127 (-2^7 to $2^7 - 1$)

short

- Size: 2 bytes (16 bits)
- Stores an 16-bit **signed two's complement** integer
- Range: -32768 to 32767 (-2^{15} to $2^{15} - 1$)

int

- Size: 4 bytes (32 bits)
- Stores an 32-bit **signed two's complement** integer
- Range: $(-2^{31}$ to $2^{31} - 1)$

long

- Size: 8 bytes (64 bits)
- Stores an 64-bit **signed two's complement** integer
- Range: $(-2^{63}$ to $2^{63} - 1)$

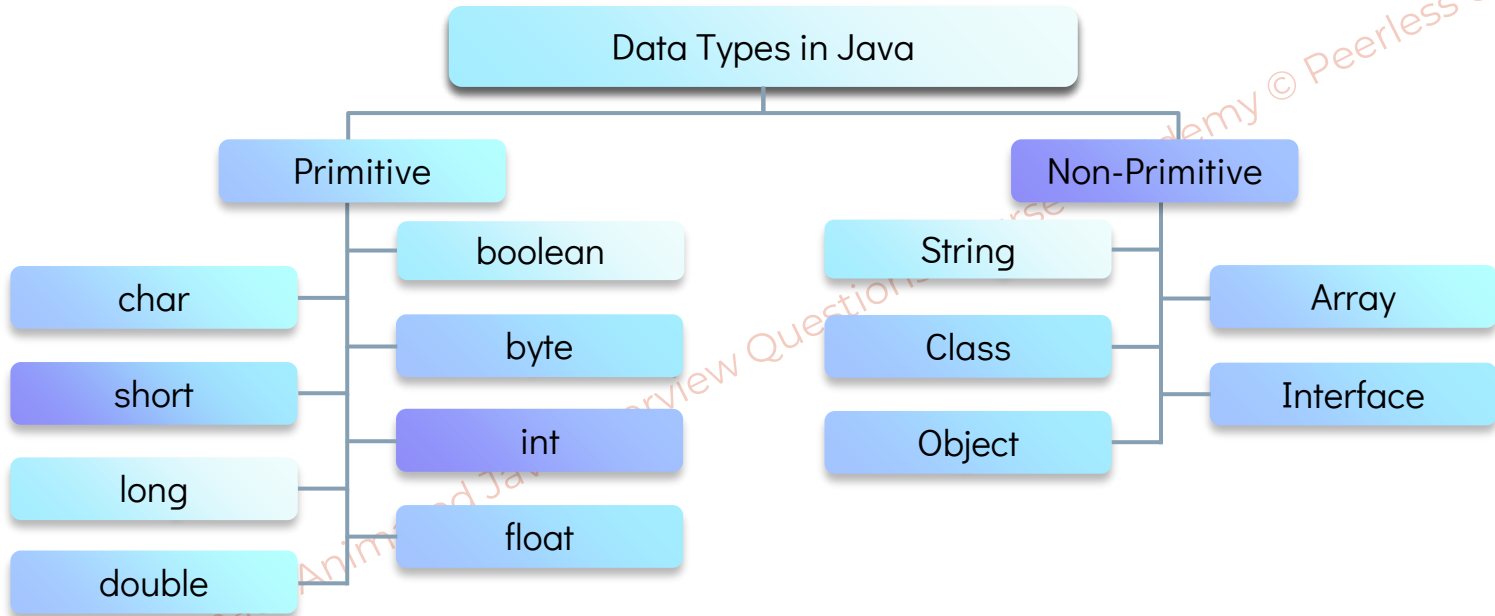
float

- Size: 4 bytes (32 bits)
- Stores a **single precision 32-bit IEEE 754 float-point**
- Has a precision of 7 decimal digits
- Range: 1.4×10^{-45} to 3.4×10^{38}

double

- Size: 8 bytes (64 bits)
- Stores a **double-precision 64-bit IEEE 754 float-point**
- Has a precision of 15 decimal digits.
- Range: 4.9×10^{-324} to 1.8×10^{308}

EXPLANATION



QUESTION

Differentiate between **Primitive** and **Non-Primitive** data types. ✎

Explanation



EXPLANATION

Primitive Data Type	Non-Primitive Data type
Primitive data types are the basic data types that are predefined by the language and are not composed of other data types.	Non-primitive data types are created by the programmer or provided by a Java Class library and are derived from primitive data types.
They are stored directly in the stack memory.	They are stored in the heap memory, and their memory addresses are stored in the stack.
They directly store the data value.	They don't store the actual data, but rather a reference (memory address) to the location
They have a fixed size. For example : an int variable occupies 4 bytes of memory.	The do not have a fixed size and the size vary depending on the data they hold.
They have default values (e.g., 0 for numeric types, false for boolean, \u0000 for char).	They have a default value of null.
They do not have methods and properties associated with them.	They can have methods and properties defined for performing various operations.
They cannot be assigned a null value.	They can be assigned a null value indicating that they do not reference any object.
Examples: byte, short, int, etc.	Examples: String, Array, Class, Interface, etc.

QUESTION

What are
Wrapper Class in
Java and why do
we need them? ✎

Explanation



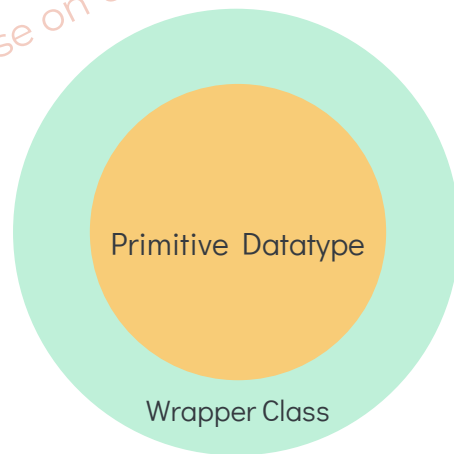
From: The Complete Java Interview Questions course on Udemy © Peerless Geek

EXPLANATION

Wrapper Class

Wrapper, in general, is referred to a larger entity that encapsulates a smaller entity with the purpose of providing an additional layer of functionality or protection.

- It is a class that encapsulates the primitive data types.
- It allows us to represent primitive data types as objects.
- It provides methods to convert between primitive types and their corresponding wrapper objects.
- It also enables compatibility with object-oriented features like generics and collections, which require objects rather than primitives. For example, ArrayList requires Integer object and not primitive int.

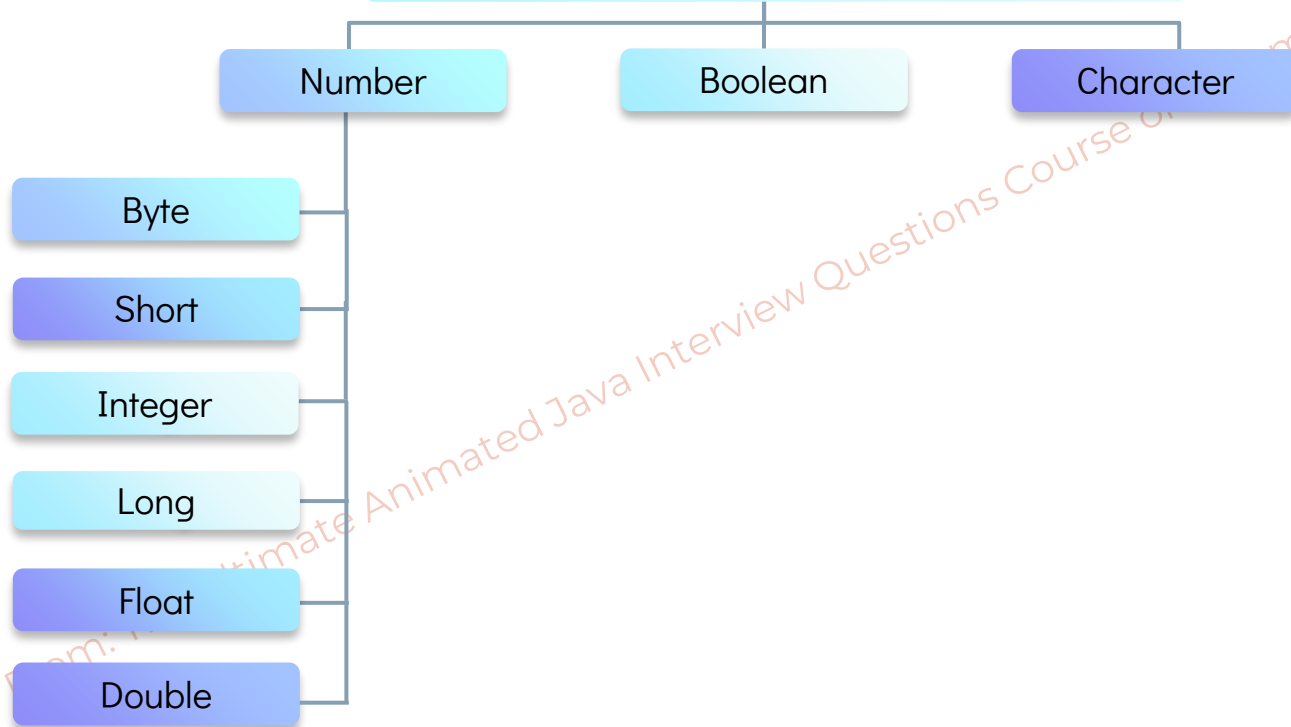


Note :

Wrapper classes are final. For example, We cannot create a custom wrapper class that extends `java.lang.Integer` to add new behavior or modify existing behavior.

EXPLANATION

Java Wrapper Class Hierarchy



Variables, Constructors & Methods

03

QUESTION

Explain the difference between an **Instance variable** and a **Class variable**.



Explanation



EXPLANATION

Instance variable vs *Class variable*

In Java, **instance variables** are *non-static fields defined within a class but outside any method, constructor, or block*, with memory allocated on the heap when an object is created—giving each object its own copy—whereas **class variables** are *static fields, also defined within the class but outside methods, constructors, or blocks, declared using the static keyword* and shared among all instances, with memory allocated only once during class loading, typically in the method area.

Instance variable (Declaration)

Access modifier – restricts access to within the same class only (other examples: **public**, **protected**, **default/no modifier**).

Data type – defines the type of variable (other examples: **int**, **double**, **boolean**, **char**).

Variable name – a user-defined identifier used to store data (other examples: **age**, **title**, **salary**).

private String name;

EXPLANATION

Instance variable vs *Class variable*

In Java, **instance variables** are *non-static fields defined within a class but outside any method, constructor, or block*, with memory allocated on the heap when an object is created—giving each object its own copy—whereas **class variables** are *static fields, also defined within the class but outside methods, constructors, or blocks, declared using the static keyword* and shared among all instances, with memory allocated only once during class loading, typically in the method area.

Class variable (Declaration)

Access modifier – restricts access to within the same class only (*other examples: public, protected, default/no modifier*).

Modifier – indicates that the variable belongs to the class rather than instances.

Data type – defines the type of variable (*other examples: int, double, boolean, char*).

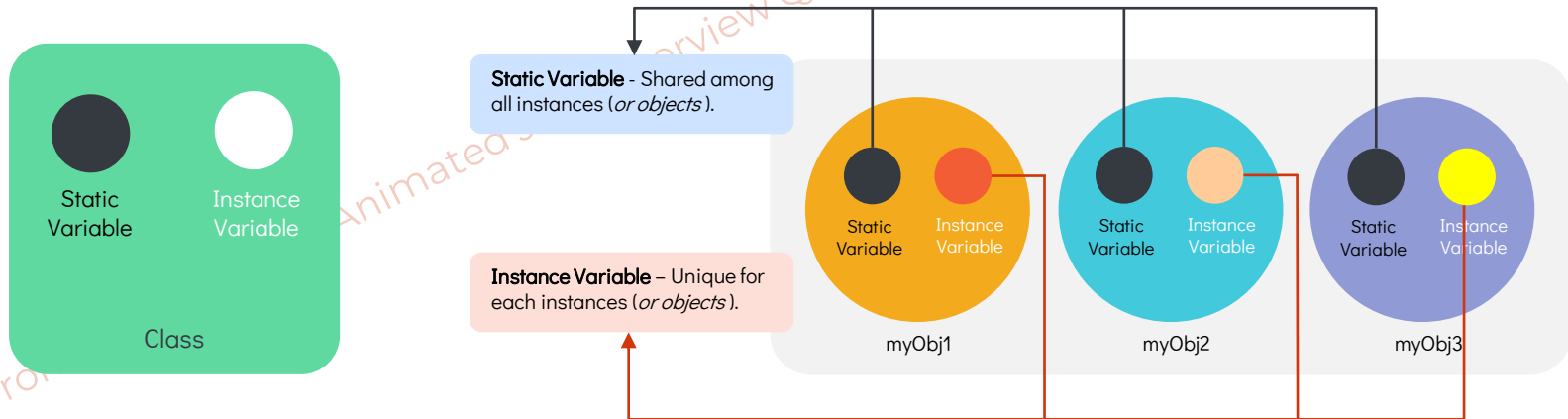
Variable name – a user-defined identifier used to store data (*other examples: age, title, salary*).

private static String name;

EXPLANATION

Instance variable vs Class variable

In Java, **instance variables** are *non-static fields defined within a class but outside any method, constructor, or block*, with memory allocated on the heap when an object is created—giving each object its own copy—whereas **class variables** are *static fields, also defined within the class but outside methods, constructors, or blocks, declared using the static keyword* and shared among all instances, with memory allocated only once during class loading, typically in the method area.



EXPLANATION

Instance Variable	Static Variable
Instance variable is declared without the static keyword within a class but outside any method, constructor, or block.	Static variable is declared with the static keyword within a class but outside any method, constructor, or block.
It is unique to each instance (<i>object</i>) of the class; each object has its own copy.	It is shared among all instances (<i>objects</i>) of the class.
It is accessible throughout the class but not in static methods. An actual object needs to be created before accessing the variable.	It is accessible throughout the class.
It is initialized when an object of the class is created using the new keyword.	It is initialized when the class is loaded into memory by the JVM.
It requires memory to be allocated each time an object is created.	It requires memory allocation only once for the class variable, regardless of the number of instances.
It is accessed using the instance (<i>object</i>) of the class.	It is accessed using the class name rather than an instance of the class.
It is used for properties and characteristics that vary from object to object.	It is used for properties and characteristics shared among all instances, such as constants, counters, or configuration settings.

QUESTION

What is a Class Variable?

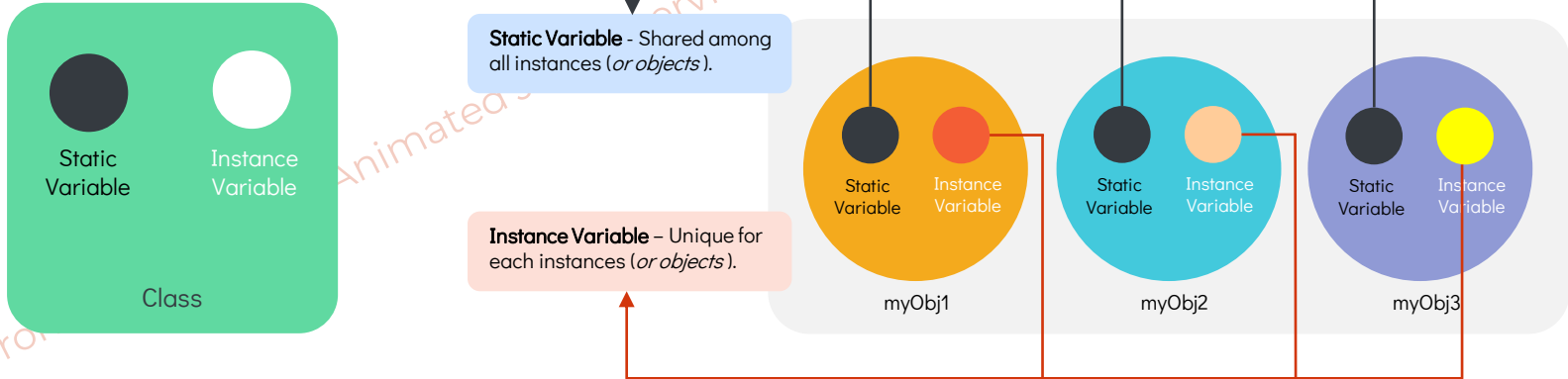
Explanation



EXPLANATION

Class variable

In Java, **class variables**—also known as **static variables**—are declared using the **static** keyword within a class but outside any method, constructor, or block, and are shared across all instances of the class, meaning only one copy exists in memory; they have class-wide scope and can be accessed directly using the class name, making them useful for storing values that should be common to all objects.



EXPLANATION

Class variable

In Java, **class variables**—also known as **static variables**—are declared using the **static** keyword within a class but outside any method, constructor, or block, and are shared across all instances of the class, meaning only one copy exists in memory; they have class-wide scope and can be accessed directly using the class name, making them useful for storing values that should be common to all objects.

```
public class Person {
    public String name;
    public int age;

    public static int personCount = 0;

    public Person(String nameValue, int
ageValue) {
        name = nameValue;
        age = ageValue;
        personCount++;
    }
}
```

```
public class Main {
    public static void main(String[] args) {
        Person p1 = new Person("Monica", 28);
        Person p2 = new Person("Joey", 26);

        // Accessing instance variable (unique per object)
        System.out.println(p1.name);
        // Accessing instance variable (unique per object)
        System.out.println(p1.age);

        // Accessing instance variable (unique per object)
        System.out.println(p2.name);
        // Accessing instance variable (unique per object)
        System.out.println(p2.age);

        // Accessing static variable (shared across all objects)
        System.out.println("Total persons created: " +
Person.personCount);
    }
}
```

QUESTION

What are the
default values
assigned to
instance and static
variables in Java? 

Explanation



From: The Android Java Interview Questions Course on Udemy © Peerless Geek

EXPLANATION

Default Values

In Java, the compiler (*javac*) assigns default values to instance variables at the time of object creation, and to static (class) variables at the time of class loading—such as *false* for *boolean*, *'\u0000'* for *char*, *0* for *numeric types*, and *null* for *reference types*—while local variables must be explicitly initialized before use.

Data type	Default Value
boolean	false
char	'\u0000'
byte	0
int	0
short	0
float	0.0
double	0.0
String	Null
Array	null
Object	null

QUESTION

Differentiate
between
instance and
local variables. 

Explanation



EXPLANATION

Instance variable vs *Local variable*

In Java, **instance variables** are *non-static fields defined within a class but outside any method, constructor, or block*, with memory allocated on the heap when an object is created—giving each object its own copy—whereas **local variables** are *declared within methods, constructors, or blocks*, exist only during the execution of that block, have no default value, and are stored in the stack, making them inaccessible outside their defined scope.

```
public class Person {
    // Instance variables
    public String name;
    public int age;
    // Static variable
    public static int personCount = 0;

    public Person(String nameValue, int ageValue) {
        // Local variable
        String greeting = "Creating a new Person
        named: " + nameValue;
        System.out.println(greeting);

        name = nameValue;
        age = ageValue;
        personCount++;
    }
}
```

EXPLANATION

Instance Variable	Local Variable
Instance variable is a non-static field declared within a class but outside any method, constructor, or block.	Local variable is a variable declared inside a method, constructor, or block.
It is associated with an instance (<i>object</i>) of the class, and each instance has its own copy.	It is not tied to any object; it exists only during the execution of the block it is declared in.
It is allowed to have access modifiers (<i>public, private, etc.</i>) and non-access modifiers (<i>final, transient, etc.</i>).	It is not allowed to have access modifiers but can be marked final .
It is accessible from any method or constructor within the class (<i>depending on its access modifier</i>).	It is accessible only within the block or method in which it is declared.
It is created when an object is instantiated and destroyed when the object is garbage collected.	It is created when the method/block is invoked and destroyed once it finishes execution.
It is stored in the heap memory as part of the object.	It is stored in the stack memory, tied to method call frames.
It is automatically assigned a default value if not explicitly initialized.	It is not given any default value and must be initialized before use.
It is used to maintain the state of an object – e.g., name, age, or balance of a Customer object.	It is used for temporary calculations or logic inside methods – e.g., sum, index, or tempResult in a loop or function.

QUESTION

What is the difference between an **instance method** and a **static (class) method**? 

Explanation



EXPLANATION

Instance method vs Static method

In Java, **instance methods** are *non-static methods defined within a class that require an object to be invoked* and *can access both instance and static members*—allowing behavior specific to each object—whereas **class methods** are *static methods declared using the static keyword, invoked without creating an object*, and *can only directly access other static members*, making them suitable for operations not tied to object state and typically shared across all instances.

Instance method (Declaration)

Access modifier – Specifies the visibility of the method. **public** means it can be accessed from any other class. Other access modifiers include **private**, **protected**, and **package-private** (no modifier).

Return type – **void** indicates that the method does not return any value; other return types include **int**, **double**, **String**, etc.

Method name – **introduce** is the name given by the programmer to identify and call the method.

Parameter list / Parentheses – **()** defines input parameters; even if there are none, parentheses are required (e.g., **(int a, int b)** for two parameters).

Method body / Curly braces – **{ }** enclose the block of code that defines what the method does; all executable statements are placed within these braces.

public void introduce () { }

EXPLANATION

Instance method vs Static method

In Java, **instance methods** are *non-static methods defined within a class that require an object to be invoked* and *can access both instance and static members*—allowing behavior specific to each object—whereas **class methods** are *static methods declared using the static keyword, invoked without creating an object*, and *can only directly access other static members*, making them suitable for operations not tied to object state and typically shared across all instances.

Static method (Declaration)

Access modifier – Specifies the visibility of the method. **public** means it can be accessed from any other class. Other access modifiers include **private**, **protected**, and package-private (no modifier).

Modifier – static indicates that the method belongs to the class itself and can be called without creating an instance of the class.

Return type – void indicates that the method does not return any value; other return types include **int**, **double**, **String**, etc.

Method name – introduce is the name given by the programmer to identify and call the method.

Parameter list / Parentheses – () defines input parameters; even if there are none, parentheses are required (e.g., **(int a, int b)** for two parameters).

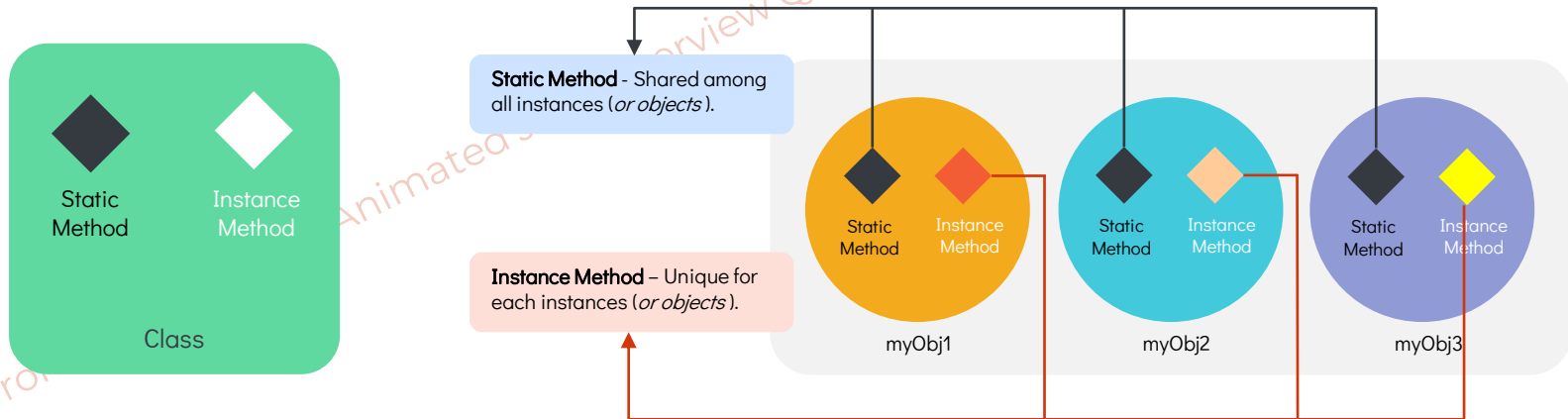
Method body / Curly braces – {} enclose the block of code that defines what the method does; all executable statements are placed within these braces.

public static void introduce () {}

EXPLANATION

Instance method vs Static method

In Java, **instance methods** are *non-static methods defined within a class that require an object to be invoked* and *can access both instance and static members*—allowing behavior specific to each object—whereas **class methods** are *static methods declared using the static keyword, invoked without creating an object*, and *can only directly access other static members*, making them suitable for operations not tied to object state and typically shared across all instances.



EXPLANATION

Instance Method	Static Method
Instance method is declared without the static keyword within a class.	Static method is declared with the static keyword within a class.
It is associated with an instance (<i>object</i>) of the class rather than with the class itself.	It is associated with the class itself rather than with an instance (<i>object</i>) of the class.
It requires an object of the class to be called.	It can be called using the class name without creating an object.
It operates on instance variables and can also access static variables.	It can only access static variables and other static methods directly.
It can use this keyword to refer to the current object.	It cannot use this since it is not associated with any object.
It is commonly used when behavior depends on instance data.	It is commonly used for utility or helper methods that don't rely on object state.

QUESTION

What is a Constructor?

Explanation



EXPLANATION

Constructor

In Java, a **constructor** is a special method that is invoked automatically when an object is created using the **new** keyword, it has the same name as the class, does not have a return type—not even **void**—and is used to initialize object state, allowing fields to be set at the time of object creation, with support for overloading to define multiple constructors for different initialization scenarios, while the absence of a user-defined constructor causes the compiler to provide a default no-argument constructor.

Constructor (Declaration)

Access modifier – Specifies the visibility of the constructor; **public** allows it to be accessed from any other class. Other access modifiers include **private**, **protected**, and package-private (no modifier).

Constructor name – Must match the class name (**Person** in this case) and is used to initialize new objects of the class.

Parameter list / Parentheses – ()
defines input parameters for the constructor; even if there are none, parentheses are required (e.g., (**String name**) for one parameter).

Constructor body / Curly braces – { }
enclose the block of code that initializes the object; all initialization logic goes inside these braces.

public Person () { }

QUESTION

What is the difference between **constructors** and **methods**? 

Explanation



EXPLANATION

Constructor	Method
Constructor is a special method that is automatically called when an object of a class is created.	Method defines the behavior or actions that objects of a class can perform.
It is used to initialize the instance variables of a class.	It is used to perform a specific task.
It is implicitly called with the new keyword.	It is explicitly called using the object's reference.
It does not have any return type, not even void.	It can have a return type (<i>including void</i>).
It has the same name as the class itself.	It can have any unique name.
It can have parameters to initialize instance variables.	It can have parameters to receive input values.
It does not have any return statement.	It uses return statement to return a value.
It is not inherited and therefore cannot be overridden by a subclass.	It can be inherited by subclasses (<i>unless marked as private</i>) and therefore can be overridden in subclasses, unless marked as <i>final</i> .

QUESTION

What happens if we do not provide a constructor in a class? 

Explanation



From: The Complete Java Interview Questions Course on Udemy © Peerless Geek

EXPLANATION

Default Constructor

In Java, if no constructors are explicitly defined in a class, *the compiler automatically provides a default no-argument constructor that has the same visibility as the class, calls the no-argument constructor of the superclass using `super()`, and initializes all fields with their default values (0 for numeric types, false for boolean, and null for object references)*; however, this constructor is not generated if any constructor is manually defined.

```
public class Person {  
    private String name;  
    private int age;  
}
```

Person.java

```
public class Person {  
    private String name;  
    private int age;  
  
    public Person() {  
        super();  
    }  
}
```

Compiled Person.class file

QUESTION

How many **types**
of constructors
are used
in Java? 

Explanation



From: The Ultimate Java Interview Questions Course on Udemy © Peerless Geek

EXPLANATION

Types of Constructors

In Java, there are **three types of constructors**—default (*no-argument*), parameterized, and copy constructor (*unlike C++, Java does not provide a default copy constructor; it must be defined manually*)—where *the default constructor initializes objects with default values, the parameterized constructor allows setting specific values during object creation, and the copy constructor creates a new object by copying the values from another object of the same class.*

```
public class Person {  
    private String name;  
    private int age;  
  
    // Default Constructor  
    public Person() {}  
  
    // Parameterized constructor  
    public Person(String name, int age) {  
        this.name = name;  
        this.age = age;  
    }  
  
    // Copy constructor  
    public Person(Person person) {  
        this.name = person.name;  
        this.age = person.age;  
    }  
}
```

QUESTION

Can we declare Pointers in Java?



Explanation



From: The Java Interview Questions Course on Udemy © Peerless Geek

EXPLANATION

Pointers

A *pointer in real life* can be understood as a reference or a link that directs you to the location of something, rather than containing the thing itself. It acts like a map, a label, or an identifier that helps you access the actual information stored elsewhere.



Address Label



Signpost



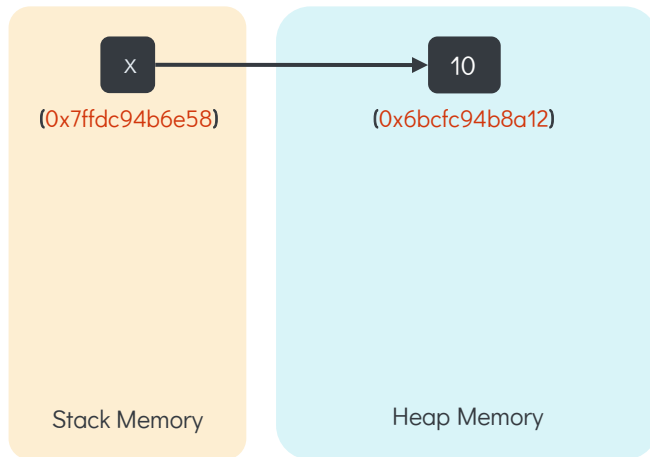
Hotel key

EXPLANATION

Pointers in Java: The Forbidden Concept

Java **DOES NOT** support explicit pointers like C or C++ does; instead, it uses references to objects which serve a similar purpose but without direct memory manipulation capabilities, as Java intentionally eliminates pointer arithmetic, dereferencing syntax, and explicit memory address access to enhance memory safety and security through its automatic garbage collection system.

```
public static void main(String[] args) {  
    // Declare an integer variable.  
    Integer x = Integer.valueOf(10);  
  
    System.out.println("The value of x is:  
" + x); // 10  
}
```



Operators

04

QUESTION

What are **operators**? How many **types of operators** are available in Java? 

Explanation



From: The Java Interview Questions Course on Udemy © Peerless Geek

EXPLANATION

Operator & its types

Operators in Java are special symbols or keywords used to perform operations on variables and values. Java provides several types of operators, including **arithmetic operators** (+, -, *, /, %), **assignment operators** (=, +=, -=, *=, /=, %=), **relational operators** (==, !=, >, <, >=, <=), **unary operators** (++ , --, !, -), **logical operators** (&&, ||, !), **ternary operator** (?:), **bitwise operators** (&, /, ^, ~, <<, >>, >>>), and the **instanceof** operator used to test object types.

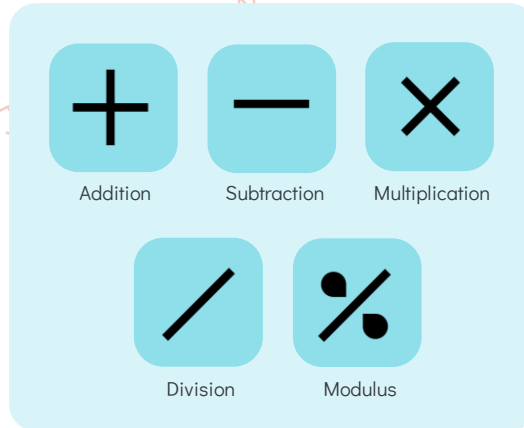
From: The Ultimate Animated Java Interview Questions Course on Udemy © Peerless Geek

EXPLANATION

Operator & its types

Operators in Java are special symbols or keywords used to perform operations on variables and values. Java provides several types of operators, including **arithmetic operators** (+, -, *, /, %), **assignment operators** (=, +=, -=, *=, /=, %=), **relational operators** (==, !=, >, <, >=, <=), **unary operators** (++ , --, !, -), **logical operators** (&&, ||, !), **ternary operator** (? :), **bitwise operators** (&, /, ^, ~, <<, >>, >>>), and the **instanceof** operator used to test object types.

Arithmetic Operators



EXPLANATION

Arithmetic Operators

Arithmetic operators are symbols used to perform mathematical calculations on numerical data types like integers and floating-point numbers.

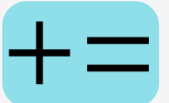
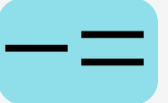
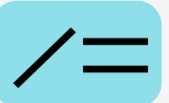
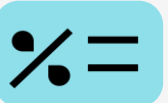
Operator	Description
Addition (+)	Adds two operands.
Subtraction (-)	Subtracts two operands.
Multiplication (*)	Multiplies two operands.
Division (/)	Divides the first operand (<i>dividend</i>) by the second operand (<i>divisor</i>) and gives the quotient as a result.
Modulus (%)	Returns the remainder when the first operand (<i>dividend</i>) is divided by the second operand (<i>divisor</i>).

EXPLANATION

Operator & its types

Operators in Java are special symbols or keywords used to perform operations on variables and values. Java provides several types of operators, including **arithmetic operators** (+, -, *, /, %), **assignment operators** (=, +=, -=, *=, /=, %=), **relational operators** (==, !=, >, <, >=, <=), **unary operators** (++ , --, !, -), **logical operators** (&&, ||, !), **ternary operator** (? :), **bitwise operators** (&, /, ^, ~, <<, >>, >>>), and the **instanceof** operator used to test object types.

Assignment Operators

		
Simple assignment	Addition and assignment	Subtraction and assignment
		
Multiplication and assignment	Division and assignment	Modulus and assignment

EXPLANATION

Assignment Operators

Assignment operators are used to assign values to variables.

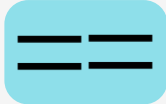
Operator	Description
Simple assignment operator (=)	Assigns the value on the right to the variable on the left.
Addition and assignment (+=)	Adds the value on the right to the current value of the variable on the left and assigns the result to the variable on the left.
Subtraction and Assignment (-=)	Subtracts the value on the right from the current value of the variable on the left and assigns the result to the variable on the left.
Multiplication and Assignment (*=)	Multiplies the value on the right with the current value of the variable on the left and assigns the result to the variable on the left.
Division and Assignment (/=)	Divides the current value of the variable on the left by the value on the right and assigns the result to the variable on the left.

EXPLANATION

Operator & its types

Operators in Java are special symbols or keywords used to perform operations on variables and values. Java provides several types of operators, including **arithmetic operators** (+, -, *, /, %), **assignment operators** (=, +=, -=, *=, /=, %=), **relational operators** (==, !=, >, <, >=, <=), **unary operators** (+++, --, !, -), **logical operators** (&&, ||, !), **ternary operator** (? :), **bitwise operators** (&, /, ^, ~, <<, >>, >>>), and the **instanceof** operator used to test object types.

Relational Operators

		
Less than	Greater than	Less than or equal to
		
Greater than or equal to	Equal to	Not Equal to

EXPLANATION

Relational Operators

Relational operators are used to compare two values or expressions and return a boolean result indicating the relationship between them.

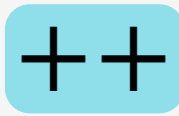
Operator	Description
Equal to (==)	Checks if two values are equal.
Not Equal to (!=)	Check if two values are not equal.
Greater than (>)	Checks if the value on the left is greater than the value on the right.
Less than (<)	Checks if the value on the left is less than the value on the right.
Greater than or equal to (>=)	Checks if the value on the left is greater than or equal to the value on the right.

EXPLANATION

Operator & its types

Operators in Java are special symbols or keywords used to perform operations on variables and values. Java provides several types of operators, including **arithmetic operators** (+, -, *, /, %), **assignment operators** (=, +=, -=, *=, /=, %=), **relational operators** (==, !=, >, <, >=, <=), **unary operators** (++ , --, !, -), **logical operators** (&&, ||, !), **ternary operator** (? :), **bitwise operators** (&, /, ^, ~, <<, >>, >>>), and the **instanceof** operator used to test object types.

Unary Operators



Increment



Decrement



NOT operator



Minus

From: The Ultimate Animated Java Questions Course on Udemy © Peerless Geek

EXPLANATION

Unary Operators

Unary operators are operators that operate on a single operand.

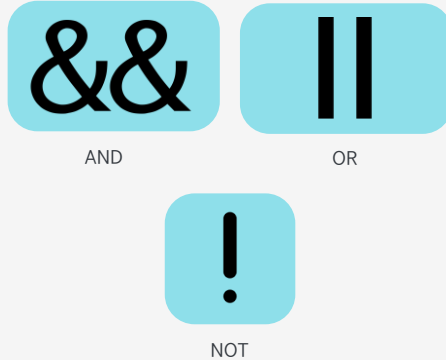
Operator	Description
Unary minus(-)	This operator is used to negate the value of an operand.
Not Operator (!) or Logical Complement Operator (!)	This is used to convert true to false and vice versa. It reverses the logical state of an operand.
Increment(++)	This is used to increment the value of an operand by 1. i) Post-increment operator (num++). ii) Pre-increment operator (++num).
Decrement(--)	This is used to decrement the value of an operand by 1. i) Post-decrement operator (num--). ii) Pre-decrement operator (--num).
Unary minus(-)	This operator is used to negate the value of an operand.

EXPLANATION

Operator & its types

Operators in Java are special symbols or keywords used to perform operations on variables and values. Java provides several types of operators, including **arithmetic operators** (+, -, *, /, %), **assignment operators** (=, +=, -=, *=, /=, %=), **relational operators** (==, !=, >, <, >=, <=), **unary operators** (++ , --, !, -), **logical operators** (&&, ||, !), **ternary operator** (?:), **bitwise operators** (&, /, ^, ~, <<, >>, >>>), and the **instanceof** operator used to test object types.

Logical Operators



EXPLANATION

Logical Operators

Logical operators are used to perform logical operations on boolean operands.

Operator	Description
Logical AND (&&)	Returns true if both operands are true, otherwise returns false.
Logical OR ()	Returns true if at least one of the operands is true, otherwise returns false.
Logical NOT (!)	Reverses the logical state of its operand. If the operand is true, it returns false; if the operand is false, it returns true.
Logical AND (&&)	Returns true if both operands are true, otherwise returns false.
Logical OR ()	Returns true if at least one of the operands is true, otherwise returns false.

EXPLANATION

Operator & its types

Operators in Java are special symbols or keywords used to perform operations on variables and values. Java provides several types of operators, including **arithmetic operators** (+, -, *, /, %), **assignment operators** (=, +=, -=, *=, /=, %=), **relational operators** (==, !=, >, <, >=, <=), **unary operators** (++ , --, !, -), **logical operators** (&&, ||, !), **ternary operator** (?:), **bitwise operators** (&, /, ^, ~, <<, >>, >>>), and the **instanceof** operator used to test object types.

Ternary Operator

A ternary operator in Java is a shorthand for an if-else statement.



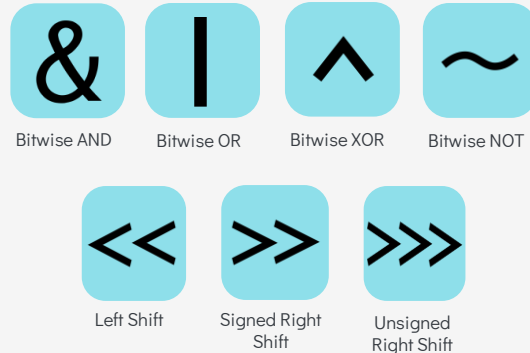
Ternary operator

EXPLANATION

Operator & its types

Operators in Java are special symbols or keywords used to perform operations on variables and values. Java provides several types of operators, including **arithmetic operators** (+, -, *, /, %), **assignment operators** (=, +=, -=, *=, /=, %=), **relational operators** (==, !=, >, <, >=, <=), **unary operators** (++ , --, !, -), **logical operators** (&&, ||, !), **ternary operator** (?:), **bitwise operators** (&, /, ^, ~, <<, >>, >>>), and the **instanceof** operator used to test object types.

Bitwise Operators



EXPLANATION

Bitwise Operators

Bitwise operators are used to perform the manipulation of individual bits of a number.

Operator	Description
Bitwise AND (&)	Performs a bitwise AND operation between the corresponding bits of two operands. If both bits are 1, the result is 1; otherwise, it's 0.
Bitwise OR ()	Performs a bitwise OR operation between the corresponding bits of two operands. If at least one of the bits is 1, the result is 1; otherwise, it's 0.
Bitwise XOR (^)	Performs a bitwise XOR (<i>exclusive OR</i>) operation between the corresponding bits of two operands. If the bits are different, the result is 1; if the bits are the same, the result is 0.
Bitwise NOT or Complement (~)	Inverts the bits of its operand. Each 0 bit becomes 1, and each 1 bit becomes 0.
Left Shift(<<)	Shifts the bits of the left operand to the left by a specified number of positions. Zeros are filled in from the right.
Signed Right Shift(>>)	Shifts the bits of the left operand to the right by the number of positions specified by the right operand. The vacant positions on the left are filled with the sign bit(0 for non-negative, and 1 for negative numbers).
Unsigned Right Shift(>>>)	Similar to the right shift (>>), but the vacant positions on the left are filled with zeros.

EXPLANATION

Operator & its types

Operators in Java are special symbols or keywords used to perform operations on variables and values. Java provides several types of operators, including **arithmetic operators** (+, -, *, /, %), **assignment operators** (=, +=, -=, *=, /=, %=), **relational operators** (==, !=, >, <, >=, <=), **unary operators** (++ , --, !, -), **logical operators** (&&, ||, !), **ternary operator** (?:), **bitwise operators** (&, /, ^, ~, <<, >>, >>>), and the **instanceof** operator used to test object types.

instanceof Operator

The instanceof operator in Java checks if an object is an instance of a specified class or subclass, returning **true** or **false**.

instanceof

QUESTION

Explain the difference between `>>` and `>>>` operators. 

Explanation



EXPLANATION

Signed right shift-operator (>>)	Unsigned right-shift operator(>>>)
Signed right-shift operator shifts the bits of a number to the right.	Unsigned right-shift operator shifts the bits of a number to the right.
It preserves the sign bit (the leftmost bit) during the shift.	It does not preserve the sign bit during the shift.
It fills the leftmost positions with zeros when shifting a positive number.	It fills the leftmost positions with zeros when shifting a positive number.
It fills the leftmost positions with ones (sign extension) when shifting a negative number, thus preserving the negative sign.	It fills the leftmost positions with zeros when shifting a negative number.

QUESTION

Which Java operator is **right associative**? 

Explanation



Associativity

Associativity refers to how operators of the same precedence are grouped in the absence of parentheses. **There are two types of associativity: *left associative* and *right associative*.**

- In left associative operators, operations are grouped from left to right.
- In right associative operators, operations are grouped from right to left.
- In Java, **the assignment operator (=), compound assignment operators** (such as +=, -=, *=, /=, %=), **ternary conditional operator** (? :), and **unary operators** (such as +, -, ++, --, !, ~) are right-associative. All other operators, including arithmetic, relational, logical, and bitwise operators, are left-associative.

QUESTION

What is the **dot operator** (•) in Java?

Explanation



EXPLANATION

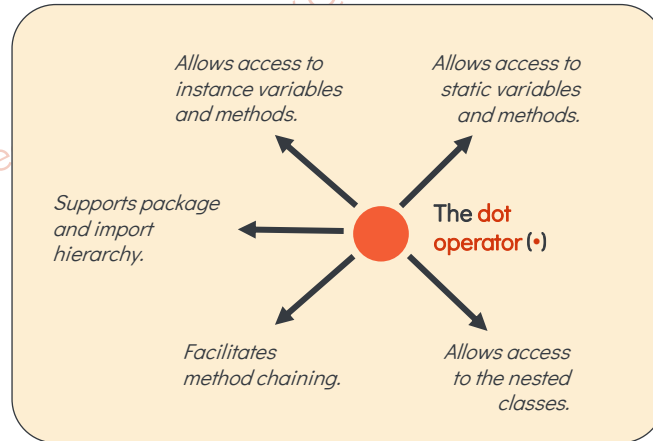
Dot Operator (●)

The *DOT* operator (●) serves as a fundamental access mechanism enabling developers to *reference instance variables* and *call methods on objects*, *access static members like fields and methods* directly from a class, *navigate nested classes* within a parent class, and *establish hierarchical relationships in package naming and imports*, ultimately functioning as an essential syntactic element that allows for proper navigation through Java's object-oriented structure.

EXPLANATION

Dot Operator (•)

The *DOT* operator (•) serves as a fundamental access mechanism enabling developers to *reference instance variables* and *call methods on objects*, *access static members like fields and methods* directly from a class, *navigate nested classes* within a parent class, and *establish hierarchical relationships in package naming and imports*, ultimately functioning as an essential syntactic element that allows for proper navigation through Java's object-oriented structure.



Strings

05

QUESTION

What is the Java String Pool?



Explanation



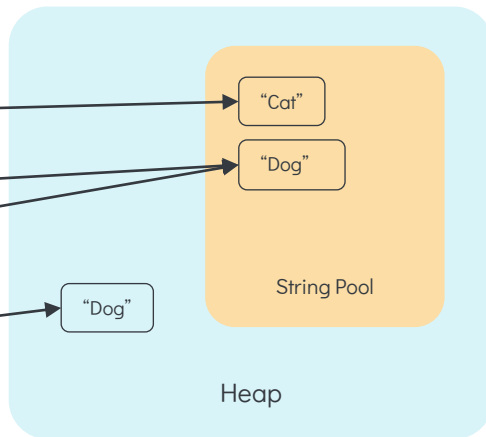
From: The Complete Java Interview Questions Course on Udemy © Peerless Geek

EXPLANATION

String Pool : Java's Memory Optimizer

The Java **String Pool**, also known as the *String Constant Pool* or *String Intern Pool*, is a special memory area in the Java Virtual Machine (JVM) that stores string literals; when you create a string literal in Java (e.g., **String str = "hello"**), the JVM checks if the string already exists in the pool, and if it does, it returns a reference to the existing string instead of creating a new object, thereby conserving memory.

```
String s1 = "Cat";  
String s2 = "Dog";  
String s3 = "Dog";  
String s4 = new String("Dog");
```



EXPLANATION

String Pool : Java's Memory Optimizer

The Java **String Pool**, also known as the *String Constant Pool* or *String Intern Pool*, is a special memory area in the Java Virtual Machine (JVM) that stores string literals; when you create a string literal in Java (e.g., **String str = "hello"**), the JVM checks if the string already exists in the pool, and if it does, it returns a reference to the existing string instead of creating a new object, thereby conserving memory.

- It is a subset of the Heap memory, not a separate memory region.
- It is part of the Java heap memory since Java 7 (previously in **PermGen** space).
- It optimizes memory by storing a single copy of each unique string literal.
- It supports **manual interning** (*storing only one copy of each unique string*) via the **intern()** method, adding strings explicitly to the pool.
- It stores literals at compile-time, while new **String().intern()** adds them at runtime.
- It does not include strings created with **new**, as they reside in the heap by default.

Note :

The statement `String s1 = new String("Cat")` will create 2 objects, one in the string pool due to string literal ("Cat") used, and another in the heap due to the **new** keyword used for string instantiation.

QUESTION

Can you tell the difference between **equals()** method and **equality operator (==)** in Java? 

Explanation



EXPLANATION

equality operator (==)	<i>equals()</i> method
The equality operator (==) in Java is a built-in language feature that cannot be modified or customized.	The equals() method is a method from the Object class, which can be overridden in custom classes to define specific equality logic based on content.
It is used for both primitives and object references.	It cannot be used with primitives, as it is specifically designed for objects.
It compares the actual values of primitives (e.g., 5 == 5 returns true), but when used with objects, it checks their memory addresses instead of their content.	It checks whether two objects have equal content instead of comparing their memory addresses, but by default (<i>inherited from the Object class</i>), it behaves like the == operator, comparing memory addresses unless overridden.

QUESTION

Why is it said that the **length()** method of **String** class doesn't return accurate results? 

Explanation



EXPLANATION

Java's *length()* Method: The Hidden Unicode Trap

Java strings use **UTF-16 encoding**, where each character is represented by **one or two 16-bit code units** (*2 bytes each*), and the **length()** method of the String class returns *the number of UTF-16 code units rather than the actual character count*, which can lead to inaccurate results with Unicode characters, emojis, or surrogate pairs, as common characters use one code unit (*2 bytes*), while special characters, symbols, and certain language scripts (*e.g., Arabic, Chinese*) may require two code units (*4 bytes*).

From: The Ultimate Animated Java Interview Questions Course © Peerless Geek

EXPLANATION

Java's `length()` Method: The Hidden Unicode Trap

Java strings use **UTF-16 encoding**, where each character is represented by **one or two 16-bit code units** (2 bytes each), and the `length()` method of the String class returns *the number of UTF-16 code units rather than the actual character count*, which can lead to inaccurate results with Unicode characters, emojis, or surrogate pairs, as common characters use one code unit (2 bytes), while special characters, symbols, and certain language scripts (e.g., *Arabic, Chinese*) may require two code units (4 bytes).

```
String s = "Welcome"; // 7
System.out.println(s.length()); // 7
```

Character	W	e	l	c	o	m	e
UTF-16 Code Unit (Decimal)	87	101	108	99	111	109	101
UTF-16 Code Unit (Hex)	U+0057	U+0065	U+006C	U+0063	U+006F	U+006D	U+0065
Position	0	1	2	3	4	5	6

EXPLANATION

Java's `length()` Method: The Hidden Unicode Trap

Java strings use **UTF-16 encoding**, where each character is represented by **one or two 16-bit code units** (2 bytes each), and the `length()` method of the String class returns *the number of UTF-16 code units rather than the actual character count*, which can lead to inaccurate results with Unicode characters, emojis, or surrogate pairs, as common characters use one code unit (2 bytes), while special characters, symbols, and certain language scripts (e.g., *Arabic, Chinese*) may require two code units (4 bytes).

```
String s = "Welcome👋"; // 8
System.out.println(s.length()); // 9
```

Character	W	e	l	c	o	m	e	👋	
UTF-16 Code Unit (Decimal)	87	101	108	99	111	109	101	129761	
UTF-16 Code Unit (Hex)	U+0057	U+0065	U+006C	U+0063	U+006F	U+006D	U+0065	0xD83E	0xDEE1
Position	0	1	2	3	4	5	6	7	8

QUESTION

How would you differentiate between a **String**, **StringBuffer**, and a **StringBuilder**? 

Explanation



EXPLANATION

String** vs **StringBuffer** vs **StringBuilder

The difference between **String**, **StringBuffer**, and **StringBuilder** in Java lies in their mutability and performance characteristics: **String** is *immutable*, meaning any modification creates a new object, whereas **StringBuffer** and **StringBuilder** are *mutable*, allowing in-place modifications; **StringBuffer** is *thread-safe* due to synchronization, making it suitable for multi-threaded environments but slower, while **StringBuilder** is *not synchronized*, offering better performance in single-threaded scenarios.

From: The Ultimate Animated Java Interview Questions Course on Udemy © Peerless Geek

EXPLANATION

String	StringBuilder	StringBuffer
String is immutable, meaning its value cannot be changed once created.	StringBuilder is a mutable object.	StringBuffer is also a mutable object.
It is inherently thread-safe.	It is not thread-safe and is preferred in single-threaded environment.	It is thread-safe as all methods are synchronized.
It is slower for frequent modifications because each change creates a new object	It is faster for single-threaded operations.	It is slower than StringBuilder due to thread-safety overhead.
It has a higher memory overhead due to immutability, as each modification creates a new object, consuming more memory.	It has a lower memory overhead for single-threaded operations.	It has a higher memory overhead due to thread-safety.
It is preferred when you don't need to modify the string frequently.	It is preferred when you need to perform frequent string modifications in a single-threaded environment	It is preferred when you need to perform frequent string modifications in a multi-threaded environment.

QUESTION

Why is
StringBuffer or
StringBuilder
called **mutable**? 

Explanation



Animated Java Interview Questions Course on Udemy © Peerless Geek

From: The

EXPLANATION

StringBuffer & *StringBuilder*: The Mutable Alternatives to *String*

StringBuffer and *StringBuilder* are called mutable because their contents can be modified after creation, allowing you to alter characters, append new text, insert or delete portions, etc., without creating a new object.

From: The Ultimate Animated Java Interview Questions Course on Udemy © Peerless Geek

Arrays

06

QUESTION

What is an array in Java?



Explanation

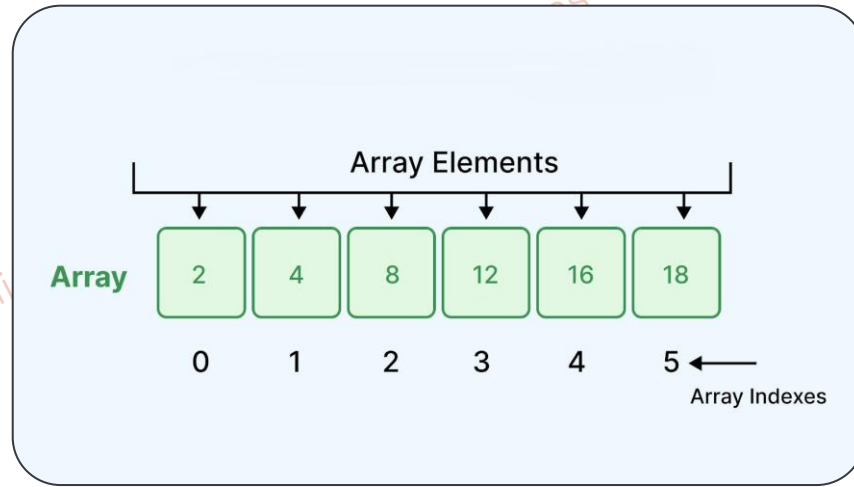


From: The
Animated Java Interview Questions Course on Udemy © Peerless Geek

EXPLANATION

Array

An **array** is a fixed-size data structure that stores multiple values of the same data type in a contiguous block of memory, where each element is accessed using an index, with its size defined at creation, making it immutable.



EXPLANATION

Array

An **array** is a fixed-size data structure that stores multiple values of the same data type in a contiguous block of memory, where each element is accessed using an index, with its size defined at creation, making it immutable.

- It is declared using square brackets ([]). For example:
int[] nums;
- It can store only elements of the same data type, such as *int[]*, *String[]*, or *double[]*.
- It can be initialized by **specifying size directly** (*int[] a = new int[5];*), **declaring first then initializing** (*int[] a; a = new int[5];*), **using new with values** (*int[] a = new int[] {1, 2, 3};*), using **literal values** (*int[] a = {1, 2, 3};*), or by **assigning values individually** (*a[0] = 10; a[1] = 20;*).
- It is initialized with default values: **0** for numeric types, **false** for boolean, and **null** for object references (e.g., *String[]*); for example, *int[] arr = new int[3];* sets *arr[0]* to 0.
- It has a fixed size that is defined at the time of creation and cannot be changed later.
- It uses **zero-based indexing**, where elements can be accessed or modified using their index, such as *arr[0]*.
- Its length can be accessed using **.length** property, for example: *arr.length*.

QUESTION

On which
memory arrays
are created in
Java? 

Explanation



EXPLANATION

Memory Allocation of *Arrays* in Java

In Java, **arrays** are always created in the **heap memory** since they are objects, while their **references** are stored in the **stack memory** (*for local variables*) or the **method area** (*for static arrays*); if an array holds objects, it stores references, while the actual objects also reside in the heap.

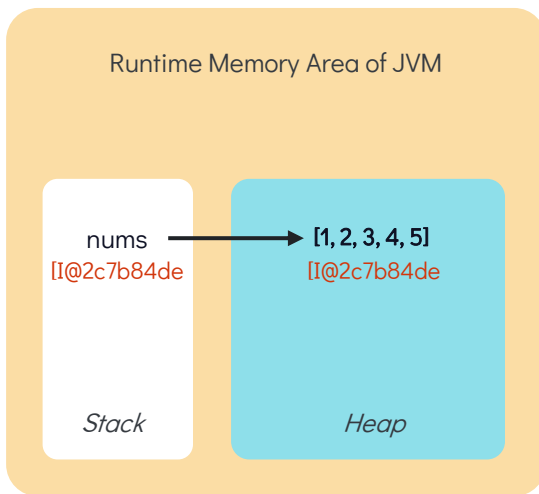
From: The Ultimate Animated Java Interview Questions Course on Udemy © Peerless Geek

EXPLANATION

Memory Allocation of *Arrays* in Java

In Java, **arrays** are always created in the **heap memory** since they are objects, while their **references** are stored in the **stack memory** (*for local variables*) or the **method area** (*for static arrays*); if an array holds objects, it stores references, while the actual objects also reside in the heap.

```
public class Main {  
    public static void main(String[] args) {  
        int[] nums = new int[] {1, 2, 3, 4, 5};  
    }  
}
```



QUESTION

What are the types of an array?

Explanation



Animated Java Interview Questions Course on Udemy © Peerless Geek

From: The

EXPLANATION

Types of *Arrays*

Arrays are categorized based on their structure into **one-dimensional arrays**, *where elements are arranged in a linear sequence*, and **multi-dimensional arrays**, which consist of *arrays of arrays* and allow the creation of tables or matrices; they are also classified by datatype into **primitive type arrays**, which *store primitive data types* like int and char, and **object type arrays**, which store instances of classes.

From: The Ultimate Animated Java Interview Questions Course on Udemy © Peerless Geek

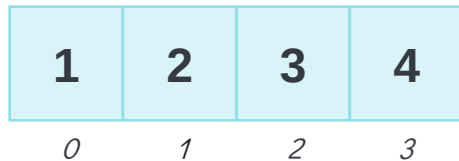
EXPLANATION

Types of *Arrays*

Arrays are categorized based on their structure into **one-dimensional arrays**, where *elements are arranged in a linear sequence*, and **multi-dimensional arrays**, which consist of *arrays of arrays* and allow the creation of tables or matrices; they are also classified by datatype into **primitive type arrays**, which *store primitive data types* like int and char, and **object type arrays**, which store instances of classes.

One-dimensional Array

```
int[] nums = {1, 2, 3, 4};
```



EXPLANATION

Types of *Arrays*

Arrays are categorized based on their structure into **one-dimensional arrays**, where elements are arranged in a linear sequence, and **multi-dimensional arrays**, which consist of arrays of arrays and allow the creation of tables or matrices; they are also classified by datatype into **primitive type arrays**, which store primitive data types like int and char, and **object type arrays**, which store instances of classes.

Two-dimensional Array

```
int[][] matrix = {  
    {1, 2, 3, 4, 5},  
    {6, 7, 8, 9, 10},  
    {11, 12, 13, 14, 15},  
    {16, 17, 18, 19, 20}  
};
```

	0	1	2	3	4
0	1	2	3	4	5
1	6	7	8	9	10
2	11	12	13	14	15
3	16	17	18	19	20

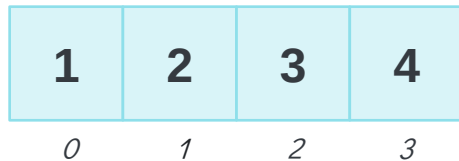
EXPLANATION

Types of *Arrays*

Arrays are categorized based on their structure into **one-dimensional arrays**, where *elements are arranged in a linear sequence*, and **multi-dimensional arrays**, which consist of *arrays of arrays* and allow the creation of tables or matrices; they are also classified by datatype into **primitive type arrays**, which *store primitive data types* like int and char, and **object type arrays**, which store instances of classes.

Primitive-type Array

```
int[] nums = {1, 2, 3, 4};
```



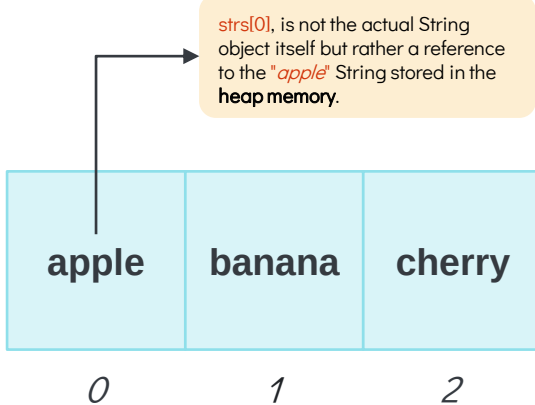
EXPLANATION

Types of *Arrays*

Arrays are categorized based on their structure into **one-dimensional arrays**, where elements are arranged in a linear sequence, and **multi-dimensional arrays**, which consist of arrays of arrays and allow the creation of tables or matrices; they are also classified by datatype into **primitive type arrays**, which store primitive data types like int and char, and **object type arrays**, which store instances of classes.

Object-type Array

```
String[] strs = {"apple", "banana", "cherry"};
```



QUESTION

Why does the
Java **array**
index start
with **0**? 

Explanation



From: The
Animated Java Interview Questions Course on Udemy © Peerless Geek

EXPLANATION

Java Array Indexing: *Why 0?*

Java arrays start at index 0 primarily because Java adopted this convention from the C programming language, where it originated due to how arrays relate to memory addressing and pointer arithmetic; this zero-based indexing makes mathematical operations more efficient, simplifies range calculations, and has become the standard across most programming languages because *the index effectively represents the offset from the beginning of the array in memory.*

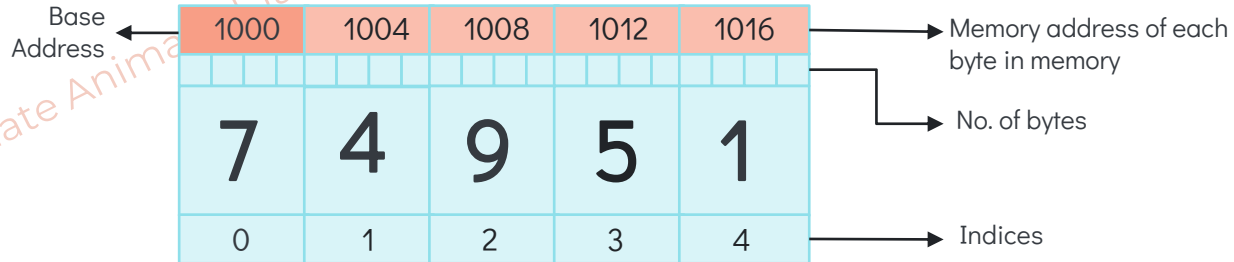
From: The Ultimate Animated Java Interview Questions Course on Udemy © Peerless Geek

EXPLANATION

Java Array Indexing: *Why 0?*

Java arrays start at index 0 primarily because Java adopted this convention from the C programming language, where it originated due to how arrays relate to memory addressing and pointer arithmetic; this zero-based indexing makes mathematical operations more efficient, simplifies range calculations, and has become the standard across most programming languages because *the index effectively represents the offset from the beginning of the array in memory.*

Address of an element at index i: $\text{Base Address} + (\text{index} \times \text{num_of_bytes})$



QUESTION

What is the
difference
between `int array[]`
and `int[] array`? 

Explanation



EXPLANATION

Java Array Declarations: *array[]* & *int[] array*

In Java, both `int array[]` and `int[] array` are *functionally identical* declarations for integer arrays, with the difference being purely syntactic—the former follows **C/C++ style** with brackets after the variable name, while the latter is the preferred **Java style** that places brackets with the type to emphasize that the variable's type is an “*array of Type*” rather than just a single element.

EXPLANATION

Java Array Declarations: *array[]* & *int[] array*

In Java, both `int array[]` and `int[] array` are *functionally identical* declarations for integer arrays, with the difference being purely syntactic—the former follows **C/C++ style** with brackets after the variable name, while the latter is the preferred **Java style** that places brackets with the type to emphasize that the variable's type is an “*array of Type*” rather than just a single element.

C-style syntax

```
int nums[] = {1, 2, 3, 4, 5};
```

Preferred Java style ✓

```
int[] nums = {1, 2, 3, 4, 5};
```

QUESTION

How to **copy**
(**clone**) an array
in Java? ✎

Explanation



From: The Complete Java Interview Questions Course on Udemy © Peerless Geek

EXPLANATION

Java *Array Cloning*: A Comparison of Methods

In Java, you can copy an array using multiple approaches: the **manual loop**, where you iterate through the array and copy elements one by one; the **clone()** method, which creates a shallow copy for one-dimensional arrays; **Arrays.copyOf()**, which copies the array with the option to resize; **Arrays.copyOfRange()**, which allows copying a specific portion of the array; and **System.arraycopy()**, a highly efficient native method for copying elements from one array to another.

From: The Ultimate Animated Java Interview Questions Course © Peerless Geek

EXPLANATION

Java *Array Cloning*: A Comparison of Methods

Approach	Description
Manual Loop	This method manually iterates over each element of sourceArray and copies it to the corresponding position in destinationArray.
<code>clone()</code>	This method is defined in the Object class, which array objects inherit. It creates a new array of the same length as the original and copies its contents into the new array.
<code>Arrays.copyOf()</code>	This method is defined in the Arrays utility class and is used to create a new array with a specific length and copies the contents of the array to the new array.
<code>Arrays.copyOfRange()</code>	This method is very similar to the <code>Arrays.copyOf()</code> method, but it also allows us to specify the range of elements to copy from the original array.
<code>System.arraycopy()</code>	This method is defined in the System utility class and allows you to specify the source array, the starting position in the source array, the destination array, the starting position in the destination array, and the number of elements to copy. It does not return anything but performs the copying operation in-place on the destination array.

QUESTION

What do you understand by the **Jagged array**? 

Explanation



From: The Java Interview Questions Course on Udemy © Peerless Geek

EXPLANATION

Jagged Array

A **jagged array** is a data structure that functions as an “*array of arrays*,” **where each sub-array can have a different length**, allowing efficient memory utilization by matching row lengths to data requirements, unlike a regular multi-dimensional array with uniform row sizes.

From: The Ultimate Animated Java Interview Questions Course © Peerless Geek

EXPLANATION

Jagged Array

A **jagged array** is a data structure that functions as an “array of arrays,” *where each sub-array can have a different length*, allowing efficient memory utilization by matching row lengths to data requirements, unlike a regular multi-dimensional array with uniform row sizes.

```
int[][] jaggedArray = {  
    {1, 2, 3},  
    {4, 5},  
    {6, 7, 8, 9, 10},  
    {11, 12, 13}  
};
```

	0	1	2	3	4
0	1	2	3		
1	4	5			
2	6	7	8	9	10
3	11	12	13	14	

QUESTION

What are the
advantages and
disadvantages of
an array? 

Explanation



From: The Java Interview Questions Course on Udemy © Peerless Geek

EXPLANATION

<i>Advantages</i> of Arrays	<i>Disadvantages</i> of Arrays
Arrays have a fixed size, ensuring efficient memory allocation.	Arrays have a fixed size that cannot be changed, leading to potential memory wastage or resizing overhead.
They can store primitive data types like int, char, and double without using wrapper classes.	They lack built-in methods like add() , remove() , or contains() , making manipulation cumbersome.
They offer fast $O(1)$ access to elements using indices.	They can lead to memory wastage if the allocated size is not fully utilized.
They allow easy modification using direct indexing.	They only support homogeneous data, meaning all elements must be of the same type.
They support multi-dimensional arrays, useful for matrices and complex computations.	They are not thread-safe, requiring manual synchronization for concurrent modifications.
They are lightweight and simple, making them efficient for scenarios where the size is known in advance.	They have limited flexibility, as resizing or dynamically growing the array requires creating a new one and copying elements.

oop (Basics)

07

QUESTION

What is an object-oriented paradigm?



Explanation



From: The Complete Java Interview Questions Course on Udemy © Peerless Geek

EXPLANATION

Object-Oriented Paradigm in Java (Intro)

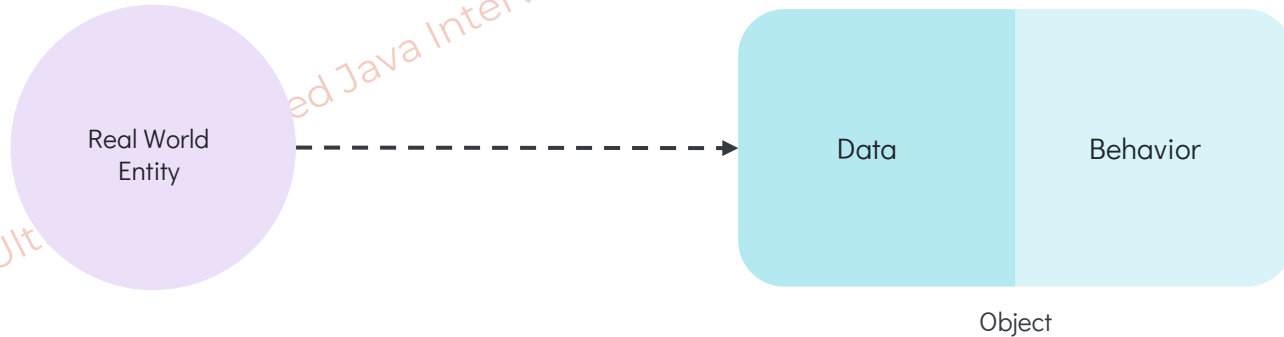
The **Object-Oriented Paradigm** (OOP) is an approach to designing and solving problems by representing real-world entities as objects, which bundle data and behavior together, enhancing code modularity, reusability, and scalability through four key principles: ***Encapsulation, Inheritance, Polymorphism,*** and ***Abstraction,*** as exemplified by a Car object that might have attributes like brand, model, and color, along with methods such as `start()` and `stop()`.

From: The Ultimate Animated Java Interview Questions Course on Udemy © Peerless Geek

EXPLANATION

Object-Oriented Paradigm in Java (Intro)

The **Object-Oriented Paradigm** (OOP) is an approach to designing and solving problems by representing real-world entities as objects, which bundle data and behavior together, enhancing code modularity, reusability, and scalability through four key principles: ***Encapsulation***, ***Inheritance***, ***Polymorphism***, and ***Abstraction***, as exemplified by a Car object that might have attributes like brand, model, and color, along with methods such as `start()` and `stop()`.



EXPLANATION

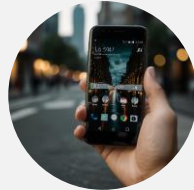
Object-Oriented Paradigm in Java (Intro)

The **Object-Oriented Paradigm** (OOP) is an approach to designing and solving problems by representing real-world entities as objects, which bundle data and behavior together, enhancing code modularity, reusability, and scalability through four key principles: ***Encapsulation***, ***Inheritance***, ***Polymorphism***, and ***Abstraction***, as exemplified by a Car object that might have attributes like brand, model, and color, along with methods such as `start()` and `stop()`.

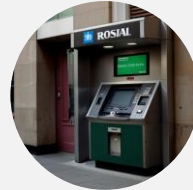
Examples



Person: A person has attributes like **name, age, gender, and address** and methods like `walk()`, `talk()`, `eat()`, and `sleep()`.



Smartphone : A smartphone has attributes like **brand, model, and battery level** and methods like `makeCall()`, `sendMessage()`, and `takePhoto()`.



ATM Machine : An ATM has attributes like **location and cash balance** and methods like `dispenseCash()` and `checkBalance()`. A customer has attributes like **name and account number** and methods like `withdraw()` and `deposit()`.

Objects (*Real-World Entities in Code*)

QUESTION

What are the main concepts of OOPs in Java?

Explanation



From: The Java Interview Questions Course on Udemy © Peerless Geek

EXPLANATION

OOP in Java: *Core Concepts*

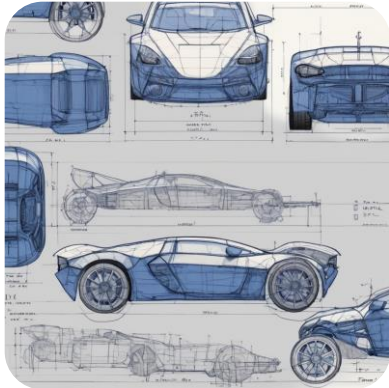
The main concepts of Object-Oriented Programming (*OOP*) in Java (*and in general*) are: ***Class*** and ***Object***, ***Encapsulation***, ***Inheritance***, ***Polymorphism***, and ***Abstraction***.

From: The Ultimate Animated Java Interview Questions Course on Udemy © Peerless Geek

EXPLANATION

Class & Object

A **class** is the practice of defining a blueprint that specifies properties and behaviors, while an **object** is an instance of a class that holds actual data and performs the defined behaviors.



Class (e.g., Car Blueprint or Template)



c1



c2



c3

Objects (Real-World Entities, e.g., Actual Car)

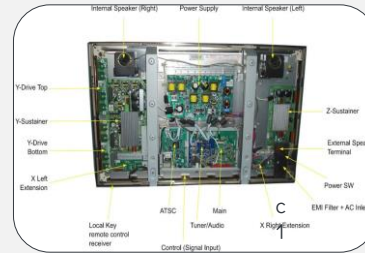
EXPLANATION

Encapsulation

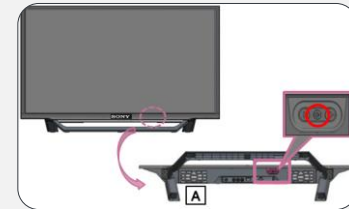
Encapsulation is the practice of bundling (*or hiding*) the internal details of an object and restricting access to certain components, exposing only necessary parts through public methods.



Television



Hidden component

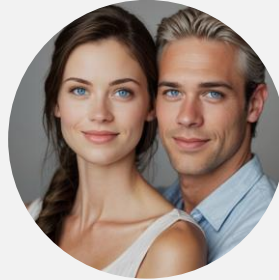


Exposed component (*volume control, channel changing*)

EXPLANATION

Inheritance

Inheritance is the practice of allowing a class (*child*) to acquire the properties and behaviors of another class (*parent*), promoting code reuse and establishing a hierarchical relationship.



Parent



The child inherits traits like blue eyes and facial features from the parents, demonstrating genetic resemblance.

EXPLANATION

Polymorphism

Polymorphism is the practice of enabling objects of different classes to respond differently to the same method call, based on their specific implementation.



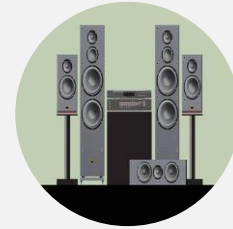
Universal Remote



Television



Air Conditioner



Home Theatre

EXPLANATION

Abstraction

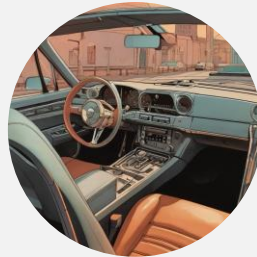
Abstraction is the practice of hiding complex implementation details and exposing only the essential features or functionalities to the user.



Remote



Mobile Phone

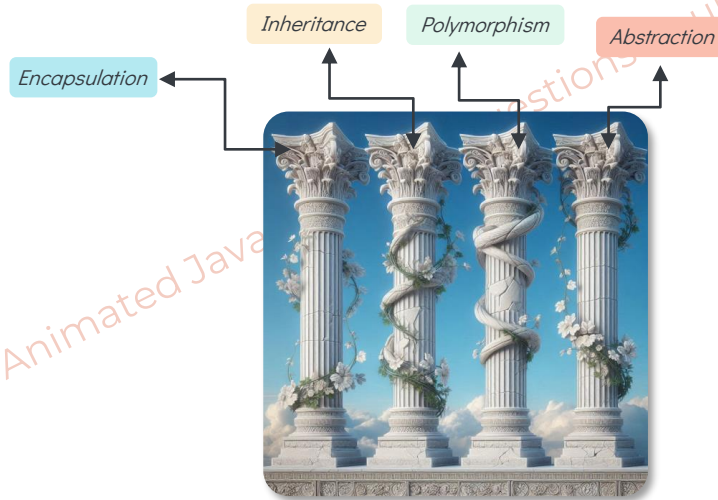


Car Dashboard

EXPLANATION

OOP in Java: *Core Concepts*

The main concepts of Object-Oriented Programming (OOP) in Java (*and in general*) are: ***Class*** and ***Object***, ***Encapsulation***, ***Inheritance***, ***Polymorphism***, and ***Abstraction***.



Four pillars of OOPs : ***Encapsulation***,
Inheritance, ***Polymorphism***, and ***Abstraction***.

QUESTION

What is the **difference** between an **object-oriented** programming language and an **object-based** programming language? 

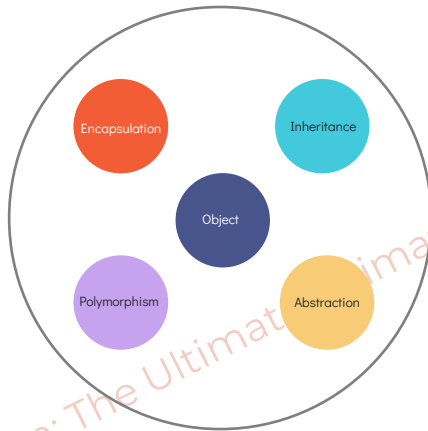
Explanation



EXPLANATION

Object Oriented Vs Object Based

An **object-oriented programming (OOP) language** supports all four OOP principles—**encapsulation**, **abstraction**, **inheritance**, and **polymorphism**—making it powerful in terms of reusability and flexibility, with examples including Java, C++, Python, and C#.



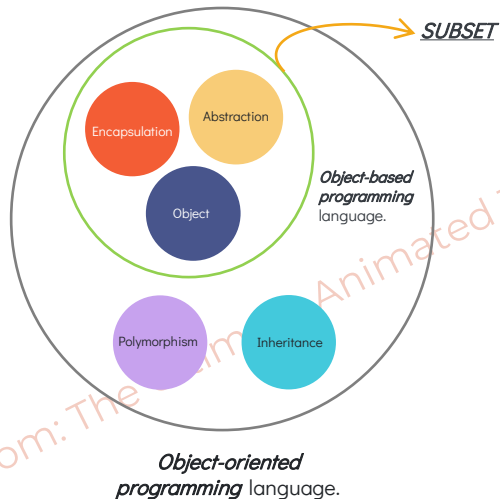
*Object-oriented
programming language.*



EXPLANATION

Object Oriented Vs Object Based

An **object-based programming language**, considered a subset of OOP languages, only *supports encapsulation and abstraction* but lacks inheritance and polymorphism, making it less versatile, with examples such as JavaScript (*ES5 and earlier*), VBScript, and early versions of ActionScript.



Classes & Objects

08

QUESTION

What are classes in Java?

Explanation



From: The Complete Java Interview Questions Course on Udemy © Peerless Geek

EXPLANATION

Class

A **class** is a *blueprint* or *template* for creating objects, defining the properties (*fields/attributes*) and behaviors (*methods*) that the objects of that class will have, much like a blueprint for a house that describes what the house will look like and what features it will have, but is not the actual house itself.

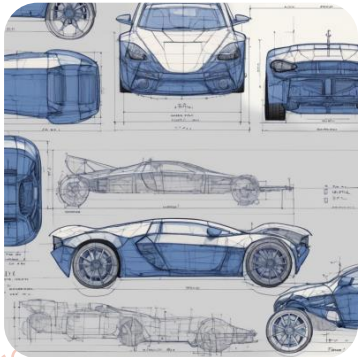
From: The Ultimate Animated Java Interview Questions Course on Udemy © Peerless Geek

EXPLANATION

Class

A **class** is a *blueprint* or *template* for creating objects, defining the properties (*fields/attributes*) and behaviors (*methods*) that the objects of that class will have, much like a blueprint for a house that describes what the house will look like and what features it will have, but is not the actual house itself.

Examples (In real-life)



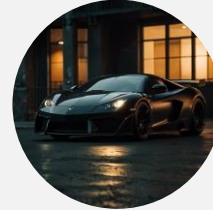
Class (e.g., Car Blueprint or Template)



c1



c2



c3

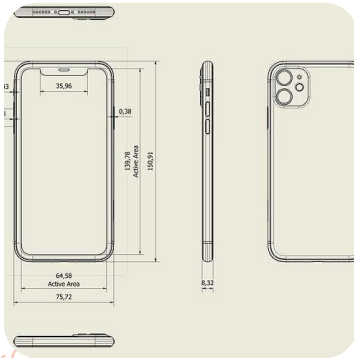
Objects (Real-World Entities, e.g., Actual Car)

EXPLANATION

Class

A **class** is a *blueprint* or *template* for creating objects, defining the properties (*fields/attributes*) and behaviors (*methods*) that the objects of that class will have, much like a blueprint for a house that describes what the house will look like and what features it will have, but is not the actual house itself.

Examples (In real-life)



Class (e.g., iPhone Blueprint or Template)



p1



p2



p3

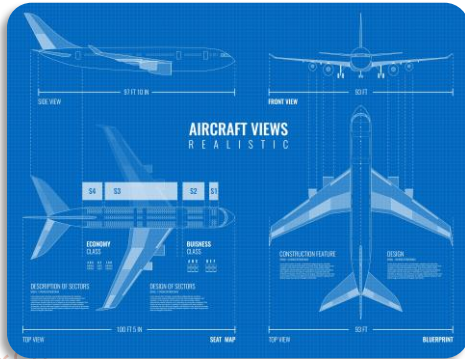
Objects (Real-World Entities, e.g., Actual iPhone)

EXPLANATION

Class

A **class** is a *blueprint* or *template* for creating objects, defining the properties (*fields/attributes*) and behaviors (*methods*) that the objects of that class will have, much like a blueprint for a house that describes what the house will look like and what features it will have, but is not the actual house itself.

Examples (In real-life)



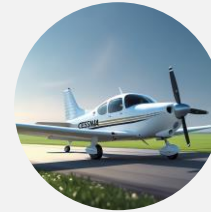
Class (e.g., Airplane Blueprint or Template)



a1



a2



a3

Objects (Real-World Entities, e.g., Actual Airplane)

EXPLANATION

Class

A **class** is a *blueprint* or *template* for creating objects, defining the properties (*fields/attributes*) and behaviors (*methods*) that the objects of that class will have, much like a blueprint for a house that describes what the house will look like and what features it will have, but is not the actual house itself.

- It is defined using the **class** keyword.
- It provides a way to model real-world entities and encapsulate their data (attributes) and behavior (methods) in a structured and reusable manner.
- Its name must follow strict rules: it can contain letters, digits (*but not start with them*), underscores, and dollar signs; it cannot use reserved keywords, must match the filename if public, and should follow **PascalCase** naming conventions for readability.
- It can be declared with only public or default access modifiers at the top level, whereas inner classes can use public, protected, default, or private access modifiers.
- It can inherit properties (*fields/variables*) and behaviors (*methods*) from other classes using the **extends** keyword.
- It can implement multiple interfaces using the **implements** keyword.
- It can be restricted from being extended by other classes using the **final** keyword.

QUESTION

What is an Object?



Explanation



From: The Java Questions Course on Udemy © Peerless Geek

EXPLANATION

Object

An **object** is a concrete instance of a class, serving as a fundamental unit of Object-Oriented Programming (*OOP*) that models real-world entities by encapsulating state (*attributes*), represented by ***instance variables*** that store data specific to the object, and behavior (*methods*), defined by ***functions*** that operate on the object's data.

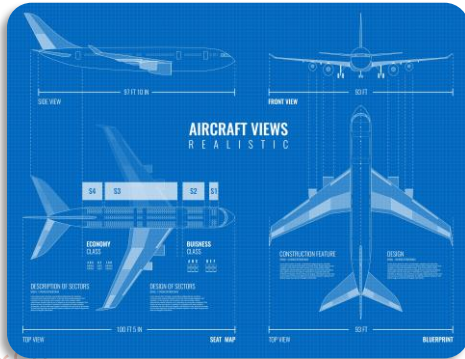
From: The Ultimate Animated Java Interview Questions Course on Udemy © Peerless Geek

EXPLANATION

Object

An **object** is a concrete instance of a class, serving as a fundamental unit of Object-Oriented Programming (*OOP*) that models real-world entities by encapsulating state (*attributes*), represented by **instance variables** that store data specific to the object, and behavior (*methods*), defined by **functions** that operate on the object's data.

Example (In real-life)



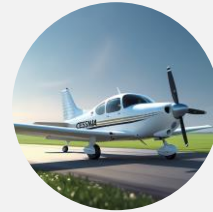
Class (e.g., Airplane Blueprint or Template)



a1



a2



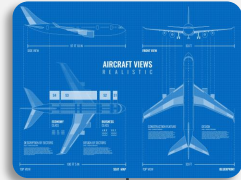
a3

Objects (Real-World Entities, e.g., Actual Airplane)

EXPLANATION

Object

An **object** is a concrete instance of a class, serving as a fundamental unit of Object-Oriented Programming (*OOP*) that models real-world entities by encapsulating state (*attributes*), represented by *instance variables* that store data specific to the object, and behavior (*methods*), defined by *functions* that operate on the object's data.



Class (Blueprint or template)



An instance (or object) of Airplane

```
public class Airplane {  
    }  
}
```

Class (Blueprint or template)

```
Airplane p1 = new Airplane();
```

An instance (or object) of Airplane

QUESTION

How is the **new** operator different from the **newInstance()** method in Java? 

Explanation



From: The Complete Java Interview Questions Course on Udemy © Peerless Geek

EXPLANATION

new operator	newInstance() method
new is an operator in Java.	newInstance() is a method of the Class class.
It is the most common and straightforward way to create objects in Java, used for creating instances (<i>objects</i>) of regular classes that are not loaded dynamically.	It is used to create a new instance (<i>object</i>) of a class dynamically at runtime, without knowing the exact class type at compile time—for example, when the class name is stored as a string or retrieved from an external source (<i>such as a command-line argument or a file</i>).
It performs compile-time checking, ensuring the class exists and the constructor parameters are valid.	It performs runtime checking, allowing the JVM to verify the class existence and the presence of a default constructor.
It directly invokes the class constructor, allowing you to specify which one to call by providing appropriate arguments.	It only calls the default (<i>no-argument</i>) constructor, throwing an InstantiationException if none exists.

Note :

The `newInstance()` method, traditionally found in the `Class` class as `Class.newInstance()`, has been deprecated since Java 9 in favor of `Constructor.newInstance()` from the `java.lang.reflect` package due to its improved exception handling and other advantages.

QUESTION

What are the
different ways to
create objects in
Java? 

Explanation



From: The
Animated Java Interview Questions Course on Udemy © Peerless Geek

EXPLANATION

Java Object Creation

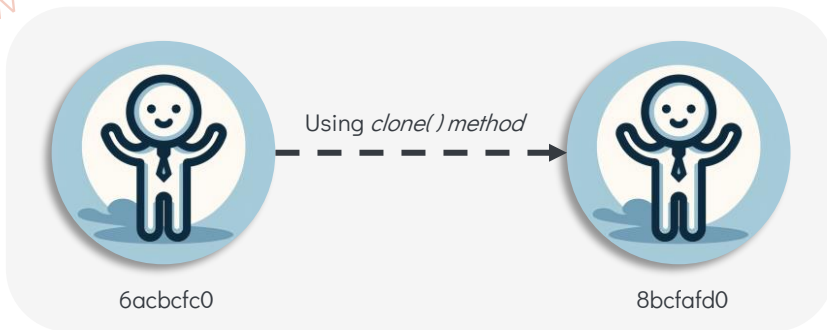
In Java, objects can be created using four main approaches: (1) ***new keyword***, which is the most common and standard way by invoking the constructor; (2) ***Class.newInstance()*** (*deprecated in Java 9+*), which allows dynamic instantiation; (3) ***clone() method***, which creates a copy of an existing object; and (4) ***deserialization***, which restores an object from a serialized stream.

EXPLANATION

Clone() method

The `clone()` method is used to create a shallow copy of an object; it is defined as `protected native Object clone() throws CloneNotSupportedException` in the `Object` class, inherited by all Java classes, and throws `CloneNotSupportedException` unless the class implements the `Cloneable` marker interface.

```
public class Object {  
    protected native Object clone()  
    throws CloneNotSupportedException;  
}
```



Clone of Person object

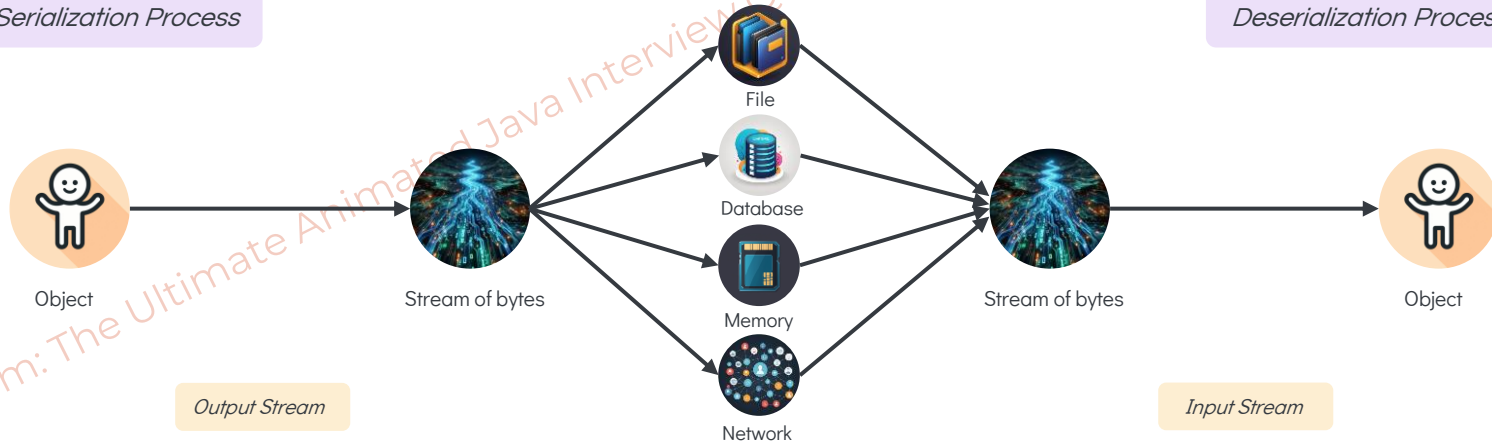
EXPLANATION

Serialization and Deserialization

Serialization is a process that *converts an object into a byte stream*, embedding its data and metadata for easy saving to a file, transmission over a network, or storage in a database, while **deserialization** *reconstructs the object from the stream*, restoring its original state; attempting to serialize or deserialize instances of a class that does not implement the Serializable interface will throw a **java.io.NotSerializableException**.

Serialization Process

Deserialization Process



EXPLANATION

Java Object Creation	
<i>Method</i>	<i>Description</i>
Using new keyword	The most common way to create an object by invoking a constructor using new .
Using Class.newInstance()	Allows dynamic object creation using reflection, but deprecated in Java 9+.
Using clone() method	Creates a shallow copy of an existing object by implementing the Cloneable interface.
Using deserialization	Restores an object from a byte stream using Java's serialization mechanism.

QUESTION

What is the use of **this** keyword in Java? 

Explanation



From: The Complete Java Interview Questions Course on Udemy © Peerless Geek

EXPLANATION

The *this* keyword

In Java, the *this* keyword refers to *the current instance of the class* and is used for four main purposes: (1) *referring to instance variables* when they are shadowed by local variables; (2) *invoking another constructor within the same class* using `this()` to avoid code duplication; (3) *returning the current instance from a method*, enabling method chaining; and (4) *passing the current instance as an argument to a method or constructor*.

From: The Ultimate Animated Java Interview Questions Course by Ujjwal Jaiswal © Peerless Geek

EXPLANATION

The *this* keyword

In Java, the *this* keyword refers to *the current instance of the class* and is used for four main purposes: (1) *referring to instance variables* when they are shadowed by local variables; (2) *invoking another constructor within the same class* using `this()` to avoid code duplication; (3) *returning the current instance from a method*, enabling method chaining; and (4) *passing the current instance as an argument to a method or constructor*.

```
public class Person {  
    private String name;  
    private int age;  
  
    public Person(String name, int age) {  
        this.name = name;  
        this.age = age;  
    }  
  
    public static void main(String[] args) {  
        Person person = new Person("Monica", 28);  
    }  
}
```



Current instance of Person object
(In heap memory)

Encapsulation

09

QUESTION

What is Encapsulation- tion?



Explanation



From: The Java Interview Questions Course on Udemy © Peerless Geek

EXPLANATION

Encapsulation

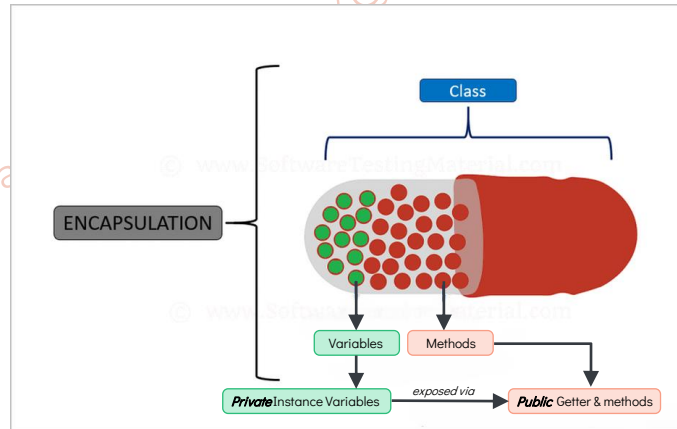
Encapsulation is one of the four core principles of Object-Oriented Programming (OOP) in Java, involving *the practice of bundling data (instance variables) and methods (instance functions) into a single unit (class) while restricting direct access to the data by making variables private and providing controlled access through public getter and setter methods*, thereby enabling data hiding, enforcing validation rules, and ensuring data integrity.

From: The Ultimate Animated Java Interview Questions Course by Peerless Geek

EXPLANATION

Encapsulation

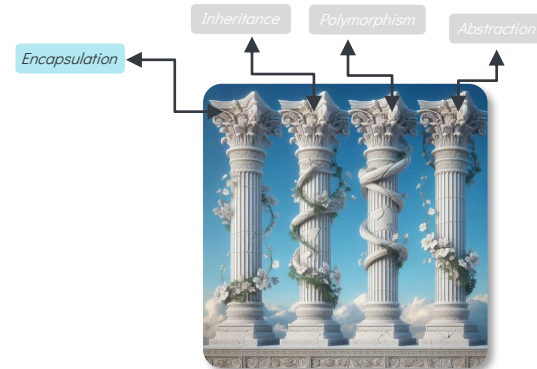
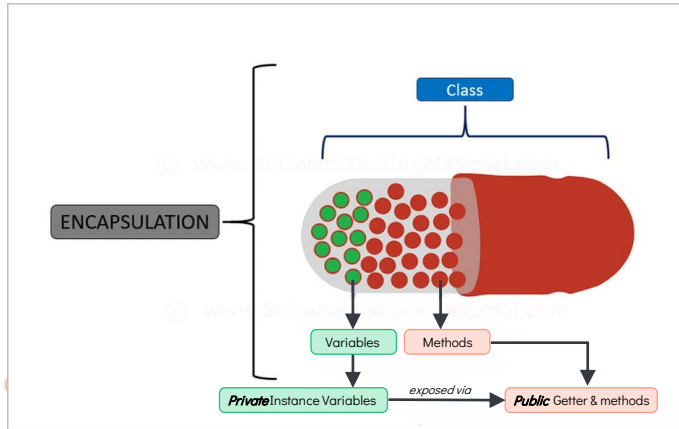
Encapsulation is one of the four core principles of Object-Oriented Programming (OOP) in Java, involving *the practice of bundling data (instance variables) and methods (instance functions) into a single unit (class) while restricting direct access to the data by making variables private and providing controlled access through public getter and setter methods*, thereby enabling data hiding, enforcing validation rules, and ensuring data integrity.



EXPLANATION

Encapsulation

Encapsulation is one of the four core principles of Object-Oriented Programming (OOP) in Java, involving *the practice of bundling data (instance variables) and methods (instance functions) into a single unit (class) while restricting direct access to the data by making variables private and providing controlled access through public getter and setter methods*, thereby enabling data hiding, enforcing validation rules, and ensuring data integrity.



Four pillars of OOPs : **Encapsulation**,
Inheritance, Polymorphism, and Abstraction.

Inheritance

10

QUESTION

Define

Inheritance.



Explanation



From: The Java Interview Questions Course on Udemy © Peerless Geek

EXPLANATION

Inheritance

- It is a fundamental concept in Object-Oriented Programming.
- The **extends** keyword is used to declare inheritance between two classes.
- It allows a class to inherit attributes and behaviors from another class.
- It helps avoid code duplication by allowing subclasses to reuse code from their superclass.
- It promotes the organization and structure of code in a hierarchical manner.

Encapsulation

Inheritance

Polymorphism

Abstraction



Four pillars of OOPs : *Encapsulation*, *Inheritance*, *Polymorphism*, and *Abstraction*.

QUESTION

Can the
constructor
be inherited in
Java? 

Explanation



From: The Ultimate Java Interview Questions Course on Udemy © Peerless Geek

EXPLANATION

Constructor Inheritance

A subclass does not inherit constructors from its superclass; each class must define its own constructor. Although constructor inheritance is not possible, a subclass can invoke a superclass constructor using the `super` keyword. This is often necessary to ensure that the superclass part of an object is properly initialized.

- *Constructors have the same name as the class.* If inherited, they would have a different name from the subclass, violating this rule.
- Constructors are meant to initialize the specific attributes of a class. A superclass constructor may not know how to properly initialize a subclass's unique attributes.
- If the superclass has a no-argument (default) constructor, it will be called implicitly by the subclass constructor if no explicit call to a superclass constructor is made.
- If the superclass doesn't have a no-argument constructor, the subclass must explicitly call one of the superclass constructors using `super()`.

Note :

The `super()` keyword in Java is not a method that is defined in a class; rather, it is a special construct provided by the Java language to facilitate calling the superclass's constructor. The actual mechanism of `super()` is handled by the Java Virtual Machine during object creation.

QUESTION

What are the different **types of inheritance** in Java? 

Explanation



EXPLANATION

Types of Inheritance

■ Single Inheritance

- A class (subclass or child class) can inherit from only one class (superclass or parent class).
- Java supports single inheritance to avoid the ambiguity that can arise from multiple inheritances.

■ Multi-level Inheritance

- A class inherits from another class, which in turn inherits from yet another class, forming a chain of inheritance.
- In multi-level inheritance, a subclass becomes the parent class of another class.

■ Hierarchical Inheritance

- Multiple classes can inherit from a single superclass.

■ Multiple Inheritance

- Java does not support multiple inheritance of classes to void the diamond problem, it supports multiple inheritance through interfaces.
- A class can implement multiple interfaces.

QUESTION

What is **multiple inheritance**? Is it supported by Java? 

Explanation

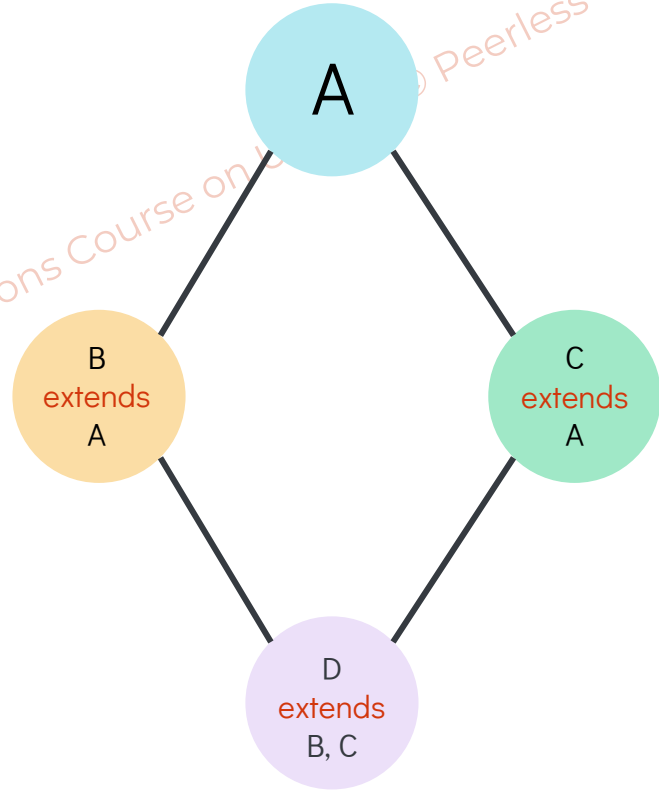


From: The Java Interview Questions Course on Udemy © Peerless Geek

EXPLANATION

Multiple Inheritance

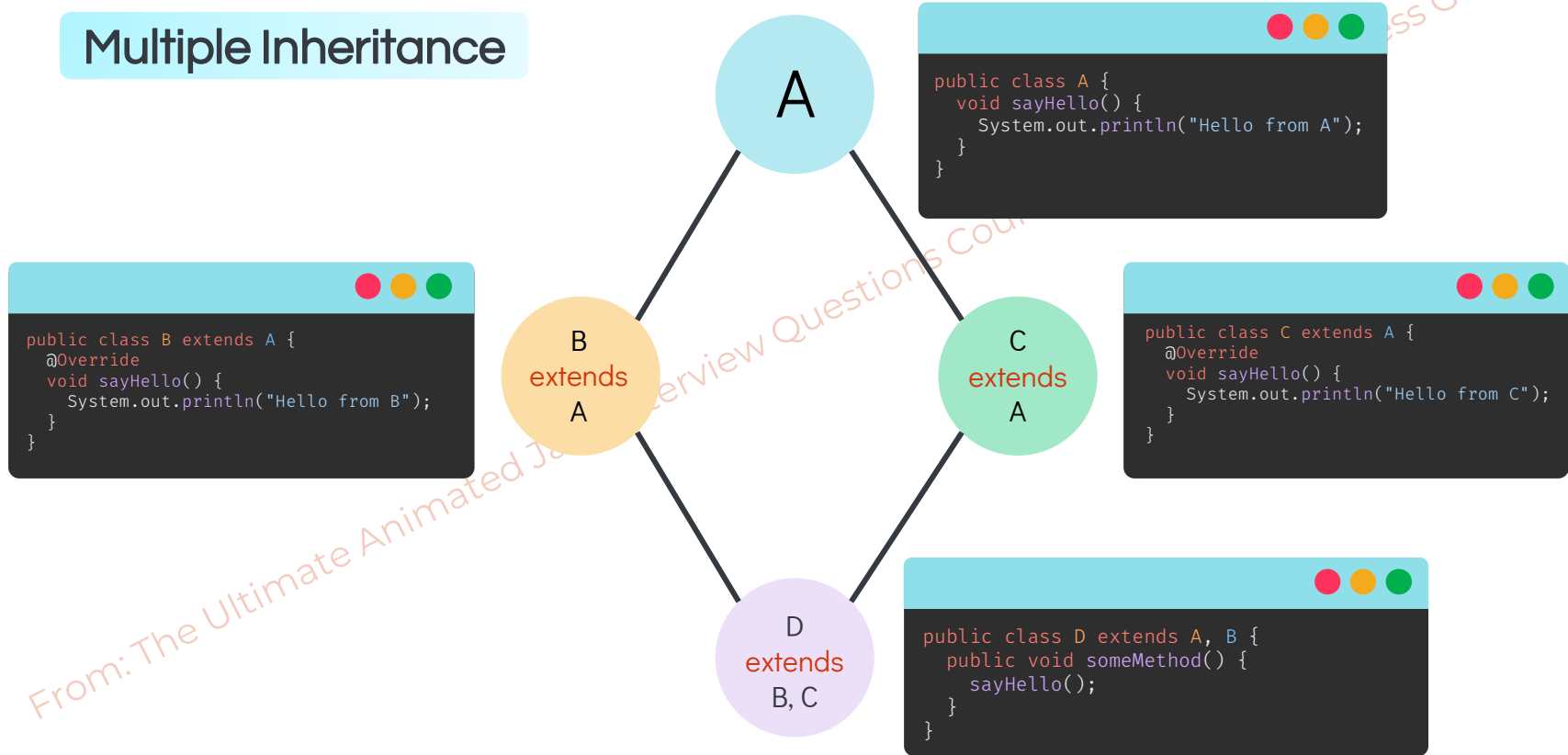
Multiple inheritance is a feature of some object-oriented programming languages where a class can inherit behaviors and properties from more than one parent class.



From: The Ultimate Animated Java Interview Questions Course on Udemy by Peerless Geek

EXPLANATION

Multiple Inheritance

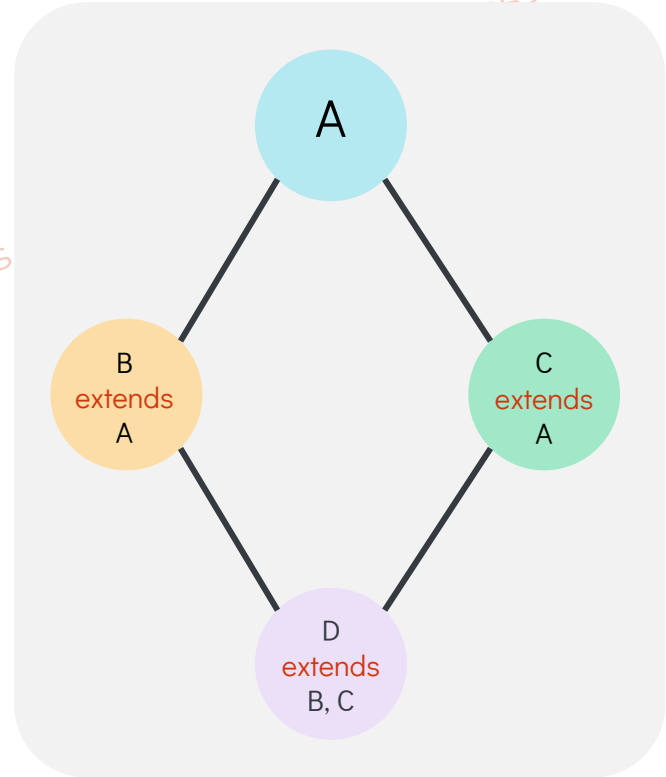


EXPLANATION

Multiple Inheritance

Multiple inheritance is a feature of some object-oriented programming languages where a class can inherit behaviors and properties from more than one parent class.

- It can lead to diamond problem where ambiguity arises if a class inherits from two classes that have a common ancestor.
- It is not supported in Java through class inheritance.
- It is supported through interfaces. A class can implement multiple interfaces, effectively inheriting method signatures from all of them.



QUESTION

How is
inheritance in
C++ different
from Java? 

Explanation



From: The C++ Interview Questions Course on Udemy © Peerless Geek

EXPLANATION

C++	Java
C++ does not enforce a common base class; classes can exist independently without a common ancestor.	Java has a single-rooted hierarchy where all classes implicitly inherit from the Object class.
It supports multiple inheritance but may lead to ambiguities like the diamond problem, which can be resolved using virtual inheritance.	It does not support multiple inheritance with classes but allows it through interfaces.
It uses a colon (:) followed by an access specifier (public, protected, private) for inheritance. Example: class Child : public Parent { }.	It uses the <i>extends</i> keyword for class inheritance and <i>implements</i> for interfaces. Example: class Child extends Parent implements Interface { }.
It uses abstract classes with pure virtual functions (= 0) to enforce derived classes to override methods.	It uses interfaces and abstract classes, where interfaces define method contracts for implementing classes.
It allows specifying inheritance access control (public, protected, private) in the class declaration. Example: class Derived : protected Base { }.	It uses public, protected, and private modifiers, but does not allow modifying inheritance access at the class level.
It requires the virtual keyword for runtime polymorphism. Example: virtual void display();	It treats all non-static methods as virtual by default, allowing runtime polymorphism without explicit keywords.

Polymorphism

11

QUESTION

What is Polymorphism?



Explanation



From: The Python Interview Questions Course on Udemy © Peerless Geek

EXPLANATION

Polymorphism

Polymorphism refers to the ability of a class to take on multiple forms, allowing objects to be treated as instances of their parent class rather than their actual class. Additionally, it also refers to the ability of a method to have many forms depending on different parameters.

There are two types of polymorphism :

Runtime polymorphism (*Method overriding*) :

- Method overriding occurs when a subclass provides a specific implementation of a method already defined in its superclass. The method in the subclass should have the same name, return type, and parameters as the method in the superclass.
- At runtime, the JVM determines which method to call based on the actual object's type, not the reference type.

Compile-time polymorphism (*Method overloading*) :

- Method overloading occurs when multiple methods in the same class have the same name but different parameters (different type, number, or both).
- At compile time, the JVM determines which method to call based on the method signature (name and parameter list).

QUESTION

What is method overriding?



Explanation



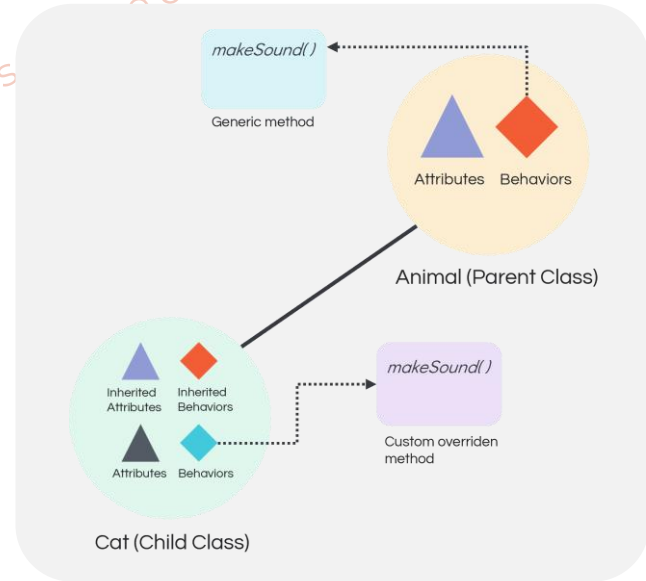
From: The Complete Interview Questions Course on Udemy © Peerless Geek

EXPLANATION

Method Overriding

Method overriding in Java is a feature that allows a subclass to provide a specific implementation for a method that is already defined in its superclass.

- It occurs in the context of inheritance, where a subclass inherits methods from its superclass.
- The overriding method in the subclass must have the same name, and parameters and the return type must be the same or a subtype of the return type in the superclass.
- The access level of the overriding method cannot be more restrictive than that of the overridden method.
- The overriding method can throw fewer or narrower exceptions, not new or broader ones.
- The `@Override` annotation is often used to indicate an overriding method, though it's not mandatory.



QUESTION

Can we **change the scope** of the overridden method in the subclass? ✎

Explanation



From: The Complete Java Interview Questions Course on Udemy © Peerless Geek

EXPLANATION

Scope of Overridden Method

In Java, when overriding a method, you cannot reduce the visibility (scope) of the method, but you can increase it.

- The access level (scope) of the overriding method in the subclass must be the same as or less restrictive than the access level of the overridden method in superclass.
 - If the overridden method in the superclass has **public** access, the overriding method in the subclass must also be **public**.
 - If the overridden method in the superclass has **protected** access, the overriding method in the subclass can have **protected** or **public** access.
 - If the overridden method in the superclass has **default** (package-private) access, the overriding method in the subclass can have **default**, **protected** or **public** access but not **private** access.

Note :

private methods in a class are not eligible for method overriding.

QUESTION

What is method overloading?



Explanation



From: The Complete Java Interview Questions Course on Udemy © Peerless Geek

EXPLANATION

Method Overloading

- It allows you to define multiple methods with the same name within a single class.
- It is distinguished by differences in the number, type, or order of parameters in the method signature.
- The return type of the methods is not considered for method overloading. Two methods with the same name, same parameter lists, but different return types will cause a compile-time error.
- Access modifiers (public, protected, private, etc.) have no impact on method overloading. Two methods with the same name and parameter lists but different access modifiers will cause a compile-time error.

```
class Calculator {  
    // Sum method  
    int sum(int x, int y) {  
        return x + y;  
    }  
  
    // Overloaded Sum method  
    int sum(int x, int y,  
    int z) {  
        return x + y + z;  
    }  
}
```

- Method overloading is a compile-time concept, meaning that the Java compiler resolves the overloaded method calls at compile time based on the method signatures.

Can we **override**
the overloaded
method? ✍️

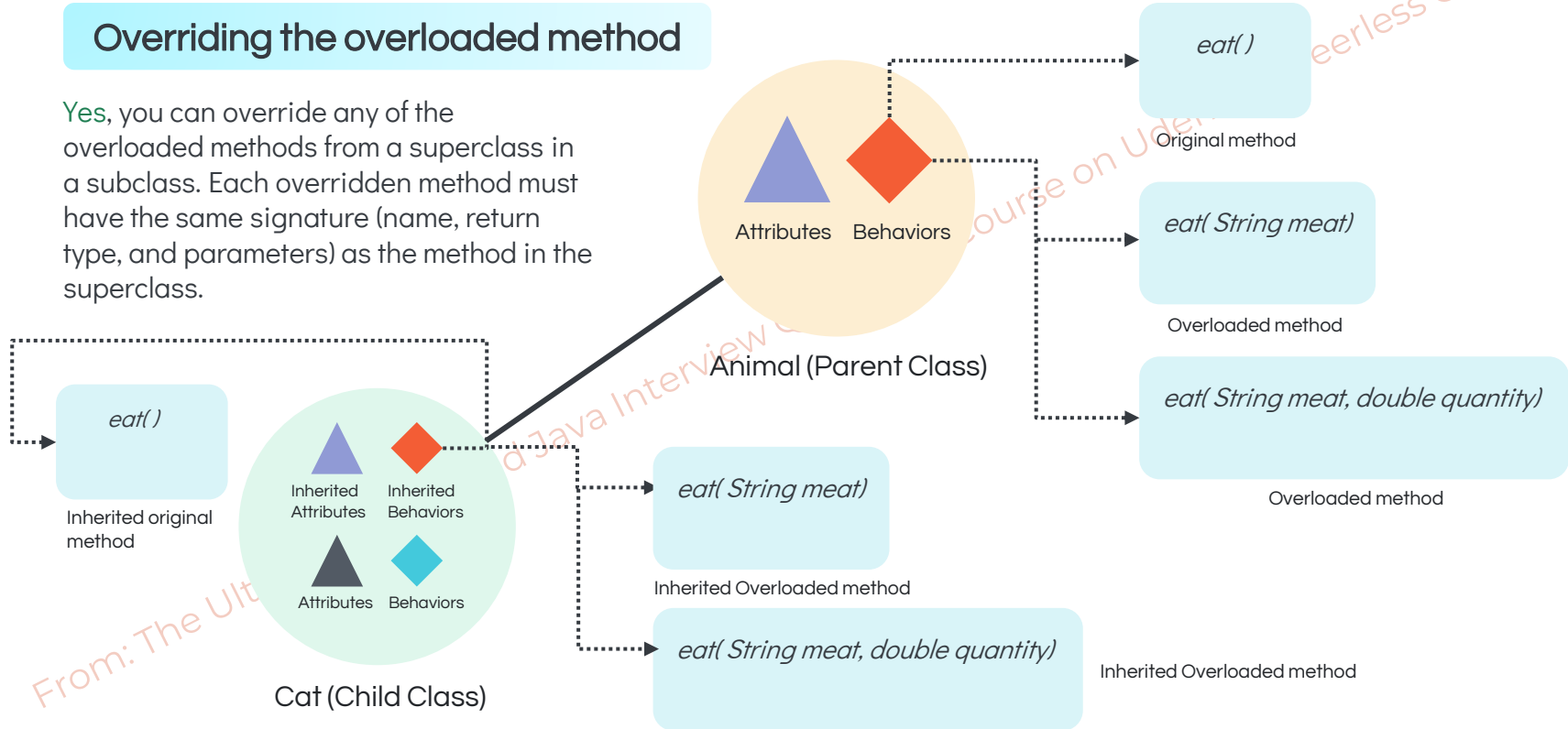
Explanation



EXPLANATION

Overriding the overloaded method

Yes, you can override any of the overloaded methods from a superclass in a subclass. Each overridden method must have the same signature (name, return type, and parameters) as the method in the superclass.



QUESTION

Can we
overload
the `main()`
method? 

Explanation



From: The Java Interview Questions Course on Udemy © Peerless Geek

EXPLANATION

Overloading the main method

Overloading the main method means you define multiple main methods with the same name but with different parameter lists which can be : **a.) Number of parameters** **b.) Type of parameters** **c.) Order of parameters**

- **Yes**, we can overload the main method, but it will not be recognized as a valid entry point and the program will not run.
- The JVM recognizes only the standard main method with the signature **public static void main(String[] args)** as the entry point for the application.
- To invoke the *OVERLOADED* main() method, we need to call it explicitly from within the *STANDARD* main() method which is not recommended.

```
public class Main {  
    // Standard `main()` method (Entry  
    point): The JVM will invoke this method  
    automatically.  
    public static void main(String[] args) {  
        System.out.println("Standard main  
        method");  
    }  
  
    // Overloaded main method.  
    public static void main(int num) {  
        System.out.println("Overloaded main  
        method with int parameter.");  
    }  
}
```


QUESTION

Can we **override**
the **private**
methods? 

Explanation



EXPLANATION

Overriding the private method

Private methods **cannot** be overridden. Overriding is a feature of inheritance, but private methods are not inherited.

- Private methods are not accessible outside the class in which they are defined. Therefore, they are not visible to any subclasses, even if the subclass is in the same package.
- If a subclass defines a method with the same name and signature as a private method in the superclass, it is treated as a new method in the subclass, not an overridden method.
- Attempting to add the **@Override** annotation to a method in the subclass that has the same name as a private method in the superclass will cause a compile-time error : "*Method does not override method from its superclass*".

```
class Superclass {  
    // Private method  
    private void sayHello() {  
        System.out.println("Hello from  
Superclass.");  
    }  
}
```

```
class Subclass extends Superclass {  
    // This method is treated as a new  
    method in the subclass.  
    private void sayHello() {  
        System.out.println("Hello from  
Subclass.");  
    }  
}
```

QUESTION

Can we **override**
the **static**
method? 

Explanation



EXPLANATION

Overriding the static method

Static methods are inherited by subclasses, allowing them to call static methods defined in their superclasses. However, static methods cannot be overridden in a subclass because they belong to the class itself, not to instances of the class.

- When a subclass defines a static method with the same signature as a static method in its superclass, it hides the superclass method. This behavior is known as **method hiding**, not overriding.
- The **@Override** annotation cannot be used with static methods. It will result in a compile-time error if used : *"Static methods cannot be annotated with @Override"*.
- Static method calls are resolved at compile-time, based on the type of the reference, not the actual object. This means the method call is determined by the class of the reference variable, not the class of the object it refers to.

```
class Superclass {  
    // Static method in superclass.  
    public static void sayHello() {  
        System.out.println("Hello from  
        Superclass.");  
    }  
}
```

```
class Subclass extends Superclass {  
    // This method hides the inherited  
    static method from its superclass.  
    public static void sayHello() {  
        System.out.println("Hello from  
        Subclass.");  
    }  
}
```

QUESTION

Can you have
virtual functions
in Java? 

Explanation



From: The Complete Java Interview Questions Course on Udemy © Peerless Geek

EXPLANATION

Virtual Function

A **virtual function** is a function or method whose behavior can be overridden within an inheriting class by a function with the same signature.

- The concept of virtual functions allows Java to implement polymorphism, enabling a method call to be resolved at runtime rather than compile time.
- In Java, all non-static methods are virtual by default, meaning they can be overridden in a subclass. This means that method overriding and dynamic dispatch are built into the language design. However, methods can be made non-virtual by using the final keyword, which prevents them from being overridden in any subclass.
- Overriding a final method will result in a compile-time error stating, "overridden method is final."

```
public class Superclass {  
    // Virtual Function (Non-final Method)  
    public void doSomethingImportant() {  
        System.out.println("This behavior should  
not be altered.");  
    }  
}
```

QUESTION

What is covariant return type?

Explanation



Animated Java Interview Questions Course on Udemy © Peerless Geek

From: The

EXPLANATION

Covariant return type

When a method in a subclass overrides a method in its superclass, the return type of the overridden method can be a subclass of the return type of the method in the superclass. This is known as a **covariant return type**.

- It allows subclasses to return more specific types, making it easier to work with subclass-specific methods and properties.
- It works only with reference types, not primitive types.
- It eliminates the necessity for explicit type casting, leading to cleaner and more readable code.

```
public class Animal {  
    public Animal reproduce() {  
        System.out.println("Reproducing  
Generic Animal...!");  
        return new Animal();  
    }  
}
```

```
public class Dog extends Animal {  
    @Override  
    public Dog reproduce() {  
        System.out.println("Dog is  
reproducing...!");  
        return new Dog();  
    }  
}
```

From: The Ultimate Animated Java Interview Questions Course by @niteshshukla96

nerless Geek

Association

12

QUESTION

What is an Association?



Explanation



EXPLANATION

Association

Association is a relationship between two separate classes that are established through their objects. It defines the way that objects of one class are connected to objects of another class.

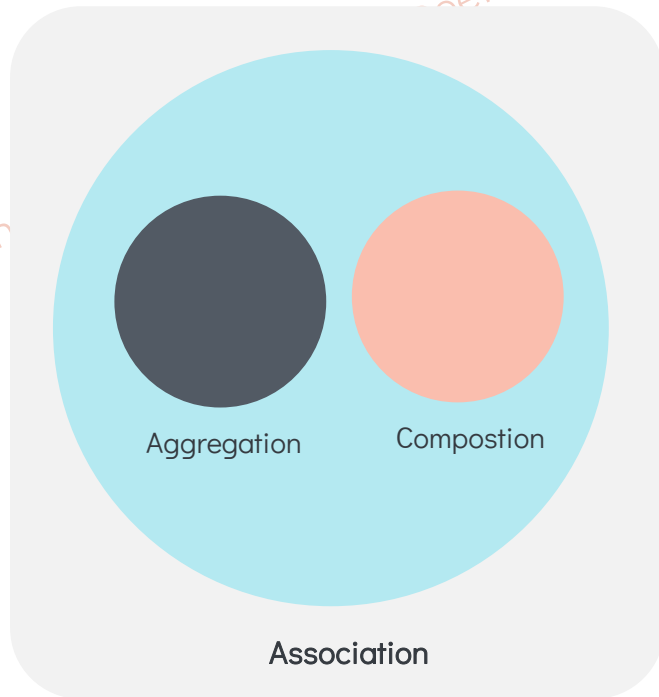
From: The Ultimate Animated Java Interview Questions Course on Udemy © Peerless Geek

EXPLANATION

Association

Association is a relationship between two separate classes that are established through their objects. It defines the way that objects of one class are connected to objects of another class.

- It is a broader term that includes both **composition** and **aggregation**.
- It represents the "uses-a" or "has-a" relationship where one class uses or has references to another class.
- It is often represented by instance variables, method parameters, that reference objects of another class.



From: The Ultimate Animated Java Interview Questions

nerless Geek

EXPLANATION

Types of Association

Associations in Java can be classified into different types based on their cardinality : *One-to-One Association*, *One-to-Many Association* and *Many-to-Many Association*.

- **One to One** : In a one-to-one association, each instance of one class is associated with exactly one instance of another class.
- **One to Many** : In a one-to-many association, one instance of a class (the "one" side) is associated with multiple instances of another class (the "many" side).
- **Many to Many** : In a many-to-many association, multiple instances of one class are associated with multiple instances of another class.

QUESTION

What do you
mean by
aggregation? 

Explanation



From: The Python Interview Questions Course on Udemy © Peerless Geek

EXPLANATION

Aggregation

Aggregation is a special form of association that represents a "has-a" relationship between two classes. It signifies that one class (*the container*) contains a reference to an object of another class (*the contained*).

- It is a weaker form of **has-a** (association) relationship.
- It is achieved by declaring an instance variable of one class inside another class.
- It is a unidirectional association, meaning the container knows about the contained object, but not vice versa.
- The lifecycle of the contained object does not depend on the lifecycle of the container object; if the container object is destroyed, it does not necessarily destroy the contained object

```
public class Book {  
    private String title;  
    private String author;  
  
    public Book(String title, String  
author) {  
        this.title = title;  
        this.author = author;  
    }  
}
```

```
public class Library {  
    private List<Book> books;  
  
    public Library(List<Book> books) {  
        this.books = books;  
    }  
}
```

QUESTION

What do you mean by Composition?



Explanation



From: The C++ Interview Questions Course on Udemy © Peerless Geek

EXPLANATION

Composition

Composition is a design principle used to build complex classes by combining objects of other classes. It allows a class to delegate some of its functionality to another class, thereby promoting code reuse and flexibility.

- It is a strong form of has-a (association) relationship.
- It is achieved by declaring an instance variable of one class inside another class.
- It promotes modularity by breaking down complex functionalities into simpler, reusable components.
- It is implemented in a unidirectional manner, similar to aggregation, but it can be bidirectional if the contained class needs to reference the container class.
- The lifecycle of the contained object is tied to the lifecycle of the container object; if the container object is destroyed, the contained object is also destroyed.

```
public class Engine {  
    public void start() {  
        System.out.println("Engine started");  
    }  
}
```

```
public class Car {  
    private Engine engine; // Composition  
  
    public Car() {  
        this.engine = new Engine();  
    }  
  
    public void startCar() {  
        engine.start();  
        System.out.println("Car started");  
    }  
}
```

QUESTION

Although **inheritance** is a popular OOP concept, it is **less advantageous** than **composition**. Explain? 

Explanation



EXPLANATION

The Limitations of Inheritance

In object-oriented programming (OOP), both inheritance and composition are techniques used to achieve code reuse, but they each have their own strengths and weaknesses.

- Inheritance follows an "is-a" relationship, while composition follows a "has-a" relationship.
- Inheritance creates tight coupling between base and derived classes, while composition promotes loose coupling between classes.
- Inheritance can lead to the "fragile base class problem," where changes in the base class can unexpectedly affect derived classes. Composition avoids this issue.
- Inheritance can lead to complex and deep hierarchies, while composition simplifies design with modular components.
- Inheritance, when used excessively, can make the code less readable and harder to understand. Developers may need to navigate through multiple levels of inheritance to understand the full behavior of a class.

Abstraction

13

QUESTION

What is an Interface?



Explanation



EXPLANATION

Interface

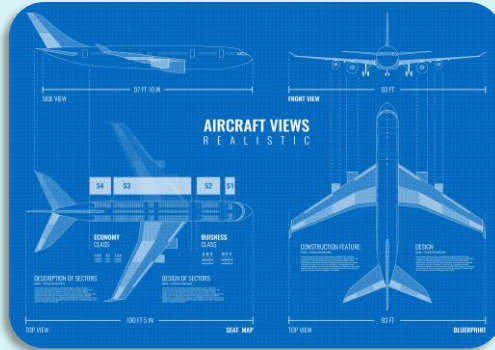
An **interface** is *a reference type that acts as a blueprint for creating classes*, defining a contract with methods that must be implemented without specifying their implementation.

From: The Ultimate Animated Java Interview Questions Course on Udemy © Peerless Geek

EXPLANATION

Interface

An **interface** is *a reference type that acts as a blueprint for creating classes*, defining a contract with methods that must be implemented without specifying their implementation.



Class (*Blueprint for the object*)

EXPLANATION

Interface

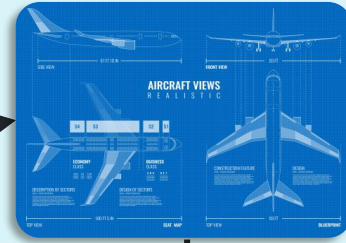
An **interface** is *a reference type that acts as a blueprint for creating classes*, defining a contract with methods that must be implemented without specifying their implementation.

We need an aircraft blueprint that can :

- i) Takeoff 
- ii) Fly 
- iii) Land 

...and perform all the essential functions an aircraft requires!

Interface (Blueprint for the Class)



Class (Blueprint for the object)



Object (Real-world entity)

EXPLANATION

Interface

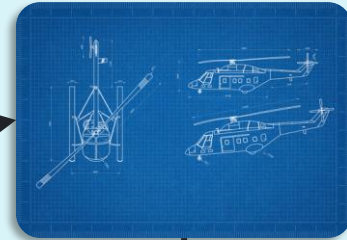
An **interface** is *a reference type that acts as a blueprint for creating classes*, defining a contract with methods that must be implemented without specifying their implementation.

We need an aircraft blueprint that can :

- i) Takeoff 
- ii) Fly 
- iii) Land 

...and perform all the essential functions an aircraft requires!

Interface (Blueprint for the Class)



Class (Blueprint for the object)



h1

h2

h3

Object (Real-world entity)

EXPLANATION

Interface

An **interface** is *a reference type that acts as a blueprint for creating classes*, defining a contract with methods that must be implemented without specifying their implementation.

- It is declared using the **interface** keyword.
- It is implemented by classes using the **implements** keyword.
- It can be extended by another interface using the **extends** keyword.
- It primarily contain **abstract methods**, which are methods without a body.
- Its declared methods are implicitly **public and abstract**, except for default and static methods.
- Its declared variables are implicitly **public, static, and final**, meaning they must be assigned a value and cannot be changed.
- It cannot have constructors because it cannot be instantiated directly.
- It can have **default** and **static methods** since *Java 8*, and **private** methods since *Java 9*.
- Its primary purpose is to achieve **Abstraction, Polymorphism and Multiple Inheritance**.

QUESTION

What is an Abstract Class?



Explanation



From: The Complete Java Interview Questions Course on Udemy © Peerless Geek

EXPLANATION

Abstract class

An **abstract class** is *a class that cannot be instantiated on its own and is meant to be subclassed, serving as a blueprint for other classes*, similar an interface or a *base class to share code among closely related classes* by defining common functionality once and inheriting it in subclasses.

From: The Ultimate Animated Java Interview Questions Course on Udemy © Peerless Geek

EXPLANATION

Abstract class

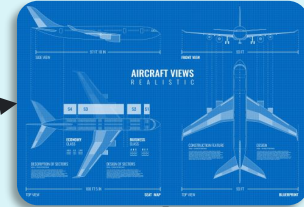
An **abstract class** is *a class that cannot be instantiated on its own and is meant to be subclassed, serving as a blueprint for other classes*, similar an interface or a *base class to share code among closely related classes* by defining common functionality once and inheriting it in subclasses.

We need an aircraft blueprint that can :

- i) Takeoff 
- ii) Fly 
- iii) Land 

...and perform all the essential functions an aircraft requires!

Interface (Blueprint for the Class)



Class (Blueprint for the object)



Object (Real-world entity)

EXPLANATION

Abstract class

An **abstract class** is *a class that cannot be instantiated on its own and is meant to be subclassed, serving as a blueprint for other classes*, similar an interface or a *base class to share code among closely related classes* by defining common functionality once and inheriting it in subclasses.

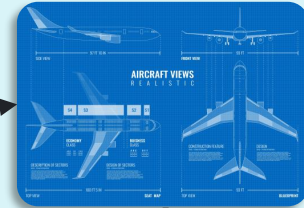
We need an aircraft blueprint that has:

- i) Model
- ii) Manufacturer
- iii) Maximum Passenger Capacity
- ... and all the essential properties an aircraft requires!

And that can :

- i) Takeoff 
- ii) Fly 
- iii) Land 
- ... and perform all the essential functions an aircraft requires!

Abstract Class (Detailed blueprint for the Class)



Class (Blueprint for the object)



a1

a2

a3

Object (Real-world entity)

EXPLANATION

Abstract class

An **abstract class** is *a class that cannot be instantiated on its own and is meant to be subclassed, serving as a blueprint for other classes*, similar an interface or a *base class to share code among closely related classes* by defining common functionality once and inheriting it in subclasses.

- It allows for partial implementation of a concept, with subclasses providing the rest.
- It is declared using the **abstract** keyword.
- It is extended by classes using the **extends** keyword.
- It can implement interfaces using the **implements** keyword and can either provide concrete implementations for their methods or leave them abstract.
- It can contain **abstract methods** that must be implemented by subclasses, and any subclass that doesn't implement all of them must also be declared abstract.
- It can also have **concrete (fully implemented) methods**.
- It can contain **instance, static and constant variables** just like regular classes.
- It can have **constructors** to initialize instance variables, which can be invoked using **super()** from a subclass, but it cannot be instantiated directly using the new keyword.
- Its constructor is called when an instance (*object*) of a subclass is created.
- It facilitates code reuse and provides support for **Abstraction** and **Polymorphism**.
- It for providing a common base for related classes without allowing direct instantiation.

QUESTION

What are the
differences between
an **interface** and an
abstract class? 

Explanation



From: The

Animated Java Interview Questions Course on Udemy © Peerless Geek

EXPLANATION

Interface	Abstract Class
Interface is a reference type that acts as a blueprint for creating classes, defining a contract with methods that must be implemented.	Abstract class cannot be instantiated on its own and is meant to be subclassed, serving as a blueprint for other classes.
It provides a method contract without implementation details.	It allows for partial implementation of a concept, with subclasses providing the rest.
It is declared using the interface keyword.	It is declared using the abstract keyword.
It is implemented by classes using the implements keyword.	It is extended by classes using the extends keyword.
It can extend another interface using the extends keyword.	It can extend another abstract class using the extends keyword and implement interfaces using the implements keyword.
It primarily contains abstract methods (<i>without a body</i>), but since Java 8 it can also include default and static methods, and since Java 9, private methods as well.	It can contain both abstract methods and concrete (<i>fully implemented</i>) methods.
Its declared variables are implicitly public, static, and final.	It can contain instance, static, and constant variables like regular classes.
It cannot have constructors.	It can have constructors to initialize instance variables.
It cannot be instantiated directly.	It cannot be instantiated directly, but its constructor is called when an instance of a subclass is created.
It supports multiple inheritance (<i>a class can implement multiple interfaces</i>).	It does not support multiple inheritance (<i>a class can extend only one abstract class</i>).
It is used to achieve Abstraction , Polymorphism , and Multiple Inheritance .	It facilitates code reuse and provides support for Abstraction and Polymorphism .

QUESTION

What is a marker interface?



Explanation



From: The Python Interview Questions Course on Udemy © Peerless Geek

EXPLANATION

Marker Interface

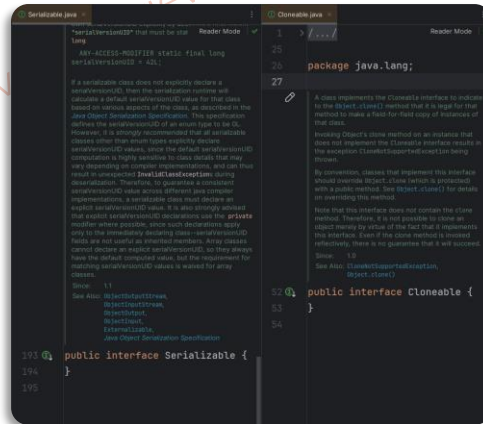
A **Marker Interface** is *an interface with no methods or fields*, used to **mark or tag** a class for specific behavior that the JVM or frameworks can recognize; for example, ***Serializable*** marks a class so its objects can be converted into a byte stream for storage or transmission, and ***Cloneable*** indicates that a class supports field-for-field copying via the `clone()` method—these interfaces act as metadata cues to the JVM, but since Java 5, annotations are often preferred due to their greater flexibility and ability to carry additional information.

From: The Ultimate Animated Java Interview Questions Course by Peerless Geek

EXPLANATION

Marker Interface

A **Marker Interface** is *an interface with no methods or fields*, used to mark or tag a class for specific behavior that the JVM or frameworks can recognize; for example, **Serializable** marks a class so its objects can be converted into a byte stream for storage or transmission, and **Cloneable** indicates that a class supports field-for-field copying via the `clone()` method—these interfaces act as metadata cues to the JVM, but since Java 5, annotations are often preferred due to their greater flexibility and ability to carry additional information.



Generics

14

QUESTION

What are
Generics, and how
do they **improve**
type safety? 

Explanation



EXPLANATION

Generics

Generics in Java empower developers to write *type-safe, reusable code* with *parameterized classes, interfaces, and methods*, enforcing compile-time type checking, eliminating explicit casting, and enhancing robustness by catching type-related errors at compilation, leading to more adaptable and reliable code.

From: The Ultimate Animated Java Interview Questions Course on Udemy © Peerless Geek

QUESTION

How do you create a generic class?



Explanation



From: The Complete Java Interview Questions Course on Udemy © Peerless Geek

EXPLANATION

Generic class

A **generic class** is a class that uses type parameters (*like T , E , K , V*) to maintain type safety at compile time, allowing it to work with different types, and is declared with angle brackets (< >) after the class name, while its members can utilize these type parameters in fields, methods, and constructors.

From: The Ultimate Animated Java Interview Questions Course on Udemy © Peerless Geek

EXPLANATION

Generic class

A **generic class** is a class that uses type parameters (*like T, E, K, V*) to maintain type safety at compile time, allowing it to work with different types, and is declared with angle brackets (<>) after the class name, while its members can utilize these type parameters in fields, methods, and constructors.



```
public class Box <T> {  
    }  
}
```

This is where we define *type parameters*.

are placeholders for the data types the class will handle.

Box<T> can represent various types, such as *Box<String>*, *Box<Integer>*, *Box<Person>*, *Box<Object>*, and more.

A generic class

From: The Ultimate

QUESTION

Can you use
primitives as
type arguments
in Generics? 

Explanation



EXPLANATION

Generics Restriction : No Primitives

No, you cannot use primitive types (*like int, double, char, etc.*) as type arguments in Java Generics. Generics only accept reference types, so you must use their corresponding wrapper classes (***Integer** instead of int, **Double** instead of double, **Character** instead of char, etc.*) when you want to use primitive types as type arguments.

From: The Ultimate Animated Java Interview Questions Course on Udemy © Peerless Geek

QUESTION

What is the difference between a **raw type** and a **parameterized type**? 

Explanation



From: The Java Interview Questions Course on Udemy © Peerless Geek

EXPLANATION

Understanding Raw Types vs Parameterized Types

A **raw type** is a generic class or interface *used without specifying a type parameter*, originating from *pre-JDK 1.5*, which lacks type safety, allows adding objects of any type, and provides no compile-time type checking, leading to potential **ClassCastException** at runtime.

From: The Ultimate Animated Java Interview Questions Course on Udemy © Peerless Geek

EXPLANATION

Understanding Raw Types vs Parameterized Types

A **raw type** is a generic class or interface *used without specifying a type parameter*, originating from *pre-JDK 1.5*, which lacks type safety, allows adding objects of any type, and provides no compile-time type checking, leading to potential `ClassCastException` at runtime. In contrast, a **parameterized type**, introduced in *Java 1.5+*, *specifies a type parameter when declaring or using a generic class* or interface, enforces type safety, enables compile-time type checking, ensures that only objects of the specified type can be added, and eliminates the need for explicit type casting when retrieving elements.



Raw container



Parameterised container

QUESTION

How do you create a generic method?

Explanation



EXPLANATION

Generic method

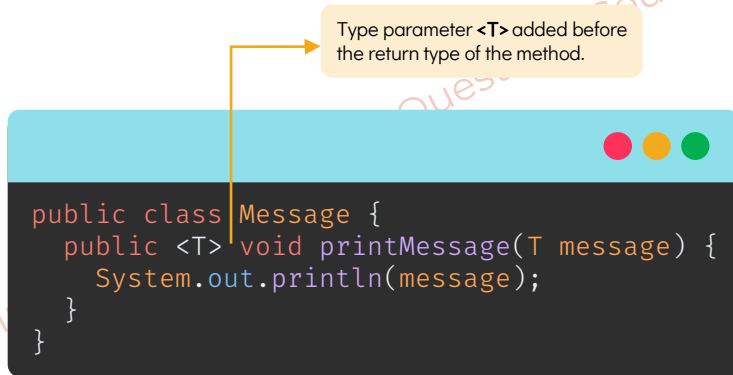
A **generic method** is a method that operates on various object types while ensuring type safety through a parameterized type specified before the return type, such as `<T>` (*or any other identifier*).

From: The Ultimate Animated Java Interview Questions Course on Udemy © Peerless Geek

EXPLANATION

Generic method

A **generic method** is a method that operates on various object types while ensuring type safety through a parameterized type specified before the return type, such as `<T>` (*or any other identifier*).



The diagram shows a code editor window with a dark background and a light blue header. The code defines a `Message` class with a generic method `printMessage`. An orange callout box with an arrow points to the `<T>` type parameter, explaining its purpose.

```
public class Message {  
    public <T> void printMessage(T message) {  
        System.out.println(message);  
    }  
}
```

Type parameter `<T>` added before the return type of the method.

A generic method (*printMessage*)

QUESTION

How do you create a **generic method** that returns a **type parameter**? 

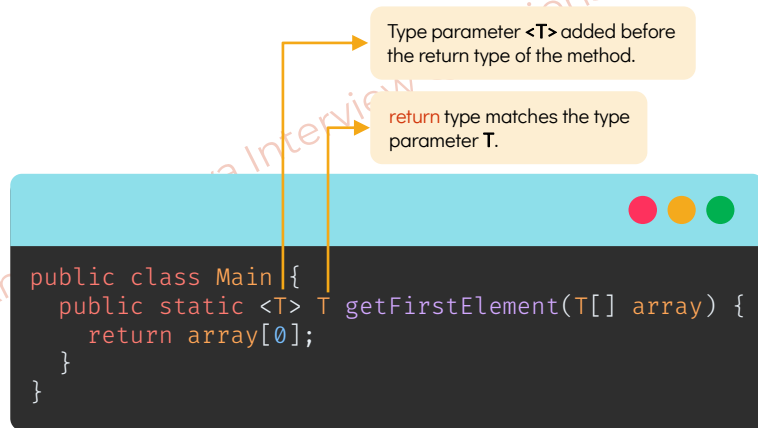
Explanation



EXPLANATION

Type Parameters as **return** values

To implement a generic method that *returns a type parameter* involves defining the type parameter `<T>` in the method signature before the return type and ensuring that the return type of the method matches the type parameter `T`.



```
public class Main {  
    public static <T> T getFirstElement(T[] array) {  
        return array[0];  
    }  
}
```

A generic method (*getFirstElement*)

QUESTION

What is **type inference** in Generics?

Explanation



From: The Complete Java Interview Questions Course on Udemy © Peerless Geek

EXPLANATION

Type Inference

Type inference is a powerful feature that allows the compiler to automatically determine the data type of a variable, method, or expression based on the context in which it is used, eliminating the need for explicit type declarations and making the code cleaner and more readable. For example, in generic methods or classes, the compiler infers the type parameter from the arguments or reference variable, reducing redundancy and improving code clarity.

From: The Ultimate Animated Java Interview Questions Course on Udemy © Peerless Geek

QUESTION

Can you
overload a
generic
method? 

Explanation



From: The Complete Java Interview Questions Course on Udemy © Peerless Geek

EXPLANATION

Java Generics : Understanding Method Overloading

Yes, you can overload a generic method by defining multiple methods with the same name but varying parameter lists, which can include *differences* in the **number**, **types**, or **order of parameters**.

From: The Ultimate Animated Java Interview Questions Course on Udemy © Peerless Geek

QUESTION

How do you create a generic constructor?

Explanation



EXPLANATION

Generic constructor

A **generic constructor** is a constructor that has its own type parameter(s) independent of the class's type parameters; it is defined by placing the type parameter(s) right before the constructor's name, enabling type parameters specific to the constructor that do not need to match those at the class level.

From: The Ultimate Animated Java Interview Questions Course on Udemy © Peerless Geek

EXPLANATION

Generic constructor

A **generic constructor** is a constructor that has its own type parameter(s) independent of the class's type parameters; it is defined by placing the type parameter(s) right before the constructor's name, enabling type parameters specific to the constructor that do not need to match those at the class level.

Type parameter **<U>** added before the constructor name.

```
public class Box {  
    private Object value;  
  
    public <U> Box(U value) {  
        this.value = value;  
    }  
  
    public Object getValue() {  
        return value;  
    }  
}
```

A generic constructor

QUESTION

Can a generic
class **extend**
another class? 

Explanation



EXPLANATION

Extending Classes with Generics

Yes, a generic class can extend both non-generic and generic classes, and it can also specify type parameters for the class it extends, allowing flexibility in defining and using type-safe relationships while reusing type parameters from inherited classes to maintain consistency in type definitions across hierarchies.

From: The Ultimate Animated Java Interview Questions Course on Udemy © Peerless Geek

QUESTION

How do you create a generic interface?

Explanation



EXPLANATION

Generic Interface

A **generic interface** is an interface defines type parameters (*e.g.*, T , E) in angle brackets (<>) after the interface name to ensure type safety at compile time, enabling it to operate with different data types and allowing its methods to use these parameters as placeholders.

From: The Ultimate Animated Java Interview Questions Course on Udemy © Peerless Geek

EXPLANATION

Generic Interface

A **generic interface** is an interface defines type parameters (*e.g.*, T , E) in angle brackets (<>) after the interface name to ensure type safety at compile time, enabling it to operate with different data types and allowing its methods to use these parameters as placeholders.

```
Public interface MyInterface<T> {  
    void print(T t);  
}
```

A generic interface

QUESTION

Can a generic class **implement** an interface? 

Explanation



EXPLANATION

Implementing Interfaces: Generic and Non-Generic Approaches

Yes, a generic class can implement an interface, whether the interface is generic or non-generic. If the interface is generic, the class must either specify a concrete type for the interface's type parameter, reuse the type parameter in its own generic declaration, or declare a new type parameter. A non-generic interface can be implemented without involving any type parameters.

From: The Ultimate Animated Java Interview Questions Course on Udemy © Peerless Geek

QUESTION

How do you create a generic enum?



Explanation



Animated Java Interview Questions Course on Udemy © Peerless Geek

From: The

EXPLANATION

Creating Generic-Like Behavior in Java Enums

You **CANNOT** directly create a generic enum in Java because the Java language does not allow enums to have type parameters. However, you can achieve similar functionality by using generics in a class or method that works with enums.

From: The Ultimate Animated Java Interview Questions Course on Udemy © Peerless Geek

EXPLANATION

Creating Generic-Like Behavior in Java Enums

You **CANNOT** directly create a generic enum in Java because the Java language does not allow enums to have type parameters. However, you can achieve similar functionality by using generics in a class or method that works with enums.

```
// Compile time error : Enum  
may not have type parameters.  
public enum Color<T> {  
    RED, GREEN, BLUE;  
}
```



```
// Define an enum named Color with  
three constants: RED, GREEN, and BLUE  
public enum Color {  
    RED, GREEN, BLUE;  
}  
  
// Generic class that works  
specifically with enums  
class EnumWrapper<T extends Enum<T>> {  
    // Add your implementation here, if  
    needed  
}
```



QUESTION

Can you use
multiple type
parameters in
Generics? 

Explanation



EXPLANATION

Multiple Type Parameters in Generics

Yes, Java Generics allows multiple type parameters, enabling the definition of classes, interfaces, or methods that operate on multiple types, with each type parameter specified within angle brackets (<>) and separated by commas.

From: The Ultimate Animated Java Interview Questions Course on Udemy © Peerless Geek

EXPLANATION

Multiple Type Parameters in Generics

Yes, Java Generics allows multiple type parameters, enabling the definition of classes, interfaces, or methods that operate on multiple types, with each type parameter specified within angle brackets (<>) and separated by commas.

```
// Generic Interface with Two  
Type Parameters  
public interface Pair<K, V> {  
    K getKey();  
    V getValue();  
}
```

Multiple type parameters specified within angle brackets (<>) and separated by commas.

QUESTION

What is type parameteriza tion?

Explanation



Animated Java Interview Questions Course on Udemy © Peerless Geek

From: The

EXPLANATION

Type Parameterization

Type parameterization is the process of introducing type parameters to *classes*, *interfaces*, or *methods*, allowing them to operate with various specified types instead of being limited to a single type, making them generic classes, interfaces, and methods, thereby enhancing reusability and ensuring type safety.

From: The Ultimate Animated Java Interview Questions Course on Udemy © Peerless Geek

EXPLANATION

Type Parameterization

Type parameterization is the process of introducing type parameters to *classes*, *interfaces*, or *methods*, allowing them to operate with various specified types instead of being limited to a single type, making them generic classes, interfaces, and methods, thereby enhancing reusability and ensuring type safety.

```
public class Box <T> {  
}
```

A generic class

```
public interface MyInterface<T> {  
    void print(T t);  
}
```

A generic interface

```
public class Message {  
    public <T> void printMessage(T message) {  
        System.out.println(message);  
    }  
}
```

A generic method (*printMessage*)

From: The Ultimate Android

Questions Course on Udemy © Peerless Geek

QUESTION

How do you
use **generics**
with **arrays**? 

Explanation



EXPLANATION

Generics with arrays

Arrays **do not support** generics directly because Java's generics are implemented using **type erasure**, which removes generic type information (e.g., `<T>`) at runtime, whereas arrays are **reified**, retaining their type information (e.g., `String[]`, `Integer[]`) at runtime. This fundamental difference means arrays enforce their type during execution, while generics do not, making `ArrayList<String>` and `ArrayList<Integer>` indistinguishable to the JVM and leading to potential type-safety issues when combining arrays and generics.

QUESTION

What are **wildcards** in Generics, and how do you use them? 

Explanation



EXPLANATION

Wildcards

Wildcards in Generics are special symbols in Java used to represent an unknown type in a generic type declaration, with the most common being the **question mark (?)**, which serves as a placeholder for an unspecified type. They provide flexibility when working with generic types, particularly in scenarios where the exact type is unknown or when operations need to be performed on a range of types.

From: The Ultimate Animated Java Interview Questions Course on Udemy © Peerless Geek

EXPLANATION

Wildcards

Wildcards in Generics are special symbols in Java used to represent an unknown type in a generic type declaration, with the most common being the **question mark (?)**, which serves as a **placeholder for an unspecified type**. They provide flexibility when working with generic types, particularly in scenarios where the exact type is unknown or when operations need to be performed on a range of types.

`List<?> list`

? -> is a **wildcard** (*unbounded*) that represents an unknown type, indicating that the list can hold references to a List of any type, such as `List<String>`, `List<Integer>`, or any other type.

From: The Ultimate Animated Java Interview Questions Course on Udemy © Peerless Geek

QUESTION

How do you use
upper bounded
wildcards? 

Explanation



EXPLANATION

Upper bounded wildcard

Upper-bounded wildcards are used in generics to restrict the type parameter to a specific **upper bound** (a *superclass or interface*), denoted by `<? extends T>`, *which represents an unknown type that is either a subclass of T or T itself*, providing read-only access to ensure type safety and is commonly used when you only need to read elements from a generic collection and do not need to modify it.

EXPLANATION

Upper bounded wildcard

Upper-bounded wildcards are used in generics to restrict the type parameter to a specific **upper bound** (a *superclass or interface*), denoted by `<? extends T>`, *which represents an unknown type that is either a subclass of T or T itself*, providing read-only access to ensure type safety and is commonly used when you only need to read elements from a generic collection and do not need to modify it.

`List<? extends Number> list`

`? extends Number` → is a **wildcard** (*upper bounded*) that represents an unknown type that is either a *subclass* of **Number**, such as **Integer**, **Double**, **Float**, etc., or **Number** itself.

Number → It is the upper bound, referring to the highest type that can be assigned to the wildcard.

QUESTION

How do you use
lower bounded
wildcards? 

Explanation



EXPLANATION

Lower bounded wildcard

Lower bounded wildcards in generics are used to restrict the type parameter to a specific **lower bound** (a superclass or interface) using `<? super T>`, *which represents an unknown type that is a superclass (or the same class) of T*, providing write access and allowing you to safely add elements to the collection and is commonly used when you need to write elements to a generic collection.

From: The Ultimate Animated Java Interview Questions Course on Udemy © Peerless Geek

EXPLANATION

Lower bounded wildcard

Lower bounded wildcards in generics are used to restrict the type parameter to a specific **lower bound** (a superclass or interface) using `<? super T>`, *which represents an unknown type that is a superclass (or the same class) of T*, providing write access and allowing you to safely add elements to the collection and is commonly used when you need to write elements to a generic collection.

`List<? super Integer> list`

`? super Integer` → is a **wildcard** (*lower bounded*) that represents an unknown type that is either a *superclass* of **Integer**, such as **Number**, **Object**, or **Integer** itself.

Integer → It is the lower bound, referring to the lowest type that can be assigned to the wildcard.

QUESTION

What is a generic type hierarchy?

Explanation



From: The Complete Java Interview Questions Course on Udemy © Peerless Geek

EXPLANATION

Generic Type Hierarchy

A **generic type hierarchy** refers to the inheritance structure involving classes, interfaces, or methods that use Java's generics, where type parameters can be inherited and refined through subclasses or implementing classes.

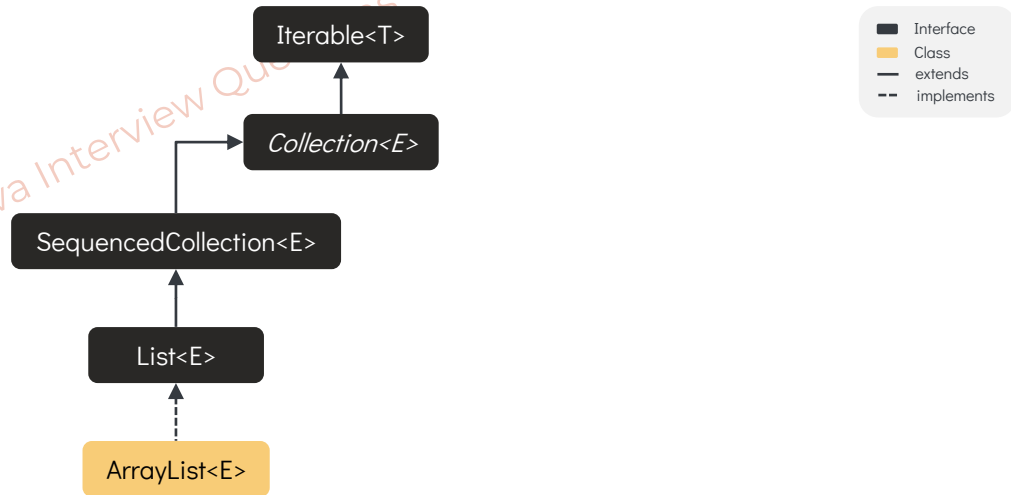
From: The Ultimate Animated Java Interview Questions Course on Udemy © Peerless Geek

EXPLANATION

Generic Type Hierarchy

A **generic type hierarchy** refers to the inheritance structure involving classes, interfaces, or methods that use Java's generics, where type parameters can be inherited and refined through subclasses or implementing classes.

ArrayList Hierarchy



QUESTION

What is **type erasure** in Generics?

Explanation



From: The Complete Java Interview Questions Course on Udemy © Peerless Geek

EXPLANATION

Type erasure

Type erasure is a process where generic type information is removed at compile-time, allowing Java to maintain backward compatibility with code written before generics were introduced in *Java 5*. During compilation, *the compiler* **replaces generic types with their bounds** (*if specified*) or **Object** (*if no bounds are provided*), ensuring type safety during compilation but removing type information at runtime.

From: The Ultimate Animated Java Interview Questions Course by Udit Garg © Peerless Geek

QUESTION

What are some
common pitfalls
to avoid with
Generics? 

Explanation



EXPLANATION

Pitfalls in Java Generics

Generics provide a way to define classes, interfaces, and methods with a placeholder for a data type, which ensures compile-time type safety and reduces code duplication. However, they also introduce challenges such as *incompatibility with primitive types, raw types, overlooking the diamond operator, type inference issues, overloading ambiguity, creating arrays of parameterized types, ignoring constraints, covariance and contravariance confusion, failing to understand generic subtyping, using generics in static contexts, and type erasure*.

From: The Ultimate Animated Java Interview Questions Course © Peerless Geek

Collections Framework

15

QUESTION

What is the Collections framework in Java?

Explanation



From: The
Animated Java Interview Questions Course on Udemy © Peerless Geek

EXPLANATION

Collections Framework

The **Collections framework** in Java is a unified architecture for representing and manipulating collections of objects and provides a set of *interfaces*, *implementations*, and *algorithms* to work with groups of objects efficiently.

From: The Ultimate Animated Java Interview Questions Course on Udemy © Peerless Geek

EXPLANATION

Collections Framework

The **Collections framework** in Java is a unified architecture for representing and manipulating collections of objects and provides a set of *interfaces*, *implementations*, and *algorithms* to work with groups of objects efficiently.

- It provides several core interfaces like **Collection**, **List**, **Set**, **Queue**, and **Map** to represent different types of collections..
- It offers concrete classes that implement these interfaces, such as **ArrayList**, **HashSet**, **PriorityQueue**, and **HashMap**.
- It provides utility classes like **Collections**, which includes utility methods for common operations such as *sorting*, *searching*, and *shuffling* elements within collections..
- It provides both **non-thread-safe** (e.g., *ArrayList*, *HashMap*) and **thread-safe** collections (e.g., *Vector*, *ConcurrentHashMap*).
- It supports older Java classes like **Enumeration**, **Vector**, **Stack**, and **Hashtable** by integrating them into its structure.
- It extensively uses generics to provide type safety at compile-time.
- It enables Collections to work together seamlessly, allowing easy conversion between different types.
- It provides a consistent API across different collection types, making it easier to learn and use.

QUESTION

What are the various **interfaces** used in the **Collections Framework**? 

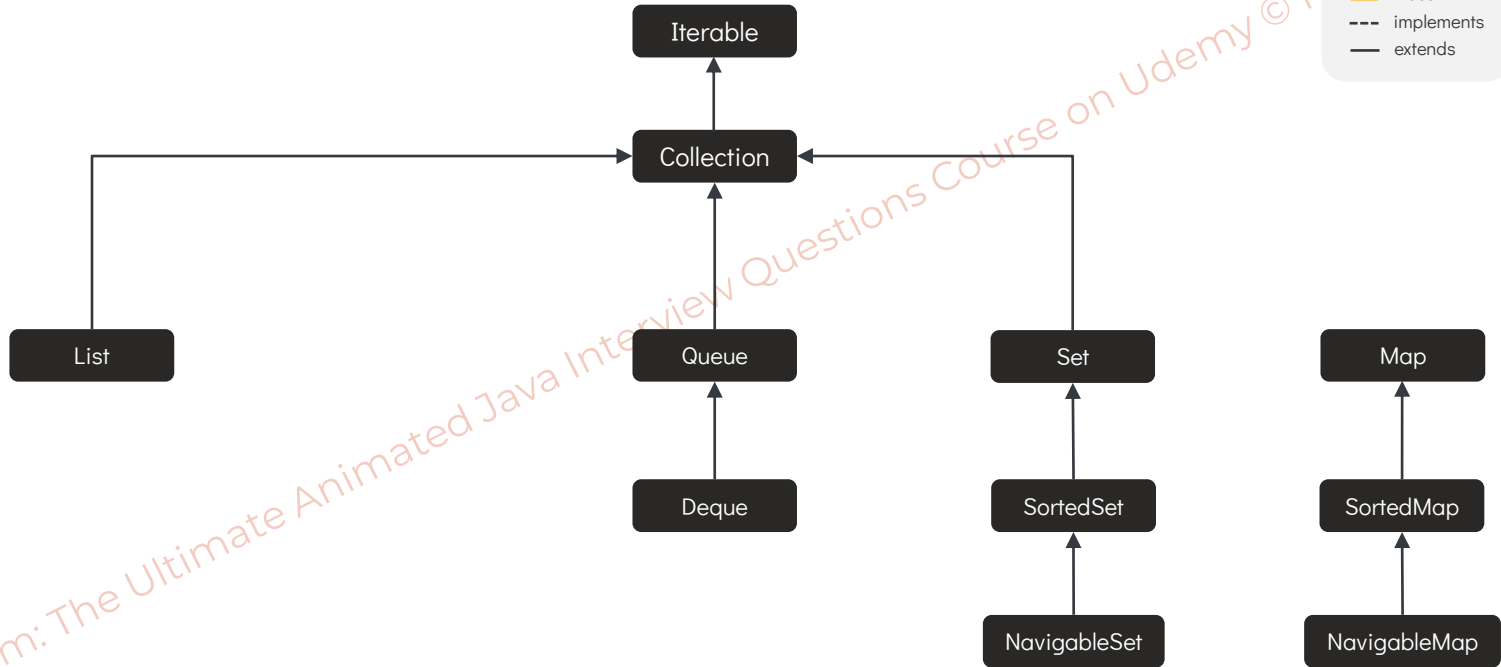
Explanation



From: The Complete Java Interview Questions Course on Udemy © Peerless Geek

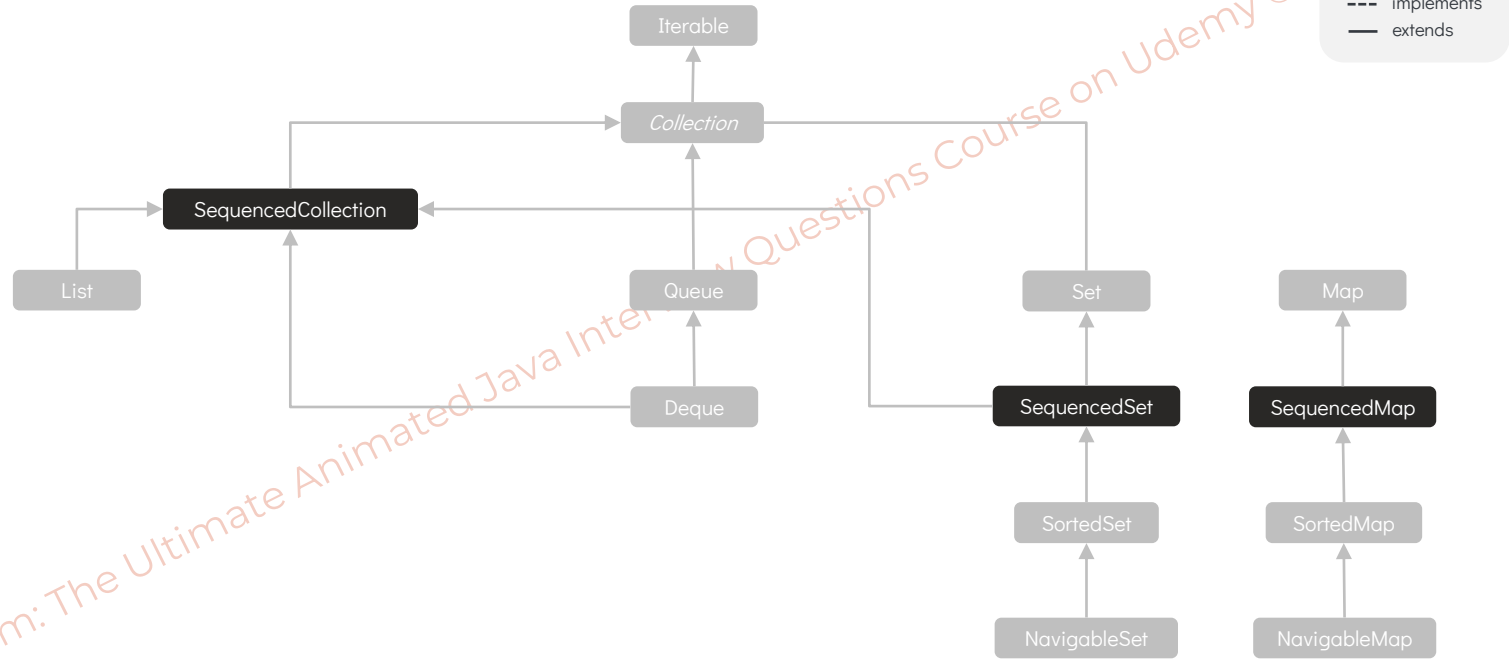
EXPLANATION

Collections Framework: Core Interfaces



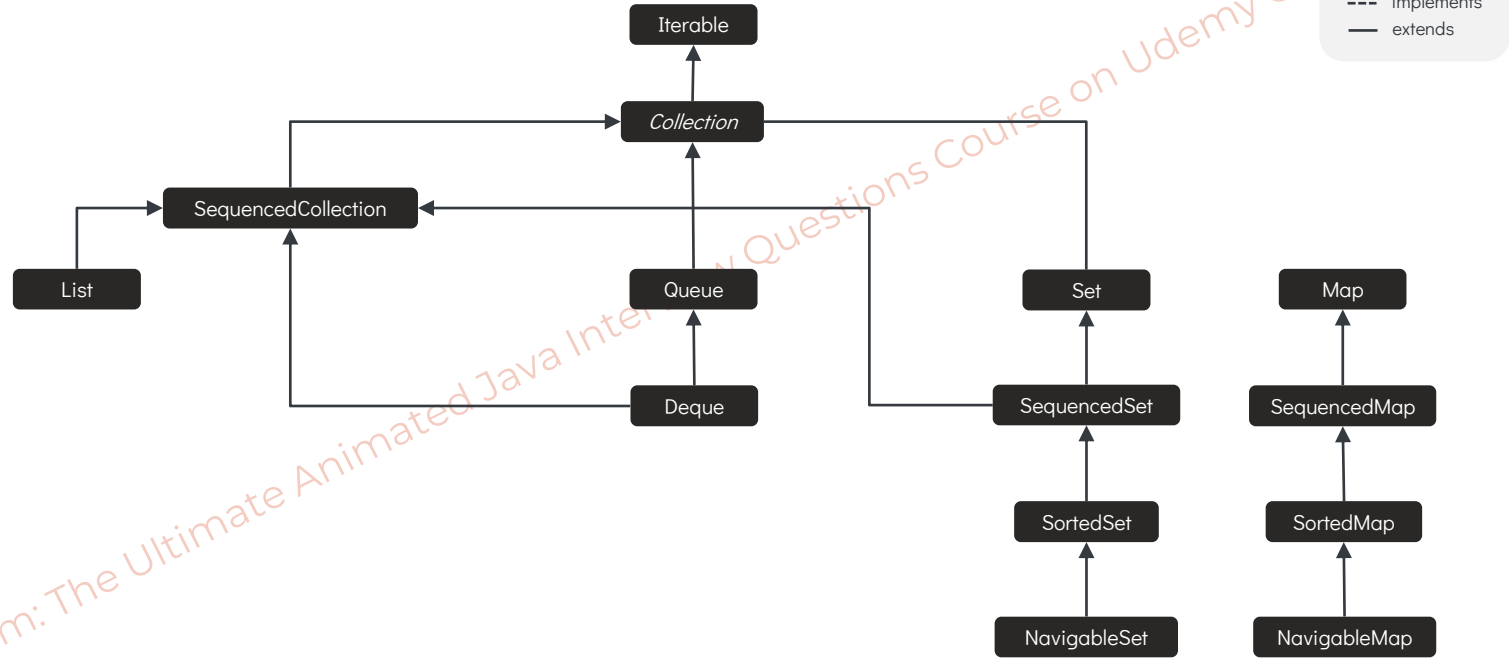
EXPLANATION

Collections Framework: Core Interfaces (After Java 21)



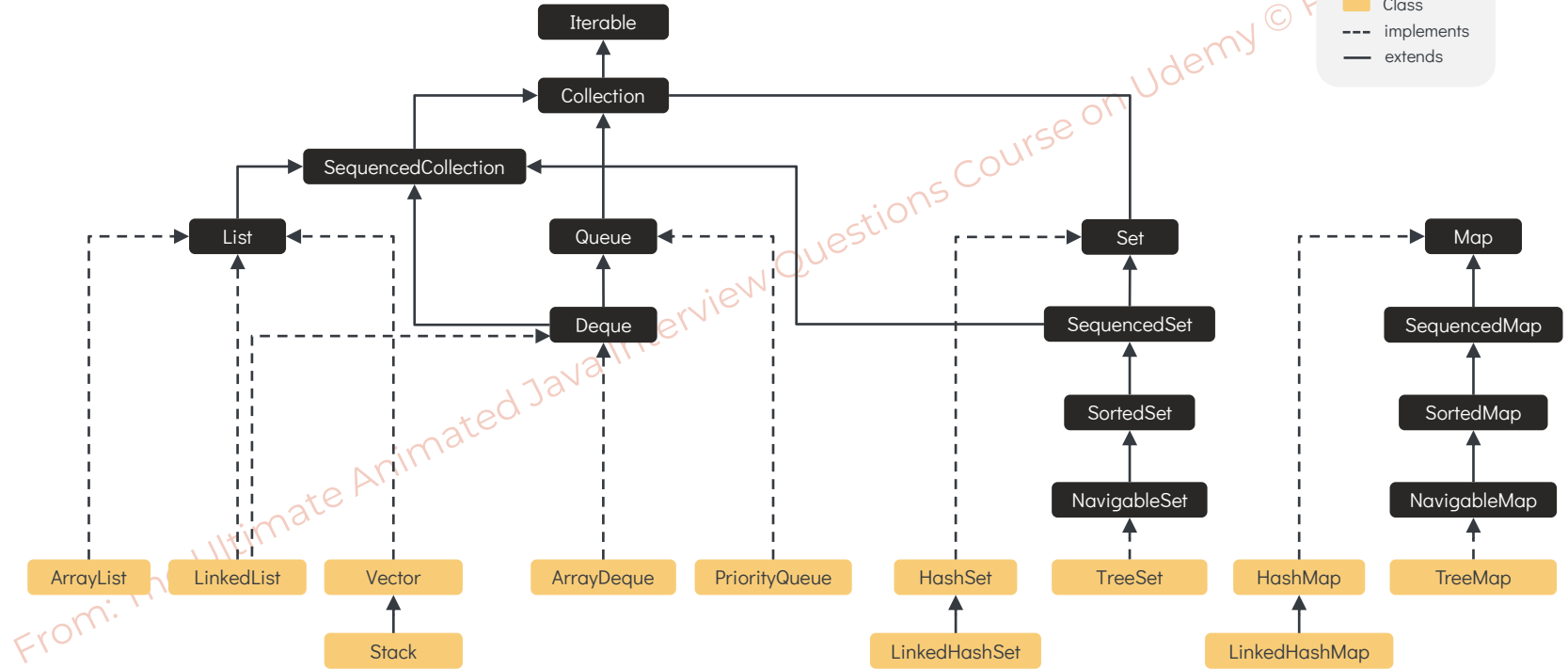
EXPLANATION

Collections Framework: Core Interfaces (After Java 21)



EXPLANATION

Collections Framework: Core Interfaces (*After Java 21*)



EXPLANATION

Interface	Description
Iterable	It is the root interface for collection classes, providing a way to iterate over elements (<i>via <code>iterator()</code></i>).
Collection	It is the root interface for representing a group of elements.
List	It is an ordered collection that allows duplicate elements and positional access.
Queue	It is a collection designed for holding elements prior to processing, following FIFO or priority order.
Deque	It is a double-ended queue allowing element insertion and removal at both ends.
Set	It is a collection that does not allow duplicate elements.
SortedSet	It is a Set that maintains its elements in ascending order.
NavigableSet	It is a SortedSet with navigation methods for retrieving closest matches.
Map	It is a collection that maps keys to values, with unique keys.
SortedMap	It is a Map that maintains its key-value pairs in ascending key order.
NavigableMap	It is a SortedMap with navigation methods for closest key lookups.
SequencedCollection (<i>introduced in Java 21</i>)	It is a collection that maintains insertion order for iteration.
SequencedSet (<i>introduced in Java 21</i>)	It is a Set that maintains insertion order without duplicates.
SequencedMap (<i>introduced in Java 21</i>)	It is a Map that maintains the insertion order of keys for iteration.

QUESTION

What is the difference between **Collection** and **Collections**? 

Explanation



EXPLANATION

Collection	Collections
Collection is an interface in the Java Collections Framework, part of the <i>java.util</i> package, and extends the Iterable interface.	Collections is a utility class in the Java Collections Framework and is also part of the <i>java.util</i> package.
It is the root interface in the Java Collections Hierarchy, extended by more specific collection interfaces like List , Set , and Queue .	It is a utility class consisting of static methods that operate on or return collections.
It represents a group of objects (elements) in a single unit.	It doesn't represent objects, but provides methods to manipulate collections.
It cannot be instantiated directly because it is an interface.	It also cannot be instantiated as its constructor is private, and all methods are static, so instantiation is unnecessary.
It offers methods such as add(E e) , remove(Object o) , contains(Object o) , isEmpty() , size() , clear() , and iterator() for managing and querying elements in the collection. Classes like ArrayList , PriorityQueue , and HashSet implement these methods by providing concrete implementations for the List , Queue , or Set interfaces.	It offers utility methods like sort() , shuffle() , max() , min() , reverse() , and synchronizedCollection() for tasks such as sorting, shuffling, finding the maximum and minimum elements, reversing the order of elements, and creating synchronized (thread-safe) collections.

QUESTION

What is **List** in the **Collections Framework**, and what are its various implementations? 

Explanation



EXPLANATION

List

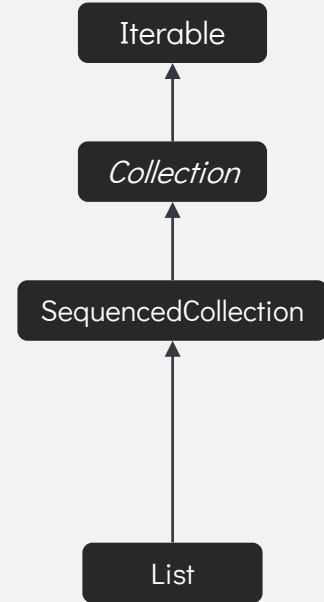
List is an interface that represents an ordered collection of elements, extends the **Collection** interface, inheriting its methods while also introducing additional methods for working with ordered collections, and is part of the *java.util* package.

From: The Ultimate Animated Java Interview Questions Course on Udemy © Peerless Geek

EXPLANATION

List

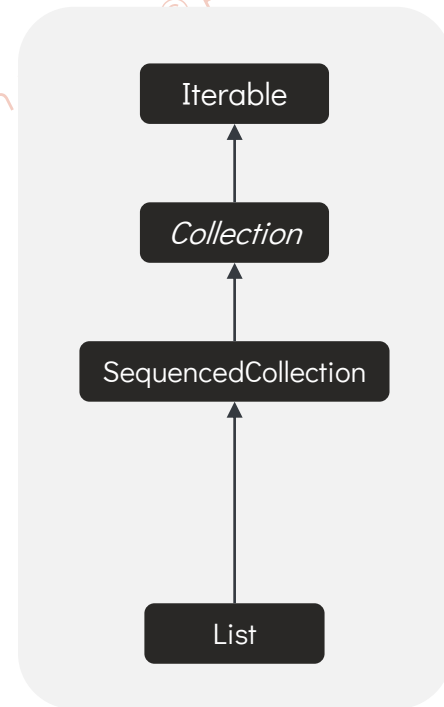
List is an interface that represents an ordered collection of elements, extends the **Collection** interface, inheriting its methods while also introducing additional methods for working with ordered collections, and is part of the `java.util` package.



EXPLANATION

Understanding Java List Methods: Hierarchical Breakdown

Category	Methods	
Inherited from Iterable	Abstract	
	Default	forEach (Consumer<? super T> action)
Inherited from Collection	Abstract	
	Default	parallelStream (), removeIf (Predicate<? super E> filter), stream (), toArray (IntFunction<T[]> generator)
Inherited from SequencedCollection	Abstract	
	Default	
Declared in List	Abstract	add (int index, E element), add (E e), addAll (int index, Collection<? extends E> c), addAll (Collection<? extends E> c), clear (), contains (Object o), containsAll (Collection<?> c), equals (Object o), get (int index), hashCode (), indexOf (Object o), isEmpty (), iterator (), lastIndexOf (Object o), listIterator (), listIterator (int index), remove (int index), remove (Object o), removeAll (Collection<?> c), retainAll (Collection<?> c), set (int index, E element), size (), subList (int fromIndex, int toIndex), toArray (), toArray (T[] a)
	Default	addFirst (E e), addLast (E e), getFirst (), getLast (), removeFirst (), removeLast (), replaceAll (UnaryOperator<E> operator), reversed (), sort (Comparator<? super E> c), splitter ()
	Static	copyOf (Collection<? extends E> coll), of (), of (E e1), of (E e1, E e2), of (E... elements)



EXPLANATION

List

List is an interface that represents an ordered collection of elements, extends the **Collection** interface, inheriting its methods while also introducing additional methods for working with ordered collections, and is part of the *java.util* package.

- It provides a way to store, access, and manage groups of elements while preserving the order in which the elements were added.
- It offers convenient access to elements by index, allows duplicates without restrictions, and generally supports null elements.
- It provides methods such as **add(index, element)** to insert at a specific index, **get(index)** to retrieve an element, **set(index, element)** to replace an element, and **remove(index)** to remove an element.
- It provides static methods like **List.of()** and **List.copyOf()** to create immutable lists.
- It extends the **SequencedCollection** interface (which itself **extends Collection**), adding methods like **getFirst()**, **getLast()**, **addFirst()**, and **addLast()** for directly accessing and manipulating elements at both the beginning and end of the list in *Java 21* and later.
- Its common implementation classes include like **ArrayList**, **LinkedList**, **Vector**, and **Stack**.

QUESTION

What is an ArrayList?



Explanation



Animated Java Interview Questions Course on Udemy © Peerless Geek

From: The

EXPLANATION

ArrayList

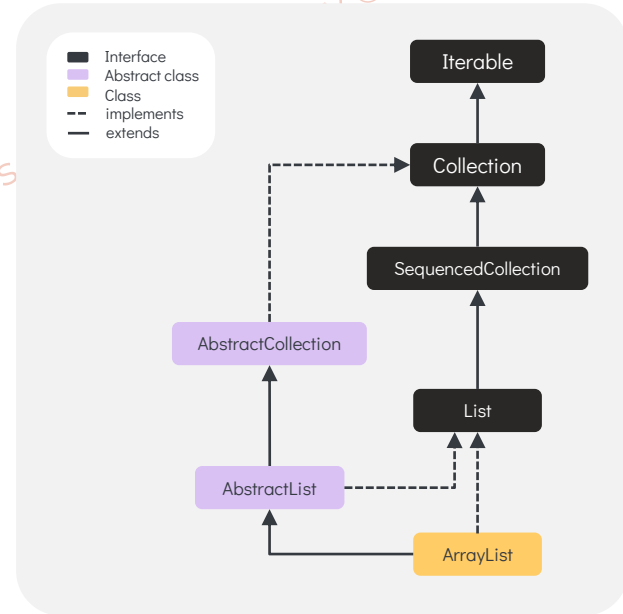
An **ArrayList** is a resizable array implementation of the **List** interface that dynamically adjusts its size as elements are added or removed, unlike regular fixed-size arrays, and is part of the *java.util* package.

From: The Ultimate Animated Java Interview Questions Course on Udemy © Peerless Geek

EXPLANATION

ArrayList

An **ArrayList** is a resizable array implementation of the **List** interface that dynamically adjusts its size as elements are added or removed, unlike regular fixed-size arrays, and is part of the *java.util* package.



EXPLANATION

ArrayList

An **ArrayList** is a resizable array implementation of the **List** interface that dynamically adjusts its size as elements are added or removed, unlike regular fixed-size arrays, and is part of the *java.util* package.

Implementation

7	4	9	5	1	6
0	1	2	3	4	5

(Dynamic array)

EXPLANATION

ArrayList

An **ArrayList** is a resizable array implementation of the **List** interface that dynamically adjusts its size as elements are added or removed, unlike regular fixed-size arrays, and is part of the *java.util* package.

- It provides concrete implementations for **List** and **Collection** interfaces by inheriting core functionality from **AbstractList** and **AbstractCollection**, ensuring streamlined and consistent behavior.
- It internally uses an array data structure to store elements and automatically resizes when elements are added or removed.
- It has an initial capacity of 10 by default, which can be specified in the constructor, and when capacity is reached, it typically increases by 50% (*$newCapacity = 1.5 \times oldCapacity$*).
- It maintains the insertion order, meaning elements are stored in the order they are added.
- It implements the **RandomAccess** interface, allowing for constant time, $O(1)$, indexed access and modification of elements.
- It is very efficient for appending elements at the end $O(1)$, but inserting or deleting elements at the beginning or middle incurs a time complexity of $O(n)$ due to the need to shift elements.
- It allows the storage of both null values and duplicate elements in the list.
- It is not thread-safe by default, meaning simultaneous modifications by multiple threads can lead to data corruption; however, it can be made thread-safe by using *Collections.synchronizedList()*.

QUESTION

How does the **size** of an **ArrayList** grow **dynamically**, and how is it implemented internally? ✎

Explanation



From: The Complete Java Interview Questions Course on Udemy © Peerless Geek

EXPLANATION

Dynamic Growth of ArrayList

An **ArrayList** in Java grows dynamically by expanding its internal array when more space is needed to accommodate new elements. *The process involves the following steps:*

- **Initial Capacity** : ArrayList starts with an initial capacity (default is 10).
- **Adding Elements** : Elements are added to the underlying array.
- **Array Resizing** : When the array is full, a new array with 50% increased capacity is created.
- **Copying Elements** : Elements are copied to the new array.
- **New Capacity** : After resizing, the ArrayList has a new, larger capacity.
- **Garbage Collection** : The old array is garbage collected by JVM.
- **Repeat** : The process repeats as needed.

QUESTION

What is the difference between the **length** of an **array** and the **size** of an **ArrayList**? 

Explanation



EXPLANATION

Array *length* vs ArrayList *size*

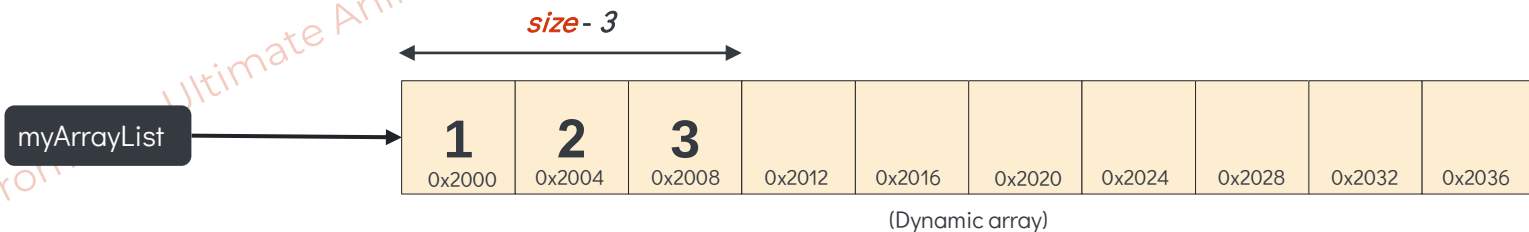
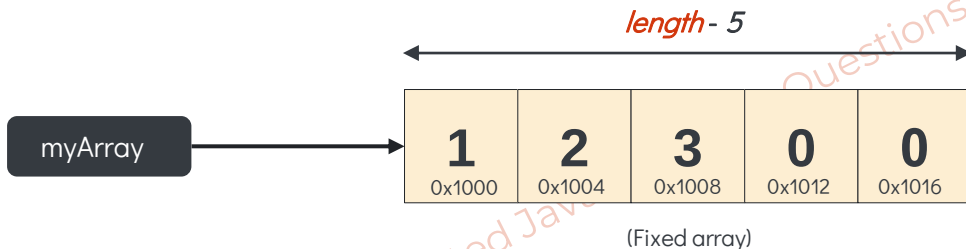
Array *length* refers to the fixed number of elements an array can hold, determined at the time of creation, while **ArrayList *size*** represents the dynamic number of elements currently stored in the ArrayList, which can grow or shrink as needed.

From: The Ultimate Animated Java Interview Questions Course on Udemy © Peerless Geek

EXPLANATION

Array *length* vs ArrayList *size*

Array *length* refers to the fixed number of elements an array can hold, determined at the time of creation, while **ArrayList *size*** represents the dynamic number of elements currently stored in the ArrayList, which can grow or shrink as needed.



EXPLANATION

Array <i>length</i>	ArrayList <i>size</i>
Length represents the total capacity of the array (i.e., how many elements the array can hold, regardless of how many positions are filled).	Size represents the actual number of elements currently stored in the ArrayList (i.e., the number of elements that have been added, not the capacity).
It is fixed , meaning the length is determined when an array is created and cannot be changed afterward.	It is dynamic , meaning the size can change as elements are added or removed.
It is accessed as a property , e.g., array.length (without parentheses, because it is not a method).	It is a method and is accessed with parentheses, e.g., arrayList.size() .

QUESTION

How are elements stored in **memory** for both **regular arrays** and **ArrayLists**? 

Explanation



From: The Complete Java Interview Questions Course on Udemy © Peerless Geek

EXPLANATION

Memory Allocation : Arrays and ArrayLists

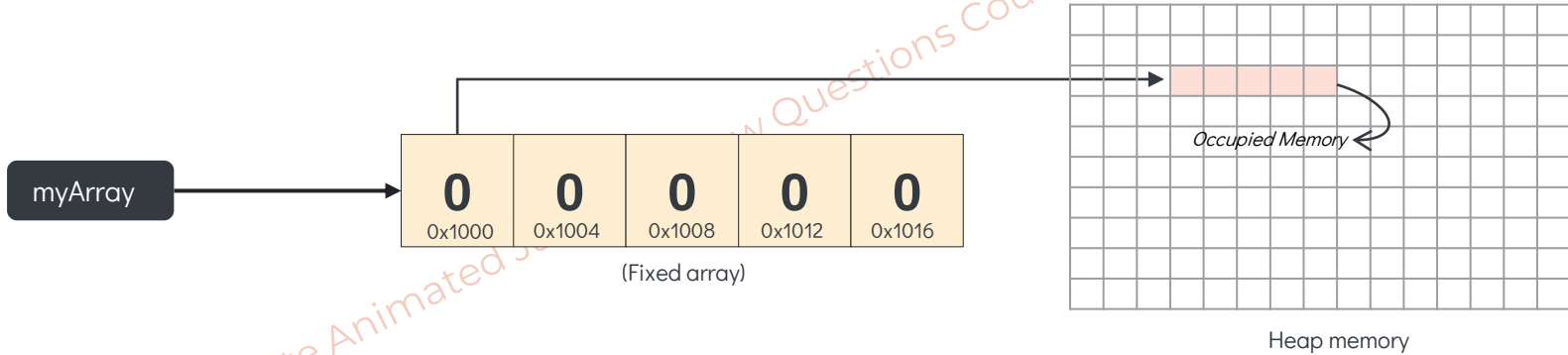
Both **arrays** and **ArrayList** store elements *contiguously* in **heap memory**, but arrays have a fixed size, while ArrayList uses a resizable internal array for dynamic growth, reserving extra memory to reduce resizing costs.

From: The Ultimate Animated Java Interview Questions Course on Udemy © Peerless Geek

EXPLANATION

Memory Allocation : Arrays and ArrayLists

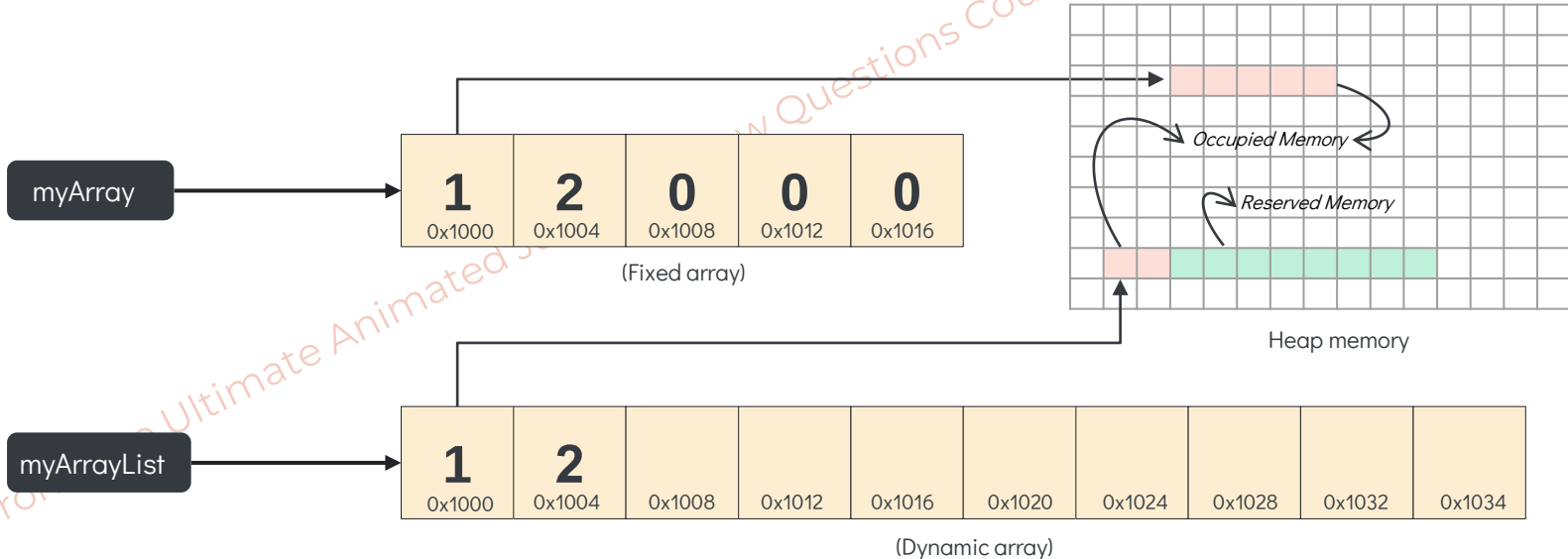
Both **arrays** and **ArrayList** store elements *contiguously* in **heap memory**, but arrays have a fixed size, while ArrayList uses a resizable internal array for dynamic growth, reserving extra memory to reduce resizing costs.



EXPLANATION

Memory Allocation : Arrays and ArrayLists

Both **arrays** and **ArrayList** store elements *contiguously* in **heap memory**, but arrays have a fixed size, while ArrayList uses a resizable internal array for dynamic growth, reserving extra memory to reduce resizing costs.



QUESTION

What is the difference between an **array** and an **ArrayList**? 

Explanation



EXPLANATION

Array	ArrayList
An array is a fixed-size, ordered collection of elements that is part of core Java and does not require an import.	An ArrayList is a resizable, ordered collection of elements that is part of <code>java.util</code> package and requires an import.
It is fixed in size; once created, its size cannot change, so adding more elements requires creating a new array.	It is dynamic in size, growing or shrinking as needed, without requiring size specification at creation.
It allocates memory once at creation, occupying a contiguous block of memory.	It allocates memory dynamically as size changes, usually reserving extra space to reduce frequent resizing.
It can hold both primitive types (<i>like int, char, etc.</i>) and object references.	It can only hold object references, not primitive types directly, but Java autoboxing converts primitives to their wrapper classes (e.g., int to Integer).
It is basic and comes with limited built-in methods, focusing primarily on fundamental operations.	It offers more built-in methods, such as <code>add()</code> , <code>remove()</code> , and <code>search()</code> , providing greater functionality.
It can contain null elements for object arrays, but primitive arrays use default values like 0 for int and cannot contain null.	It can contain null elements and offers more flexible null handling due to its object nature.
It can be iterated using traditional for loops and enhanced for loops.	It can be iterated using for loops, enhanced for loops, or iterators (<i>Iterator and ListIterator</i>).
It is generally faster for accessing and modifying elements, as there's no overhead of dynamic resizing.	It is slightly slower due to overhead from dynamic resizing and additional methods.

QUESTION

How do you
convert an
ArrayList to an
array and an **array**
to an **ArrayList**? 

Explanation



From: The Complete Java Interview Questions Course on Udemy © Peerless Geek

EXPLANATION

Array to ArrayList		ArrayList to Array	
Method	Description	Method	Description
Manual loop	This approach involves manually iterating over the array and adding each element to a new ArrayList.	Manual loop	This approach involves manually iterating over the ArrayList and copying each element to a new array.
Arrays.asList() and new ArrayList<>()	This approach first converts the array to a fixed-size List using Arrays.asList(), then uses that List to initialize a new ArrayList. Note: Arrays.asList() returns a fixed-size list backed by the original array, allowing modifications to existing elements but not to the size of the list.	toArray()	This approach uses the toArray() method to convert the ArrayList into an array.
List.of() and new ArrayList<>()	This approach uses the List.of() method to create an immutable List from the array, then uses that to initialize a new ArrayList. Note: List.of() returns an immutable list with the specified elements, disallowing any modifications to the list's content or size.	toArray(T[])	This approach uses the generic version toArray(T[] a) to convert the ArrayList to an array of a specific type. Note: If the array a is large enough, the existing array is used; otherwise, a new array is allocated.
Collections.addAll()	This approach creates an empty ArrayList and then uses Collections.addAll() to add all elements from the array to the list.		

QUESTION

How do you
make an
ArrayList
read-only? 

Explanation



Animated Java Interview Questions Course on Udemy © Peerless Geek

From: The

EXPLANATION

Read-only ArrayLists: Ensuring Data Protection

ArrayList objects can be made **read-only** in three ways: by using `Collections.unmodifiableList()` to create a read-only view of an existing list (*which still reflects changes to the original list*), or by using `List.of()` or `List.copyOf()` to create a truly immutable list that cannot be modified or changed in any way.

From: The Ultimate Animated Java Interview Questions Course on Udemy © Peerless Geek

EXPLANATION

Read-only ArrayLists: Ensuring Data Protection

<i><code>Collections.unmodifiableList()</code></i>	<i><code>List.Of()</code></i>	<i><code>List.copyOf()</code></i>
<i><code>Collections.unmodifiableList()</code> wraps the original list and returns an unmodifiable view of the list.</i>	<i><code>List.Of()</code> creates an immutable list from the specified elements via varargs (<i>variable arguments</i>).</i>	<i><code>List.copyOf()</code> creates an immutable list by copying elements from an existing Collection.</i>
It reflects changes to the original list (<i>since it's a view of the original</i>).	It doesn't reflect any future changes to the original list (<i>if the source is mutable</i>).	It doesn't reflect any future changes to the original list (<i>if the source is mutable</i>).
It allows null elements in the list.	It does not allow null as an element; if passed, it throws a <code>NullPointerException</code> .	It does not allow null in the collection; if present, it throws a <code>NullPointerException</code> .
It is used you want to expose a list to other classes without allowing them to modify it.	It is used when you need a list with predefined contents that should remain constant.	It is used when want to make a snapshot of an existing list and prevent any changes to it.

QUESTION

How do you
remove
duplicates from
an **ArrayList**? 

Explanation



EXPLANATION

ArrayLists: Strategies for Removing Duplicate Entries

Method	Description	Time & Space Complexity
Nested Loop (<i>Brute Force</i>)	The algorithm uses nested loops to compare each element with subsequent ones in the ArrayList. Duplicates are removed when found, and the inner loop index is adjusted to account for the shortened list.	TC: $O(n^2)$ SC: $O(1)$
Sorting and Single Loop	This optimized approach leverages sorting to reduce comparisons by first sorting the ArrayList using <code>Collections.sort()</code> , which brings duplicates adjacent, and then uses a single loop to remove adjacent duplicates by comparing each element with its neighbor, adjusting the index as needed.	TC: $O(n \log n)$ SC: $O(1)$
LinkedHashSet	This approach uses a LinkedHashSet to remove duplicates from the ArrayList while preserving element order. It converts the list to a LinkedHashSet and back, automatically eliminating duplicates.	TC: $O(n)$ SC: $O(n)$

QUESTION

How do you reverse an ArrayList?

Explanation



From: The Complete Java Interview Questions Course on Udemy © Peerless Geek

EXPLANATION

Reversing an ArrayList

An **ArrayList** can be reversed by using `Collections.reverse()`, the `ArrayList`'s **`reverse()`** method (*Java 21+*), or a **custom algorithm** that swaps elements from both ends towards the center.

Approach	Description
<code>Collections.reverse()</code>	Uses <code>Collections.reverse()</code> to reverse the order of elements in an <code>ArrayList</code> in-place.
<code>ArrayList</code> 's <code>reverse()</code> Method (<i>Java 21+</i>)	Reverses the <code>ArrayList</code> using its built-in <code>reverse()</code> method, introduced in <i>Java 21</i> .
Custom Algorithm (<i>Swapping Elements</i>)	Implements a custom algorithm that swaps elements from both ends of the <code>ArrayList</code> towards the center.

QUESTION

How do you **sort**
an **ArrayList** in
descending
order? 

Explanation



From: The Complete Java Interview Questions Course on Udemy © Peerless Geek

EXPLANATION

Sorting an ArrayList in Descending Order in Java

An `ArrayList` can be sorted in descending order using `Collections.sort()` with a comparator, the `ArrayList's sort()` method (*Java 8+*), or a **custom algorithm** that sorts the list in reverse order.

Approach	Description
<code>Collections.sort()</code> with Comparator	Uses <code>Collections.sort()</code> combined with <code>Collections.reverseOrder()</code> to sort the elements in place in descending order.
<code>ArrayList's sort()</code> with Comparator (<i>Java 8+</i>)	Uses the <code>sort()</code> method of <code>ArrayList</code> with <code>Collections.reverseOrder()</code> to sort elements in place in descending order.
Custom Algorithm (Bubble Sort Descending)	Implements a custom sorting algorithm, such as Bubble Sort , to arrange the <code>ArrayList</code> in descending order.

QUESTION

What is a LinkedList?



Explanation



Animated Java Interview Questions Course on Udemy © Peerless Geek

From: The

EXPLANATION

LinkedList

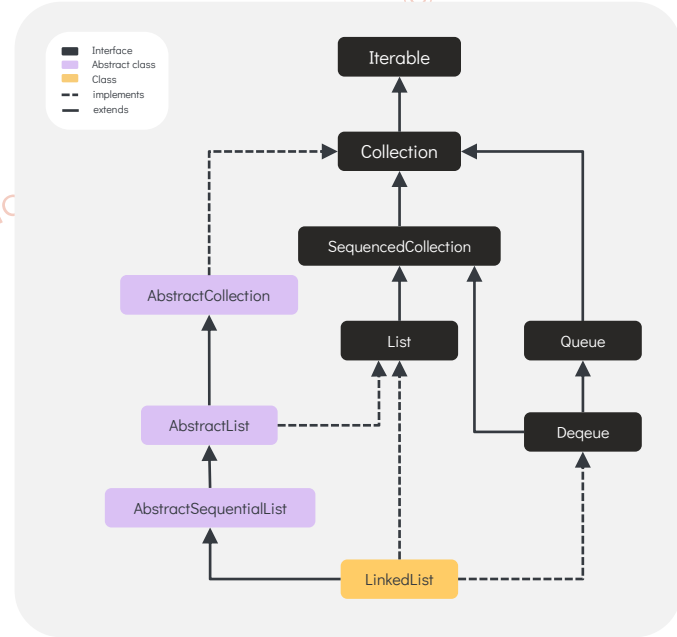
A **LinkedList** is a doubly linked list implementation of the **List**, and **Deque** interfaces, allowing efficient insertions and deletions from both ends of the list, and is part of the *java.util* package.

From: The Ultimate Animated Java Interview Questions Course on Udemy © Peerless Geek

EXPLANATION

LinkedList

A **LinkedList** is a doubly linked list implementation of the **List**, and **Deque** interfaces, allowing efficient insertions and deletions from both ends of the list, and is part of the *java.util* package.

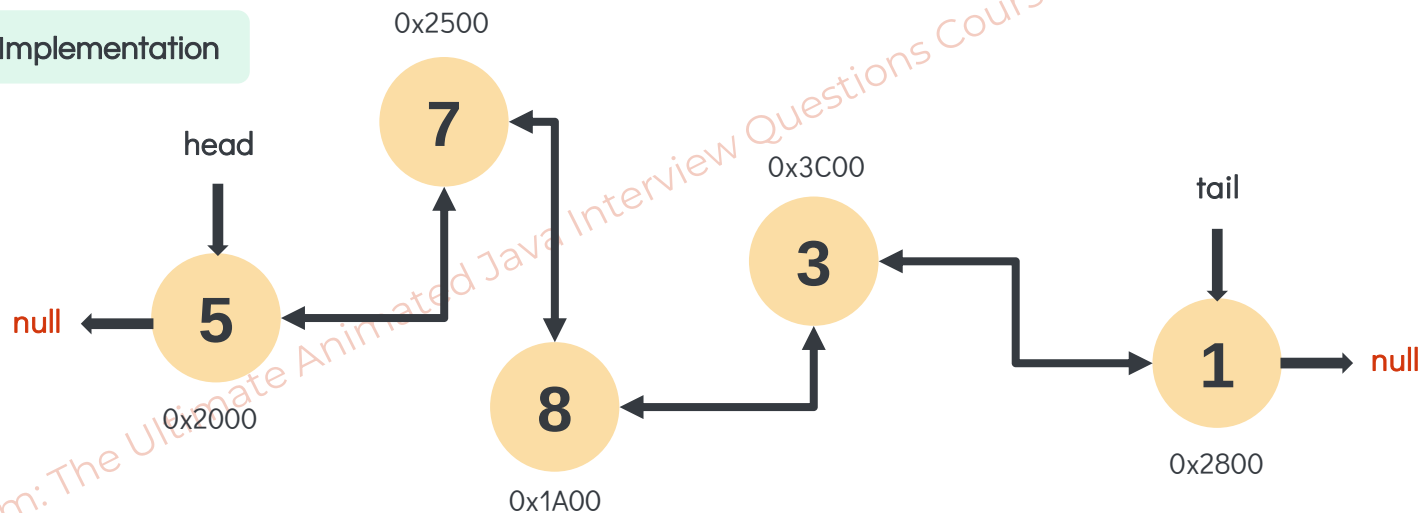


EXPLANATION

LinkedList

A **LinkedList** is a doubly linked list implementation of the **List**, and **Deque** interfaces, allowing efficient insertions and deletions from both ends of the list, and is part of the *java.util* package.

Implementation



Doubly Linked List (In memory)

EXPLANATION

LinkedList

A **LinkedList** is a doubly linked list implementation of the **List**, and **Deque** interfaces, allowing efficient insertions and deletions from both ends of the list, and is part of the *java.util* package.

- It provides concrete implementations for the **List** and **Collection** interfaces by inheriting core functionality from **AbstractList** and **AbstractCollection**, while also extending **AbstractSequentialList** to support sequential access.
- It internally uses a **doubly linked list** data structure where each element (*node*) contains a reference (*memory address*) to the previous and next elements.
- It maintains the insertion order, meaning elements are stored in the order they are added.
- It requires linear time, $O(n)$, to access its elements, as this involves traversing from either the start or the end of the list.
- It offers $O(1)$ insertion or deletion at the beginning and end due to pointer updates, but $O(n)$ in the middle because of traversal.
- It implements the **Deque** interface, allowing it to function as both a queue and a stack by supporting operations at both ends.
- It allows the storage of both null values and duplicate elements in the list.
- It is not thread-safe by default, so multiple threads modifying it simultaneously can cause data corruption. It can be made thread-safe using *Collections.synchronizedList()*.

QUESTION

What is the
difference between
ArrayList and
LinkedList? 

Explanation



EXPLANATION

ArrayList	LinkedList
ArrayList internally uses a dynamic array as its underlying data structure to store elements.	LinkedList internally uses a doubly linked list as its underlying data structure to store elements.
It uses contiguous memory allocation.	It uses non-contiguous memory, with each element (node) containing references to previous and next elements.
It has an initial capacity and resizes when it reaches capacity.	It has no fixed capacity and doesn't need resizing as nodes are added or removed as needed.
It provides constant $O(1)$ access time to elements due to its indexing.	It provides $O(n)$ access time for elements because it requires traversing the list from either the start or the end.
It is efficient for insertions at the end (<i>amortized $O(1)$</i>), but inefficient for insertions in the middle or at the beginning, as they require $O(n)$ time due to shifting elements.	It is efficient for insertions at both ends with constant time $O(1)$, but insertions in the middle require $O(n)$ time due to the need to traverse the list to the insertion point.
It has lower memory overhead as it only stores data and not additional pointers..	It has higher memory overhead, storing elements plus previous and next references for each node.
It is ideal for applications where random access and iteration are more frequent than insertion/deletion.	It is suitable for scenarios where frequent insertions and deletions are required.

QUESTION

What is a Vector?



Explanation



EXPLANATION

Vector

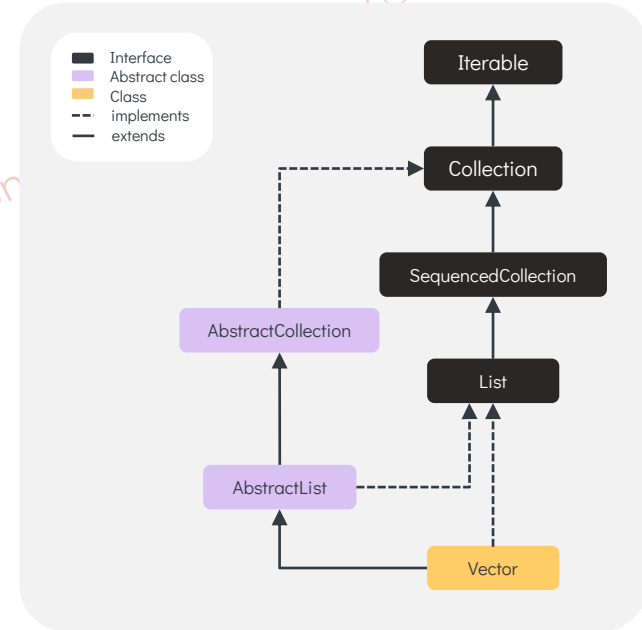
A **Vector** is a legacy class from the early versions of Java and is a thread-safe and resizable array implementation of the **List** interface that dynamically adjusts its size as elements are added or removed, similar to **ArrayList**, and is part of the *java.util* package.

From: The Ultimate Animated Java Interview Questions Course on Udemy © Peerless Geek

EXPLANATION

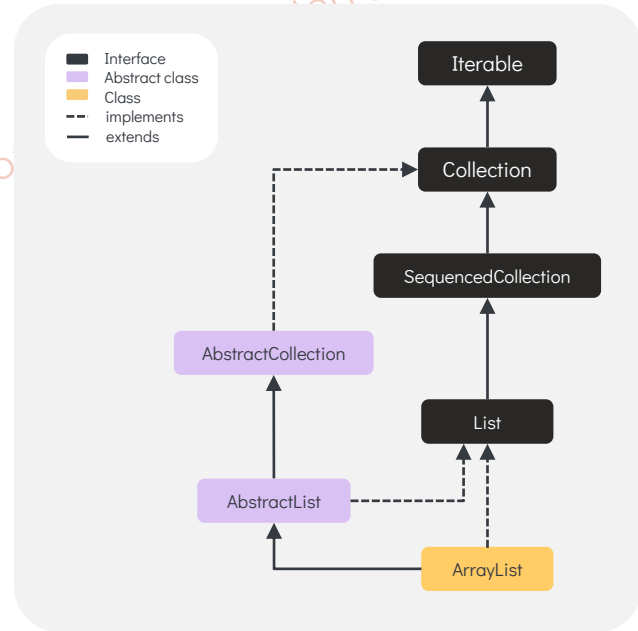
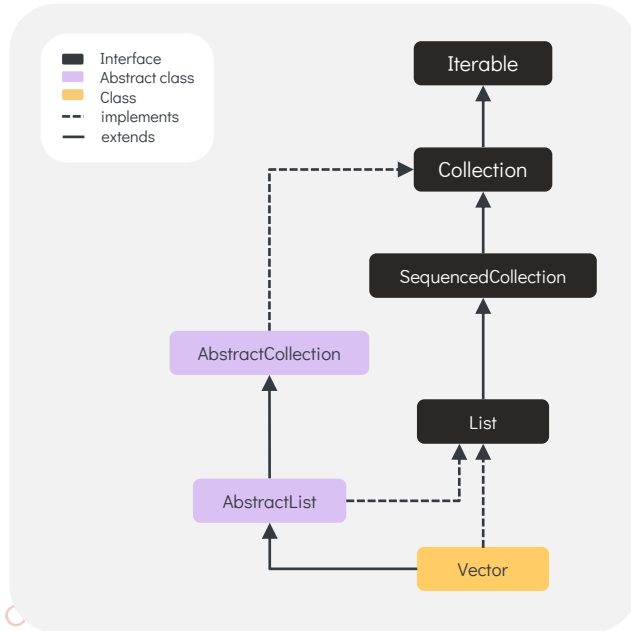
Vector

A **Vector** is a legacy class from the early versions of Java and is a thread-safe and resizable array implementation of the **List** interface that dynamically adjusts its size as elements are added or removed, similar to **ArrayList**, and is part of the *java.util* package.



EXPLANATION

Class Hierarchy (*Vector* and *ArrayList*)



EXPLANATION

Vector

A **Vector** is a legacy class from the early versions of Java and is a thread-safe and resizable array implementation of the **List** interface that dynamically adjusts its size as elements are added or removed, similar to **ArrayList**, and is part of the *java.util* package.

Implementation

7	4	9	5	1	6
0	1	2	3	4	5

(Dynamic array)

EXPLANATION

Vector

A **Vector** is a legacy class from the early versions of Java and is a thread-safe and resizable array implementation of the **List** interface that dynamically adjusts its size as elements are added or removed, similar to **ArrayList**, and is part of the *java.util* package.

- It provides concrete implementations for **List** and **Collection** interfaces by inheriting core functionality from **AbstractList** and **AbstractCollection**.
- It internally uses an array data structure to store elements and automatically resizes when elements are added or removed.
- It has an initial capacity of 10 by default, which can be specified in the constructor, and when capacity is reached, it typically doubles its size ($\text{newCapacity} = 2 \times \text{oldCapacity}$).
- It maintains the insertion order, meaning elements are stored in the order they are added.
- It implements the **RandomAccess** interface, allowing for constant time, $O(1)$, indexed access and modification of elements.
- It is less efficient, $O(n)$, to insert or delete elements at the beginning or middle of a list due to the need to shift other elements.
- It allows the storage of both null values and duplicate elements in the list.
- Its methods are synchronized, making them safe for use in multi-threaded environments without the need for external synchronization.

QUESTION

What is the difference between **ArrayList** and **Vector**? 

Explanation



EXPLANATION

ArrayList	Vector
ArrayList , introduced in <i>JDK 1.2</i> as part of the Collections framework, is the more modern and preferred choice for most applications.	Vector , introduced in <i>JDK 1.0</i> , predates the Collections framework and is considered legacy but still supported for backward compatibility.
It is not synchronized by default; external synchronization is required if multiple threads access it concurrently and at least one modifies it structurally.	It is thread-safe as every method is synchronized, allowing safe concurrent access and modification without external synchronization.
It is generally faster in single-threaded scenarios due to the absence of synchronization overhead, making operations like add() , remove() , and get() more efficient.	It is slightly slower due to synchronization overhead on each method call, which can impact performance in high-demand applications.
It grows by 50% of its current size when it reaches capacity; for example, if the size is 10, it grows to 15.	It doubles its size by default when it reaches capacity, or grows by the specified capacity increment if provided.
It is ideal for single-threaded applications where performance is crucial and thread safety is not a concern.	It is suitable for legacy codebases or scenarios where automatic thread safety is required.

QUESTION

What is the **Stack** class, and what are its various methods? 

Explanation



EXPLANATION

Stack

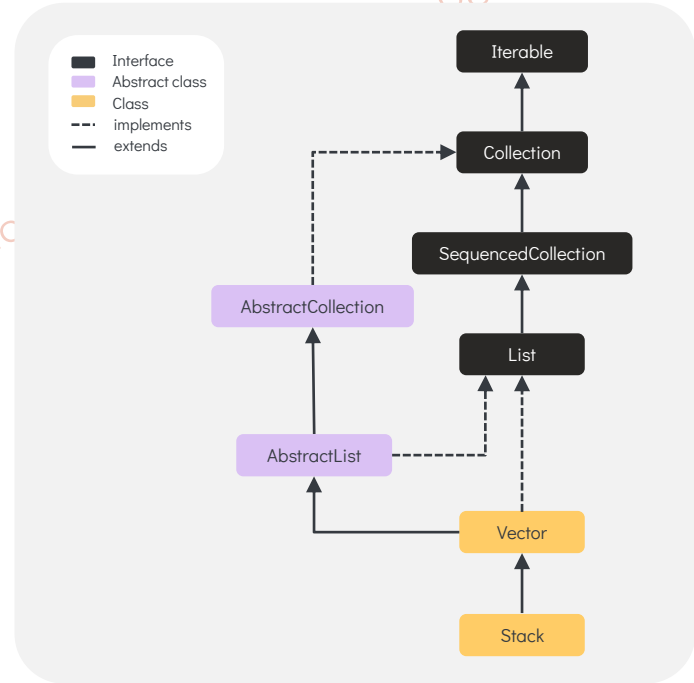
A **Stack** is a Last-In-First-Out (*LIFO*) data structure commonly used to represent a collection where elements are retrieved in the reverse order of their insertion and is part of the *java.util* package.

From: The Ultimate Animated Java Interview Questions Course on Udemy © Peerless Geek

EXPLANATION

Stack

A **Stack** is a Last-In-First-Out (*LIFO*) data structure commonly used to represent a collection where elements are retrieved in the reverse order of their insertion and is part of the `java.util` package.

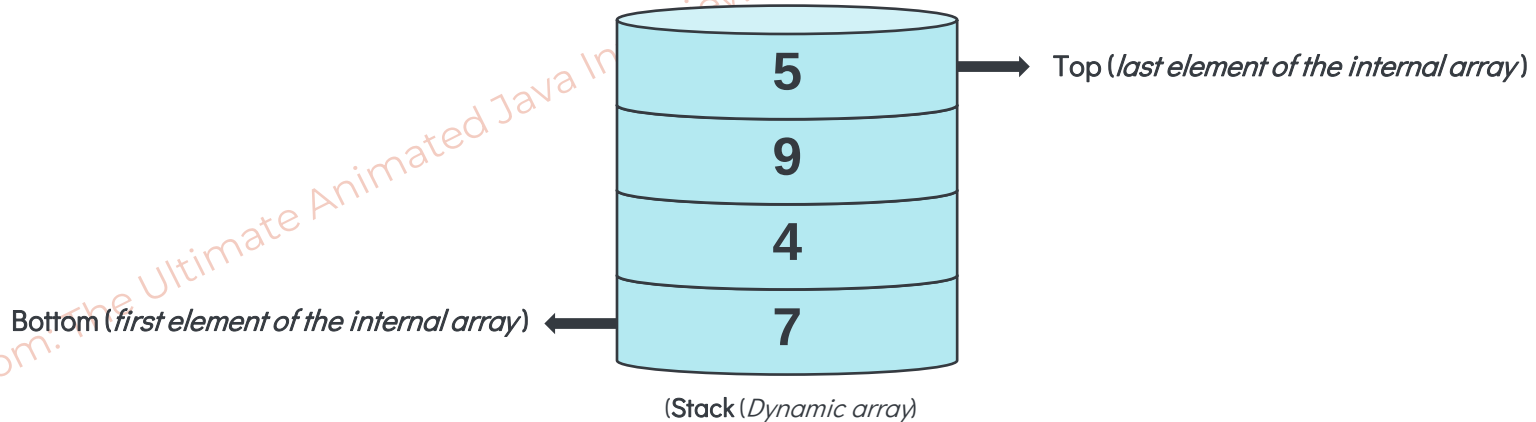


EXPLANATION

Stack

A **Stack** is a Last-In-First-Out (*LIFO*) data structure commonly used to represent a collection where elements are retrieved in the reverse order of their insertion and is part of the *java.util* package.

Implementation



EXPLANATION

Stack

A **Stack** is a legacy class in Java that represents a Last-In-First-Out (*LIFO*) data structure commonly used to represent a collection where elements are retrieved in the reverse order of their insertion and is part of the *java.util* package.

- It internally uses a **Vector** (*which uses a dynamic array*) to store elements, allowing dynamic resizing as elements are pushed onto or popped off the stack.
- It does not have a fixed capacity and grows dynamically as needed to accommodate new elements, with resizing handled by the underlying Vector.
- It preserves insertion order, but it allows access only to the most recent element, following Last-In-First-Out (LIFO) order.
- It offers stack-specific methods like **push**(E item) to add, **pop**() to remove and return the top element, and **peek**() to inspect the top element without removing it.
- It provides the flexibility to store both null values and duplicate elements.
- It is thread-safe, like **Vector**, because it is internally synchronized.

QUESTION

How can you synchronize an ArrayList?



Explanation



Animated Java Interview Questions Course on Udemy © Peerless Geek

From: The

EXPLANATION

ArrayList Synchronization

An **ArrayList** can be **synchronized** using *Collections.synchronizedList()*, which wraps the original list and makes individual operations like add, remove, and get thread-safe by synchronizing them internally; however, it does not synchronize iteration, meaning that if one thread modifies the list while another is iterating, it can lead to unpredictable behavior or a **ConcurrentModificationException**.

From: The Ultimate Animated Java Interview Questions Course on Udemy © Peerless Geek

QUESTION

What is a **Set** in the **Collections Framework**, and what are its **various implementations**? 

Explanation



EXPLANATION

Set

Set is an interface that represents an unordered collection of unique elements, extends the **Collection** interface, inheriting its methods while introducing additional constraints on element uniqueness and is part of the *java.util* package.

From: The Ultimate Animated Java Interview Questions Course on Udemy © Peerless Geek

EXPLANATION

Set

Set is an interface that represents an unordered collection of unique elements, extends the **Collection** interface, inheriting its methods while introducing additional constraints on element uniqueness and is part of the *java.util* package.

Iterable

Collection

Set

From: The Ultimate Animated Java Interview Questions Course on Peerless Geek

EXPLANATION

Understanding Java Set Methods: Hierarchical Breakdown

Category	Methods	
Inherited from Iterable	Abstract	
Inherited from Iterable	Abstract	
	Default	forEach (Consumer<? super T> action)
Inherited from Collection	Abstract	
	Default	parallelStream() , removeIf (Predicate<? super E> filter), stream() , toArray (IntFunction<T[]> generator)
Declared in Set	Abstract	add (E e), addAll (Collection<? extends E> c), clear() , contains (Object o), containsAll (Collection<?> c), equals (Object o), hashCode() , isEmpty() , iterator() , remove (Object o), removeAll (Collection<?> c), retainAll (Collection<?> c), size() , toArray() , toArray (T[] a)
	Default	splititerator()
	Static	copyOf (Collection<? extends E> coll), of() , of (E e1), of (E e1, E e2), of (E... elements)



EXPLANATION

Set

Set is an interface that represents an unordered collection of unique elements, extends the **Collection** interface, inheriting its methods while introducing additional constraints on element uniqueness and is part of the *java.util* package.

- It provides a way to store, access, and manipulate groups of unique elements.
- It does not guarantee the order of elements, meaning elements may not be retrieved in the order they were added.
- It inherits methods from the **Collection** interface, such as **add(*element*)**, **remove(*element*)**, and **contains(*element*)**, but it overrides behaviors like **add(*element*)** to enforce *element uniqueness*.
- It does not support indexed access or methods like **get(*index*)** because the elements are not ordered, unlike a List.
- It provides static methods like **Set.of()** and **Set.copyOf()** (introduced in Java 9) to create immutable sets.
- Its common implementation classes include **HashSet**, **LinkedHashSet**, and **TreeSet**.

QUESTION

What is the **HashSet** class, and how does it store elements? 

Explanation



EXPLANATION

HashSet

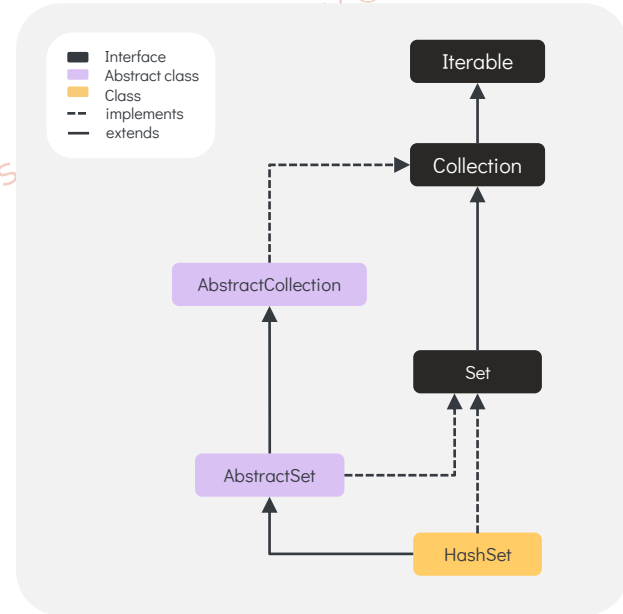
A **HashSet** is an implementation of the **Set** interface that stores unique elements without preserving any specific order of insertion and is part of the *java.util* package.

From: The Ultimate Animated Java Interview Questions Course on Udemy © Peerless Geek

EXPLANATION

HashSet

A **HashSet** is an implementation of the **Set** interface that stores unique elements without preserving any specific order of insertion and is part of the *java.util* package.



EXPLANATION

HashSet

A **HashSet** is an implementation of the **Set** interface that stores unique elements without preserving any specific order of insertion and is part of the *java.util* package.

Implementation

0	
1	
2	
3	
4	
5	
6	
7	
8	
9	

(hash table)

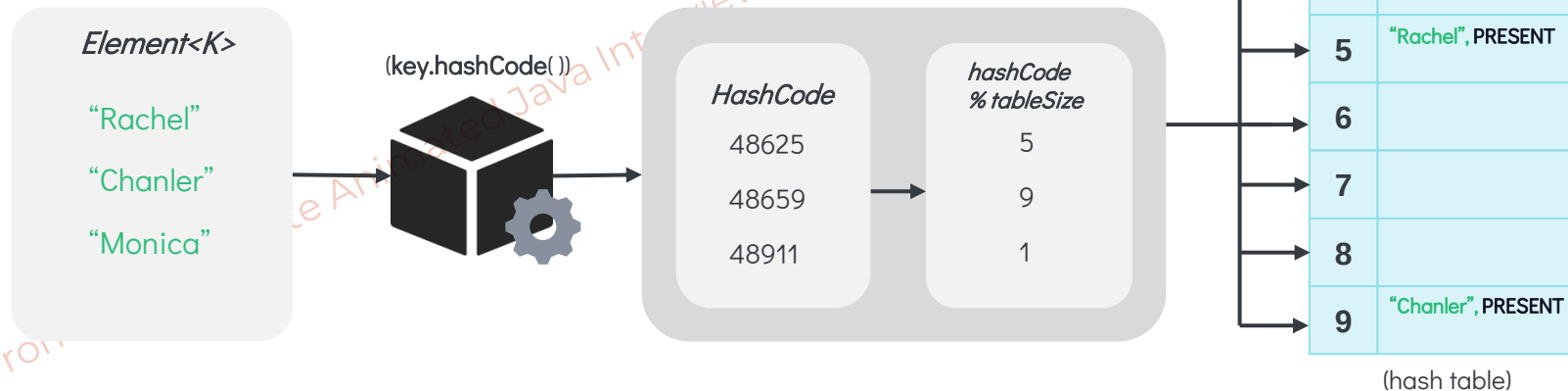
From: The Ultimate Animated Java Interview Questions Course on Udemy © Peerless Geek

EXPLANATION

HashSet

A **HashSet** is an implementation of the **Set** interface that stores unique elements without preserving any specific order of insertion and is part of the `java.util` package.

Adding an element in a hashtable : $O(1)$



EXPLANATION

HashSet

A **HashSet** is an implementation of the **Set** interface that stores unique elements without preserving any specific order of insertion and is part of the *java.util* package.

- It provides a concrete implementation of the **Set** interface by extending **AbstractSet**, which offers a skeletal implementation of core Set functionality.
- It uses a **hash table** (*backed by a **HashMap***) to store elements as keys with a constant dummy value (**PRESENT**), ensuring average constant-time performance ($O(1)$) for add, remove, and contains operations.
- It has a default load factor of **0.75**, meaning it will resize when the number of entries exceeds 75% of its current capacity.
- It has an initial capacity of **16** by default, which can be specified in the constructor, and when the threshold is reached (*based on the load factor*), it typically increases by doubling the capacity (*$newCapacity = 2 \times oldCapacity$*).
- It does not maintain insertion order, so elements are stored in an unpredictable order that may change over time due to rehashing.
- It does not allow duplicate elements and silently ignores attempts to add duplicates, without throwing an exception.
- It allows the storage of one null element, adhering to the uniqueness constraint.
- It is not thread-safe by default, meaning simultaneous modifications by multiple threads can lead to data corruption; however, it can be made thread-safe by using ***Collections.synchronizedSet()***.

QUESTION

What is a
LinkedHashSet in
the Collections
Framework? 

Explanation



EXPLANATION

LinkedHashSet

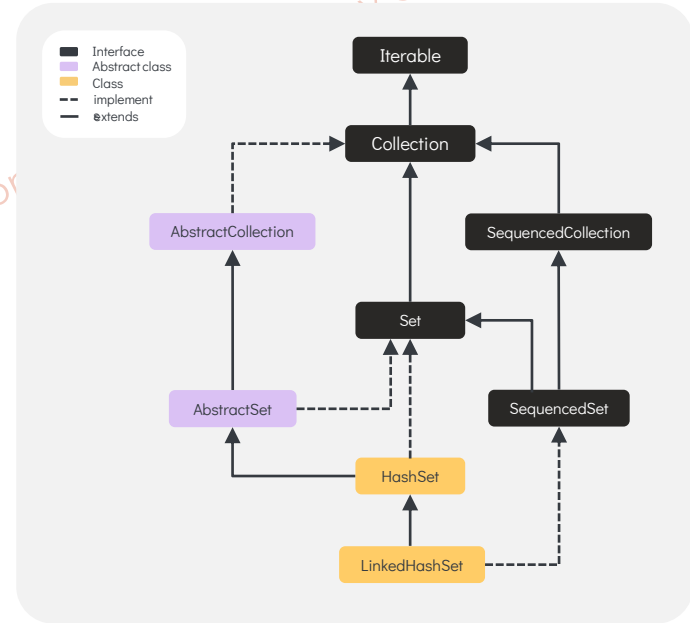
A **LinkedHashSet** is an implementation of the **Set** interface that stores unique elements and maintains insertion order and is part of the *java.util* package.

From: The Ultimate Animated Java Interview Questions Course on Udemy © Peerless Geek

EXPLANATION

LinkedHashSet

A **LinkedHashSet** is an implementation of the **Set** interface that stores unique elements and maintains insertion order and is part of the *java.util* package.

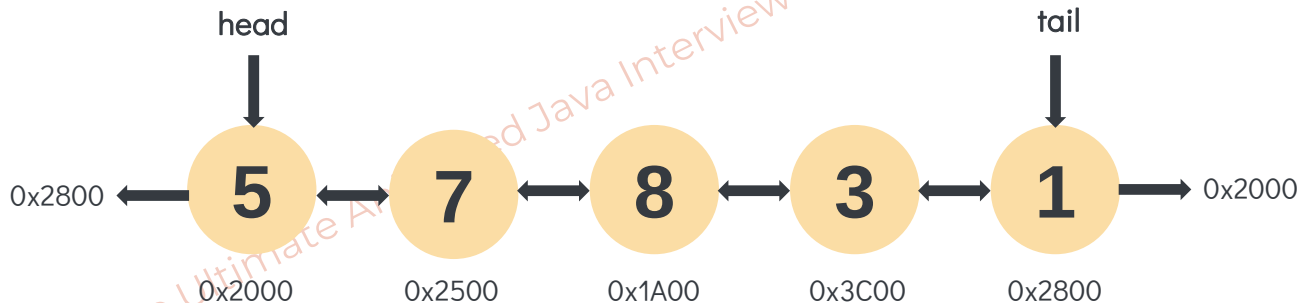


EXPLANATION

LinkedHashSet

A **LinkedHashSet** is an implementation of the **Set** interface that stores unique elements and maintains insertion order and is part of the *java.util* package.

Implementation



Doubly Linked List

0	
1	1, PRESENT
2	
3	3, PRESENT
4	
5	5, PRESENT
6	
7	7, PRESENT
8	8, PRESENT
9	

(hash table)

EXPLANATION

LinkedHashSet

A **LinkedHashSet** is an implementation of the **Set** interface that stores unique elements and maintains insertion order and is part of the *java.util* package.

- It extends **HashSet** for efficient hashing and implements **SequencedSet** to maintain predictable iteration order based on element insertion.
- It uses a combination of a **hash table** (from *HashMap*) for storage, and a **doubly-linked list** to maintain the insertion order of elements.
- It performs like **HashSet** but with slightly higher overhead due to the linked list, while `add()`, `remove()`, and `contains()` still run in $O(1)$ time.
- It has a default initial capacity of **16** and a load factor of **0.75**, similar to **HashSet**.
- It disallows duplicate elements, silently ignoring duplicates while allowing one null element, similar to **HashSet**.
- It is not thread-safe by default, meaning simultaneous modifications by multiple threads can lead to data corruption; however, it can be made thread-safe by using `Collections.synchronizedSet()`.

QUESTION

What is the **TreeSet** class, and how does it store elements? 

Explanation



EXPLANATION

TreeSet

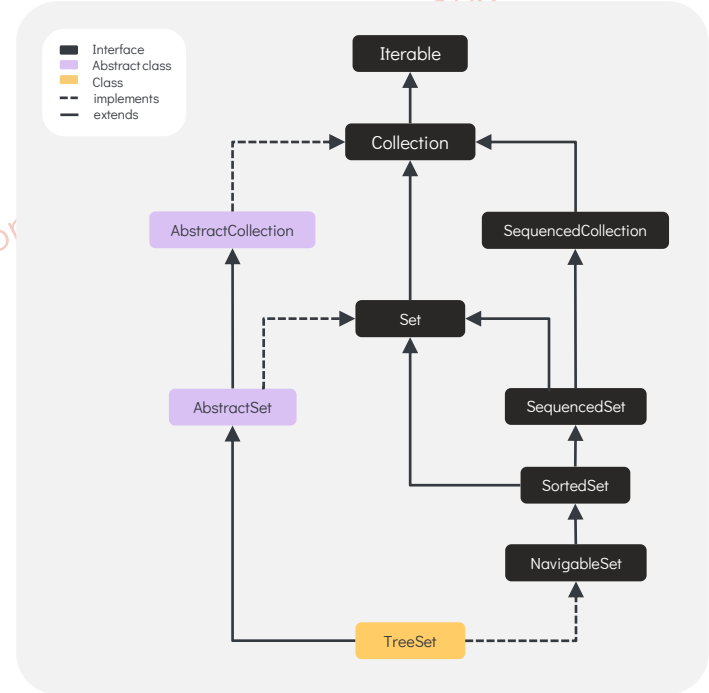
A **TreeSet** is a collection implementation of the **Set** interface that stores unique elements while maintaining their natural ordering or a custom comparator-defined order and is part of the *java.util* package.

From: The Ultimate Animated Java Interview Questions Course on Udemy © Peerless Geek

EXPLANATION

TreeSet

A **TreeSet** is a collection implementation of the **Set** interface that stores unique elements while maintaining their natural ordering or a custom comparator-defined order and is part of the *java.util* package.

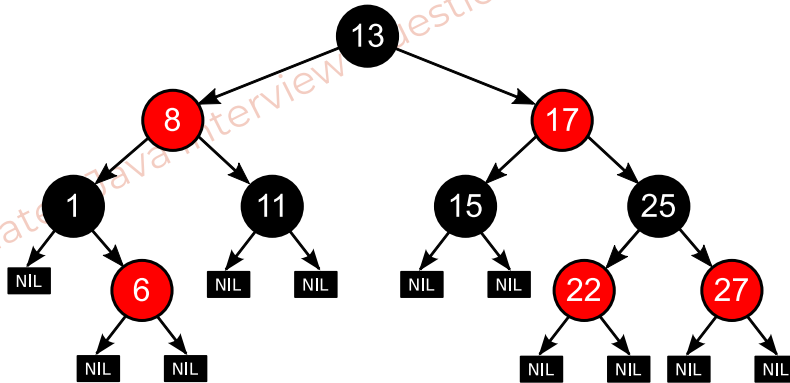


EXPLANATION

TreeSet

A **TreeSet** is a collection implementation of the **Set** interface that stores unique elements while maintaining their natural ordering or a custom comparator-defined order and is part of the *java.util* package.

Implementation



(Red Black Tree)

EXPLANATION

TreeSet

A **TreeSet** is a collection implementation of the **Set** interface that stores unique elements while maintaining their natural ordering or a custom comparator-defined order and is part of the *java.util* package.

- It extends **AbstractSet** for core set operations and implements **NavigableSet** to offer sorted, navigable functionality using a self-balancing tree structure.
- It uses a **Red-Black tree** (a self-balancing BST) to store elements in sorted order, ensuring $O(\log n)$ time complexity for add, remove, and contains operations.
- It stores elements in sorted order, either in their natural order (as defined by *Comparable*) or by a custom comparator (as specified in the constructor).
- It doesn't allow duplicate elements and ignores attempts to add duplicates.
- It does not allow null elements, and adding null results in a **NullPointerException** since null cannot be compared in a sorted set.
- It is not thread-safe by default, meaning concurrent modifications by multiple threads can lead to data corruption. However, it can be made thread-safe using *Collections.synchronizedSortedSet()*.

QUESTION

What is the
difference
between HashSet
and TreeSet? 

Explanation



EXPLANATION

HashSet	TreeSet
HashSet internally uses a hash table (backed by a <i>HashMap</i>) data structure to store elements.	TreeSet internally uses a red-black tree (backed by a <i>TreeMap</i>) data structure to store elements.
It doesn't guarantee a specific order of elements, as they are stored based on their hash codes.	It stores elements in natural order (as defined by <i>Comparable</i>) or according to a custom comparator if provided.
It offers constant-time performance ($O(1)$) for basic operations like add() , remove() , and contains() , assuming there are no collisions.	It provides logarithmic time performance ($O(\log n)$) for operations like add() , remove() , and contains() since it uses a tree structure.
It does not require elements to implement <i>Comparable</i> or provide a <i>Comparator</i> ; relies on equals() and hashCode() for hashing and equality checks.	It uses the compareTo() method for natural ordering or a provided <i>Comparator</i> , so elements must implement Comparable or be supplied with a <i>Comparator</i> for ordering.
It allows storing null elements, but only one null element is permitted.	It does not allow null elements; attempting to add null results in a NullPointerException , as comparisons are required to order elements, and null cannot be compared.
It generally offers better performance than <i>TreeSet</i> due to its constant-time operations, making it ideal when ordering is not required.	It is slower compared to <i>HashSet</i> because it maintains sorting, which requires logarithmic time.

QUESTION

Can you add a
null element into
a **TreeSet** or
HashSet? 

Explanation



EXPLANATION

Null in Java Sets: A Comparison of TreeSet and HashSet

A **TreeSet** does not allow null elements because it uses a **Red-Black Tree** (*inherited from **TreeMap***) data structure, which requires elements to be comparable for sorting. Since null is not comparable, attempting to add null will result in a ***NullPointerException***.

From: The Ultimate Animated Java Interview Questions Course on Udemy © Peerless Geek

QUESTION

What is EnumSet?



Explanation



From: The Java Interview Questions Course on Udemy © Peerless Geek

EXPLANATION

EnumSet

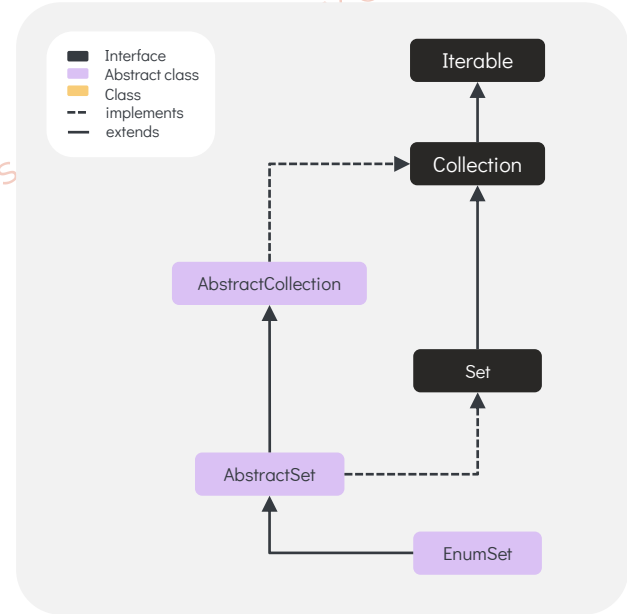
An **EnumSet** is a specialized Set designed for use with enum types, allowing only elements from a single enum type in the collection.

From: The Ultimate Animated Java Interview Questions Course on Udemy © Peerless Geek

EXPLANATION

EnumSet

An **EnumSet** is a specialized Set designed for use with enum types, allowing only elements from a single enum type in the collection.



EXPLANATION

EnumSet

An **EnumSet** is a specialized Set designed for use with enum types, allowing only elements from a single enum type in the collection.

- It provides a concrete implementation of the **Set** interface by extending **AbstractSet**, which offers a skeletal implementation of essential Set functionality.
- It uses bit vectors for efficient storage, allowing constant-time operations ($O(1)$) for add, remove, and contains.
- Its capacity is **fixed** at the number of enum constants in the specified enum type, as the internal representation of an EnumSet uses a bit vector where each bit represents one of the enum constants.
- It maintains the order of its elements based on the order of the enum constants as they are defined in the enum class.
- It does not allow duplicate elements; attempts to add duplicate enum constants will silently fail without throwing an exception.
- It does not permit null elements; attempting to add a null element will throw a **NullPointerException**.
- It is not synchronized and therefore not thread-safe; external synchronization, such as using **Collections.synchronizedSet()**, is required if multiple threads access it concurrently and at least one modifies it.

QUESTION

What does the
**Collections.
emptySet()
return?** 

Explanation



From: The Java Interview Questions Course on Udemy © Peerless Geek

EXPLANATION

`Collections.emptySet()`

Collections.emptySet() returns an *immutable, generic, singleton instance* of an empty set that cannot be modified, making it useful for representing a set that contains no elements without the overhead of creating a new set instance.

From: The Ultimate Animated Java Interview Questions Course on Udemy © Peerless Geek

QUESTION

What is the difference between **List** and **Set**? 

Explanation



From: The Python Interview Questions Course on Udemy © Peerless Geek

EXPLANATION

List	Set
List is an ordered collection of elements that allows duplicates.	Set is an unordered collection of elements that does not allow duplicates.
It maintains the insertion order, meaning elements are stored and accessed in the same sequence in which they were added.	It does not guarantee insertion order, as it is unordered, although some implementations like LinkedHashSet maintain insertion order.
It provides access to elements by their index using methods like get(int index) .	It does not provide indexed access; you must iterate through the set to access elements.
It allows duplicates, storing all instances of the same element.	It does not allow duplicates; attempts to add a duplicate element are ignored, keeping the original.
It can have multiple null elements, except for CopyOnWriteArrayList , which does not allow any null elements.	It can hold a single null element, with exceptions in implementations such as TreeSet , EnumSet , and CopyOnWriteArraySet .
It provides faster access to elements by index, especially for ArrayList , where access time is constant ($O(1)$).	It offers $O(1)$ performance for searching, adding, and removing elements with hash-based implementations like HashSet , as they do not preserve order.
It is typically used when the order matters, duplicates are allowed, and index-based access is required.	It is suitable when a collection of unique elements is needed, and the order does not matter.

QUESTION

What is a **Queue** in the **Collections Framework**, and what are its various implementations? ✎

Explanation



From: The Java Interview Questions Course on Udemy © Peerless Geek

EXPLANATION

Queue

The **Queue** is an interface that represents a collection designed for holding elements prior to processing, extends the **Collection** interface, inherits its methods while introducing additional methods for managing elements in a **FIFO** manner, and is part of the *java.util* package.

From: The Ultimate Animated Java Interview Questions Course on Udemy © Peerless Geek

EXPLANATION

Queue

The **Queue** is an interface that represents a collection designed for holding elements prior to processing, extends the **Collection** interface, inherits its methods while introducing additional methods for managing elements in a **FIFO** manner, and is part of the `java.util` package.



EXPLANATION

Understanding Java Queue Methods: Hierarchical Breakdown

Category	Methods	
Inherited from Iterable	Abstract	
	Default	forEach (Consumer<? super T> action)
Inherited from Collection	Abstract	addAll (Collection<? extends E> c), clear (), contains (Object o), containsAll (Collection<?> c), equals (Object o), hashCode (), isEmpty (), iterator (), remove (Object o), removeAll (Collection<?> c), retainAll (Collection<?> c), size (), toArray (), toArray(T[] a)
	Default	parallelStream (), removeIf (Predicate<? super E> filter), splititerator () , stream (), toArray(IntFunction<T[]> generator)
Defined in Queue	Abstract	add (E e), element (), offer (E e), peek (), poll (), remove ()
	Default	



EXPLANATION

Queue

The **Queue** is an interface that represents a collection designed for holding elements prior to processing, extends the **Collection** interface, inherits its methods while introducing additional methods for managing elements in a **FIFO** manner, and is part of the *java.util* package.

- It follows **FIFO order**, where elements are added at the end (*tail*) and removed from the front (*head*) of the queue.
- It provides a way to store and process elements in the order in which they are added.
- It offers two methods to insert elements: **add()** (*throws exception on failure*) and **offer()** (*returns false on failure*); two methods to retrieve and remove elements: **remove()** (*throws exception if empty*) and **poll()** (*returns null if empty*); and two methods to peek at the head without removing it: **element()** (*throws exception if empty*) and **peek()** (*returns null if empty*).
- It allows duplicate elements, meaning the same value can appear multiple times in the queue.
- It generally does not permit null (*except LinkedList*), as null is used to signal an empty queue in methods like **poll()** and **peek()**.
- Its common implementation classes include **LinkedList**, **PriorityQueue**, and **ArrayDeque**.

QUESTION

What is a Priority Queue?

Explanation



EXPLANATION

PriorityQueue

A **PriorityQueue** is a *special type* of queue that orders elements by natural ordering or a provided Comparator, ensuring higher-priority elements are served before others, unlike a normal *FIFO* queue.

From: The Ultimate Animated Java Interview Questions Course on Udemy © Peerless Geek

EXPLANATION

PriorityQueue

A **PriorityQueue** is a *special type* of queue that orders elements by natural ordering or a provided Comparator, ensuring higher-priority elements are served before others, unlike a normal *FIFO* queue.

Highest priority Lowest priority
↓ ↓
[25, 2, 50, 3, 10]

(Priority Queue (*Integers*))

EXPLANATION

PriorityQueue

A **PriorityQueue** is a *special type* of queue that orders elements by natural ordering or a provided Comparator, ensuring higher-priority elements are served before others, unlike a normal *FIFO* queue.

Lowest priority



Highest priority



[Tiger, Zebra, Rabbit, Elephant, Lion]

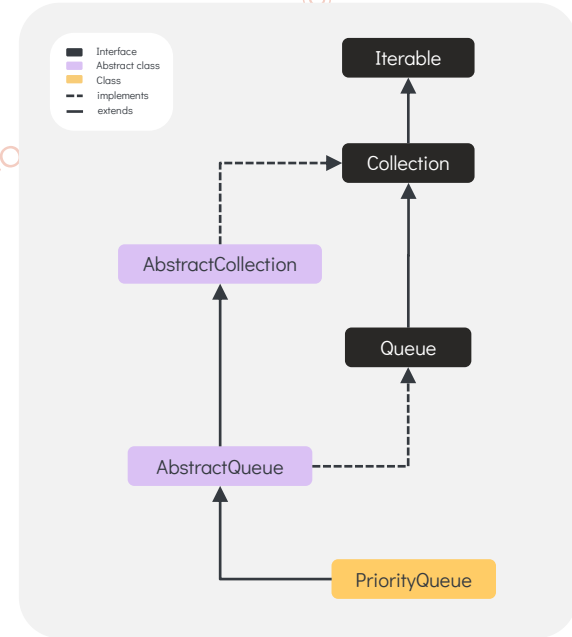
(Priority Queue (Strings))

From: The Ultimate Android Java Interview Questions Course on Udemy © Peerless Geek

EXPLANATION

PriorityQueue

A **PriorityQueue** is a *special type* of queue that orders elements by natural ordering or a provided Comparator, ensuring higher-priority elements are served before others, unlike a normal *FIFO* queue.

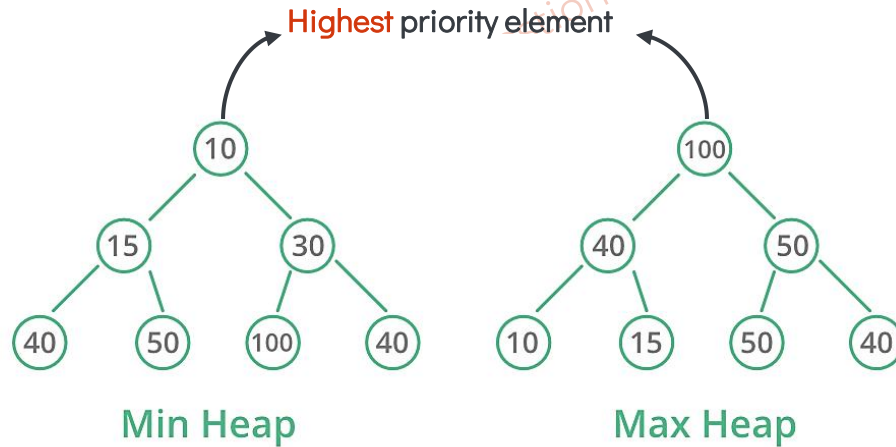


EXPLANATION

PriorityQueue

A **PriorityQueue** is a *special type* of queue that orders elements by natural ordering or a provided Comparator, ensuring higher-priority elements are served before others, unlike a normal *FIFO* queue.

Implementation



From: The Ultimate

EXPLANATION

PriorityQueue

A **PriorityQueue** is a *special type* of queue that orders elements by natural ordering or a provided Comparator, ensuring higher-priority elements are served before others, unlike a normal *FIFO* queue.

- It extends **AbstractQueue**, providing a skeletal implementation of Queue methods to simplify priority-based ordering logic.
- It internally uses a **binary heap** to efficiently maintain elements for quick retrieval of the smallest element, based on natural ordering or a custom comparator, but the elements are not necessarily sorted.
- It has an initial capacity of **11** by default, which can be specified in the constructor, and when capacity is reached, it typically increases by 50% (**$newCapacity = 1.5 \times oldCapacity$**).
- It provides $O(\log n)$ time for insertion and removal, and $O(1)$ for peeking at the head, as it's always the root of the heap.
- It allows duplicate elements and stores them based on their priority order.
- It does not permit null elements, as they cannot be compared, and inserting a null will result in a ***NullPointerException***.
- It is not thread-safe by default but can be made thread-safe using ***Collections.synchronizedCollection()***, or you can use **PriorityBlockingQueue** for thread safety.

QUESTION

What is the difference between **PriorityQueue** and **TreeSet**? 

Explanation



EXPLANATION

PriorityQueue	TreeSet
PriorityQueue uses a binary heap data structure to store its elements, ensuring efficient retrieval of the highest or lowest priority element.	TreeSet uses a Red-Black Tree data structure to store its elements, ensuring efficient operations.
It doesn't guarantee a specific order, except that the highest-priority element (based on natural ordering or a comparator) will be at the front.	It stores elements in a sorted order using a Red-Black tree , which organizes them based on their natural ordering or a provided Comparator.
It offers $O(\log n)$ time for insertion and removal of elements, and $O(1)$ for peeking at the head of the queue because the head is always the root of the heap.	It offers $O(\log n)$ time for access, insertion, and removal of elements.
It allows duplicate elements and stores them based on priority.	It disallows duplicates and maintains a collection of unique elements.
It does not permit null elements because null would have ambiguous priority, and any attempt to insert a null element will result in a NullPointerException .	It does not allow null elements, and adding null results in a NullPointerException since null cannot be compared in a sorted set.
It is ideal for scenarios where you need to prioritize elements and access the highest priority element quickly (<i>like task scheduling or algorithms like Dijkstra's shortest path</i>).	It is suitable when you need a collection of unique, sorted elements, and often need to perform operations like subset retrieval.

QUESTION

What is the difference between Queue and Stack? 

Explanation



EXPLANATION

Queue	Stack
Queue is an interface that extends Collection, introduces FIFO element management methods, and is part of the <i>java.util</i> package.	Stack is a class that extends Vector, operates in a LIFO manner, and is part of the <i>java.util</i> package.
It represents a collection designed to hold elements for processing in a First-In-First-Out (FIFO) order, meaning the first element added is the first to be removed.	It represents a collection where elements are managed in a Last-In-First-Out (LIFO) order, meaning the last element added is the first to be removed.
It offers common methods like offer() to add an element, poll() to retrieve and remove the head, and peek() to retrieve without removing the head.	It provides common methods like push() to add an element to the top, pop() to remove and return the top element, and peek() to view the top element without removing it.
It is ideal for scenarios where the order of processing matters, like task scheduling, breadth-first search, or managing resources in a sequential manner.	It is suitable for scenarios where you need to reverse operations or backtrack, such as undo functionality, depth-first search, or recursive method call handling.

QUESTION

What is a
Deque and its
implementing
classes? 

Explanation



EXPLANATION

Deque

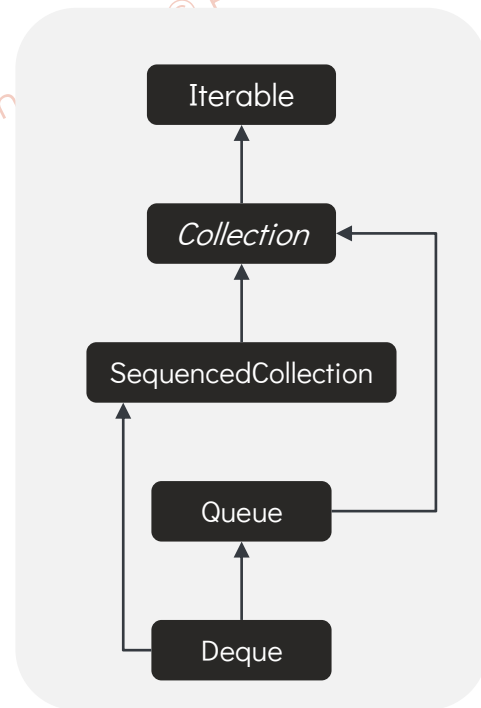
The **Deque** interface represents a double-ended queue, allowing elements to be added or removed from both ends, functioning as both a **Queue** (*FIFO*) and a **Stack** (*LIFO*), and is part of the *java.util* package.

From: The Ultimate Animated Java Interview Questions Course on Udemy © Peerless Geek

EXPLANATION

Understanding Java Deque Methods: Hierarchical Breakdown

Category	Methods	
Inherited from Iterable	Abstract	
	Default	<code>forEach(Consumer<? super T> action)</code>
Inherited from Collection	Abstract	<code>clear()</code> , <code>containsAll(Collection<?> c)</code> , <code>equals(Object o)</code> , <code>hashCode()</code> , <code>isEmpty()</code> , <code>iterator()</code> , <code>removeAll(Collection<?> c)</code> , <code>retainAll(Collection<?> c)</code> , <code>toArray()</code> , <code>toArray(T[] a)</code>
	Default	<code>parallelStream()</code> , <code>removeIf(Predicate<? super E> filter)</code> , <code>splitterator()</code> , <code>stream()</code> , <code>toArray(IntFunction<T[]> generator)</code>
Inherited from SequencedCollection	Abstract	
	Default	
Inherited from Queue	Abstract	<code>add(E e)</code> , <code>element()</code> , <code>offer(E e)</code> , <code>peek()</code> , <code>poll()</code> , <code>remove()</code>
	Default	
Declared in Deque	Abstract	<code>add(E e)</code> , <code>addAll(Collection<? extends E> c)</code> , <code>addFirst(E e)</code> , <code>addLast(E e)</code> , <code>contains(Object o)</code> , <code>descendingIterator()</code> , <code>element()</code> , <code>getFirst()</code> , <code>getLast()</code> , <code>iterator()</code> , <code>offer(E e)</code> , <code>offerFirst(E e)</code> , <code>offerLast(E e)</code> , <code>peek()</code> , <code>peekFirst()</code> , <code>peekLast()</code> , <code>poll()</code> , <code>pollFirst()</code> , <code>pollLast()</code> , <code>pop()</code> , <code>push(E e)</code> , <code>remove()</code> , <code>remove(Object o)</code> , <code>removeFirst()</code> , <code>removeFirstOccurrence(Object o)</code> , <code>removeLast()</code> , <code>removeLastOccurrence(Object o)</code> , <code>reversed()</code> , <code>size()</code>
	Default	<code>reversed()</code>



EXPLANATION

Deque

The **Deque** interface represents a double-ended queue, allowing elements to be added or removed from both ends, functioning as both a **Queue** (*FIFO*) and a **Stack** (*LIFO*), and is part of the *java.util* package.

- It **extends** the **Queue** interface and inherits its methods, allowing FIFO (*First-In-First-Out*) behavior and also introduces additional methods to support LIFO (*Last-In-First-Out*) behavior as well.
- It inherits queue-specific methods from **Queue**, such as **offer()** (*inserts element at the tail*), **poll()** (*retrieves and removes element from the head*), and **peek()** (*retrieves element from the head*).
- It defines stack-specific methods such as **push()** (*inserts element at the head*) and **pop()** (*retrieves and removes element from the head*).
- It inherits methods from **SequencedCollection** for manipulating first and last elements, such as **addFirst()** (*adds element at the head*), **addLast()** (*adds element at the tail*), **removeFirst()** (*removes element from the head*), **removeLast()** (*removes element from the tail*), **getFirst()** (*retrieves element from the head*), and **getLast()** (*retrieves element from the tail*).
- It defines methods such as **offerFirst()** (*adds element at the head*), **offerLast()** (*adds element at the tail*), **pollFirst()** (*removes and retrieves element from the head*), **pollLast()** (*removes and retrieves element from the tail*), **peekFirst()** (*retrieves element from the head*), and **peekLast()** (*retrieves element from the tail*).
- Its implementations include **ArrayDeque** and **LinkedList**.

QUESTION

What is **Map** in the Collections Framework, and what are its **various implementations**? 

Explanation



EXPLANATION

Map

The **Map** is an interface that represents a collection of key-value pairs (*entries*), where each key is unique, and it maps to a specific value and is part of the *java.util* package.

From: The Ultimate Animated Java Interview Questions Course on Udemy © Peerless Geek

EXPLANATION

Understanding Java Map Methods: Hierarchical Breakdown

Category	Methods	
Declared in Map	Abstract	clear() , containsKey (Object key), containsValue (Object value), entrySet() , equals (Object o), get (Object key), hashCode() , isEmpty() , keySet() , put (K key, V value), putAll (Map<? extends K,? extends V> m), remove (Object key), size() , values()
	Default	compute (K key, BiFunction<? super K,? super V,? extends V> remappingFunction), computeIfAbsent (K key, Function<? super K,? extends V> mappingFunction), computeIfPresent (K key, BiFunction<? super K,? super V,? extends V> remappingFunction), forEach (BiConsumer<? super K,? super V> action), getOrDefault (Object key, V defaultValue), merge (K key, V value, BiFunction<? super V,? super V,? extends V> remappingFunction), putIfAbsent (K key, V value), remove (Object key, Object value), replace (K key, V value), replace (K key, V oldValue, V newValue), replaceAll (BiFunction<? super K,? super V,? extends V> function)
	Static	copyOf (Map<? extends K,? extends V> map), entry (K k, V v), of() , of (K k1, V v1), of (K k1, V v1, K k2, V v2), ofEntries (Map.Entry<? extends K,? extends V>... entries)

Map

EXPLANATION

Map

The **Map** is an interface that represents a collection of key-value pairs (*entries*), where each key is unique, and it maps to a specific value and is part of the *java.util* package.

- It does not extend the **Collection** interface and therefore does not inherit any of its methods.
- It allows storing, accessing, and managing *key-value* pairs where each key is unique.
- It defines several important methods like **put()** to insert key-value pairs, **get()** to retrieve the value associated with a key, **remove()** to delete a key-value pair, and **containsKey()** to check if a specific key exists in the map.
- It provides three collection views: **keySet()** for the keys, **values()** for the values, and **entrySet()** for the key-value mappings.
- It defines static methods like **Map.of()**, which is for creating a new map from a set of key-value pairs, for creating small, unmodifiable maps, and **Map.copyOf()** for creating an unmodifiable copy of an existing map.
- Its common implementation classes include **HashMap**, **LinkedHashMap**, **TreeMap** and **Hashtable**.

QUESTION

Why Map doesn't extend the Collection Interface?

Explanation



From: The Complete Java Interview Questions Course on Udemy © Peerless Geek

EXPLANATION

Decoding the Separation of Map and Collection

The **Map** interface *doesn't extend* **Collection** because it handles key-value pairs rather than individual elements, making Collection methods like `add()` incompatible and leading to inefficient operations or confusing semantics; however, it provides views of its data as Collection through methods like `values()`, `keySet()`, and `entrySet()`.

From: The Ultimate Animated Java Interview Questions Course on Udemy © Peerless Geek

QUESTION

What is **Hashing**
and How Does a
Hash Function Work
in a **hash table**? 

Explanation



From: The Complete Java Interview Questions Course on Udemy © Peerless Geek

EXPLANATION

Hash Function

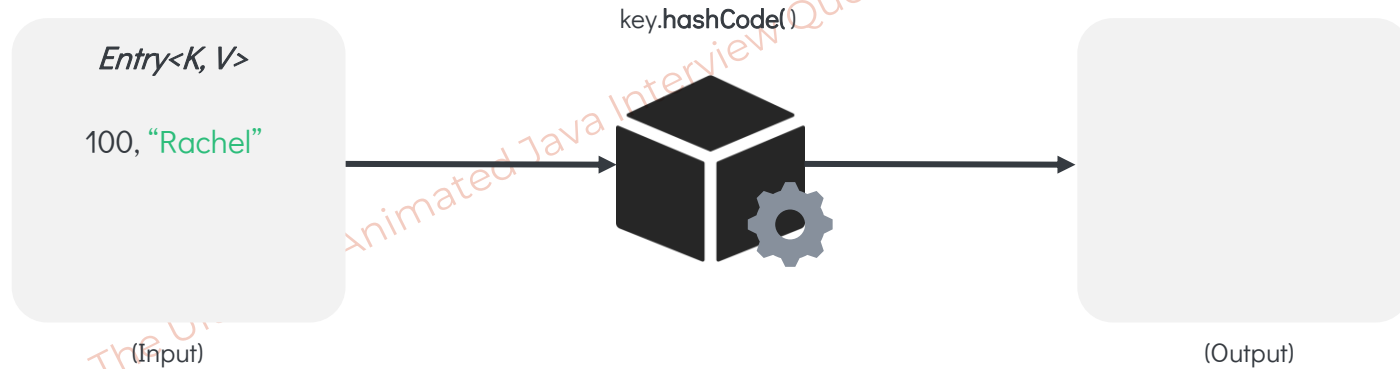
A **hash function** is a mathematical algorithm or method that takes an input (*often an object or data*) and returns a fixed-size value, called a **hash code**.

From: The Ultimate Animated Java Interview Questions Course on Udemy © Peerless Geek

EXPLANATION

Hash Function

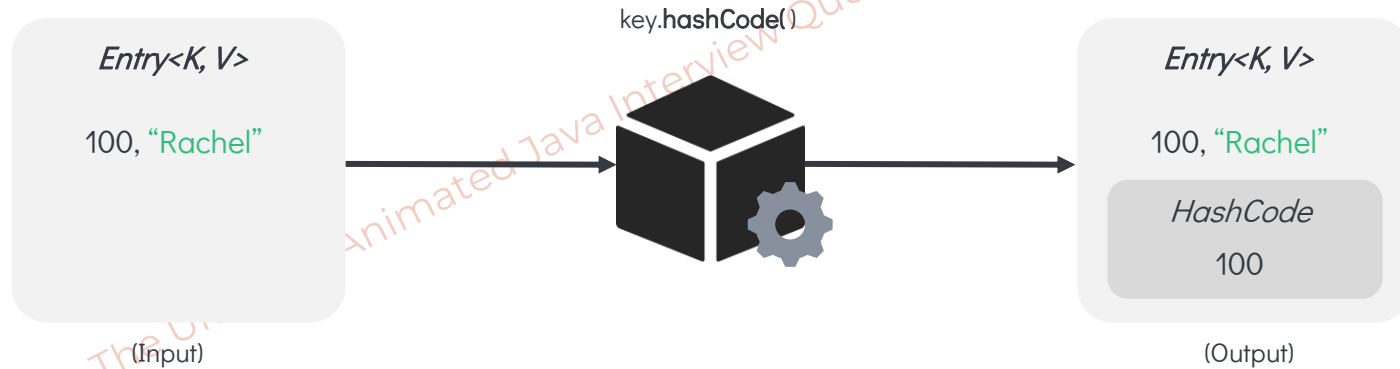
A **hash function** is a mathematical algorithm or method that takes an input (*often an object or data*) and returns a fixed-size value, called a **hash code**.



EXPLANATION

Hash Function

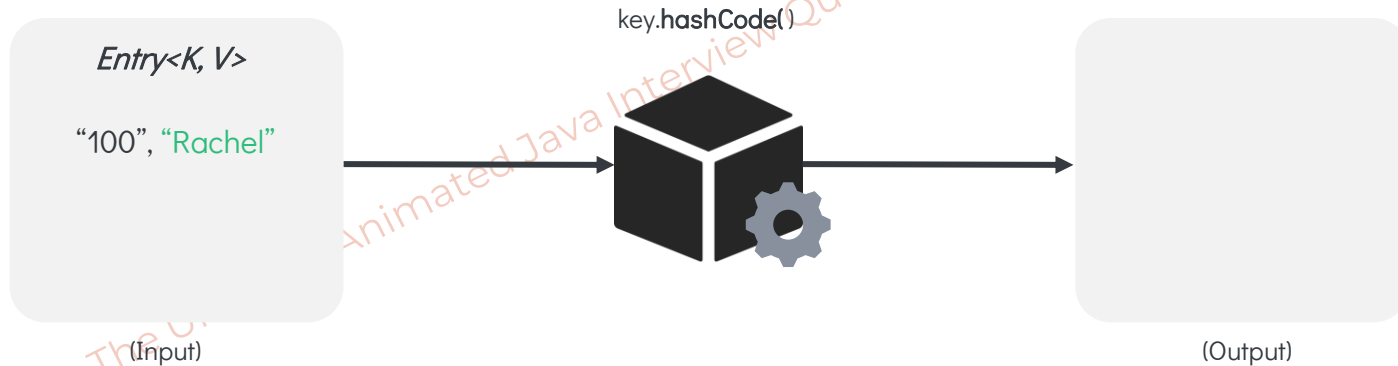
A **hash function** is a mathematical algorithm or method that takes an input (*often an object or data*) and returns a fixed-size value, called a **hash code**.



EXPLANATION

Hash Function

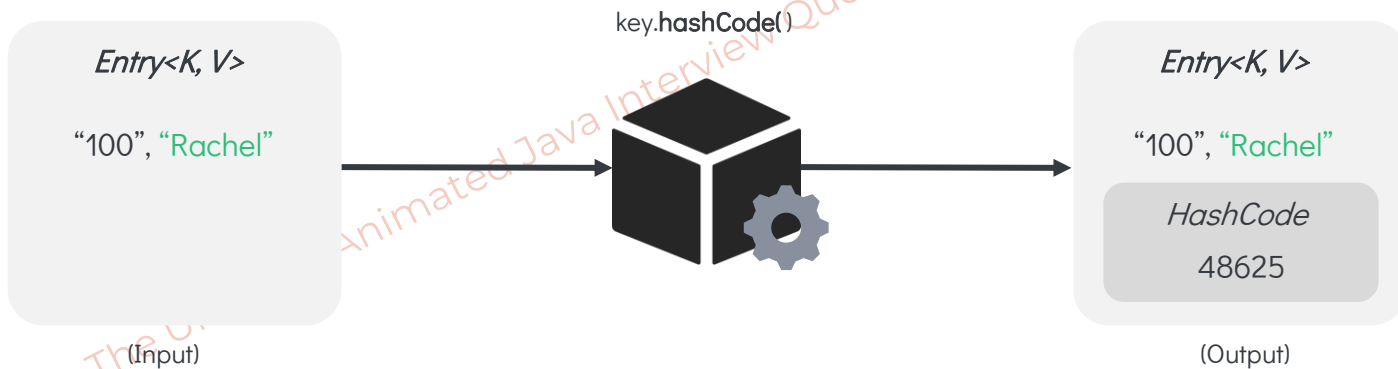
A **hash function** is a mathematical algorithm or method that takes an input (*often an object or data*) and returns a fixed-size value, called a **hash code**.



EXPLANATION

Hash Function

A **hash function** is a mathematical algorithm or method that takes an input (*often an object or data*) and returns a fixed-size value, called a **hash code**.



EXPLANATION

Hashing

Hashing is a technique used to convert an object (typically a key) into a fixed-size integer, which represents its location in a **hash table** (*a collection of key-value pairs* and each key-value pair is known as an **Entry**).

From: The Ultimate Animated Java Interview Questions Course on Udemy © Peerless Geek

EXPLANATION

Hashing

Hashing is a technique used to convert an object (typically a key) into a fixed-size integer, which represents its location in a **hash table** (*a collection of key-value pairs* and each key-value pair is known as an **Entry**).

0	
1	
2	
3	
4	
5	
6	
7	
8	
9	

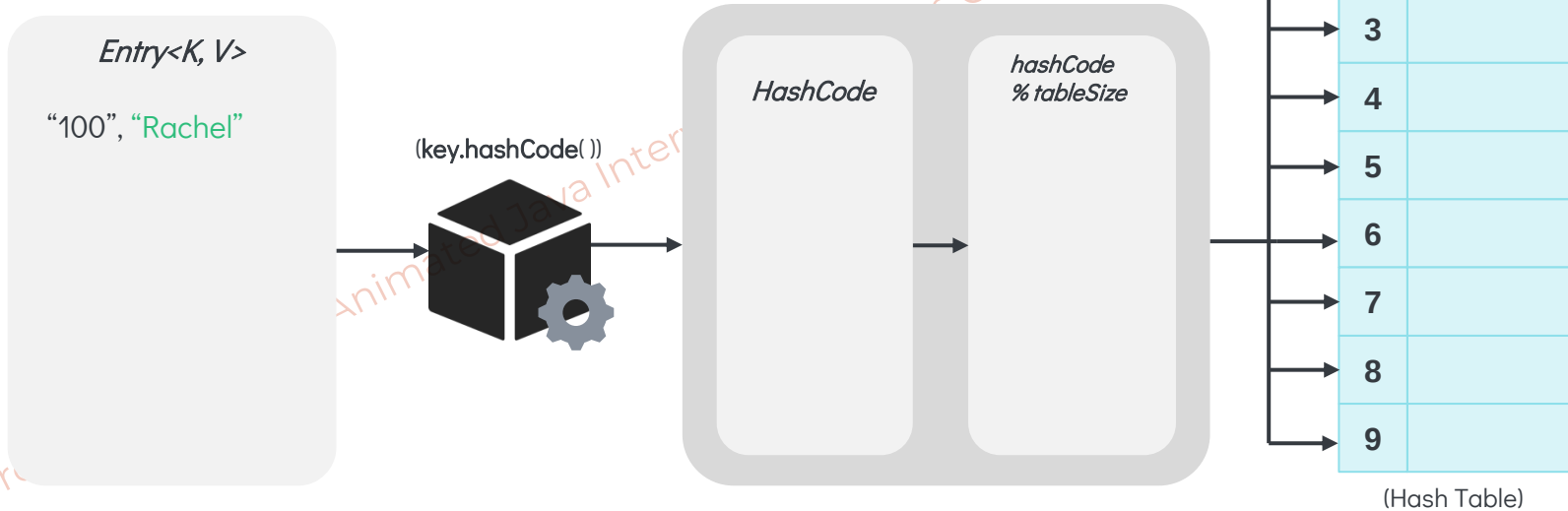
(Hash Table)

From: The Ultimate Animated Java Interview Questions Course on Udemy © Peerless Geek

EXPLANATION

Hashing

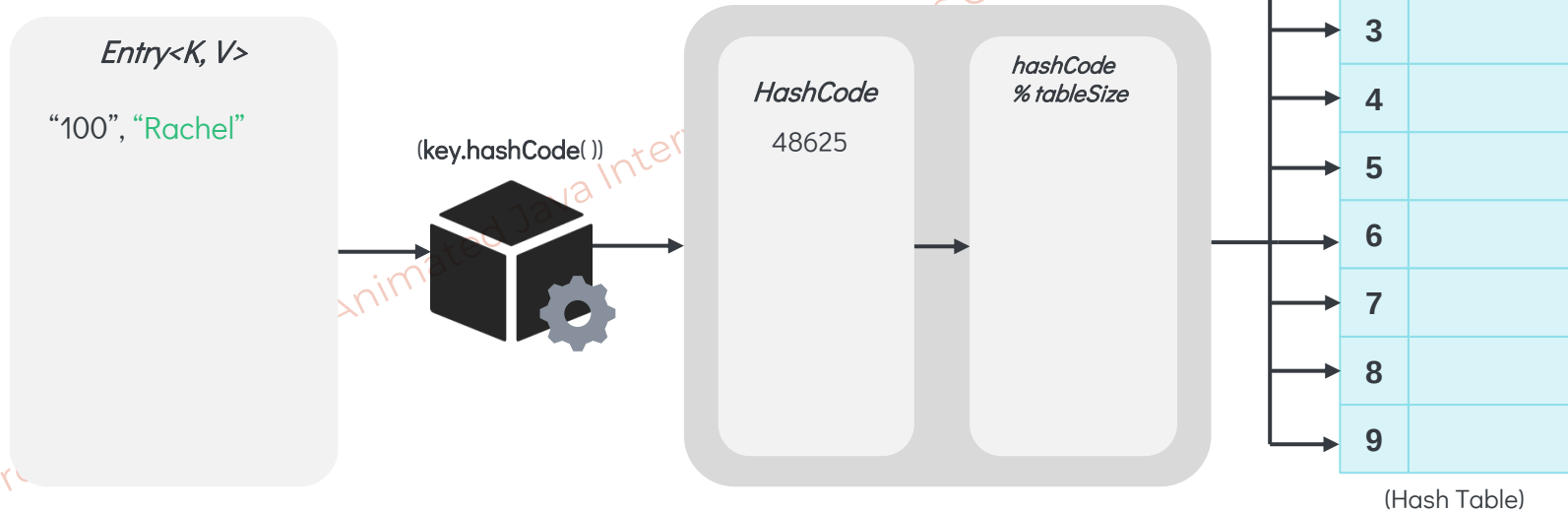
Hashing is a technique used to convert an object (typically a key) into a fixed-size integer, which represents its location in a **hash table** (a collection of key-value pairs and each key-value pair is known as an **Entry**).



EXPLANATION

Hashing

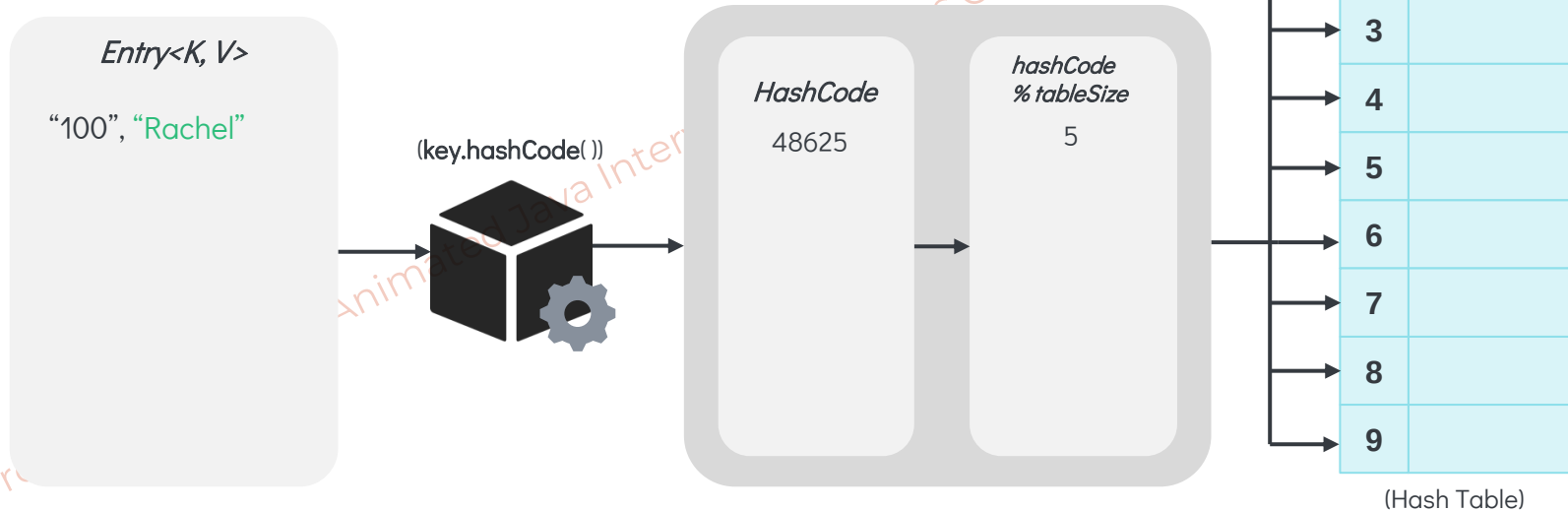
Hashing is a technique used to convert an object (typically a key) into a fixed-size integer, which represents its location in a **hash table** (a collection of key-value pairs and each key-value pair is known as an **Entry**).



EXPLANATION

Hashing

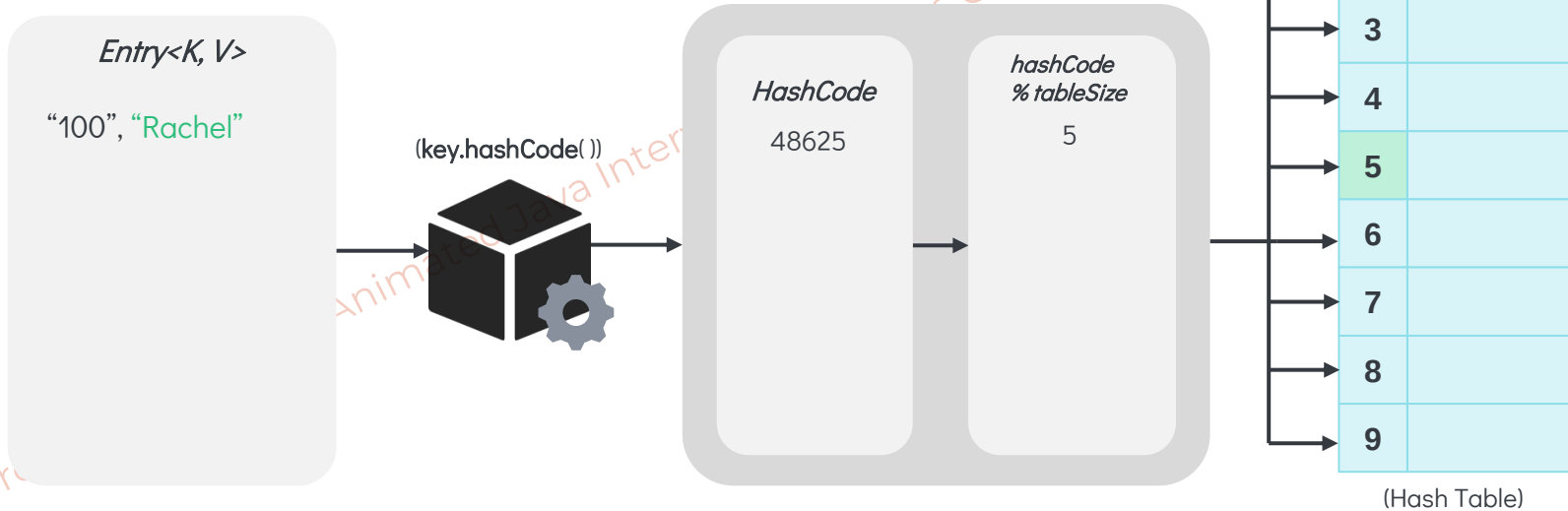
Hashing is a technique used to convert an object (typically a key) into a fixed-size integer, which represents its location in a **hash table** (a collection of key-value pairs and each key-value pair is known as an **Entry**).



EXPLANATION

Hashing

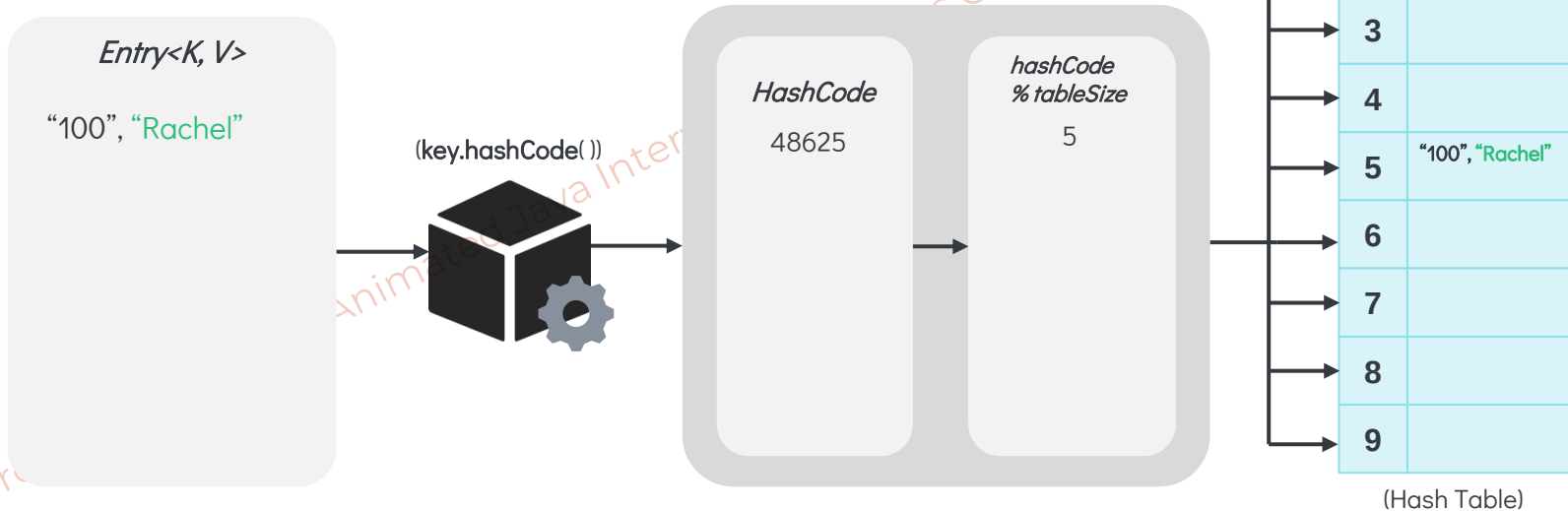
Hashing is a technique used to convert an object (typically a key) into a fixed-size integer, which represents its location in a **hash table** (*a collection of key-value pairs* and each key-value pair is known as an **Entry**).



EXPLANATION

Hashing

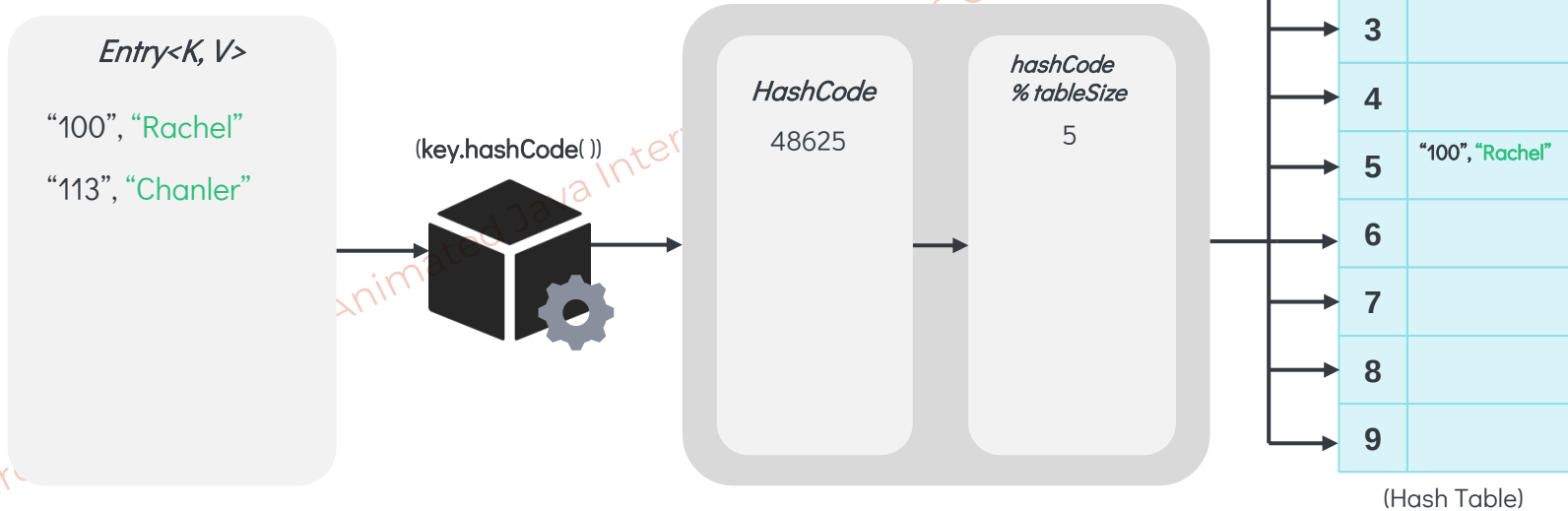
Hashing is a technique used to convert an object (typically a key) into a fixed-size integer, which represents its location in a **hash table** (a collection of key-value pairs and each key-value pair is known as an **Entry**).



EXPLANATION

Hashing

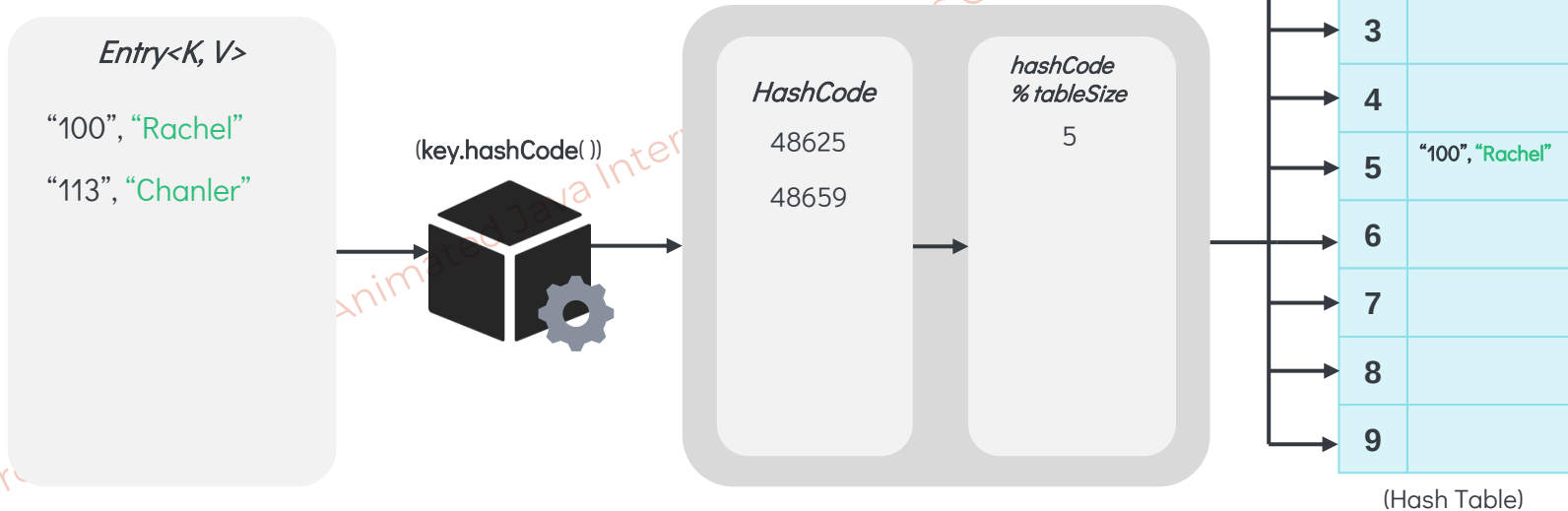
Hashing is a technique used to convert an object (typically a key) into a fixed-size integer, which represents its location in a **hash table** (*a collection of key-value pairs* and each key-value pair is known as an **Entry**).



EXPLANATION

Hashing

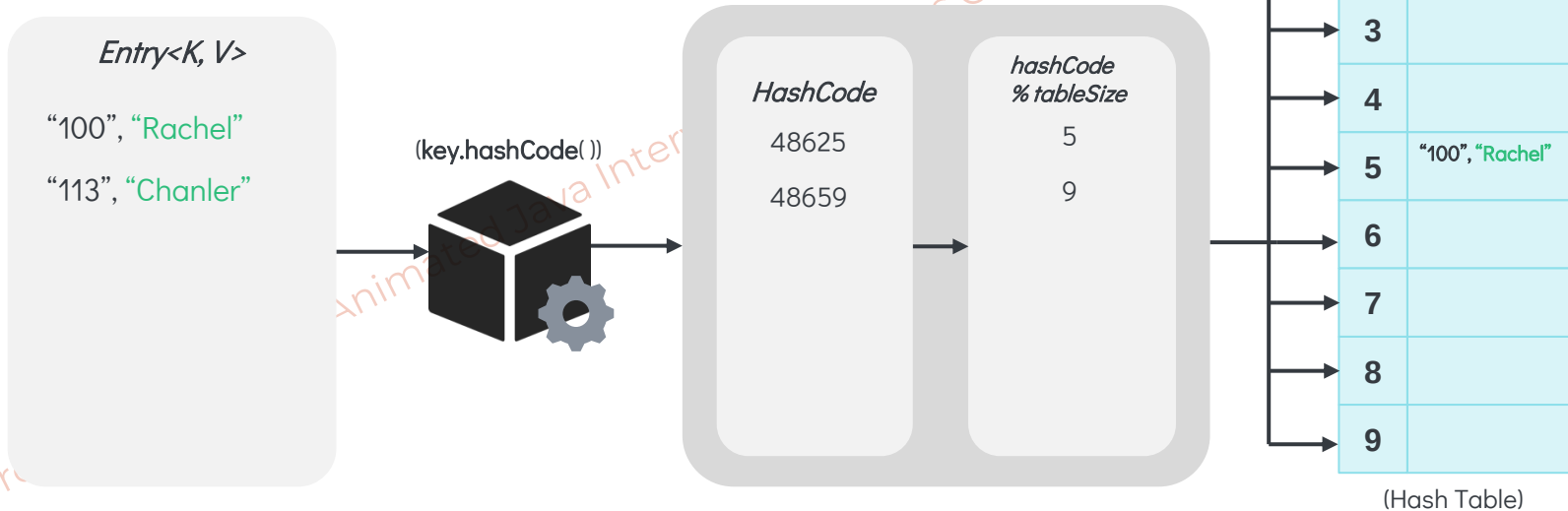
Hashing is a technique used to convert an object (typically a key) into a fixed-size integer, which represents its location in a **hash table** (a collection of key-value pairs and each key-value pair is known as an **Entry**).



EXPLANATION

Hashing

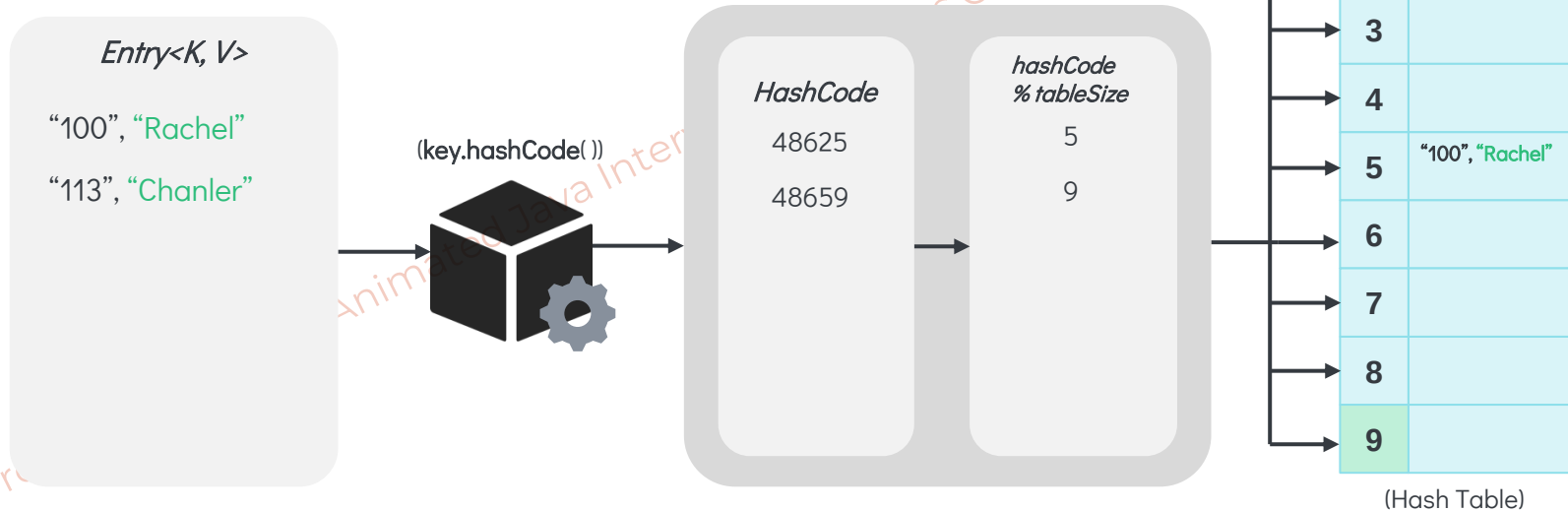
Hashing is a technique used to convert an object (typically a key) into a fixed-size integer, which represents its location in a **hash table** (a collection of key-value pairs and each key-value pair is known as an **Entry**).



EXPLANATION

Hashing

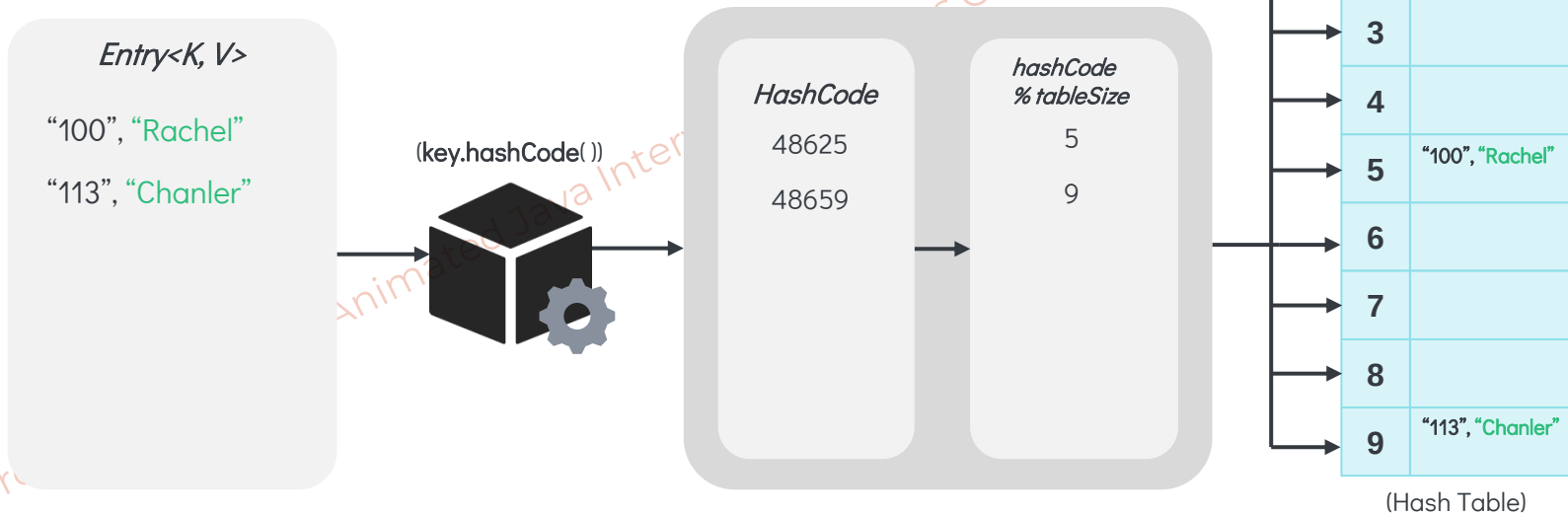
Hashing is a technique used to convert an object (typically a key) into a fixed-size integer, which represents its location in a **hash table** (a collection of key-value pairs and each key-value pair is known as an **Entry**).



EXPLANATION

Hashing

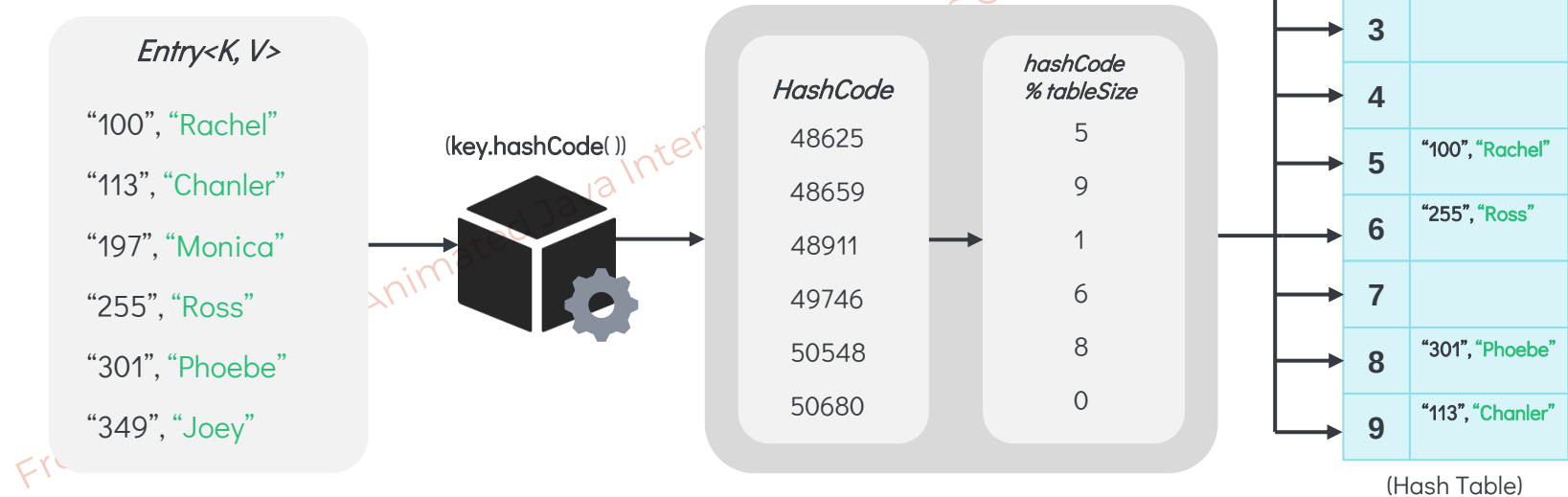
Hashing is a technique used to convert an object (typically a key) into a fixed-size integer, which represents its location in a **hash table** (a collection of key-value pairs and each key-value pair is known as an **Entry**).



EXPLANATION

Hashing

Hashing is a technique used to convert an object (typically a key) into a fixed-size integer, which represents its location in a **hash table** (a collection of key-value pairs and each key-value pair is known as an **Entry**).



QUESTION

What is a **hash collision** in a **Hashtable**, and how is it handled? 

Explanation



EXPLANATION

Hash Collision

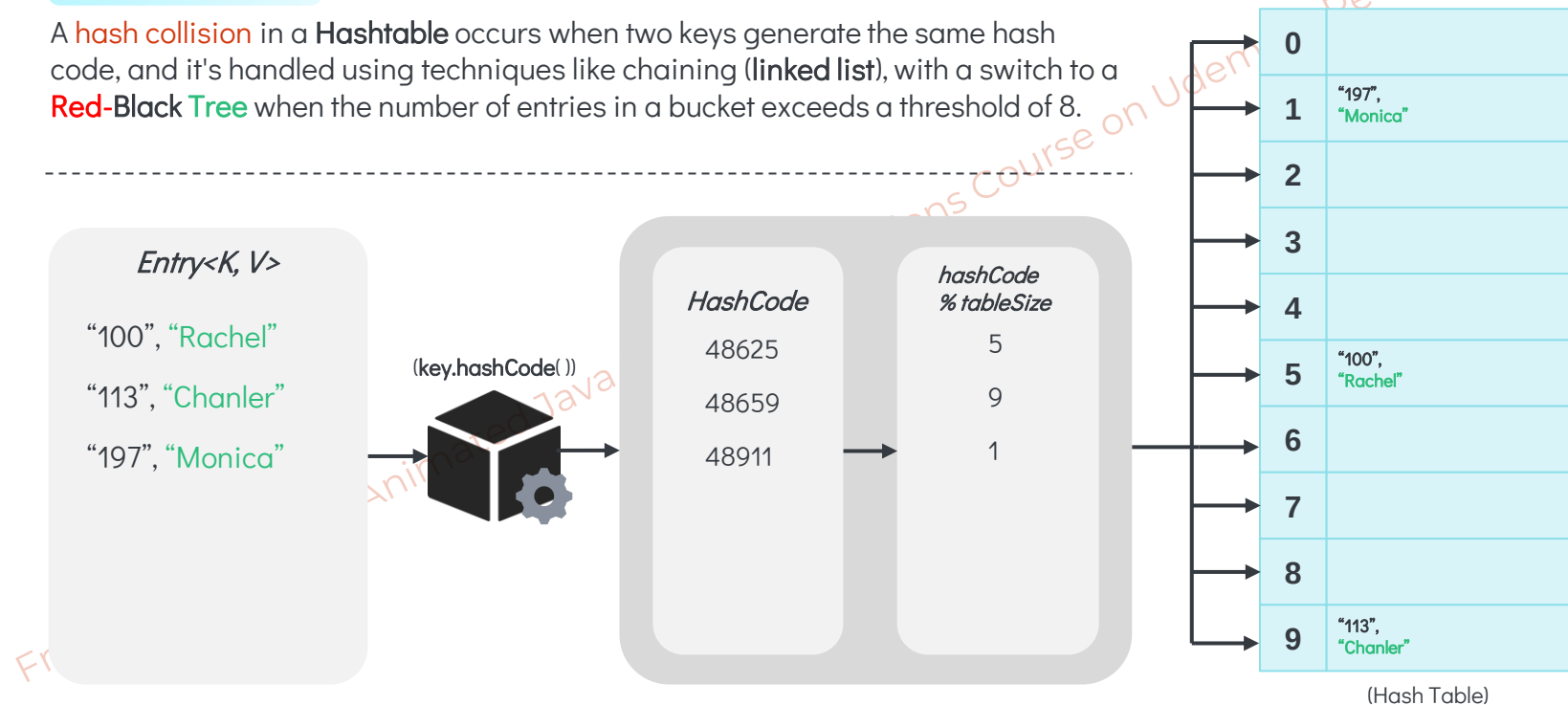
A **hash collision** in a **Hashtable** occurs when two keys generate the same **hash code**, and it's handled using techniques like chaining (**linked list**), with a switch to a **Red-Black Tree** when the number of entries in a bucket exceeds a threshold of 8.

From: The Ultimate Animated Java Interview Questions Course on Udemy @ Peerless Geek

EXPLANATION

Hash Collision

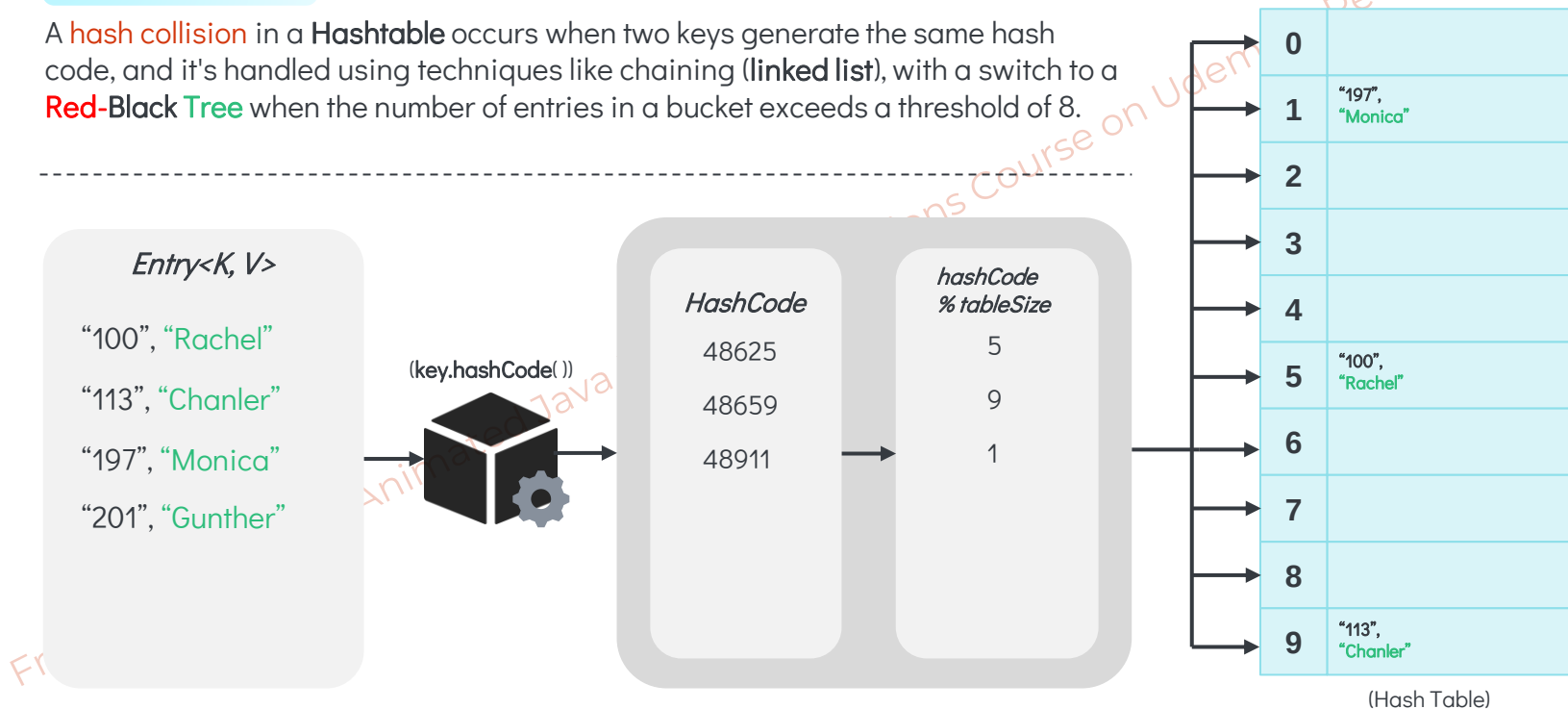
A **hash collision** in a **Hashtable** occurs when two keys generate the same hash code, and it's handled using techniques like chaining (**linked list**), with a switch to a **Red-Black Tree** when the number of entries in a bucket exceeds a threshold of 8.



EXPLANATION

Hash Collision

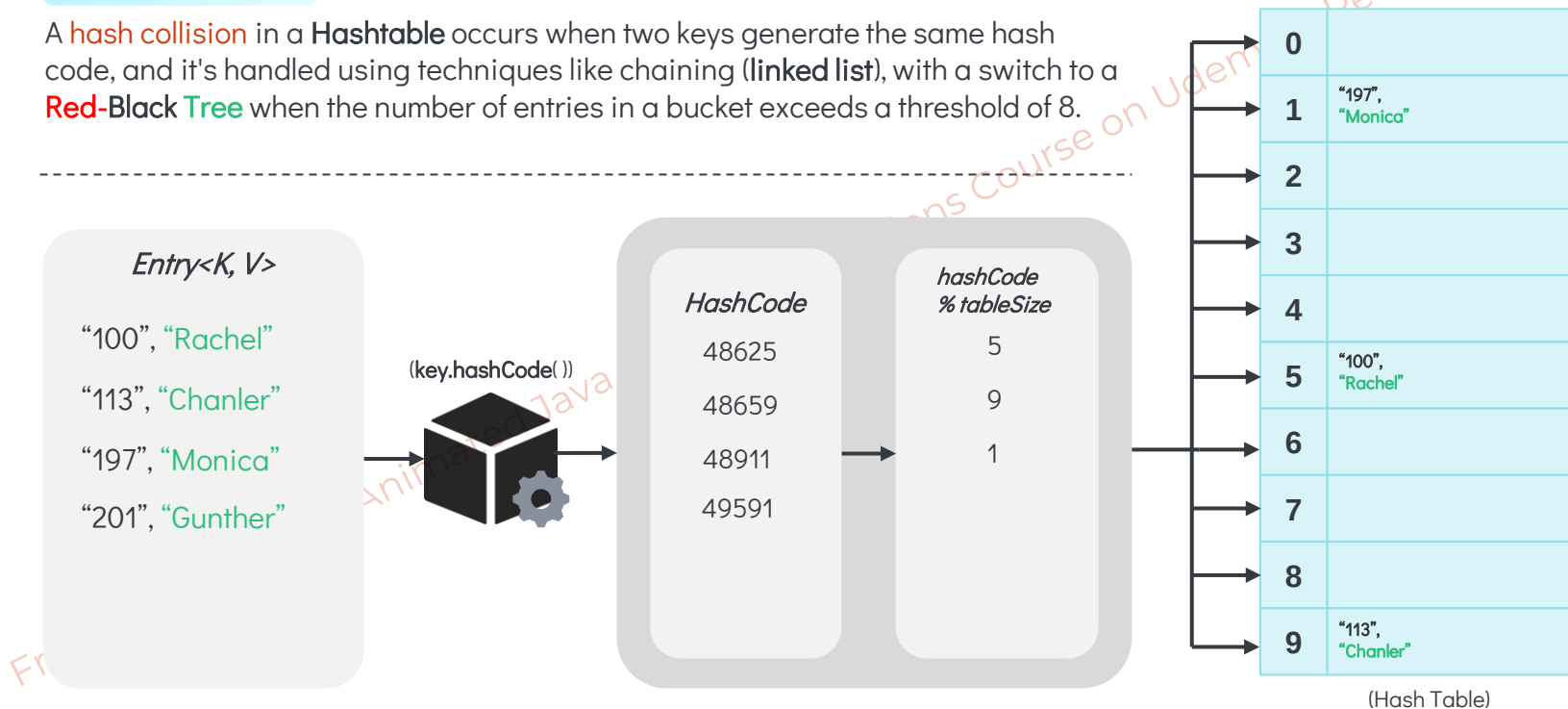
A **hash collision** in a **Hashtable** occurs when two keys generate the same hash code, and it's handled using techniques like chaining (**linked list**), with a switch to a **Red-Black Tree** when the number of entries in a bucket exceeds a threshold of 8.



EXPLANATION

Hash Collision

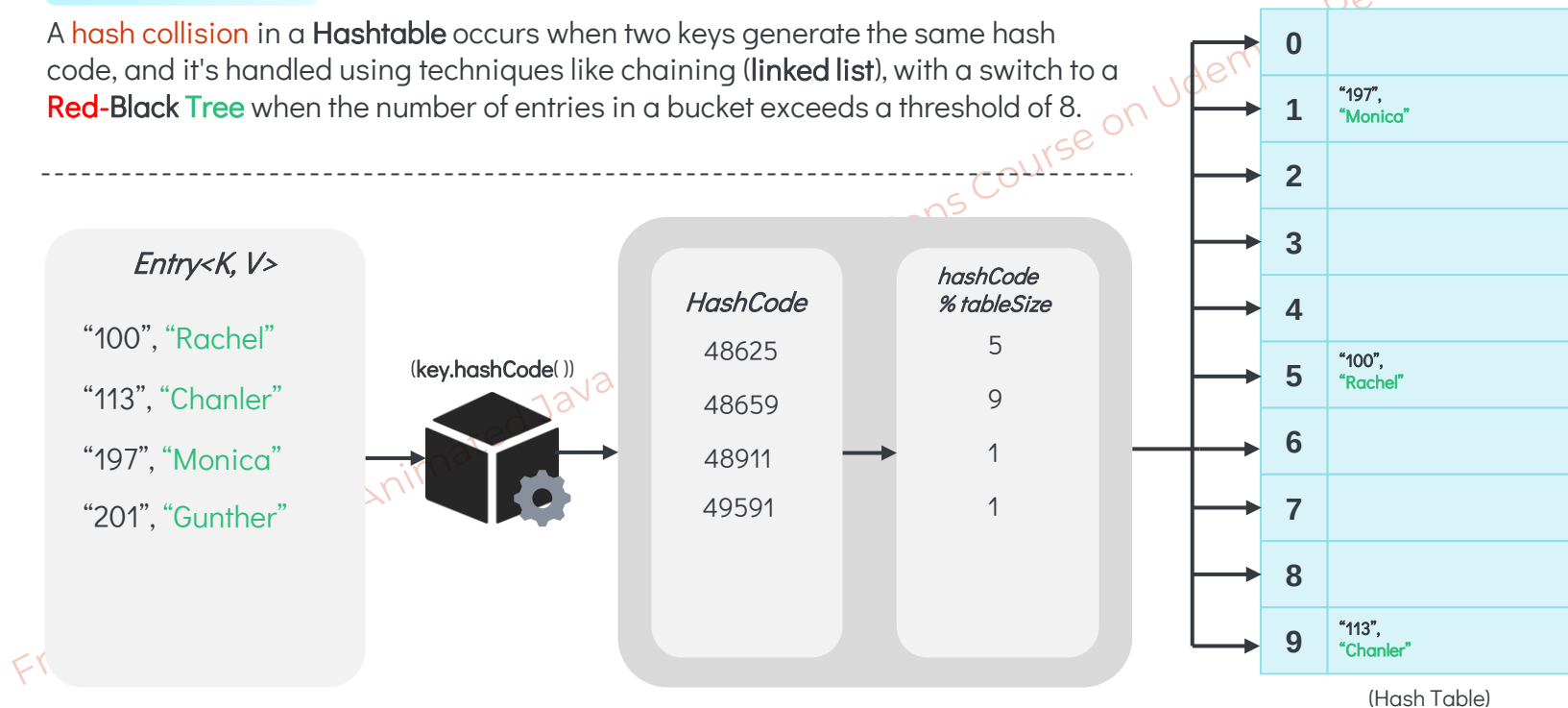
A **hash collision** in a **Hashtable** occurs when two keys generate the same hash code, and it's handled using techniques like chaining (**linked list**), with a switch to a **Red-Black Tree** when the number of entries in a bucket exceeds a threshold of 8.



EXPLANATION

Hash Collision

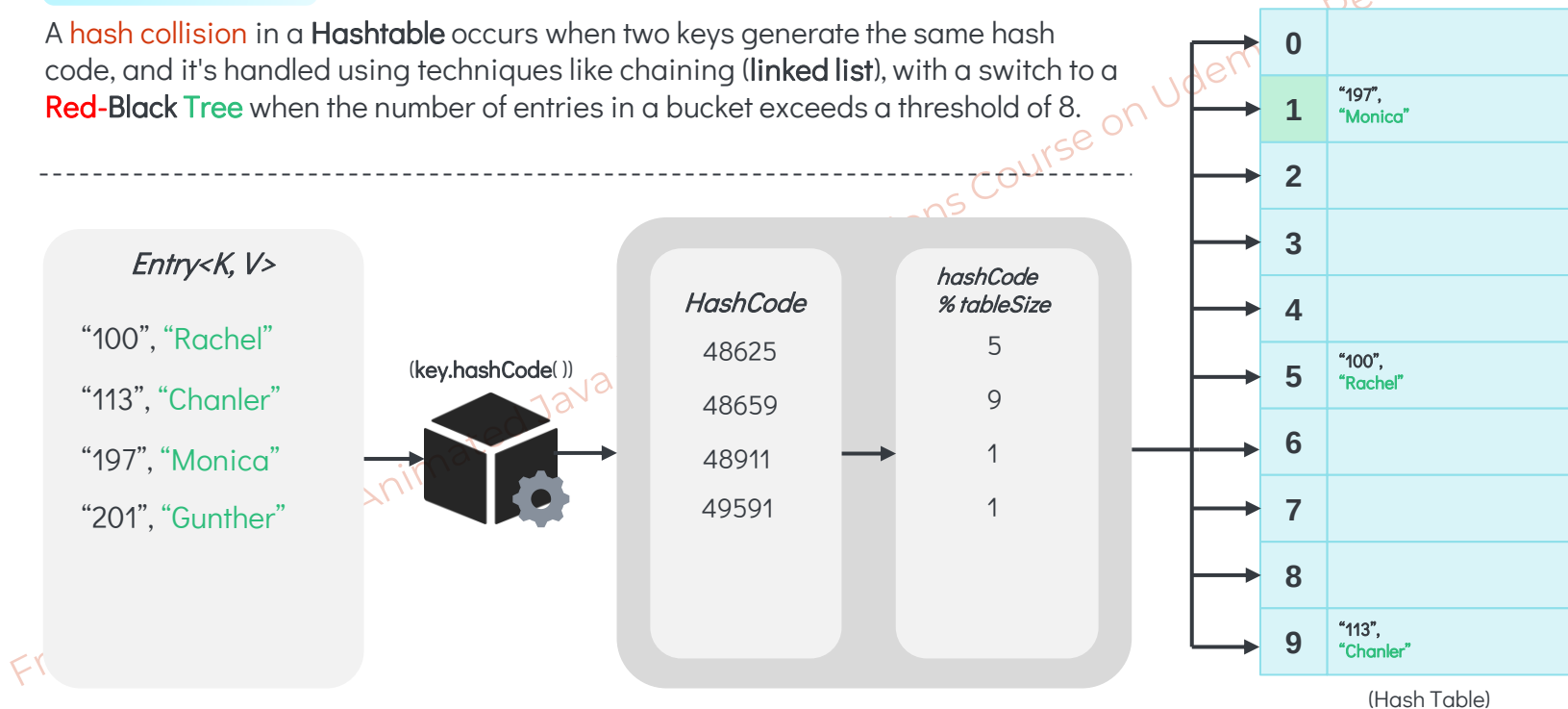
A **hash collision** in a **Hashtable** occurs when two keys generate the same hash code, and it's handled using techniques like chaining (**linked list**), with a switch to a **Red-Black Tree** when the number of entries in a bucket exceeds a threshold of 8.



EXPLANATION

Hash Collision

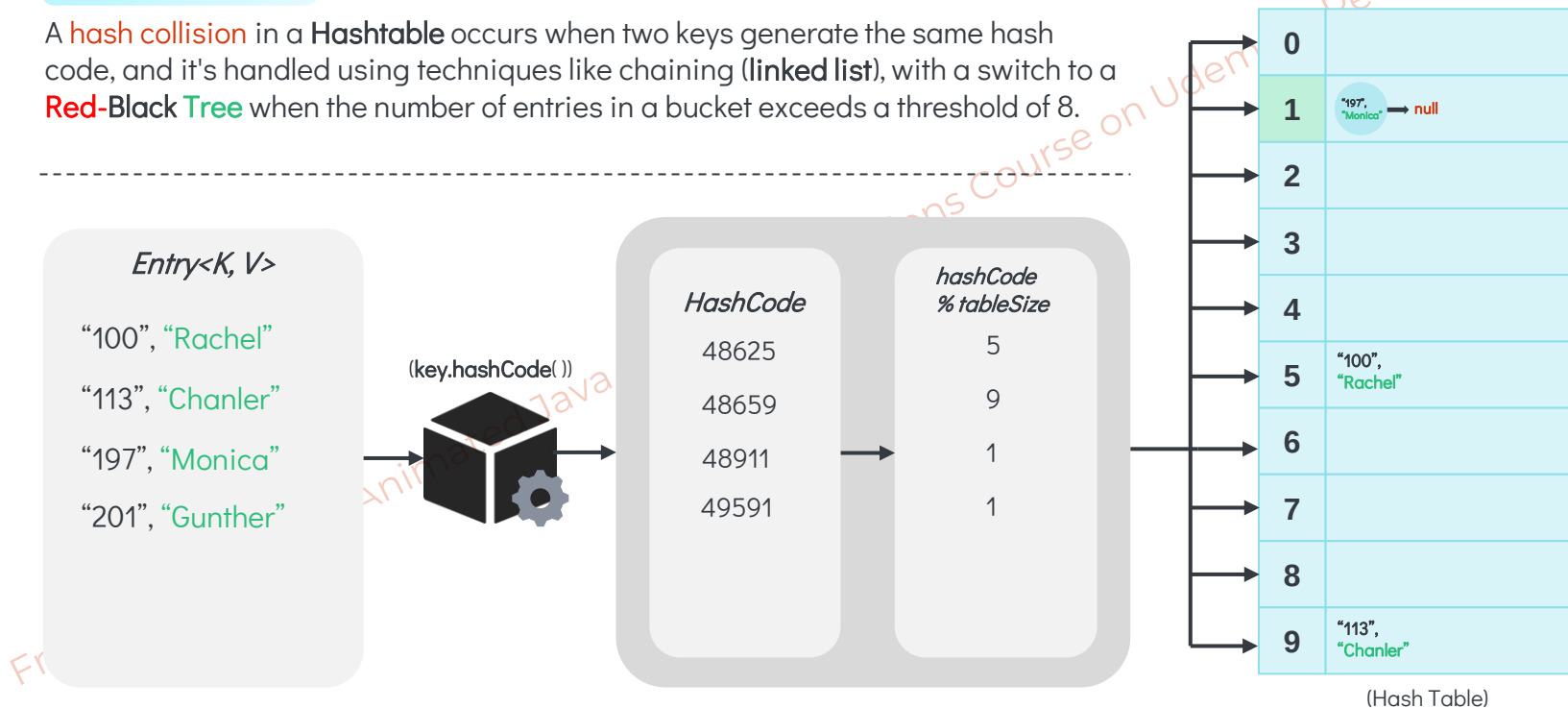
A **hash collision** in a **Hashtable** occurs when two keys generate the same hash code, and it's handled using techniques like chaining (**linked list**), with a switch to a **Red-Black Tree** when the number of entries in a bucket exceeds a threshold of 8.



EXPLANATION

Hash Collision

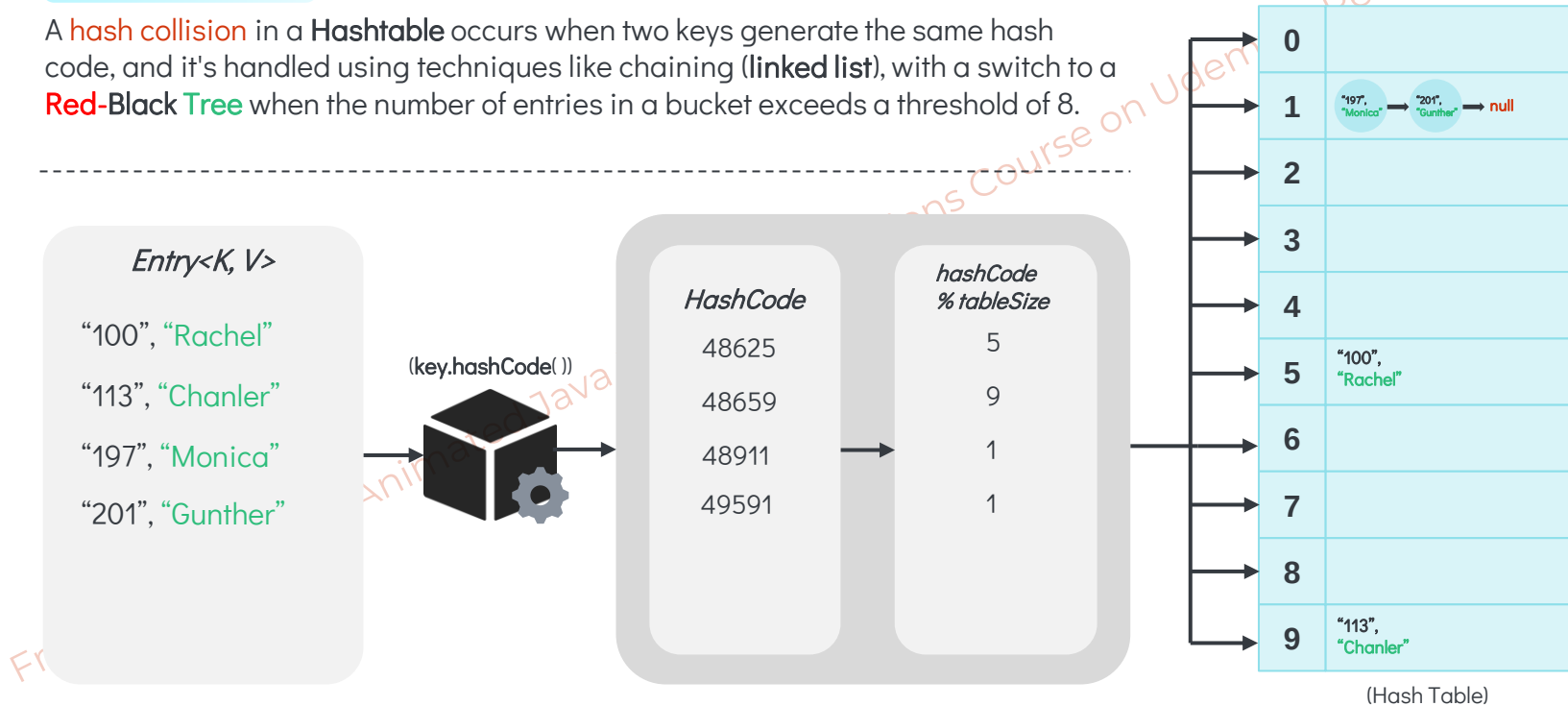
A **hash collision** in a **Hashtable** occurs when two keys generate the same hash code, and it's handled using techniques like chaining (**linked list**), with a switch to a **Red-Black Tree** when the number of entries in a bucket exceeds a threshold of 8.



EXPLANATION

Hash Collision

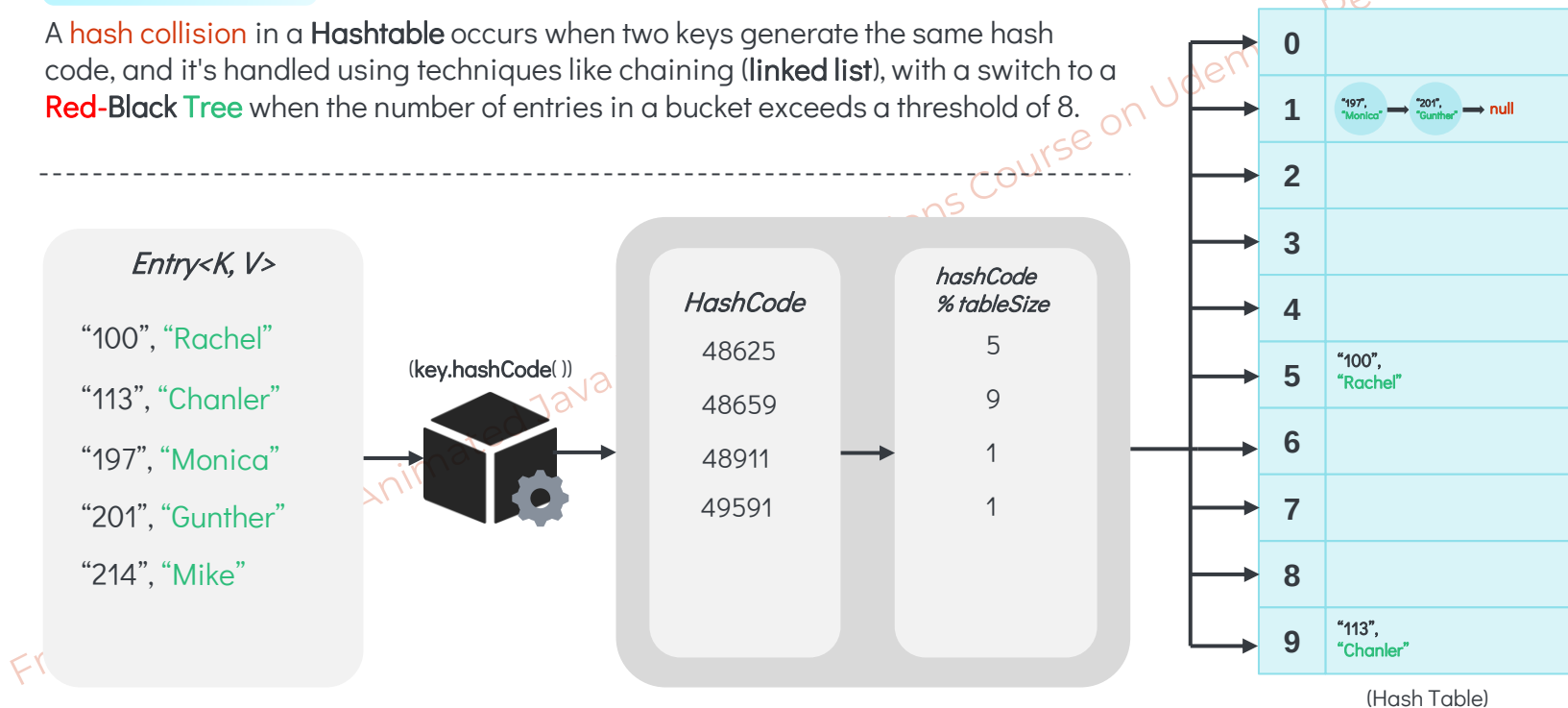
A **hash collision** in a **Hashtable** occurs when two keys generate the same hash code, and it's handled using techniques like chaining (**linked list**), with a switch to a **Red-Black Tree** when the number of entries in a bucket exceeds a threshold of 8.



EXPLANATION

Hash Collision

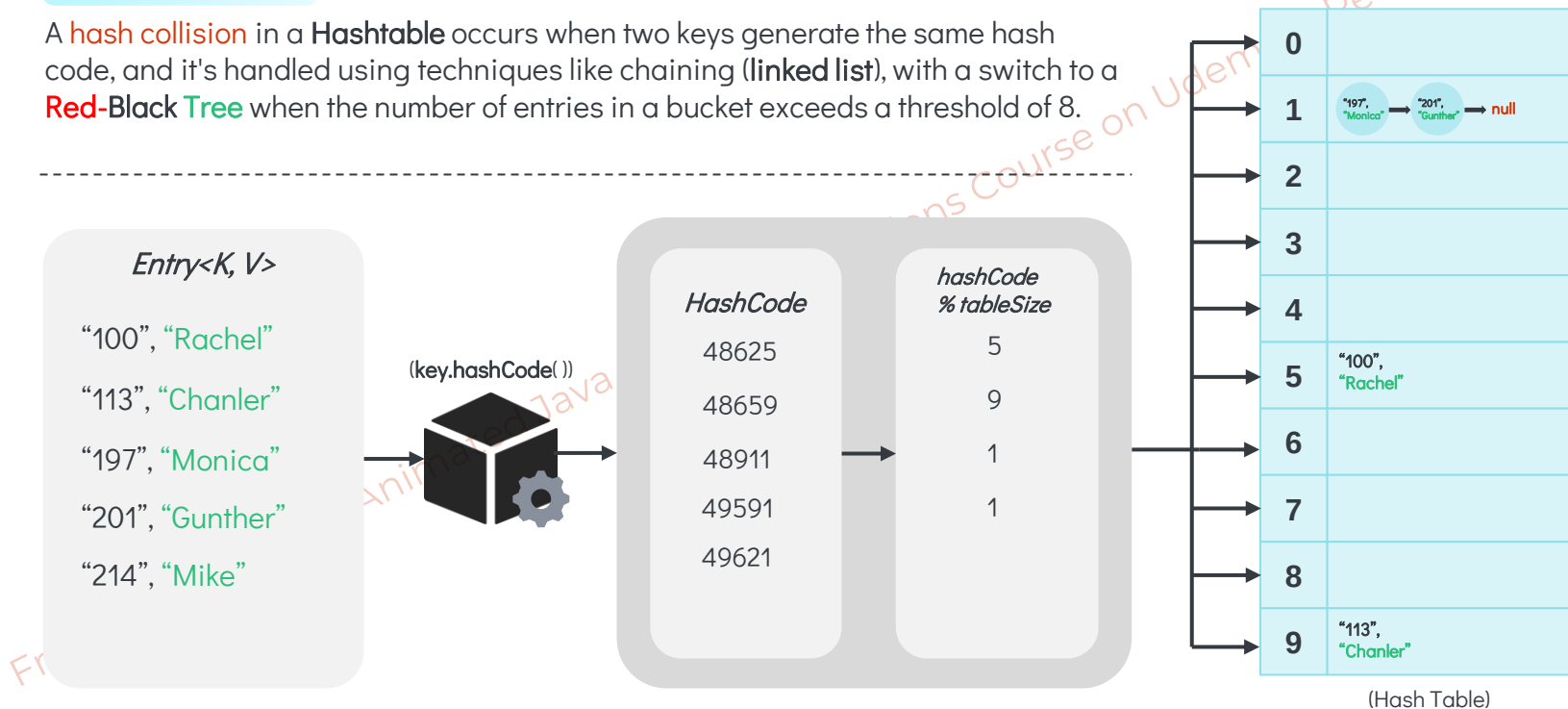
A **hash collision** in a **Hashtable** occurs when two keys generate the same hash code, and it's handled using techniques like chaining (**linked list**), with a switch to a **Red-Black Tree** when the number of entries in a bucket exceeds a threshold of 8.



EXPLANATION

Hash Collision

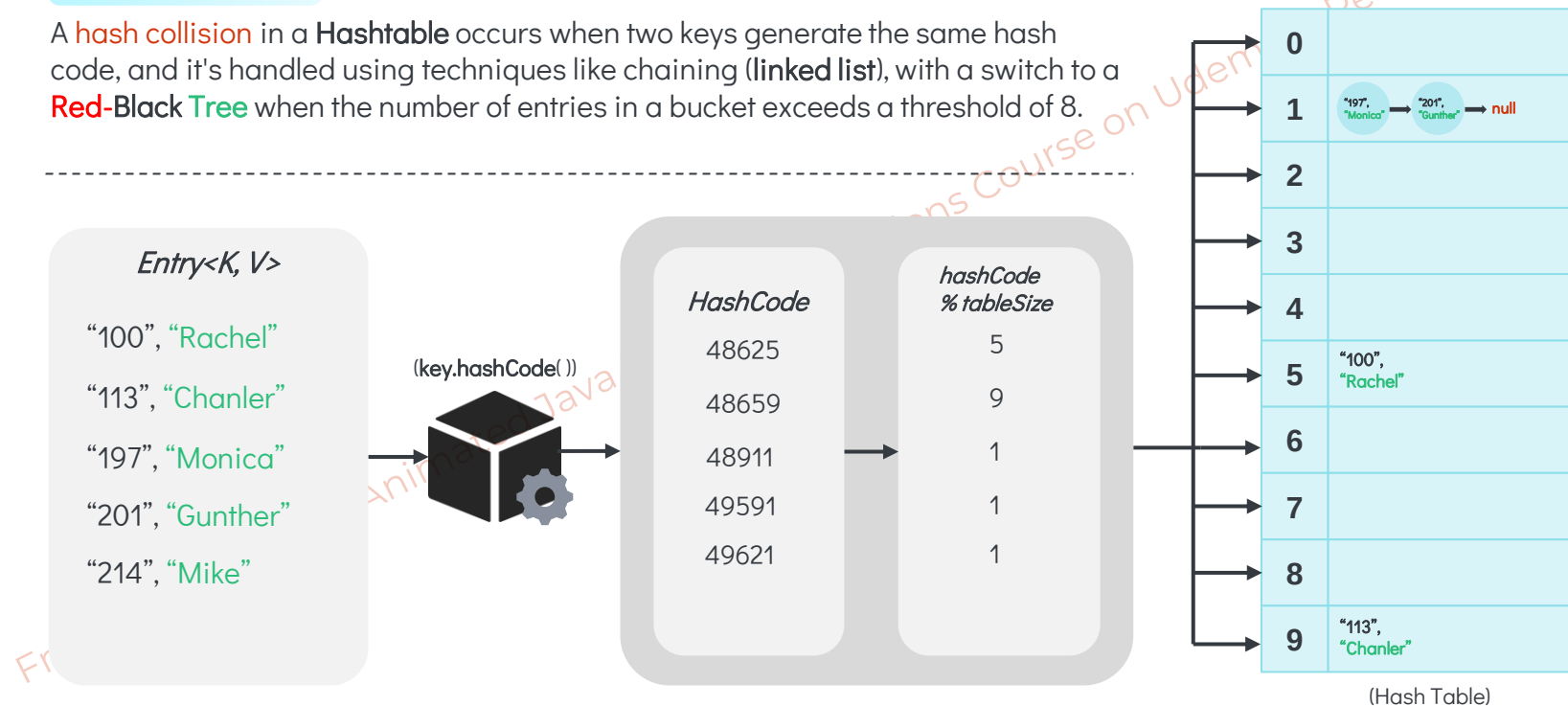
A **hash collision** in a **Hashtable** occurs when two keys generate the same hash code, and it's handled using techniques like chaining (**linked list**), with a switch to a **Red-Black Tree** when the number of entries in a bucket exceeds a threshold of 8.



EXPLANATION

Hash Collision

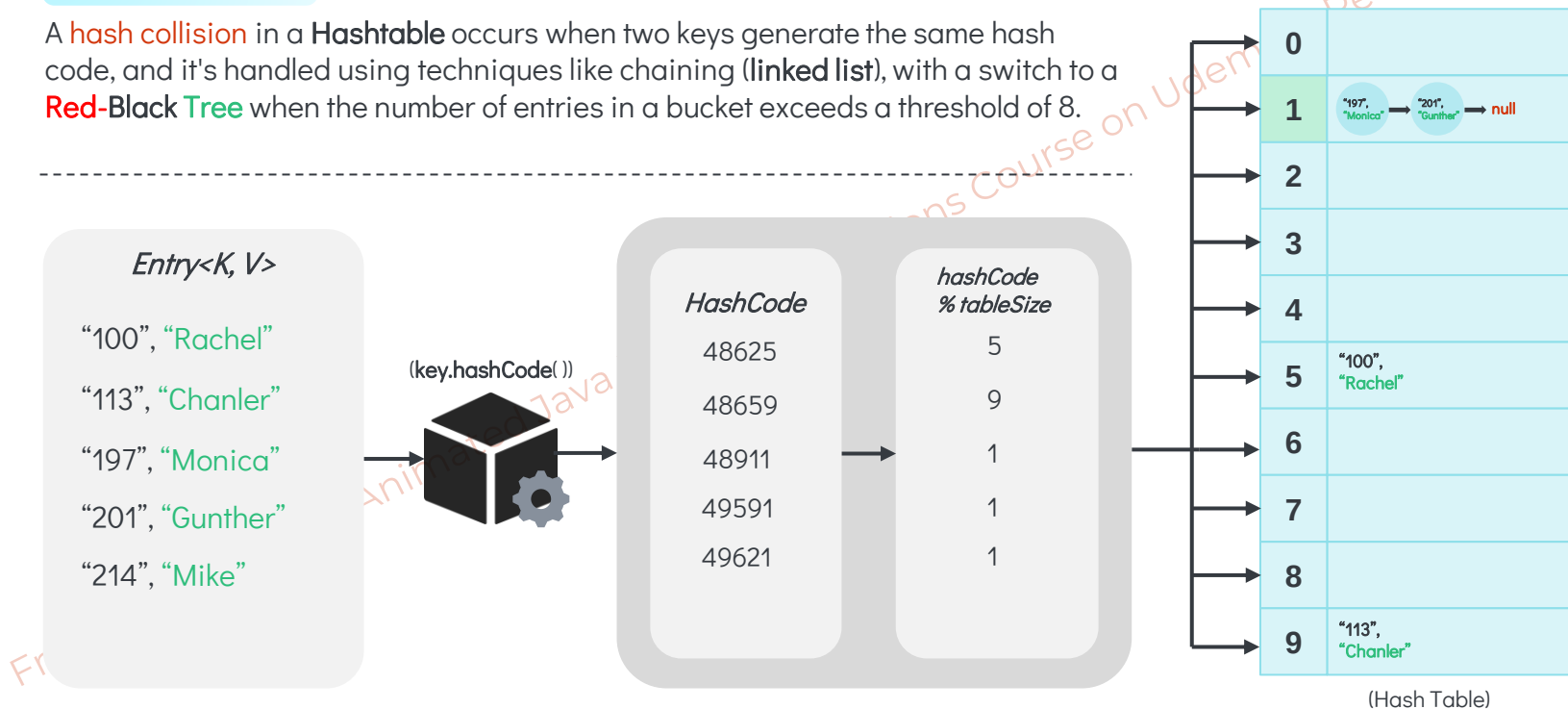
A **hash collision** in a **Hashtable** occurs when two keys generate the same hash code, and it's handled using techniques like chaining (**linked list**), with a switch to a **Red-Black Tree** when the number of entries in a bucket exceeds a threshold of 8.



EXPLANATION

Hash Collision

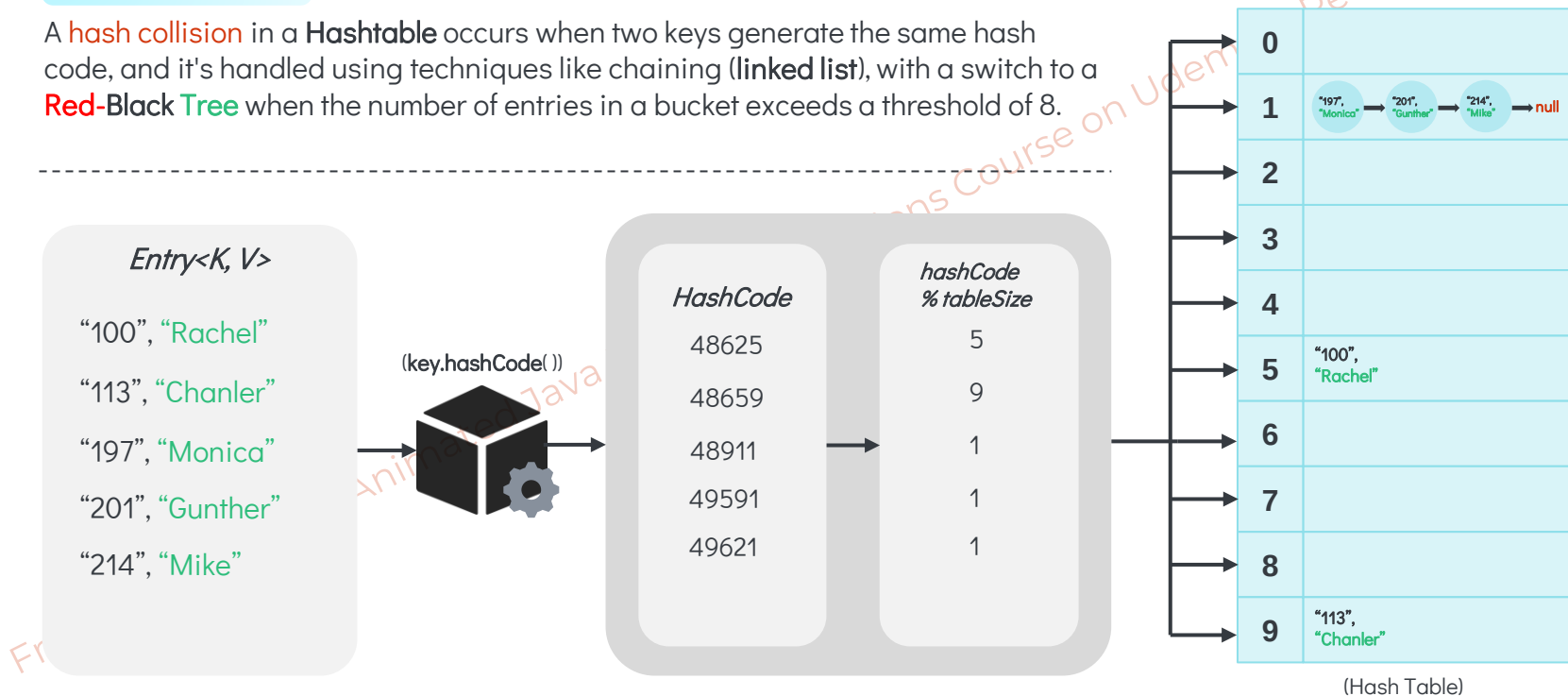
A **hash collision** in a **Hashtable** occurs when two keys generate the same hash code, and it's handled using techniques like chaining (**linked list**), with a switch to a **Red-Black Tree** when the number of entries in a bucket exceeds a threshold of 8.



EXPLANATION

Hash Collision

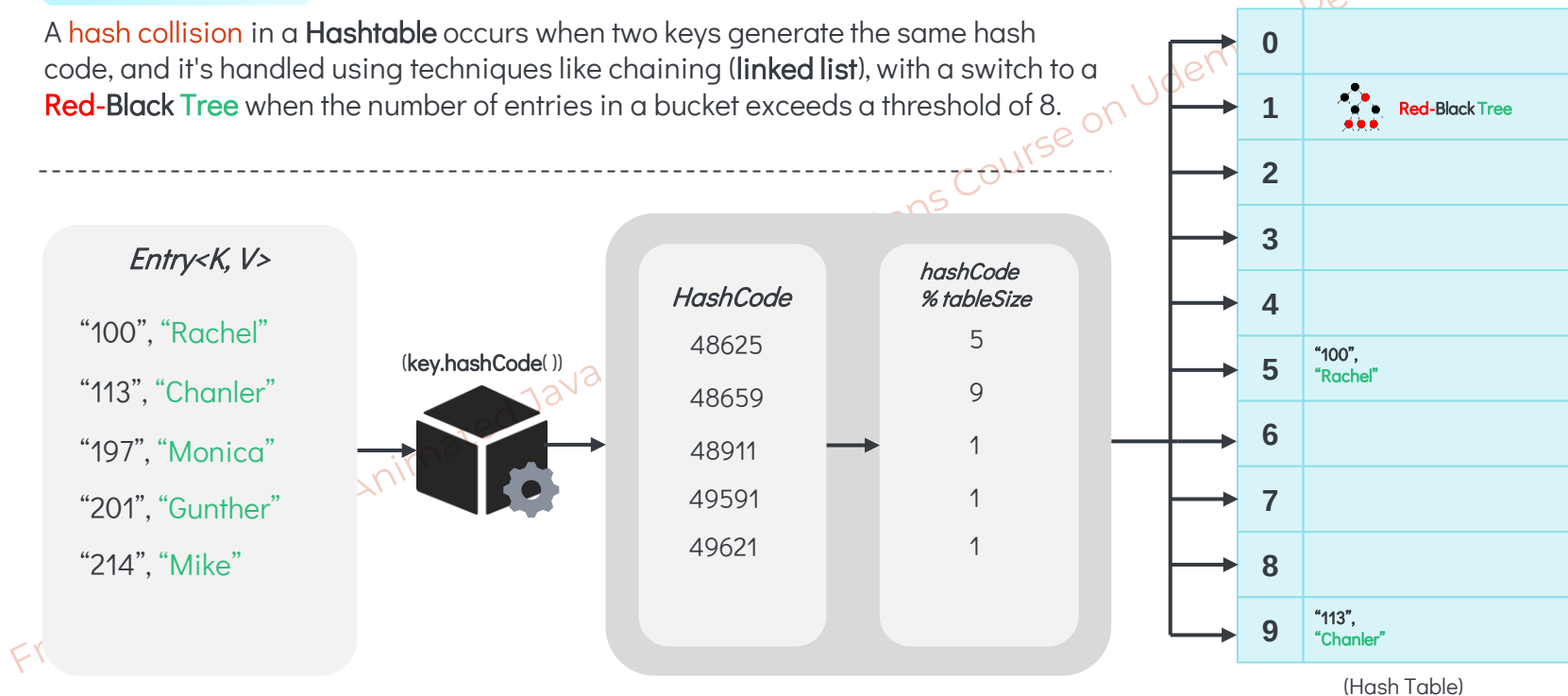
A **hash collision** in a **Hashtable** occurs when two keys generate the same hash code, and it's handled using techniques like chaining (**linked list**), with a switch to a **Red-Black Tree** when the number of entries in a bucket exceeds a threshold of 8.



EXPLANATION

Hash Collision

A **hash collision** in a **Hashtable** occurs when two keys generate the same hash code, and it's handled using techniques like chaining (**linked list**), with a switch to a **Red-Black Tree** when the number of entries in a bucket exceeds a threshold of 8.



QUESTION

What is a HashMap?



Explanation



Animated Java Interview Questions Course on Udemy © Peerless Geek

From: The

EXPLANATION

HashMap

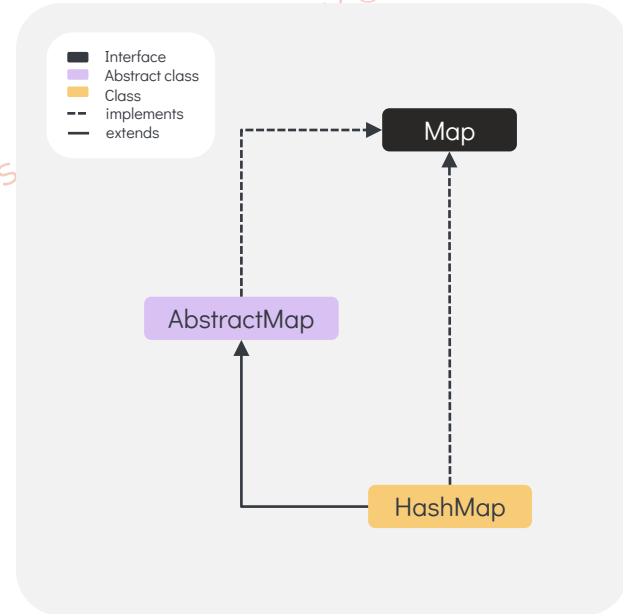
HashMap is a collection implementation of the **Map** interface that stores key-value pairs, and is part of the *java.util* package.

From: The Ultimate Animated Java Interview Questions Course on Udemy © Peerless Geek

EXPLANATION

HashMap

HashMap is a collection implementation of the **Map** interface that stores key-value pairs, and is part of the *java.util* package.



EXPLANATION

HashMap

HashMap is a collection implementation of the **Map** interface that stores key-value pairs, and is part of the *java.util* package.

Implementation

0	
1	
2	
3	
4	
5	
6	
7	
8	
9	

(Hash Table)

From: The Ultimate Animated Java Interview Questions Course on Udemy © Peerless Geek

EXPLANATION

HashMap

HashMap is a collection implementation of the **Map** interface that stores key-value pairs, and is part of the *java.util* package.

- It provides a concrete implementation of the Map interface by extending **AbstractMap**, which offers a skeletal implementation of core Map functionality.
- It uses a **hash table** to store key-value pairs, ensuring average constant time performance ($O(1)$) for put, get, and remove operations.
- It has a default load factor of **0.75**, meaning it will resize when the number of entries exceeds 75% of its current capacity.
- It has an initial capacity of **16** by default, which can be specified in the constructor, and when the threshold is reached (based on the load factor), it typically increases by doubling the capacity (***newCapacity = 2 x oldCapacity***).
- It does not maintain any order of elements; keys and values may change order, especially after rehashing, as elements are stored based on hash codes
- It does not allow duplicate keys; adding an existing key replaces its previous value with the new one.
- It permits the storage of one null key and multiple null values, adhering to its uniqueness constraint for keys.
- It is not thread-safe by default, meaning simultaneous modifications by multiple threads can lead to data corruption; however, it can be made thread-safe by using **Collections.synchronizedMap()**.

QUESTION

What is the **default size of the load factor** in the **hashing-based collection**? 

Explanation



EXPLANATION

Understanding Load Factors in Hash Collections

The **load factor** of a hashing-based collection like **HashMap**, when initialized without specifying an initial capacity and load factor, defaults to 0.75, meaning it will resize when the number of entries exceeds 12 ($16 * 0.75$), with an initial capacity of 16.

From: The Ultimate Animated Java Interview Questions Course on Udemy © Peerless Geek

QUESTION

What is the Dictionary class?



Explanation



From: The Complete Java Interview Questions Course on Udemy © Peerless Geek

EXPLANATION

Dictionary

The **Dictionary** class is a legacy class that represents an abstract data structure consisting of a collection of key-value pairs, introduced in Java 1.0 and is part of the *java.util* package.

From: The Ultimate Animated Java Interview Questions Course on Udemy © Peerless Geek

EXPLANATION

Understanding Java Dictionary Methods: Hierarchical Breakdown

Category	Methods	
Declared in Dictionary	Abstract	<u>elements</u> () , <u>get</u> (Object key), <u>isEmpty</u> () , <u>keys</u> () , <u>put</u> (K key, V value), <u>remove</u> (Object key), <u>size</u> ()
	Default	
	Static	

Dictionary

EXPLANATION

Dictionary

The **Dictionary** class is a legacy class that represents an abstract data structure consisting of a collection of key-value pairs, introduced in Java 1.0 and is part of the *java.util* package.

- It allows storing, accessing, and managing key-value pairs where each key is unique.
- It defines important methods like **put**(Object key, Object value) to insert key-value pairs, **get**(Object key) to retrieve the value associated with a key, and **remove**(Object key) to delete a key-value pair.
- It has been deprecated since *Java 1.2* and has largely been replaced by the more versatile Map interface.
- Its common implementation classes include **Hashtable** and **Properties**.

EXPLANATION

Dictionary

The **Dictionary** class is a legacy class that represents an abstract data structure consisting of a collection of key-value pairs, introduced in Java 1.0 and is part of the *java.util* package.

- It allows storing, accessing, and managing key-value pairs where each key is unique.
- It defines important methods like **put**(Object key, Object value) to insert key-value pairs, **get**(Object key) to retrieve the value associated with a key, and **remove**(Object key) to delete a key-value pair.
- It has been deprecated since *Java 1.2* and has largely been replaced by the more versatile Map interface.
- Its common implementation classes include **Hashtable** and **Properties**.

Note :

Most Java developers no longer use Dictionary and instead rely on Map and its implementations such as HashMap or TreeMap.

QUESTION

What is a Hashtable?



Explanation



From: The Complete Java Interview Questions Course on Udemy © Peerless Geek

EXPLANATION

Hashtable

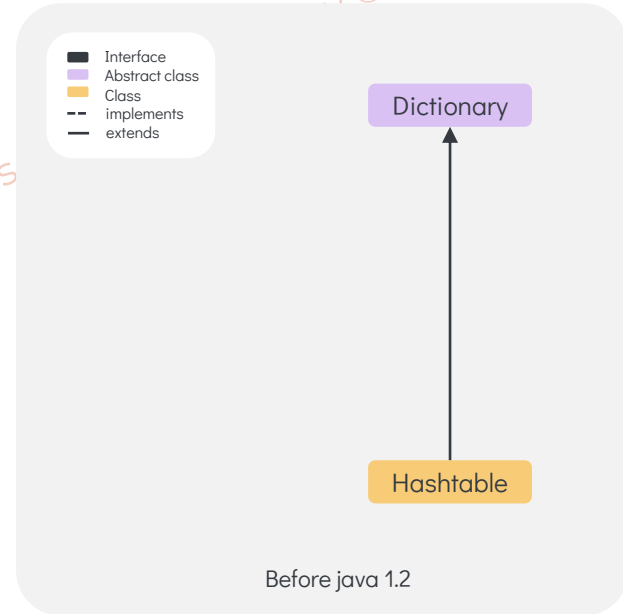
Hashtable is a class that extends the **Dictionary** abstract class, providing foundational functionality for storing key-value pairs and was retrofitted to implement the **Map** interface and is part of the *java.util* package.

From: The Ultimate Animated Java Interview Questions Course on Udemy © Peerless Geek

EXPLANATION

Hashtable

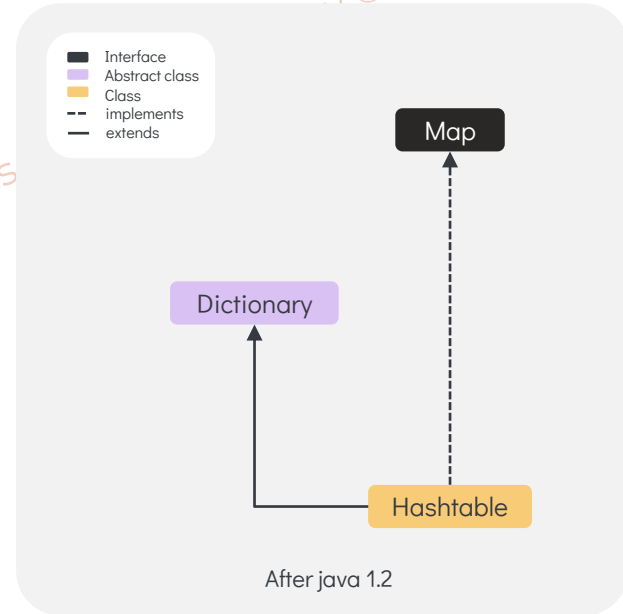
Hashtable is a class that extends the **Dictionary** abstract class, providing foundational functionality for storing key-value pairs and was retrofitted to implement the **Map** interface and is part of the *java.util* package.



EXPLANATION

Hashtable

Hashtable is a class that extends the **Dictionary** abstract class, providing foundational functionality for storing key-value pairs and was retrofitted to implement the **Map** interface and is part of the *java.util* package.



EXPLANATION

Hashtable

Hashtable is a class that extends the **Dictionary** abstract class, providing foundational functionality for storing key-value pairs and was retrofitted to implement the **Map** interface and is part of the *java.util* package.

Implementation

0	
1	
2	
3	
4	
5	
6	
7	
8	
9	

(hash table)

From: The Ultimate Animated Java Interview Questions Course on Udemy © Peerless Geek

EXPLANATION

Hashtable

Hashtable is a class that extends the **Dictionary** abstract class, providing foundational functionality for storing key-value pairs and was retrofitted to implement the **Map** interface and is part of the *java.util* package.

- It implements the **Map** interface by extending **Dictionary**, providing standardized key-value handling that bridges legacy and modern collection practices.
- It uses a **hash table** to store key-value pairs, ensuring average constant time performance ($O(1)$) for put, get, and remove operations.
- It has an initial capacity of **11** by default, which can be specified in the constructor, and when the capacity is reached, it typically doubles ($newCapacity = 2 \times oldCapacity$).
- It has a default load factor of **0.75**, meaning it will resize when the number of entries exceeds 75% of its current capacity.
- It does not maintain any order of its elements; the order of keys and values may change over time, especially after rehashing, as elements are stored based on their hash codes.
- It does not allow duplicate keys; adding an existing key replaces its previous value with the new one.
- It does not permit null keys or values; inserting a null key or value throws a **NullPointerException**.
- It is thread-safe by default, meaning it can handle simultaneous modifications by multiple threads without data corruption.

QUESTION

What is the
difference between
HashMap and
Hashtable? 

Explanation



EXPLANATION

HashMap	Hashtable
HashMap , introduced in Java 2 (<i>JDK 1.2</i>) as part of the Collections Framework, extends the AbstractMap class.	Hashtable is a legacy class from Java 1.0 that extends the Dictionary (<i>which is considered obsolete</i>) class.
It has a default capacity of 16 and a load factor of 0.75 .	It has a default capacity of 11 and a load factor of 0.75 .
It is not synchronized and is not thread-safe.	It is synchronized and thread-safe.
It allows one null key and multiple null values.	It doesn't allow null keys or values.
It has a subclass LinkedHashMap , which maintains insertion order.	It does not have a direct subclass maintaining order.
It is generally faster due to lack of synchronization.	It is slower because of the overhead of synchronization.
It is preferred in modern Java applications.	It is considered legacy and its use is generally discouraged.

QUESTION

What is the **Properties** class, and what are the advantages of using a **Properties** file? 

Explanation



EXPLANATION

Properties

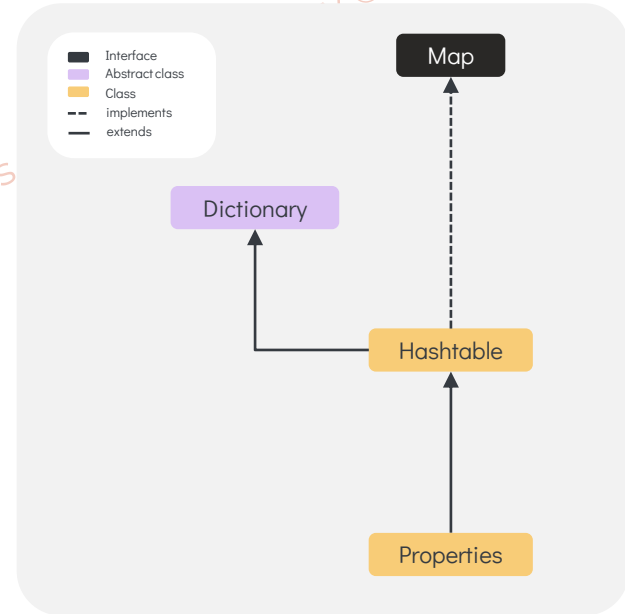
The **Properties** class is a subclass of **Hashtable** that is used for storing key-value pairs as String objects, typically in *.properties files*, providing easy loading, saving, and management of configuration and system settings.

From: The Ultimate Animated Java Interview Questions Course on Udemy © Peerless Geek

EXPLANATION

Properties

The **Properties** class is a subclass of **Hashtable** that is used for storing key-value pairs as String objects, typically in *.properties files*, providing easy loading, saving, and management of configuration and system settings.



From: The Ultimate Animated Java Interview Questions

EXPLANATION

Properties Files

Properties files are human-readable plain text files with a **.properties** extension, that stores commonly used configuration settings as key-value pairs, such as **database credentials**, **application settings**, **server details**, **user preferences**, **API keys**, **file paths**, and **feature flags**.

```
#Database Configuration:
db.url=jdbc:mysql://localhost:3306/mydb
db.username=root
db.password=secret

#Application Settings
app.name=MyApplication
app.version=1.0.0
app.log.level=DEBUG

#Server Configuration:
server.host=localhost
server.port=8080

#User Preferences:
theme=dark
language=en

#API Keys:
api.key=YOUR_API_KEY

#File Paths:
config.path=/etc/myapp/config.properties
data.path=/usr/local/myapp/data

#Feature Flags:
feature.enableNewFeature=true
feature.enableBetaTesting=false
```

(A typical Properties file)

QUESTION

What is a Linked HashMap?



Explanation



Animated Java Interview Questions Course on Udemy © Peerless Geek

From: The

EXPLANATION

LinkedHashMap

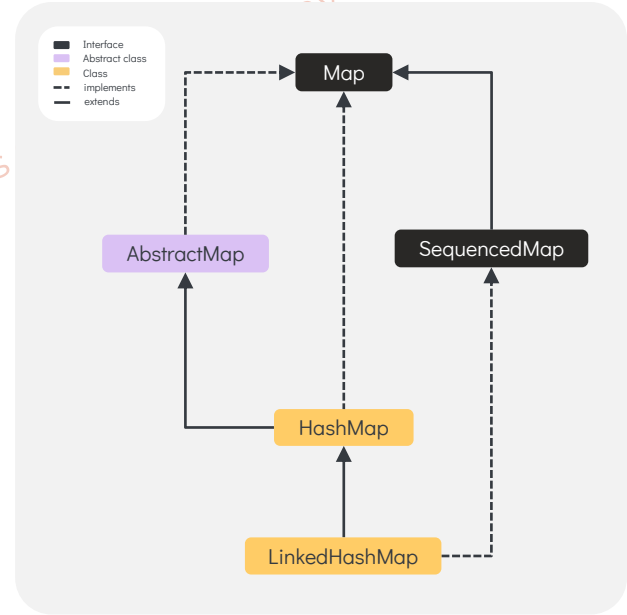
A **LinkedHashMap** is an implementation of the **Map** interface that stores key-value pairs while maintaining the insertion order of keys and is part of the *java.util* package.

From: The Ultimate Animated Java Interview Questions Course on Udemy © Peerless Geek

EXPLANATION

LinkedHashMap

A **LinkedHashMap** is an implementation of the **Map** interface that stores key-value pairs while maintaining the insertion order of keys and is part of the *java.util* package.

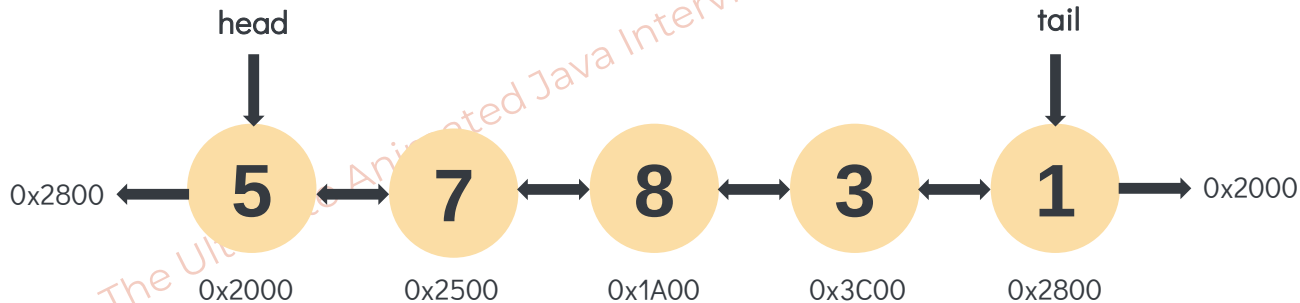


EXPLANATION

LinkedHashMap

A **LinkedHashMap** is an implementation of the **Map** interface that stores key-value pairs while maintaining the insertion order of keys and is part of the *java.util* package.

Implementation



Doubly Linked List

0	
1	
2	
3	
4	
5	
6	
7	
8	
9	

(hash table)

EXPLANATION

LinkedHashMap

A **LinkedHashMap** is an implementation of the **Map** interface that stores key-value pairs while maintaining the insertion order of keys and is part of the *java.util* package.

- It extends **HashMap** for efficient hashing and implements the **SequencedMap** interface to maintain predictable iteration order based on key insertion.
- It uses a combination of a **hash table** (inherited from **HashMap**) for storage and a **doubly-linked list** to maintain the insertion order of elements.
- It performs like **HashMap** but with slightly higher overhead due to the linked list, while `put()`, `remove()`, and `containsKey()` still run in $O(1)$ time.
- It has a default initial capacity of **16** and a load factor of **0.75**, similar to **HashMap**.
- It does not allow duplicate keys (*it replaces the old value with the new one*) and permits one null key along with multiple null values, similar to a **HashMap**.
- It is not thread-safe by default, meaning simultaneous modifications by multiple threads can lead to data corruption; however, it can be made thread-safe by using **Collections.synchronizedMap()**.

QUESTION

What is a TreeMap?



Explanation



Animated Java Interview Questions Course on Udemy © Peerless Geek

From: The

EXPLANATION

TreeMap

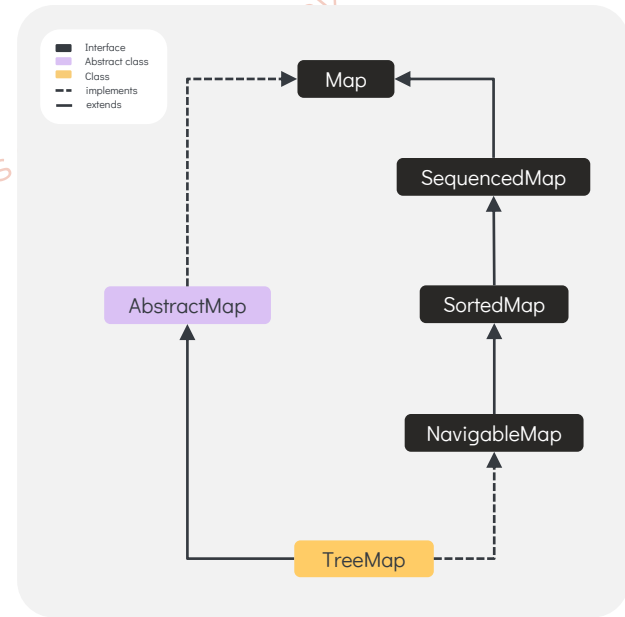
A **TreeMap** is a collection implementation of the **Map** interface that stores key-value pairs in a sorted order, based on the natural ordering of the keys or a custom comparator-defined order, and is part of the *java.util* package.

From: The Ultimate Animated Java Interview Questions Course on Udemy © Peerless Geek

EXPLANATION

TreeMap

A **TreeMap** is a collection implementation of the **Map** interface that stores key-value pairs in a sorted order, based on the natural ordering of the keys or a custom comparator-defined order, and is part of the *java.util* package.

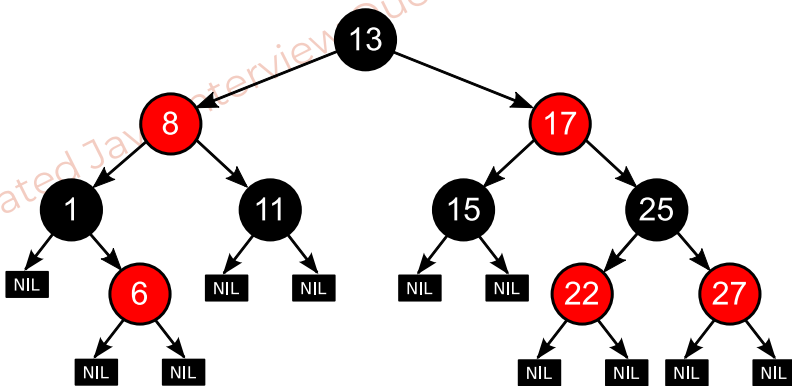


EXPLANATION

TreeMap

A **TreeMap** is a collection implementation of the **Map** interface that stores key-value pairs in a sorted order, based on the natural ordering of the keys or a custom comparator-defined order, and is part of the *java.util* package.

Implementation



(Red Black Tree)

EXPLANATION

TreeMap

A **TreeMap** is a collection implementation of the **Map** interface that stores key-value pairs in a sorted order, based on the natural ordering of the keys or a custom comparator-defined order, and is part of the *java.util* package.

- It extends **AbstractMap** for core map operations and implements **NavigableMap** to offer sorted, navigable functionality using a self-balancing tree structure.
- It uses a **Red-Black tree** (a self-balancing BST) to store elements in sorted order, ensuring $O(\log n)$ time complexity for add, remove, and contains operations.
- It stores keys in sorted order, either by their natural order (as defined by *Comparable*) or by a custom comparator (as specified in the constructor).
- It doesn't allow duplicate keys and replaces the existing value if the same key is added.
- It doesn't allow null keys but permits multiple null values, throwing a **NullPointerException** for null keys.
- It is not thread-safe by default, meaning concurrent modifications by multiple threads can lead to data corruption. However, it can be made thread-safe using **Collections.synchronizedSortedMap()**.

QUESTION

What is the
difference between
HashMap and
TreeMap? 

Explanation



EXPLANATION

HashMap	TreeMap
HashMap implements the Map interface using a hash table .	TreeMap implements the Map interface using a red-black tree.
It does not maintain element order; iteration order is unpredictable.	It maintains a sorted order based on the natural ordering of keys or a custom comparator.
Its keys are not required to implement the Comparable interface.	Its keys must be comparable or a custom Comparator must be provided.
It provides $O(1)$ average time complexity for basic operations like get, put, and remove.	It provides $O(\log n)$ time complexity for basic operations like get, put, and remove.
It allows one null key and multiple null values.	It does not allow null keys (throws NullPointerException) but allows multiple null values.
It is preferred when fast access and no ordering are required.	It is preferred when a sorted order of keys is needed.

QUESTION

What is Identity HashMap?



Explanation



Animated Java Interview Questions Course on Udemy © Peerless Geek

From: The

EXPLANATION

IdentityHashMap

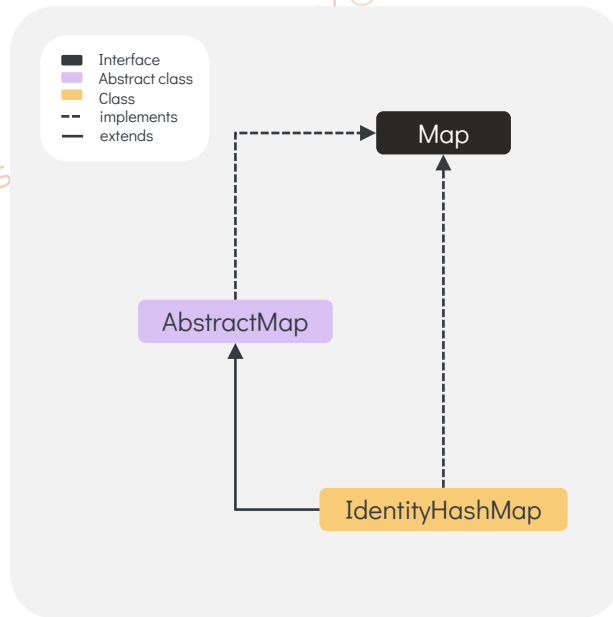
IdentityHashMap is a specialized implementation of the **Map** interface that compares keys using reference equality (`obj1 == obj2`) instead of object equality (`obj1.equals(obj2)`).

From: The Ultimate Animated Java Interview Questions Course on [Udemy](#) © Peerless Geek

EXPLANATION

IdentityHashMap

IdentityHashMap is a specialized implementation of the **Map** interface that compares keys using reference equality ($obj1 == obj2$) instead of object equality ($obj1.equals(obj2)$).



From: The Ultimate Animated Java Interview Questions

© Peerless Geek

EXPLANATION

IdentityHashMap

IdentityHashMap is a specialized implementation of the **Map** interface that compares keys using reference equality (*obj1 == obj2*) instead of object equality (*obj1.equals(obj2)*).

Implementation

0	
1	
2	
3	
4	
5	
6	
7	
8	
9	

(hash table)

From: The Ultimate Animated Java Interview Questions Course on Udemy © Peerless Geek

EXPLANATION

IdentityHashMap

IdentityHashMap is a specialized implementation of the **Map** interface that compares keys using reference equality (`obj1 == obj2`) instead of object equality (`obj1.equals(obj2)`).

- It provides a concrete implementation of the **Map** interface by extending **AbstractMap**, which offers a skeletal implementation of core Map functionality.
- It uses a **hash table** to store key-value pairs, hashing keys based on reference equality (*using == for comparison*) and relies on `System.identityHashCode()` to compute hash codes, ensuring average constant time ($O(1)$) performance for put, get, and remove operations, similar to **HashMap**.
- Its default initial capacity is **32** key-value pairs, and instead of using a load factor like other maps, it resizes when it reaches approximately two-thirds (67%) of its capacity.
- It does not maintain any order of elements; the order of keys and values may change, especially after rehashing.
- It allows for multiple keys that are logically equal but not identical by reference (*e.g., two different objects that are .equals() but not ==*).
- It permits one null key and multiple null values, but the null key is also subject to reference comparison.
- It is not thread-safe by default, meaning simultaneous modifications by multiple threads can lead to data corruption; however, it can be made thread-safe by using **Collections.synchronizedMap()**.

QUESTION

What is Weak HashMap?



Explanation



Animated Java Interview Questions Course on Udemy © Peerless Geek

From: The

EXPLANATION

WeakHashMap

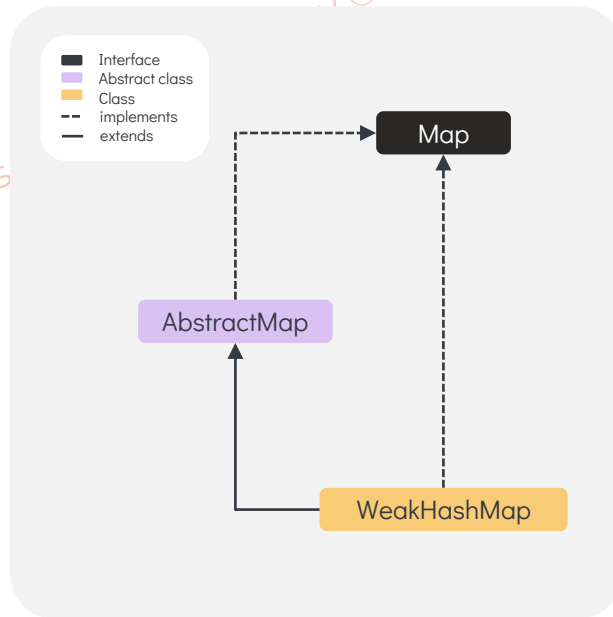
A **WeakHashMap** is a special implementation of the **Map** interface where the keys are weakly referenced, allowing them to be garbage collected when no strong references exist.

From: The Ultimate Animated Java Interview Questions Course on Udemy © Peerless Geek

EXPLANATION

WeakHashMap

A **WeakHashMap** is a special implementation of the **Map** interface where the keys are weakly referenced, allowing them to be garbage collected when no strong references exist.



EXPLANATION

WeakHashMap

A **WeakHashMap** is a special implementation of the **Map** interface where the keys are weakly referenced, allowing them to be garbage collected when no strong references exist.

Implementation

0	
1	
2	
3	
4	
5	
6	
7	
8	
9	

(hash table)

From: The Ultimate Animated Java Interview Questions Course on Udemy © Peerless Geek

EXPLANATION

WeakHashMap

A **WeakHashMap** is a special implementation of the **Map** interface where the keys are weakly referenced, allowing them to be garbage collected when no strong references exist.

- It provides a concrete implementation of the **Map** interface by extending **AbstractMap**, which offers a skeletal implementation of core Map functionality.
- It uses a **hash table** to store key-value pairs, ensuring average constant time performance ($O(1)$) for put, get, and remove operations.
- It allows keys to be garbage-collected when no longer in use, automatically removing entries once keys are no longer referenced outside the map.
- It has a default initial capacity of **16** and a load factor of **0.75**.
- It does not maintain any order of elements; the order of keys and values may change after rehashing.
- It does not allow duplicate keys; adding an existing key replaces its previous value with the new one.
- It permits the storage of one null key and multiple null values, adhering to its uniqueness constraint for keys.
- It is not thread-safe by default, meaning simultaneous modifications by multiple threads can lead to data corruption; however, it can be made thread-safe by using `Collections.synchronizedMap()`.

QUESTION

What is EnumMap?



Explanation



From: The Complete Java Interview Questions Course on Udemy © Peerless Geek

EXPLANATION

EnumMap

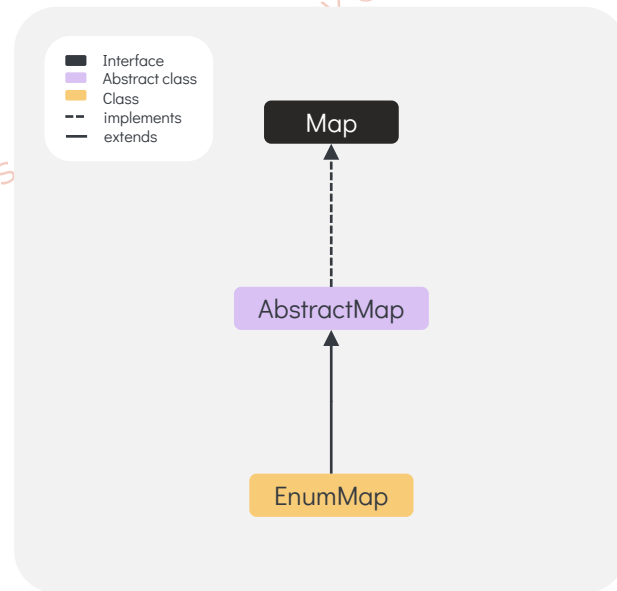
An **EnumMap** in Java is a specialized Map that uses enum constants as keys, requiring all keys to belong to a single enum type.

From: The Ultimate Animated Java Interview Questions Course on Udemy © Peerless Geek

EXPLANATION

EnumMap

An **EnumMap** in Java is a specialized Map that uses enum constants as keys, requiring all keys to belong to a single enum type.



From: The Ultimate Animated Java Interview Questions

© Peerless Geek

EXPLANATION

EnumMap

An **EnumMap** in Java is a specialized Map that uses enum constants as keys, requiring all keys to belong to a single enum type.

Implementation

0	1	2	3	4	5

(Static array)

EXPLANATION

EnumMap

An **EnumMap** in Java is a specialized Map that uses enum constants as keys, requiring all keys to belong to a single enum type.

- It provides a concrete implementation of the **Map** interface by extending **AbstractMap**, which offers a skeletal implementation of essential Map functionality.
- It stores keys internally as an array, indexed by the ordinal values of the enum constants, ensuring $O(1)$ time complexity for operations like **get()**, **put()**, **containsKey()**, and **remove()**.
- Its internal array length is set to match the number of constants in the enum.
- It maintains the natural order of elements based on the order of the enum constants as they are defined in the enum class.
- It does not allow duplicate keys; if an existing key is added, the previous value is replaced with the new value.
- It does not allow null keys, throwing a **NullPointerException** if one is inserted, but it permits multiple null values associated with different keys.
- It is not synchronized and therefore not thread-safe; external synchronization, such as using **Collections.synchronizedMap()**, is required if multiple threads access it concurrently and at least one modifies it.

QUESTION

Can you use
any class as a
key in a Map? 

Explanation



Animated Java Interview Questions Course on Udemy © Peerless Geek

From: The

EXPLANATION

Key Constraints : Classes as Map Keys

No, not every class can be used as a key in a Java Map; for hash-based Maps like **HashMap**, it must implement **hashCode()** and **equals()**, for sorted Maps like **TreeMap**, it must implement **Comparable** or use a **Comparator**, and “*immutable keys*” are preferred for consistency.

From: The Ultimate Animated Java Interview Questions Course on Udemy © Peerless Geek

QUESTION

What is the
difference between
Comparable and
Comparator? 

Explanation



EXPLANATION

Comparable	Comparator
Comparable is an interface that is part of the <i>java.lang</i> package.	Comparator is an interface that is part of the <i>java.util</i> package.
It defines a single method: int compareTo(T o) , which is used to compare the current object with the specified object o .	It defines a single method: int compare(T o1, T o2) , which is used to compare two objects o1 and o2 .
It is used to define the natural ordering of objects within the class itself.	It is used to define custom or multiple ordering strategies for objects without modifying the class such as sorting by name, age, or any other field.
It requires modification of the class whose objects are being sorted by implementing the Comparable interface.	It does not modify the class; sorting logic is defined in a separate comparator class (<i>a separate class that implements the Comparator interface</i>).
Objects of the class that implement Comparable can be sorted using <i>Collections.sort()</i> or <i>Arrays.sort()</i> directly, as the natural order is already defined.	Objects of the class can be sorted by passing an instance of the comparator to <i>Collections.sort()</i> or <i>Arrays.sort()</i> .
It is suitable when a single, default comparison logic is sufficient.	It is suitable when multiple comparison strategies are needed or if you do not have control over the class source code.

QUESTION

What is the difference between **HashSet** and **HashMap**? 

Explanation



EXPLANATION

HashSet	HashMap
HashSet internally uses a HashMap to store its elements.	HashMap uses a hash table data structure to store key-value pairs.
It stores a collection of unique elements as keys in the backing HashMap , with a constant dummy value (typically a <i>PRESENT</i> object).	It stores key-value pairs, ensuring that each key is unique.
It uses add(E e) to add elements, remove(Object o) to delete, and contains(Object o) to check for existence.	It uses put(K key, V value) to add or update key-value pairs, remove(Object key) to delete entries, and get(Object key) to retrieve values by key.
It does not allow duplicate elements.	It does not allow duplicate keys, but values can be duplicated.
It allows one null element.	It allows one null key and multiple null values.
It generally uses less memory than a HashMap , as it only stores keys with a dummy value instead of key-value pairs.	It requires more memory to store both keys and values.
It is preferred when you need to ensure uniqueness of elements.	It is considered when you need to associate values with keys for quick lookups.

QUESTION

How to make a Collection Read-Only?

Explanation



From: The Complete Java Interview Questions Course on Udemy © Peerless Geek

EXPLANATION

Immutable Collection : Ensuring Data Protection

Collections can be made **read-only** in Java using `Collections.unmodifiableCollection()` for generic types, or `Collections.unmodifiableList()`, `Collections.unmodifiableSet()`, and `Collections.unmodifiableMap()` for **List**, **Set**, and **Map** respectively.

From: The Ultimate Animated Java Interview Questions Course on Udemy © Peerless Geek

QUESTION

What is Unsupported Operation Exception?

Explanation



From: The Complete Java Interview Questions Course on Udemy © Peerless Geek

EXPLANATION

UnsupportedOperationException

UnsupportedOperationException is a *RUNTIME* exception that indicates a particular operation is not supported, commonly encountered when trying to modify **immutable collections** such as those created by `List.of()`, `List.copyOf()`, `Set.of()`, `Set.copyOf()`, `Map.of()`, `Map.copyOf()` or returned by methods such as `Collections.unmodifiableCollections()`, `Collections.unmodifiableList()`, `Collections.unmodifiableSet()` and `Collections.unmodifiableMap()`.

QUESTION

What is the difference between **Set** and **Map**? 

Explanation



From: The Complete Java Interview Questions Course on Udemy © Peerless Geek

EXPLANATION

Set	Map
Set is an unordered collection of elements that does not allow duplicates.	Map is an unordered collection of key-value pairs, called entries , where each key is unique and maps to exactly one value.
It does not guarantee insertion order, as it is unordered, although some implementations like LinkedHashSet maintain insertion order.	It does not guarantee insertion order, as it is unordered; however, some implementations, like LinkedHashMap , maintain insertion order.
It does not allow duplicate elements.	It does not allow duplicate keys, but values can be duplicated.
It can hold a single null element, with exceptions in implementations such as TreeSet , EnumSet , and CopyOnWriteArraySet .	It can have a single null key and multiple null values, except in implementations like TreeMap (<i>no null keys</i>), EnumMap (<i>no null keys</i>), Hashtable (<i>no null keys or values</i>) and ConcurrentHashMap (<i>no null keys or values</i>).
It is typically used when a collection of unique elements is needed, and the order may or may not matter depending on the implementation.	It is suitable when a collection of key-value pairs is needed, where each key is unique, and the order of entries may or may not matter depending on the implementation.

QUESTION

What is the difference between **List**, **Set**, and **Map**? 

Explanation



From: The Complete Java Interview Questions Course on Udemy © Peerless Geek

EXPLANATION

List	Set	Map
List is an ordered collection of elements that allows duplicates.	Set is an unordered collection of elements that does not allow duplicates.	Map is an unordered collection of key-value pairs, called entries , where each key is unique and maps to exactly one value.
It maintains the insertion order, meaning elements are stored and accessed in the same sequence in which they were added.	It does not guarantee insertion order, as it is unordered, although some implementations like LinkedHashSet maintain insertion order.	It does not guarantee insertion order, as it is unordered; however, some implementations, like LinkedHashMap , maintain insertion order.
It allows duplicate elements.	It does not allow duplicate elements.	It does not allow duplicate keys, but values can be duplicated.
It can hold multiple null elements, except for CopyOnWriteArrayList , which does not allow any null elements.	It can hold a single null element, with exceptions in implementations such as TreeSet , EnumSet , and CopyOnWriteArraySet .	It can have a single null key and multiple null values, except in implementations like TreeMap (<i>no null keys</i>), EnumMap (<i>no null keys</i>), Hashtable (<i>no null keys or values</i>), ConcurrentHashMap (<i>no null keys or values</i>).
It is typically used when the order matters, duplicates are allowed, and index-based access is required.	It is suitable when a collection of unique elements is needed, and the order may or may not matter depending on the implementation.	It is suitable when a collection of key-value pairs is needed, where each key is unique, and the order of entries may or may not matter depending on the implementation.

QUESTION

What is an Enumeration?



Explanation



From: The Complete Java Interview Questions Course on Udemy © Peerless Geek

EXPLANATION

Enumeration

Enumeration is an interface in the *java.util* package used to iterate through elements in a collection, mainly in legacy code.

- It was introduced in Java 1.0, predating the newer and more versatile **Iterator** interface.
- It defines only two methods: **hasMoreElements()** which returns true if more elements are available to iterate, and **nextElement()** which returns the next element in the enumeration.
- It allows read-only access to the elements in the collection, with no methods to remove or modify them.
- It can only traverse forward through a collection.
- It is mainly used with legacy classes like **Vector**, **Stack**, and **Hashtable**, which have an **elements()** method returning an Enumeration of their elements.
- It can be converted to an Iterator using *myEnumeration.asIterator()*.
- It's considered outdated, so for new code, it's better to use Iterator or the enhanced for-loop with Iterable collections.

QUESTION

What is an **Iterator**?

Explanation



From: The Complete Interview Questions Course on Udemy © Peerless Geek

EXPLANATION

Iterator

Iterator is an interface in the *java.util* package for iterating through collection elements, offering more flexibility than Enumeration.

- It was introduced in *Java 1.2* as part of the Collections Framework, replacing the older Enumeration interface and provides a uniform way to iterate over different types of collections.
- It can be used with any collection that implements the **Iterable** interface, such as **ArrayList**, **LinkedList**, and **HashSet**.
- It provides read and remove capabilities with three core methods: **hasNext()** to check for more elements, **next()** to return the next element, and **remove()** to remove the last element returned.
- It can be obtained from a collection directly using its **iterator()** method, e.g., *myCollection.iterator()*.
- It allows forward-only traversal, with no way to revisit elements, similar to Java's Enumeration interface.
- It is used behind the scenes by Java's enhanced for loop (*or for-each loop*), which leverages the Iterable interface.
- It can be categorized based on its behavior during modifications to the underlying collection during iteration, specifically into two types: **fail-fast** and **fail-safe**.

QUESTION

What are the
difference between
Iterator and
Enumeration? 

Explanation



EXPLANATION

Enumeration	Iterator
Enumeration is an interface in the <i>java.util</i> package, introduced in Java 1.0, that provides a way to sequentially iterate through elements in a collection.	Iterator is an interface in the <i>java.util</i> package, introduced in Java 1.2, that provides a way to traverse elements in a collection while allowing safe element removal during iteration.
It defines only two methods: hasMoreElements() to check for more elements, and nextElement() to retrieve the next element.	It defines three methods: hasNext() to check for more elements, next() to retrieve the next element, and remove() to remove the current element.
It provides read-only access and does not support removing or modifying elements.	It allows both reading and removal of elements from the collection.
It does not support the fail-fast behavior.	It supports fail-fast behavior.
It is considered outdated and mostly replaced by Iterator or the enhanced for-loop.	It is preferred for modern Java code, especially with the enhanced for-loop.
It is commonly used with legacy classes like Vector , Stack , and Hashtable , which return an Enumeration from the elements() method.	It can be used with any collection that implements the Iterable interface, such as ArrayList , LinkedList , and HashSet .

QUESTION

Explain the **Fail-Fast** iterator and **Fail-Safe** iterator. 

Explanation



From: The Java Interview Questions Course on Udemy © Peerless Geek

EXPLANATION

Fail-fast	Fail-safe
Fail-fast iterator is an iterator that immediately throws a <i>ConcurrentModificationException</i> if the underlying collection is structurally modified during iteration.	Fail-safe iterator is an iterator that does not throw a <i>ConcurrentModificationException</i> even if the collection is modified during iteration.
It is returned by <i>non-concurrent</i> collections like ArrayList , HashSet , HashMap , and others.	It is returned by <i>concurrent</i> collections like CopyOnWriteArrayList , ConcurrentHashMap and others.
It iterates directly on the original collection.	It iterates over a clone or copy of the collection, which does not reflect the changes made to the original collection.
It is more efficient than fail-safe iterators, as no cloning and thread-safety is involved.	It is less efficient than fail-fast iterators due to the cloning operation and thread-safety overhead.
It is more suitable for single-threaded environments or when the collection is not expected to be modified during iteration.	It is more suitable for multi-threaded environments where concurrent modifications to the collections are expected and desired.

QUESTION

What is Concurrent Modification Exception?

Explanation



From: The Complete Java Interview Questions Course on Udemy © Peerless Geek

EXPLANATION

ConcurrentModificationException

A **ConcurrentModificationException** is a *RUNTIME* exception thrown when modifying a **non-concurrent collection** (like *ArrayList* or *HashSet*) during iteration except through iterator's own `remove()` method, while **concurrent collections** (like *CopyOnWriteArrayList*) are designed to handle such modifications safely.

From: The Ultimate Animated Java Interview Questions Course on Udemy © Peerless Geek

QUESTION

What are the differences between **Iterator** and **ListIterator**? 

Explanation



EXPLANATION

Iterator	ListIterator
Iterator is an interface in the <i>java.util</i> package that provides a way to traverse elements in a collection while allowing safe element removal during iteration.	ListIterator is an interface in the <i>java.util</i> package that extends Iterator , allowing bidirectional traversal and safe insertion, removal, and replacement of elements in a list.
It is a unidirectional iterator, meaning it can only move forward in the collection.	It is a bidirectional iterator, which means it can move forward and backward in a list.
It provides basic methods such as hasNext() and next() for iterating and remove() for removing elements from the collection.	In addition to the methods inherited from Iterator , it has more methods like hasPrevious() , previous() , nextIndex() , previousIndex() , set() and add() .
It can only remove elements from the Collection while iterating.	It can add, modify and remove elements from the list while iterating.
It does not provide positional information of the iterator in the Collection.	It provides methods to get the current position (index) of the iterator in the List.
It can be used to iterate over any Collection implementation, including ArrayList , HashSet , HashMap and others.	It can only be used with List implementations (<i>like ArrayList or LinkedList</i>).

Exception Handling

16

QUESTION

What is Exception Handling?



Explanation



From: The Complete Python Interview Questions Course on Udemy © Peerless Geek

EXPLANATION

Exception

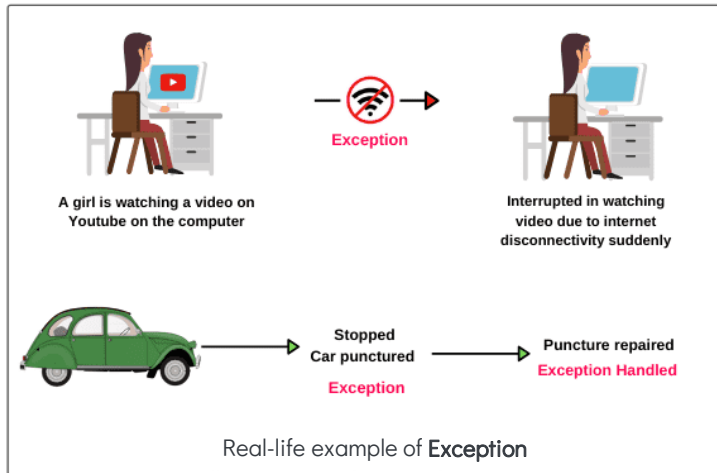
Exception is an unwanted or unexpected event or occurrence that interrupts the normal flow of a program's execution.

From: The Ultimate Animated Java Interview Questions Course on Udemy © Peerless Geek

EXPLANATION

Exception

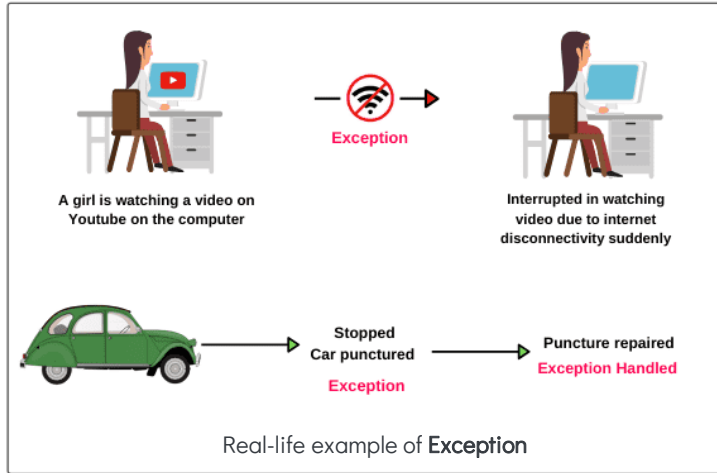
Exception is an unwanted or unexpected event or occurrence that interrupts the normal flow of a program's execution.



EXPLANATION

Exception

Exception is an unwanted or unexpected event or occurrence that interrupts the normal flow of a program's execution. When an exception occurs within a method, it creates an object called the **exception object** that contains information about the exception, such as the **type** and **description**.



www.QuestionsCourse.com Peerless Geek

The **Video** is not playing because the **Wi-Fi** is not working.

The **Car** is not moving properly because a **tyre** got punctured.

Exception Object

EXPLANATION

Exception Handling

Exception handling is a mechanism for dealing with runtime errors and unexpected situations in a controlled manner.

- It is a mechanism to deal with runtime errors such as `NullPointerException`, `ArrayIndexOutOfBoundsException`, `IndexOutOfBoundsException`, `ClassCastException`, `IllegalArgumentException`, `ArithmeticException`, `NumberFormatException`, etc.
- It allows developers to gracefully handle unexpected situations and maintain the normal flow of the application.
- It involves using a **try-catch** block to handle exceptions. The block of code that might throw an exception is placed inside the try block. If an exception is thrown, it is caught and handled by the catch block.

```
public static void main(String[] args) {  
    System.out.println("Program started!");  
  
    try {  
        int result = 10 / 0;  
    } catch (ArithmeticException e) {  
        System.out.println("Error caught...");  
        System.out.println(e.getClass());  
        System.out.println(e.getMessage());  
    }  
  
    System.out.println("Program ended!");  
}
```

(Standard try-catch block)

QUESTION

Is it mandatory for a **catch** block to be followed after a **try** block? 

Explanation



EXPLANATION

Try-Catch-Finally Block Relationship

A **try block** in Java is not mandatory to be followed by a catch block, as a finally block can be used instead.

- It is most commonly followed by one or more catch blocks to handle specific exceptions.
- It can be followed by a finally block if there is no catch block.
- It can also be followed by both catch and finally blocks, with the finally block always executing after the catch block(s), even if an exception is thrown.

```
try {  
    // Code that might throw an  
    exception.  
} catch (ExceptionType e) {  
    // Code to handle the exception  
} finally {  
    // Code that will run after the  
    try block, even if an exception  
    occurs.  
}
```

try with catch and finally blocks

Note :

A try block in Java is required to have a catch block, a finally block, or both. Omitting both will cause a compilation error.

QUESTION

What is the
difference between
Checked Exception
and **Unchecked**
Exception? 

Explanation



EXPLANATION

Checked Exception	Unchecked Exception
Checked Exception inherits from <code>java.lang.Exception</code> (excluding <code>RuntimeException</code> and its subclasses).	Unchecked Exception inherits from <code>java.lang.RuntimeException</code> .
It represents anticipated conditions that a well-written application should handle, typically used when recovery from the error is possible or required.	It indicates programming bugs, such as incorrect logic, often leading to program termination rather than recovery.
It is checked by the compiler at compile-time but occurs at runtime.	It is not checked by the compiler at compile-time, but like a <code>CheckedException</code> , it occurs at runtime.
It ensures that the code will not compile unless the exception is handled.	It allows the JVM to compile the code even if the exception is not handled.
It requires the method throwing checked exceptions to either handle them using a <code>try-catch</code> block or declare them using the throws keyword in the method signature.	It is not required for methods that throw unchecked exceptions to handle them since these exceptions are automatically thrown by the JVM at runtime when an exceptional condition occurs during program execution.
Examples: <code>IOException</code> , <code>SQLException</code> , and <code>ClassNotFoundException</code> .	Examples: <code>ArithmeticException</code> , <code>NullPointerException</code> , <code>ArrayIndexOutOfBoundsException</code> and <code>IllegalArgumentException</code> .

QUESTION

Explain Runtime Exceptions.



Explanation



From: The Complete Java Interview Questions Course on Udemy © Peerless Geek

EXPLANATION

Runtime Exception

Runtime exceptions are a category of exceptions that occur during the execution of a program, as opposed to checked exceptions, which are verified by the compiler at compile-time.

- They inherit from *java.lang.RuntimeException*.
- They are also known as unchecked exceptions because the compiler doesn't check them at compile-time, allowing the JVM to run the code even if they aren't handled.
- They indicate programming bugs, such as incorrect logic, often leading to program terminations rather than recoveries.
- They are automatically thrown by the JVM at runtime when exceptional conditions occur during program executions.
- Examples: **ArithmeticException**, **NullPointerException**, **ArrayIndexOutOfBoundsException**, **IllegalArgumentException** and others.

java.lang.RuntimeException

UncheckedExceptions Classes

- NullPointerException,
- IndexOutOfBoundsException,
 - StringIndexOutOfBoundsException,
 - ArrayIndexOutOfBoundsException,
- ClassCastException,
- ConcurrentModificationException,
- IllegalArgumentException, and **others**.

QUESTION

How many **types of exceptions** can occur in a Java program? 

Explanation



From: The Complete Java Interview Questions Course on Udemy © Peerless Geek

EXPLANATION

Built-In Exceptions	User-Defined Exceptions
Built-In Exceptions are predefined exceptions in Java's core libraries and include both checked and unchecked exceptions.	User-Defined Exceptions are defined by creating a new class that extends either the Exception class or one of its subclasses.
The exception classes are typically defined in <code>java.lang</code> or other standard Java packages like <code>java.io</code> , <code>java.sql</code> , <code>java.util</code> etc., and they extend the <code>java.lang.Exception</code> class.	They're part of the application's package structure.
They're designed to handle common runtime errors and exceptions, like arithmetic errors, null references, and array index issues.	They're designed to address specific error conditions that are not covered by the built-in exceptions.
These exceptions are automatically thrown by the JVM or runtime environment when a specific error occurs.	These exceptions must be manually thrown in the code using the <code>throw</code> keyword.
<i>Examples:</i> i) Checked Exceptions: SQLException, IOException. ii) Unchecked Exceptions: ArrayIndexOutOfBoundsException, NullPointerException, ArithmeticException, ConcurrentModificationException.	<i>Examples:</i> InsufficientFundsException, InvalidInputException, AgeNotValidException.

QUESTION

What is the
difference between
an **Error** and an
Exception? 

Explanation



EXPLANATION

Exception	Error
Exceptions indicate exceptional (unexpected) conditions that might occur that can disrupt the program's normal flow.	Errors indicate severe and usually unrecoverable problems in the application.
The exception classes are typically defined in java.lang or other standard Java packages like java.io , java.sql , java.util etc., and they extend the java.lang.Exception class.	The error classes in Java are typically defined in the java.lang package and extend the java.lang.Error class, a subclass of java.lang.Throwable .
They're caused by programming errors.	They're generally caused by external factors or system failures rather than issues in the application's code.
They must be caught (for checked exceptions) or can be caught (for unchecked exceptions) using try-catch blocks to prevent the program from terminating abruptly.	They are not expected to be caught or handled, and the program is usually allowed to terminate if an error occurs.
They always occur at runtime, but checked exceptions are known to the compiler, while unchecked exceptions are not.	They also occur at runtime, but they represent serious problems that are typically beyond the control of the application.
<i>Examples:</i> i) Checked Exceptions: <code>SQLException</code> , <code>IOException</code> . ii) Unchecked Exceptions: <code>ArrayIndexOutOfBoundsException</code> , <code>NullPointerException</code> , <code>ArithmeticException</code> , <code>ConcurrentModificationException</code> .	<i>Examples:</i> <code>OutOfMemoryError</code> , <code>StackOverflowError</code> , etc.

QUESTION

What is NullPointerException Exception?

Explanation



EXPLANATION

NullPointerException

NullPointerException is an unchecked exception (runtime exception) that occurs when a program attempts to access and manipulate an object whose reference is **null**.

From: The Ultimate Animated Java Interview Questions Course on Udemy © Peerless Geek

EXPLANATION

NullPointerException

NullPointerException is an unchecked exception (runtime exception) that occurs when a program attempts to access and manipulate an object whose reference is **null**.

Course on Udemy © Peerless Geek

```
public static void main(String[] args) {  
    int[] nums1 = {1, 2, 3, 4, 5};  
    int[] nums2 = null;  
  
    System.out.println(nums1[0]); // 1  
    System.out.println(nums2[0]); // This  
    will throw a `NullPointerException`.  
}
```

```
public static void main(String[] args) {  
    String str = null;  
  
    // This will throw a  
    `NullPointerException`.  
    System.out.println(str.length());  
}
```

QUESTION

When is the
**ArrayStore
Exception**
thrown? 

Explanation



From: The Complete Java Interview Questions Course on Udemy © Peerless Geek

EXPLANATION

ArrayStoreException

ArrayStoreException is an unchecked exception (runtime exception) which typically occurs when an attempt is made to store an element of an incompatible type in an array.

From: The Ultimate Animated Java Interview Questions Course on Udemy © Peerless Geek

EXPLANATION

ArrayStoreException

ArrayStoreException is an unchecked exception (runtime exception) which typically occurs when an attempt is made to store an element of an incompatible type in an array.

`strs` is pointing to a String object whose memory address is `0x7FFF5FBFF6A8`

```
Object[] strs = new String[5];
```

```
str[0] = "Java";  
str[0] = 10; // This statement  
Will produce a runtime exception  
called `ArrayStoreException`.
```

Java

(0x7FFF5FBFF6A8)

Java Heap

QUESTION

How does an exception
propagate in the
code? 

Explanation



From: The Complete Java Interview Questions Course on Udemy © Peerless Geek

EXPLANATION

Exception Propagation

Exception propagation, also known as exception throwing or exception raising, refers to the process of *passing an exception from one method to another* in the call stack until it is caught and handled by an appropriate exception handler or until it reaches the bottom of the call stack.

From: The Ultimate Animated Java Interview Questions Course on Udemy © Peerless Geek

EXPLANATION

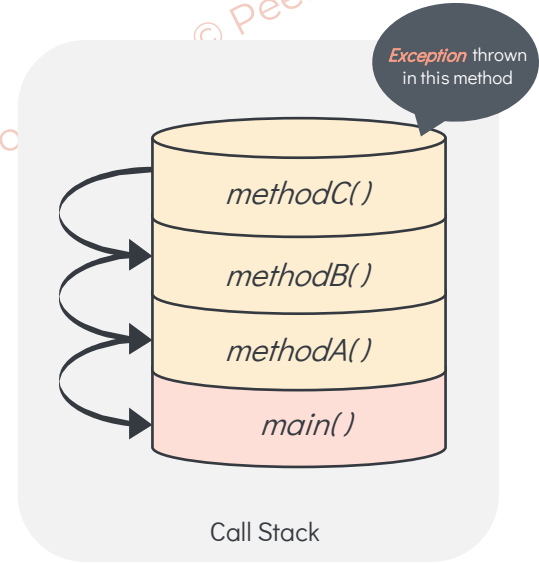
Exception Propagation

Exception propagation, also known as **exception throwing** or **exception raising**, refers to the process of *passing an exception from one method to another* in the call stack until it is caught and handled by an appropriate exception handler or until it reaches the bottom of the call stack.

- When an exception is thrown in a method, it is propagated to the calling method.
- If the calling method does not catch the exception, the exception is further propagated to the calling method of the calling method.
- This process continues until the exception is caught or until it reaches the bottom of the call stack.

Note :

If an exception propagates all the way up to the main method and if the main method also does not catch and handle the exception, the Java Virtual Machine (JVM) will terminate the program abruptly, and print the stack trace of the exception.



QUESTION

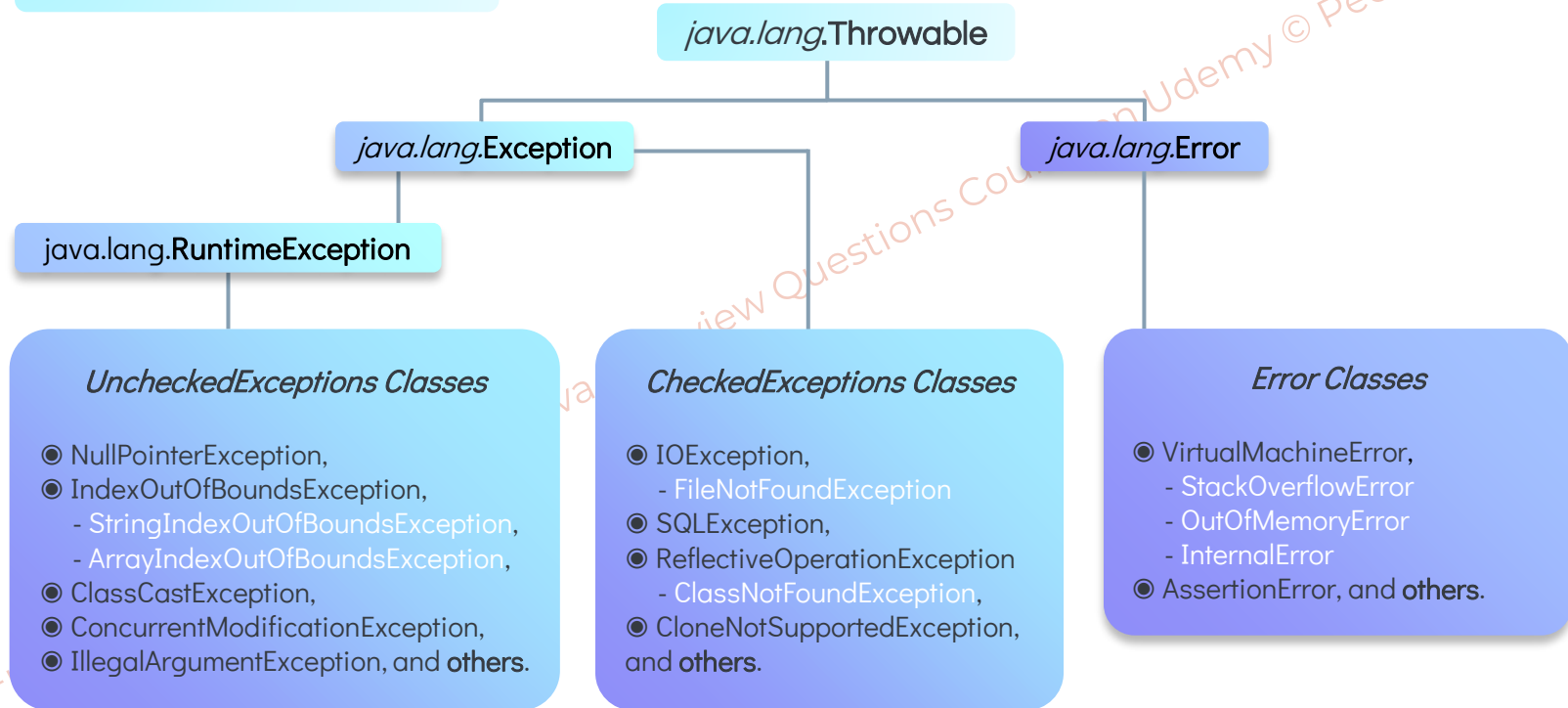
Explain the
hierarchy of Java
Exception and
Error classes. 

Explanation



EXPLANATION

Java Exception Hierarchy



QUESTION

What will happen if you put `System.exit(0)` on the `try` or `catch` `block`? Will `finally` block execute? 

Explanation



EXPLANATION

`exit(int status)`

The `exit(int status)` method is a part of the **System** class in Java, which belongs to the *java.lang* package. It is used to terminate the currently running Java Virtual Machine (JVM), which immediately stops the program's execution.

- It is generally reserved for situations where you want to forcefully terminate a program, such as when encountering a critical **error**.
- It takes a parameter '**status**' that represents the exit status of the program, with **0** typically indicating **normal termination** and a **non-zero** status indicating an error or **abnormal termination**.
- It bypasses normal exception handling and method return mechanisms, causing the finally block and any remaining code in the current method to be skipped, even if it's placed inside a try or catch block.

Miscellaneous

17

QUESTION

What is the use of the **final** keyword? 

Explanation



From: The Complete Java Interview Questions Course on Udemy © Peerless Geek

EXPLANATION

final keyword

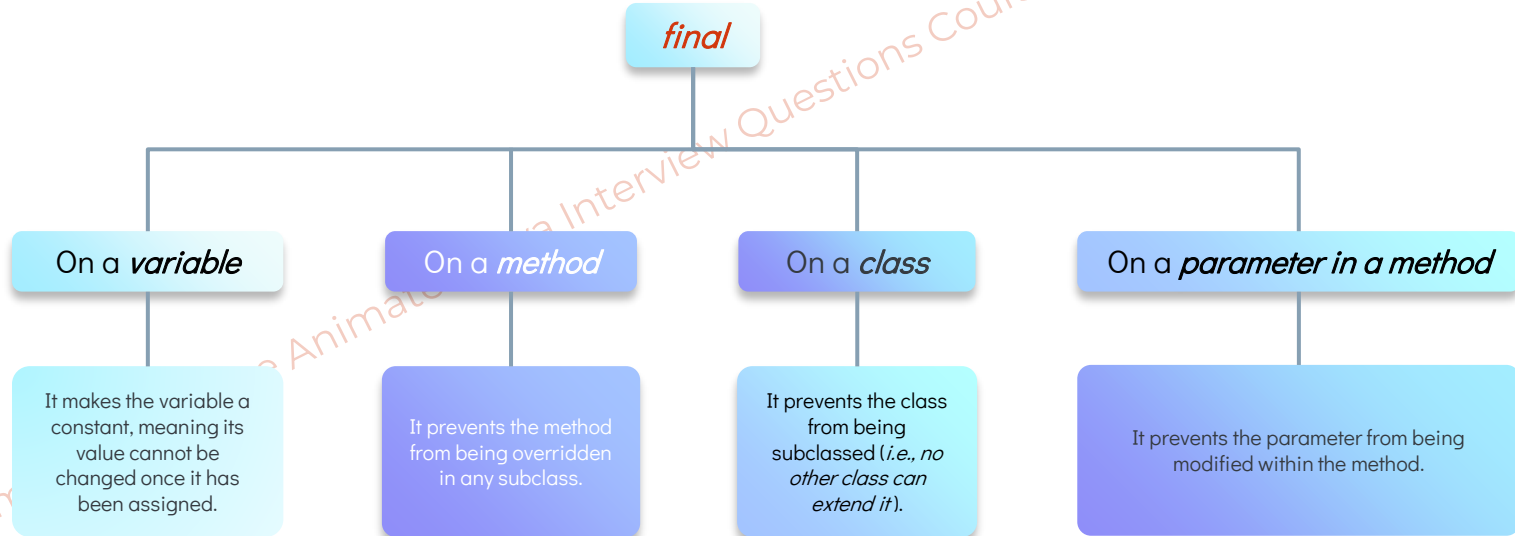
The **final** keyword is used to **impose restrictions** on **variables, methods, and classes** and ensures *immutability, prevents modification, and restricts inheritance or overriding*.

From: The Ultimate Animated Java Interview Questions Course on Udemy © Peerless Geek

EXPLANATION

final keyword

The **final** keyword is used to **impose restrictions** on **variables, methods, and classes** and ensures *immutability, prevents modification, and restricts inheritance or overriding*.



QUESTION

What is blank final variable?

Explanation



From: The Complete Java Interview Questions Course on Udemy © Peerless Geek

EXPLANATION

blank final variable

A **blank final variable** is a **final** variable that is declared but not initialized at the point of declaration. Unlike regular final variables, which must be assigned a value when declared, blank final variables are assigned a value later, typically within a **constructor** or an **instance initializer block**, ensuring that it gets a value before an object is created.

From: The Ultimate Animated Java Interview Questions Course on Udemy © Peerless Geek

QUESTION

What is the difference between **break** and **continue** statements? ✎

Explanation



EXPLANATION

break	continue
The break statement terminates the loop and exits it entirely.	The continue statement skips the current iteration and moves to the next iteration within the loop.
After a break statement, the program control moves to the statement immediately following the loop.	After a continue statement, the program control moves to the next iteration of the loop.
In nested loops, a break statement exits only the innermost loop.	In nested loops, a continue statement skips the current iteration of the innermost loop.
It is typically used when a specific condition is met, and you want to exit the loop prematurely.	It is used when you want to skip a particular iteration based on a condition but continue with the remaining iterations.

QUESTION

What do
you understand
by **Object
cloning**? 

Explanation



Animated Java Interview Questions Course on Udemy © Peerless Geek

From: The

EXPLANATION

Object cloning

Object cloning is the process of creating an exact copy of an existing object. In Java, there are two ways to achieve object cloning: **shallow cloning** and **deep cloning**.

- **Shallow Copy:** In a shallow copy, only the object itself and its primitive data members are duplicated. If the object contains references to other objects, the references are copied, but the actual objects are not.
- **Deep Copy:** In a deep copy, not only the object itself but also all the objects it references (directly or indirectly) are duplicated recursively.
- We can achieve shallow cloning by implementing the **Cloneable** interface and overriding the **clone()** method from the **Object** class

Note :

If the Cloneable interface is not implemented and just the method is overridden, it results in **CloneNotSupportedException** in Java.

QUESTION

What purpose do the keywords **final**, **finally**, and **finalize** fulfill? 

Explanation



From: The Ultimate Java Interview Questions Course on Udemy © Peerless Geek

EXPLANATION

final and finally

- The **final** keyword is used to restrict the behavior of classes, methods, and variables.
- The **finally** block is used with a try-catch block to define a block of code that will be executed after the try block, regardless of whether an exception is thrown or not.

finalize

- **finalize** is a method (deprecated) in the Object class that is called by the garbage collector before an object is reclaimed (garbage collected).
- It was traditionally used to release resources or perform cleanup operations before an object is destroyed.
- It is not recommended in modern Java, and instead, the use of try-with-resources and other mechanisms is preferred for resource management.

QUESTION

What are the possible ways of making objects eligible for **garbage collection** in Java? 

Explanation



From: The Java Interview Questions Course on Udemy © Peerless Geek

EXPLANATION

Garbage Collection

In Java, objects become eligible for **garbage collection** (GC) when there are no more references to them, meaning they can no longer be accessed by any part of the program.

From: The Ultimate Animated Java Interview Questions Course on Udemy © Peerless Geek

EXPLANATION

Garbage Collection

In Java, objects become eligible for **garbage collection** (GC) when there are no more references to them, meaning they can no longer be accessed by any part of the program.

- **Nullifying References:** If an object reference is assigned a null value, it becomes unreachable and qualifies for garbage collection.
- **Resigning References:** If an object reference is updated to point to a different object, the original object becomes eligible for garbage collection, provided no other references exist.
- **Exiting Scope:** If an object is instantiated within a method or block, it becomes eligible for garbage collection once execution of the method or block completes and no remaining references exist.

END

A Heartfelt Thank You ❤️

Thank you for making it to the end — it truly means a lot.

This guide, like the full course, was created with genuine love, care, and a deep passion for teaching. I know how challenging and overwhelming Java interview preparation can be — I've walked that path myself. That's why I set out to make the learning experience feel less stressful, and more clear, visual, and even enjoyable.

If these summary notes helped you in any way — brought you clarity, saved you time, or gave you the confidence to move forward — I'm genuinely glad they served their purpose.

If you feel these notes could help someone else, you're more than welcome to share them with friends, classmates, or anyone preparing for Java interviews. And if you think the full course might support someone's learning journey, sharing that too would mean a lot.

Lastly, if the course added value to your journey and you choose to leave a review on Udemy, I'd be truly grateful. Your thoughts and feedback encourage me to keep going, and to keep creating better, more thoughtful learning experiences for students like you.

Keep learning. Keep coding. And never stop believing in your potential.

With gratitude,

Peerless Geek

Java Educator | Udemy Instructor